

Section 100 Editorial — Grandmaster Finale

01. Temporal Path Median

Restatement and constraints

For every tree path, output its lower median and the minimum sum of absolute deviations from one integer. With n, q up to 200000, extracting and sorting the path separately is impossible.

The two requested values are connected: every median minimizes the sum of absolute deviations. We therefore need the median plus enough aggregate information to evaluate its cost.

How to find the representation

Root the tree at 1. For each vertex v , let $R[v]$ describe the multiset of values on the root-to- v path. If $w = \text{LCA}(u, v)$ and $\text{parent}(w)$ has the empty root when $w=1$, then the multiset on $u \rightarrow v$ is

$$R[u] + R[v] - R[w] - R[\text{parent}(w)].$$

This identity is coefficient-wise: for every compressed value, it gives its exact frequency on the path. Consecutive root paths differ by one inserted value, so $R[v]$ can be a persistent segment tree obtained by cloning one root-to-leaf path from $R[\text{parent}(v)]$.

Store both a count and a sum in every segment-tree node. Counts locate the $k = (\text{length} + 1) / 2$ -th value. While descending, whenever the answer lies in the right child, accumulate the entire left child's count and sum. At the answer leaf, include every occurrence of the median. We now know:

```
count_low = number of path values <= median
sum_low   = sum of path values <= median
sum_all   = sum of every path value.
```

The required cost is

$$\text{median} * \text{count_low} - \text{sum_low} + (\text{sum_all} - \text{sum_low}) - \text{median} * (\text{length} - \text{count_low}).$$

Sample walk-through

On path 4 → 3, the values are 3, 1, 5, 9, sorted as 1, 3, 5, 9. The lower median is 3. The lower group has count 2 and sum 4; the total is 18. The formula gives $3 * 2 - 4 + (18 - 4) - 3 * 2 = 10$.

Algorithm

```
compress every vertex value
root the tree; build binary-lifting ancestors
R[root] = persistent_insert(empty, value[root])
for every other vertex v in parent-before-child order:
    R[v] = persistent_insert(R[parent(v)], value[v])

for each query u,v:
    w = LCA(u,v)
    p = parent(w), or the empty root if w is the tree root
    use roots R[u],R[v],R[w],R[p]
    descend by their combined left counts to find the lower median
    accumulate counts and sums strictly left of the descent
    add the answer leaf, then evaluate the absolute-deviation formula
```

Correctness proof

Lemma 1. $R[v]$ stores exactly the multiset on the root-to- v path.

Proof. The root version inserts only the root. For any other vertex, the induction hypothesis gives the path ending at its parent; inserting $a[v]$ adds exactly the final vertex. Persistence leaves the parent version intact.

Lemma 2. The four-root combination gives the multiset on path $u \rightarrow v$.

Proof. The two root paths contain every vertex on $u \rightarrow v$. Their shared root-to- w prefix is counted twice. Subtracting $R[w]$ and $R[\text{parent}(w)]$ removes two copies of the strict prefix and one extra copy of w , leaving every path vertex exactly once.

Lemma 3. The descent returns the lower median and the accumulated pair is the count and sum of all values at most that median.

Proof. At each value interval, its combined left count is the number of path elements in that half by Lemma 2. The standard order-statistic decision keeps exactly the half containing rank k . A skipped left half lies wholly below the answer and is accumulated. The final leaf contains every value equal to the median, so adding it completes the $\leq$ median aggregates.

Lemma 4. The reported cost is minimum.

Proof. Moving z upward by one changes the objective by the number of values below z minus the number above z . At a median this change changes sign, so a median minimizes the convex objective. Splitting values at the chosen median and expanding the absolute values gives exactly the reported formula.

Theorem. Every query output is the requested lower median and minimum sum.

Proof. Lemmas 1–3 establish the median and its aggregates. Lemma 4 proves both optimality and the evaluated objective.

Complexity

Coordinate compression costs $O(n \log n)$. Building all persistent versions uses $O(n \log n)$ time and nodes. Binary lifting uses $O(n \log n)$ memory and preprocessing. Each query takes $O(\log n)$ time.

Implementation notes and wrong answers

- The path formula subtracts $R[LCA]$ and $R[\text{parent}(LCA)]$, not two copies of $R[LCA]$.
- Use 64-bit sums even though values fit in 32 bits.
- For an even path, use rank $(\text{length}+1)/2$ with one-based ranks.
- Accumulate the complete median leaf; duplicates equal to the median do not belong partly to each side for the cost formula used here.
- Python uses packed `array` storage because millions of ordinary Python integer objects would consume far more memory.

Tests that matter include a one-vertex tree, all equal values, negative values, ancestor-descendant paths, even lengths, a star, and a chain.

C++

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    long long sum = 0;
    int left = 0;
    int right = 0;
    int count = 0;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
    vector<long long> value(n), coordinates;
    for (long long &x : value) cin >> x;
    coordinates = value;
    sort(coordinates.begin(), coordinates.end());
    coordinates.erase(unique(coordinates.begin(), coordinates.end()), coordinates.end());

    vector<vector<int>> graph(n);
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    int log = 1;
    while ((1 << log) <= n) ++log;
    vector<vector<int>> up(log, vector<int>(n));
```

```

vector<int> depth(n), order{0};
up[0][0] = 0;
for (size_t at = 0; at < order.size(); ++at) {
    int u = order[at];
    for (int v : graph[u]) {
        if (v == up[0][u] && u != 0) continue;
        up[0][v] = u;
        depth[v] = depth[u] + 1;
        order.push_back(v);
    }
}
for (int bit = 1; bit < log; ++bit)
    for (int v = 0; v < n; ++v)
        up[bit][v] = up[bit - 1][up[bit - 1][v]];

auto lca = [&](int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int difference = depth[u] - depth[v];
    for (int bit = 0; bit < log; ++bit)
        if (difference >> bit & 1) u = up[bit][u];
    if (u == v) return u;
    for (int bit = log - 1; bit >= 0; --bit) {
        if (up[bit][u] != up[bit][v]) {
            u = up[bit][u];
            v = up[bit][v];
        }
    }
    return up[0][u];
};

vector<Node> tree(1);
tree.reserve(static_cast<size_t>(n) * (log + 2));
auto insert = [&](auto &&self, int previous, int left, int right,
                  int position, long long x) -> int {
    int current = static_cast<int>(tree.size());
    tree.push_back(tree[previous]);
    ++tree[current].count;
    tree[current].sum += x;
    if (right - left == 1) return current;
    int middle = (left + right) / 2;
    if (position < middle)
        tree[current].left = self(self, tree[previous].left, left, middle,
                                   position, x);
    else
        tree[current].right = self(self, tree[previous].right, middle, right,
                                    position, x);
    return current;
};

vector<int> root(n);
for (int v : order) {

```

```

    int previous = v == 0 ? 0 : root[up[0][v]];
    int position = lower_bound(coordinates.begin(), coordinates.end(), value[v])
        - coordinates.begin();
    root[v] = insert(insert, previous, 0, coordinates.size(), position, value[v]);
}

auto combined_count = [&](int a, int b, int c, int d) {
    return tree[a].count + tree[b].count - tree[c].count - tree[d].count;
};

auto combined_sum = [&](int a, int b, int c, int d) {
    return tree[a].sum + tree[b].sum - tree[c].sum - tree[d].sum;
};

while (q--) {
    int u, v;
    cin >> u >> v;
    --u;
    --v;
    int ancestor = lca(u, v);
    int before = ancestor == 0 ? 0 : root[up[0][ancestor]];
    int a = root[u], b = root[v], c = root[ancestor], d = before;
    int length = combined_count(a, b, c, d);
    long long total = combined_sum(a, b, c, d);
    int wanted = (length + 1) / 2;
    int left = 0, right = coordinates.size();
    long long lower_sum = 0;
    int lower_count = 0;

    while (right - left > 1) {
        int al = tree[a].left, bl = tree[b].left;
        int cl = tree[c].left, dl = tree[d].left;
        int count_left = combined_count(al, bl, cl, dl);
        int middle = (left + right) / 2;
        if (wanted <= count_left) {
            a = al;
            b = bl;
            c = cl;
            d = dl;
            right = middle;
        } else {
            wanted -= count_left;
            lower_count += count_left;
            lower_sum += combined_sum(al, bl, cl, dl);
            a = tree[a].right;
            b = tree[b].right;
            c = tree[c].right;
            d = tree[d].right;
            left = middle;
        }
    }
    lower_count += combined_count(a, b, c, d);
}

```

```

        lower_sum += combined_sum(a, b, c, d);
        long long median = coordinates[left];
        long long cost = median * lower_count - lower_sum
                        + (total - lower_sum) - median * (length - lower_count);
        cout << median << ' ' << cost << '\n';
    }
}

```

Python

```

import sys
from array import array

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    n, q = data[index], data[index + 1]
    index += 2
    values = data[index:index + n]
    index += n
    coordinates = sorted(set(values))
    compressed = {x: i for i, x in enumerate(coordinates)}

    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v = data[index] - 1, data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    log = n.bit_length()
    parent = array("i", [0]) * n
    depth = array("i", [0]) * n
    order = [0]
    for u in order:
        for v in graph[u]:
            if v == parent[u] and u != 0:
                continue
            parent[v] = u
            depth[v] = depth[u] + 1
            order.append(v)
    up = [parent]
    for _ in range(1, log):
        previous = up[-1]
        up.append(array("i", (previous[previous[v]] for v in range(n))))

    left_child = array("i", [0])

```

```

right_child = array("i", [0])
count = array("i", [0])
sums = array("q", [0])

def clone(node: int) -> int:
    left_child.append(left_child[node])
    right_child.append(right_child[node])
    count.append(count[node])
    sums.append(sums[node])
    return len(count) - 1

def insert(previous: int, position: int, x: int) -> int:
    new_root = clone(previous)
    current = new_root
    old = previous
    low, high = 0, len(coordinates)
    while True:
        count[current] += 1
        sums[current] += x
        if high - low == 1:
            return new_root
        middle = (low + high) // 2
        if position < middle:
            old_child = left_child[old]
            new_child = clone(old_child)
            left_child[current] = new_child
            old, current, high = old_child, new_child, middle
        else:
            old_child = right_child[old]
            new_child = clone(old_child)
            right_child[current] = new_child
            old, current, low = old_child, new_child, middle

roots = array("i", [0]) * n
for v in order:
    previous = 0 if v == 0 else roots[parent[v]]
    roots[v] = insert(previous, compressed[values[v]], values[v])

def lca(u: int, v: int) -> int:
    if depth[u] < depth[v]:
        u, v = v, u
    difference = depth[u] - depth[v]
    bit = 0
    while difference:
        if difference & 1:
            u = up[bit][u]
            difference >>= 1
        bit += 1
    if u == v:
        return u
    for bit in range(log - 1, -1, -1):

```

```

        if up[bit][u] != up[bit][v]:
            u, v = up[bit][u], up[bit][v]
        return parent[u]

output = []
for _ in range(q):
    u, v = data[index] - 1, data[index + 1] - 1
    index += 2
    ancestor = lca(u, v)
    before = 0 if ancestor == 0 else roots[parent[ancestor]]
    a, b, c, d = roots[u], roots[v], roots[ancestor], before

    length = count[a] + count[b] - count[c] - count[d]
    total = sums[a] + sums[b] - sums[c] - sums[d]
    wanted = (length + 1) // 2
    low, high = 0, len(coordinates)
    lower_count = 0
    lower_sum = 0
    while high - low > 1:
        al, bl = left_child[a], left_child[b]
        cl, dl = left_child[c], left_child[d]
        count_left = count[al] + count[bl] - count[cl] - count[dl]
        middle = (low + high) // 2
        if wanted <= count_left:
            a, b, c, d = al, bl, cl, dl
            high = middle
        else:
            wanted -= count_left
            lower_count += count_left
            lower_sum += sums[al] + sums[bl] - sums[cl] - sums[dl]
            a, b = right_child[a], right_child[b]
            c, d = right_child[c], right_child[d]
            low = middle

    lower_count += count[a] + count[b] - count[c] - count[d]
    lower_sum += sums[a] + sums[b] - sums[c] - sums[d]
    median = coordinates[low]
    cost = (median * lower_count - lower_sum
            + total - lower_sum - median * (length - lower_count))
    output.append(f"{median} {cost}")
print("\n".join(output))

if __name__ == "__main__":
    main()

```


02. Bipartite Timeline

Restatement and decisive observations

Constraints of the form $\text{bit}[u] \text{ xor } \text{bit}[v] = p$ are inserted and deleted. At every query we need the number of complete assignments satisfying the active system.

In one connected consistent constraint component, choosing one vertex's bit forces every other bit, so there are exactly two assignments. With c components there are 2^c . A parity DSU can maintain both connectivity and the xor from a vertex to its root, detecting a contradiction when a new edge joins vertices already in the same component with the wrong implied parity.

Deletion is the obstacle. Each inserted constraint is active on one time interval $[\text{insert_time}, \text{remove_time})$. Put that interval into a segment tree over time. During a DFS of this segment tree, add all constraints stored at the current node and roll the DSU back on exit. A leaf then sees exactly the constraints active at its time.

Why ordinary DSU is insufficient

Path compression rewrites an unbounded set of parent pointers and is awkward to undo. Use union by size without compression. Its height is logarithmic. Record every merge, consistent redundant edge, and contradiction on a history stack. A snapshot is just the stack length.

For a constraint between u and v , let x_u and x_v be their xor values to their roots. When different roots are joined, the child root must receive

```
parity_to_parent =  $x_u \text{ xor } x_v \text{ xor required}$ .
```

Sample walk-through

Initially three isolated components give $2^3=8$. Constraint $1 \text{ xor } 2=0$ merges two components, leaving 4 assignments. Adding $2 \text{ xor } 3=1$ leaves one component and 2 assignments. The constraint $1 \text{ xor } 3=0$ contradicts the already implied xor of 1, so the answer becomes zero. Removing that exact interval restores the earlier snapshot and the answer returns to two.

Algorithm

```
read operations and match every insertion with its removal
add each active interval to a segment tree over operation times
precompute powers_of_two[0..n]

DFS(time_node):
    snapshot = history size
    add every parity constraint stored at this node
    if this is a query leaf:
        print 0 if contradictions > 0 else powers_of_two[components]
    otherwise visit both children
    rollback to snapshot
```

Correctness proof

Lemma 1. At a time-segment leaf τ , the DSU contains exactly the constraints active after operation τ .

Proof. An interval is stored in a disjoint collection of segment-tree nodes whose ranges partition that interval. The root-to-leaf path for τ contains exactly one of those nodes iff τ lies in the interval. DFS rollback removes all constraints belonging only to a completed sibling range.

Lemma 2. The parity DSU reports a contradiction exactly when the inserted constraints are inconsistent.

Proof. In each DSU component, stored xor potentials realize every forest constraint. Joining different roots extends that realization using the stated root parity. An edge inside one component is satisfiable exactly when its required xor equals the xor already implied by the two root potentials.

Lemma 3. A consistent DSU state with c components has 2^c assignments.

Proof. Choose one root bit independently for every component. Potentials then force every vertex bit, giving one assignment per root-bit vector. Conversely, every satisfying assignment restricts to exactly one such vector.

Theorem. Every ? answer is the number of assignments satisfying precisely the active constraints.

Proof. Lemma 1 identifies the active system, Lemma 2 distinguishes the zero case, and Lemma 3 counts every consistent case.

Complexity

Each constraint interval occupies $O(\log q)$ time-tree nodes. Union by size gives $O(\log n)$ worst-case root searches, so total time is $O(q \log q \log n)$ and stored interval data is $O(q \log q)$. Rollback DSU state uses $O(n+q \log q)$ memory including the active history.

Implementation notes and wrong answers

- A removal at time τ makes the interval right endpoint exclusive.
- A self-constraint with parity one is an immediate contradiction.
- Redundant consistent edges must still place a history marker so snapshots undo exactly one addition at a time.
- Track the number of contradictory active edges, not just a Boolean; several contradictions can overlap in time.
- Count isolated vertices as components.

Stress with parallel constraints, repeated contradictory edges, self-edges, intervals ending at the final operation, no active constraints, and alternating insert/remove operations.

C++

```
#include <bits/stdc++.h>
using namespace std;

constexpr int MOD = 1'000'000'007;

struct Constraint {
    int u, v, parity;
```

```

};

class RollbackParityDSU {
    struct Change {
        int type;
        int child = -1;
        int root = -1;
        int root_size = 0;
        int child_size = 0;
        int child_parity = 0;
    };

    vector<int> parent;
    vector<int> parity_to_parent;
    vector<Change> history;

public:
    int components;
    int contradictions = 0;

    explicit RollbackParityDSU(int n)
        : parent(n, -1), parity_to_parent(n), components(n) {}

    pair<int, int> find(int x) const {
        int parity = 0;
        while (parent[x] >= 0) {
            parity ^= parity_to_parent[x];
            x = parent[x];
        }
        return {x, parity};
    }

    int snapshot() const { return static_cast<int>(history.size()); }

    void add(int u, int v, int required) {
        auto [root_u, parity_u] = find(u);
        auto [root_v, parity_v] = find(v);
        if (root_u == root_v) {
            if ((parity_u ^ parity_v) != required) {
                ++contradictions;
                history.push_back({1});
            } else {
                history.push_back({0});
            }
        }
        return;
    }

    if (-parent[root_u] < -parent[root_v]) {
        swap(root_u, root_v);
        swap(parity_u, parity_v);
    }
    history.push_back({2, root_v, root_u, parent[root_u], parent[root_v],

```

```

        parity_to_parent[root_v]});
parent[root_u] += parent[root_v];
parent[root_v] = root_u;
parity_to_parent[root_v] = parity_u ^ parity_v ^ required;
--components;
}

void rollback(int target) {
    while (static_cast<int>(history.size()) > target) {
        Change change = history.back();
        history.pop_back();
        if (change.type == 1) {
            --contradictions;
        } else if (change.type == 2) {
            parent[change.root] = change.root_size;
            parent[change.child] = change.child_size;
            parity_to_parent[change.child] = change.child_parity;
            ++components;
        }
    }
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
    vector<vector<Constraint>> timeline(4 * q + 4);
    vector<bool> is_query(q);
    vector<int> start(q, -1);
    vector<Constraint> inserted(q);

    auto add_interval = [&](auto &&self, int node, int left, int right,
                            int query_left, int query_right,
                            Constraint constraint) -> void {
        if (query_right <= left || right <= query_left) return;
        if (query_left <= left && right <= query_right) {
            timeline[node].push_back(constraint);
            return;
        }
        int middle = (left + right) / 2;
        self(self, node * 2, left, middle, query_left, query_right, constraint);
        self(self, node * 2 + 1, middle, right, query_left, query_right, constraint);
    };

    for (int time = 0; time < q; ++time) {
        char type;
        cin >> type;
        if (type == '+') {

```

```

        int u, v, parity;
        cin >> u >> v >> parity;
        inserted[time] = {u - 1, v - 1, parity};
        start[time] = time;
    } else if (type == '-') {
        int identifier;
        cin >> identifier;
        --identifier;
        add_interval(add_interval, 1, 0, q, start[identifier], time,
                     inserted[identifier]);
        start[identifier] = -1;
    } else {
        is_query[time] = true;
    }
}

for (int identifier = 0; identifier < q; ++identifier)
    if (start[identifier] != -1)
        add_interval(add_interval, 1, 0, q, start[identifier], q,
                     inserted[identifier]);

vector<int> powers(n + 1, 1);
for (int i = 1; i <= n; ++i)
    powers[i] = static_cast<long long>(powers[i - 1]) * 2 % MOD;

RollbackParityDSU dsu(n);
auto solve = [&](auto &&self, int node, int left, int right) -> void {
    int saved = dsu.snapshot();
    for (Constraint constraint : timeline[node])
        dsu.add(constraint.u, constraint.v, constraint.parity);
    if (right - left == 1) {
        if (is_query[left])
            cout << (dsu.contradictions ? 0 : powers[dsu.components]) << '\n';
    } else {
        int middle = (left + right) / 2;
        self(self, node * 2, left, middle);
        self(self, node * 2 + 1, middle, right);
    }
    dsu.rollback(saved);
};
solve(solve, 1, 0, q);
}

```

Python

```

import sys

MOD = 1_000_000_007

class RollbackParityDSU:

```

```

def __init__(self, n: int) -> None:
    self.parent = [-1] * n
    self.parity = [0] * n
    self.components = n
    self.contradictions = 0
    self.history = []

def find(self, x: int) -> tuple[int, int]:
    parity = 0
    while self.parent[x] >= 0:
        parity ^= self.parity[x]
        x = self.parent[x]
    return x, parity

def add(self, u: int, v: int, required: int) -> None:
    root_u, parity_u = self.find(u)
    root_v, parity_v = self.find(v)
    if root_u == root_v:
        if parity_u ^ parity_v != required:
            self.contradictions += 1
            self.history.append((1,))
        else:
            self.history.append((0,))
        return
    if -self.parent[root_u] < -self.parent[root_v]:
        root_u, root_v = root_v, root_u
        parity_u, parity_v = parity_v, parity_u
    self.history.append((2, root_v, root_u, self.parent[root_u],
                        self.parent[root_v], self.parity[root_v]))
    self.parent[root_u] += self.parent[root_v]
    self.parent[root_v] = root_u
    self.parity[root_v] = parity_u ^ parity_v ^ required
    self.components -= 1

def rollback(self, target: int) -> None:
    while len(self.history) > target:
        change = self.history.pop()
        if change[0] == 1:
            self.contradictions -= 1
        elif change[0] == 2:
            _, child, root, root_size, child_size, child_parity = change
            self.parent[root] = root_size
            self.parent[child] = child_size
            self.parity[child] = child_parity
            self.components += 1

def main() -> None:
    lines = sys.stdin.buffer.readlines()
    if not lines:
        return

```

```

n, q = map(int, lines[0].split())
size = 1
while size < q:
    size *= 2
timeline = [[] for _ in range(2 * size)]
queries = [False] * q
start = [-1] * q
inserted = [None] * q

def add_interval(left: int, right: int, constraint: tuple[int, int, int]) -> None:
    left += size
    right += size
    while left < right:
        if left & 1:
            timeline[left].append(constraint)
            left += 1
        if right & 1:
            right -= 1
            timeline[right].append(constraint)
    left //= 2
    right //= 2

for time, raw in enumerate(lines[1:q + 1]):
    parts = raw.split()
    if parts[0] == b"+":
        constraint = (int(parts[1]) - 1, int(parts[2]) - 1, int(parts[3]))
        inserted[time] = constraint
        start[time] = time
    elif parts[0] == b"-":
        identifier = int(parts[1]) - 1
        add_interval(start[identifier], time, inserted[identifier])
        start[identifier] = -1
    else:
        queries[time] = True
for identifier in range(q):
    if start[identifier] != -1:
        add_interval(start[identifier], q, inserted[identifier])

powers = [1] * (n + 1)
for i in range(1, n + 1):
    powers[i] = powers[i - 1] * 2 % MOD

dsu = RollbackParityDSU(n)
output = []
stack = [(1, 0, size, 0)]
snapshots = {}
while stack:
    node, left, right, phase = stack.pop()
    if phase == 0:
        saved = len(dsu.history)
        snapshots[node] = saved

```

```

    for u, v, parity in timeline[node]:
        dsu.add(u, v, parity)
    if right - left == 1:
        if left < q and queries[left]:
            output.append("0" if dsu.contradictions
                           else str(powers[dsu.components]))
        dsu.rollback(saved)
    else:
        middle = (left + right) // 2
        stack.append((node, left, right, 1))
        stack.append((node * 2 + 1, middle, right, 0))
        stack.append((node * 2, left, middle, 0))
    else:
        dsu.rollback(snapshots.pop(node))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

03. Versioned Convex DP

Restatement and recognition

Every operation is a node in a version tree. An added line is active exactly in the subtree of its operation node. A query asks for the lexicographically smallest pair (value, line_id) at one coordinate.

A persistent Li Chao tree is possible, but full persistence is unnecessary: all operations are known offline. DFS the version tree, mutate one Li Chao tree, and roll every overwritten node back when leaving a subtree. This is the same lifetime idea as a recursion-stack variable.

Only query coordinates matter. Compress them, and build a discrete Li Chao tree over that sorted set. At each Li Chao node, retain the line best at the middle coordinate. Of the displaced line, at most one side can still beat the retained line because two affine functions cross at most once.

Trace

Version 1 adds $2x+3$ below version 0. At $x=-4$ it beats the permanent zero line, giving $(-5,1)$. When DFS returns to version 0, rollback removes that line. The sibling version 3 adds $-x+5$; it cannot see line 1, and at $x=10$ returns $(-5,3)$. Descending from version 1 later restores line 1 before line 5 is inserted.

Algorithm

```

read all operations; make parent -> children adjacency
compress coordinates from Q operations
initialize the Li Chao tree with line 0

DFS(version):

```



```

    snapshot = change-stack size
    if A: Li-Chao-insert its line, logging every overwritten node
    if Q: query its compressed x and store the answer by operation number
    DFS every child
    restore overwritten nodes until snapshot

print stored query answers in operation order

```

Correctness proof

Lemma 1. On entry to a version node, the Li Chao structure contains exactly the lines added by its strict version-tree ancestors, plus line zero.

Proof. DFS adds the current node's line before visiting descendants. On exit, rollback restores every changed Li Chao node to its entry value, so a line cannot leak into a sibling subtree. Induction on DFS depth proves the claim.

Lemma 2. Li Chao insertion preserves the minimum line at every compressed coordinate.

Proof. The better line at the interval midpoint is retained at the current node. The two lines' value difference is affine, so after their midpoint order is fixed, the displaced line can be better on at most one side. Recursing into that side preserves it wherever necessary; all other previously stored lines retain the same guarantee inductively. Comparing $(\text{value}, \text{id})$ also implements the required tie rule.

Theorem. Every reported pair is the minimum value and smallest attaining identifier among exactly the lines in that version.

Proof. Lemma 1 identifies the active set. Lemma 2 makes the root-to-leaf Li Chao query return its best pair at the requested coordinate.

Complexity, traps, and tests

Each insertion changes $O(\log q)$ Li Chao nodes and each query visits $O(\log q)$ nodes. Total time is $O(q \log q)$ and memory is $O(q)$ besides the version tree and rollback history.

Do not traverse operations in input order as if versions formed one timeline. Log swaps as actual node assignments; rollback must restore the previous line, not merely erase the new line. Include the identifier in every comparison. Test equal lines with different identifiers, equal values at one coordinate, all-add instances, deep chains, wide version stars, and extreme signed slopes.

C++

```

#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

struct Line {
    int64 slope = 0;
    int64 intercept = 0;
    int identifier = -1;
};

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    vector<int> parent(q + 1);
    vector<char> type(q + 1);
    vector<int64> first(q + 1), second(q + 1);
    vector<vector<int>> children(q + 1);
    vector<int64> coordinates;
    for (int version = 1; version <= q; ++version) {
        cin >> parent[version] >> type[version] >> first[version];
        if (type[version] == 'A') cin >> second[version];
        else coordinates.push_back(first[version]);
        children[parent[version]].push_back(version);
    }
    if (coordinates.empty()) coordinates.push_back(0);
    sort(coordinates.begin(), coordinates.end());
    coordinates.erase(unique(coordinates.begin(), coordinates.end()), coordinates.end());

    vector<Line> tree(4 * coordinates.size() + 4);
    vector<pair<int, Line>> history;
    auto value = [](const Line &line, int64 x) {
        return line.slope * x + line.intercept;
    };
    auto better = [&](const Line &a, const Line &b, int64 x) {
        if (a.identifier == -1) return false;
        if (b.identifier == -1) return true;
        return pair{value(a, x), a.identifier} < pair{value(b, x), b.identifier};
    };
    auto assign = [&](int node, Line line) {
        history.push_back({node, tree[node]});
        tree[node] = line;
    };
    auto insert = [&](auto &&self, int node, int left, int right, Line line) -> void {
        if (tree[node].identifier == -1) {
            assign(node, line);
            return;
        }
        int middle = (left + right) / 2;
        if (better(line, tree[node], coordinates[middle])) {
            Line displaced = tree[node];
            assign(node, line);
            line = displaced;
        }
        if (right - left == 1) return;
        if (better(line, tree[node], coordinates[left]))
            self(self, node * 2, left, middle, line);
        else if (better(line, tree[node], coordinates[right - 1]))
            self(self, node * 2 + 1, middle, right, line);
    };
}

```

```

};
auto query = [&](int position) {
    int node = 1, left = 0, right = coordinates.size();
    Line answer;
    while (true) {
        if (better(tree[node], answer, coordinates[position])) answer = tree[node];
        if (right - left == 1) break;
        int middle = (left + right) / 2;
        if (position < middle) {
            node *= 2;
            right = middle;
        } else {
            node = node * 2 + 1;
            left = middle;
        }
    }
    return answer;
};

auto rollback = [&](int snapshot) {
    while (static_cast<int>(history.size()) > snapshot) {
        auto [node, old_line] = history.back();
        history.pop_back();
        tree[node] = old_line;
    }
};

insert(insert, 1, 0, coordinates.size(), {0, 0, 0});
history.clear();
vector<pair<int64, int>> answers(q + 1);
vector<tuple<int, int, int>> stack;
for (auto it = children[0].rbegin(); it != children[0].rend(); ++it)
    stack.push_back({*it, 0, 0});
while (!stack.empty()) {
    auto [version, phase, snapshot] = stack.back();
    stack.pop_back();
    if (phase == 0) {
        snapshot = history.size();
        if (type[version] == 'A') {
            insert(insert, 1, 0, coordinates.size(),
                {first[version], second[version], version});
        } else {
            int position = lower_bound(coordinates.begin(), coordinates.end(),
                first[version]) - coordinates.begin();
            Line line = query(position);
            answers[version] = {value(line, first[version]), line.identifier};
        }
        stack.push_back({version, 1, snapshot});
        for (auto it = children[version].rbegin(); it != children[version].rend(); ++it)
            stack.push_back({*it, 0, 0});
    } else {
        rollback(snapshot);
    }
}

```

```

    }
}
for (int version = 1; version <= q; ++version)
    if (type[version] == 'Q')
        cout << answers[version].first << ' ' << answers[version].second << '\n';
}

```

Python

```

import bisect
import sys

def main() -> None:
    lines = sys.stdin.buffer.readlines()
    if not lines:
        return
    q = int(lines[0])
    parent = [0] * (q + 1)
    kind = [b""] * (q + 1)
    first = [0] * (q + 1)
    second = [0] * (q + 1)
    children = [[] for _ in range(q + 1)]
    coordinates = []
    for version, raw in enumerate(lines[1:q + 1], 1):
        parts = raw.split()
        parent[version] = int(parts[0])
        kind[version] = parts[1]
        first[version] = int(parts[2])
        if parts[1] == b"A":
            second[version] = int(parts[3])
        else:
            coordinates.append(first[version])
            children[parent[version]].append(version)
    coordinates = sorted(set(coordinates)) or [0]
    tree = [None] * (4 * len(coordinates) + 4)
    history = []

    def key(line: tuple[int, int, int], x: int) -> tuple[int, int]:
        return line[0] * x + line[1], line[2]

    def better(a, b, x: int) -> bool:
        return a is not None and (b is None or key(a, x) < key(b, x))

    def assign(node: int, line) -> None:
        history.append((node, tree[node]))
        tree[node] = line

    def insert(line) -> None:
        node, left, right = 1, 0, len(coordinates)

```

```

while True:
    if tree[node] is None:
        assign(node, line)
        return
    middle = (left + right) // 2
    if better(line, tree[node], coordinates[middle]):
        displaced = tree[node]
        assign(node, line)
        line = displaced
    if right - left == 1:
        return
    if better(line, tree[node], coordinates[left]):
        node *= 2
        right = middle
    elif better(line, tree[node], coordinates[right - 1]):
        node = node * 2 + 1
        left = middle
    else:
        return

def query(position: int):
    node, left, right = 1, 0, len(coordinates)
    answer = None
    while True:
        if better(tree[node], answer, coordinates[position]):
            answer = tree[node]
        if right - left == 1:
            return answer
        middle = (left + right) // 2
        if position < middle:
            node *= 2
            right = middle
        else:
            node = node * 2 + 1
            left = middle

insert((0, 0, 0))
history.clear()
answers = [None] * (q + 1)
stack = [(version, 0, 0) for version in reversed(children[0])]
while stack:
    version, phase, snapshot = stack.pop()
    if phase == 0:
        snapshot = len(history)
        if kind[version] == b"A":
            insert((first[version], second[version], version))
        else:
            position = bisect.bisect_left(coordinates, first[version])
            line = query(position)
            answers[version] = key(line, first[version])
    stack.append((version, 1, snapshot))

```

```

        stack.extend((child, 0, 0) for child in reversed(children[version]))
    else:
        while len(history) > snapshot:
            node, old_line = history.pop()
            tree[node] = old_line
print("\n".join(f"{answers[i][0]} {answers[i][1]}"
                for i in range(1, q + 1) if kind[i] == b"Q"))

if __name__ == "__main__":
    main()

```

04. Historical Rectangle Selection

From quadratic DP to a geometric query

Let $dp[i]$ be the maximum score of a valid subsequence ending at record i , and $ways[i]$ its number of realizations. A predecessor j is legal exactly when

```
j < i, x[j] < x[i], and y[j] < y[i].
```

Thus each transition asks for the best state among earlier points in an open lower-left rectangle. A Fenwick tree can answer the y prefix after points are ordered by x , but the historical condition $j < i$ prevents one global sort.

CDQ divide and conquer keeps time order as its recursion dimension. After the left half has been solved, sweep its points and the right-half query points by x . Insert a left point only while its x is strictly smaller, then query the Fenwick prefix strictly below the right point's y . Apply these cross-half transitions before recursively solving the right half.

The Fenwick payload is $(best_score, ways)$. Combining unequal scores keeps the larger; combining equal positive scores adds their counts modulo MOD .

Why counts are not duplicated

For any predecessor pair $j < i$, there is one unique CDQ node where j lies in the left child and i in the right child. Their transition is considered exactly at that node. It is not considered in descendants, where they are in different recursive calls, or ancestors, where they lie on the same side or are represented by another unique split.

Algorithm

```

initialize dp[i]=w[i], ways[i]=1
compress all y coordinates

CDQ(l,r):
    if one record: return
    mid=(l+r)/2
    CDQ(l,mid)                # left dp values are now final

```

```

sort left and right indices by x
sweep right indices:
    insert left indices with x_left < x_right into y-Fenwick
    candidate = Fenwick.query(y_right-1)
    relax (dp[right],ways[right]) with candidate + w[right]
clear every touched Fenwick cell
CDQ(mid,r)

```

combine all endpoint states for the global answer

Correctness proof

Lemma 1. During a CDQ merge, the Fenwick query for right record i combines exactly the left-half records satisfying both strict coordinate inequalities.

Proof. The x sweep inserts precisely points with smaller x . A prefix at compressed rank $y[i]-1$ retains precisely inserted points with smaller y .

Lemma 2. Every legal DP transition is applied exactly once.

Proof. Use the unique recursion node described in the counting argument above. Lemma 1 accepts it there, and no other node separates the pair in the required left-to-right direction.

Lemma 3. On return from $CDQ(l,r)$, every state in that interval includes all legal predecessors within the interval and all transitions already passed from ancestor left halves; its score and count are exact.

Proof. The left half is exact by induction. The merge applies all and only left-to-right transitions by Lemmas 1–2. The right recursive call then closes all transitions internal to the right half. Max-and-count combination is the DP recurrence, and positive rewards make the singleton base explicit.

Theorem. The final pair is the maximum score and number of optimal index subsequences.

Proof. Every nonempty subsequence has one final index. Lemma 3 gives the exact best score and count for each endpoint; combining equal global maxima counts these disjoint endpoint classes.

Complexity, traps, and tests

The direct implementation sorts at every recursion level, taking $O(n \log^2 n)$ time and $O(n)$ working memory. A merge-sorted CDQ can reduce this to $O(n \log n)$, but is not needed for the limits.

Use strict comparisons for both coordinates. Clear every Fenwick cell touched by a merge before processing its sibling. Do not merge equal zero sentinel states as real empty subsequences. Count index subsequences, so equal-coordinate records remain distinct. Test all equal points, negative coordinates, many equal optimal predecessors, a decreasing sequence, and rewards that force 64-bit totals.

C++

```

#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

constexpr int MOD = 1'000'000'007;

```

```

struct State {
    int64 score = 0;
    int ways = 0;
};

State combine(State a, State b) {
    if (a.score != b.score) return a.score > b.score ? a : b;
    if (a.score == 0) return {0, 0};
    return {a.score, static_cast<int>((a.ways + static_cast<long long>(b.ways)) % MOD)};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> x(n), y(n), compressed_y;
    vector<int64> reward(n), dp(n);
    vector<int> ways(n, 1);
    for (int i = 0; i < n; ++i) {
        cin >> x[i] >> y[i] >> reward[i];
        compressed_y.push_back(y[i]);
        dp[i] = reward[i];
    }
    sort(compressed_y.begin(), compressed_y.end());
    compressed_y.erase(unique(compressed_y.begin(), compressed_y.end()),
                        compressed_y.end());
    vector<int> rank_y(n);
    for (int i = 0; i < n; ++i)
        rank_y[i] = lower_bound(compressed_y.begin(), compressed_y.end(), y[i])
                    - compressed_y.begin() + 1;

    vector<State> bit(compressed_y.size() + 1);
    vector<int> touched;
    auto update = [&](int position, State state) {
        for (int i = position; i < static_cast<int>(bit.size()); i += i & -i) {
            bit[i] = combine(bit[i], state);
            touched.push_back(i);
        }
    };

    auto query = [&](int position) {
        State result;
        for (int i = position; i > 0; i -= i & -i)
            result = combine(result, bit[i]);
        return result;
    };

    auto cdq = [&](auto &&self, int left, int right) -> void {
        if (right - left == 1) return;
        int middle = (left + right) / 2;

```



```

    self(self, left, middle);
    vector<int> first, second;
    first.reserve(middle - left);
    second.reserve(right - middle);
    for (int i = left; i < middle; ++i) first.push_back(i);
    for (int i = middle; i < right; ++i) second.push_back(i);
    sort(first.begin(), first.end(), [&](int a, int b) {
        return pair{x[a], y[a]} < pair{x[b], y[b]};
    });
    sort(second.begin(), second.end(), [&](int a, int b) {
        return pair{x[a], y[a]} < pair{x[b], y[b]};
    });
    size_t pointer = 0;
    for (int j : second) {
        while (pointer < first.size() && x[first[pointer]] < x[j]) {
            int i = first[pointer++];
            update(rank_y[i], {dp[i], ways[i]});
        }
        State predecessor = query(rank_y[j] - 1);
        if (predecessor.score == 0) continue;
        int64 candidate = predecessor.score + reward[j];
        if (candidate > dp[j]) {
            dp[j] = candidate;
            ways[j] = predecessor.ways;
        } else if (candidate == dp[j]) {
            ways[j] = (ways[j] + static_cast<long long>(predecessor.ways)) % MOD;
        }
    }
    for (int position : touched) bit[position] = {};
    touched.clear();
    self(self, middle, right);
};
cdq(cdq, 0, n);

State answer;
for (int i = 0; i < n; ++i)
    answer = combine(answer, {dp[i], ways[i]});
cout << answer.score << ' ' << answer.ways << '\n';
}

```

Python

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:

```

```

    return
n = data[0]
x = [0] * n
y = [0] * n
reward = [0] * n
at = 1
for i in range(n):
    x[i], y[i], reward[i] = data[at:at + 3]
    at += 3
ys = sorted(set(y))
rank_y = [0] * n
position = {value: i + 1 for i, value in enumerate(ys)}
for i in range(n):
    rank_y[i] = position[y[i]]

dp = reward.copy()
ways = [1] * n
bit_score = [0] * (len(ys) + 1)
bit_ways = [0] * (len(ys) + 1)
touched = []

def update(pos: int, score: int, count: int) -> None:
    while pos < len(bit_score):
        if score > bit_score[pos]:
            bit_score[pos] = score
            bit_ways[pos] = count
        elif score == bit_score[pos] and score:
            bit_ways[pos] = (bit_ways[pos] + count) % MOD
        touched.append(pos)
        pos += pos & -pos

def query(pos: int) -> tuple[int, int]:
    score = count = 0
    while pos:
        if bit_score[pos] > score:
            score, count = bit_score[pos], bit_ways[pos]
        elif bit_score[pos] == score and score:
            count = (count + bit_ways[pos]) % MOD
        pos -= pos & -pos
    return score, count

sys.setrecursionlimit(1_000_000)

def cdq(left: int, right: int) -> None:
    if right - left == 1:
        return
    middle = (left + right) // 2
    cdq(left, middle)
    first = sorted(range(left, middle), key=lambda i: (x[i], y[i]))
    second = sorted(range(middle, right), key=lambda i: (x[i], y[i]))
    pointer = 0

```

```

    for j in second:
        while pointer < len(first) and x[first[pointer]] < x[j]:
            i = first[pointer]
            pointer += 1
            update(rank_y[i], dp[i], ways[i])
        best, count = query(rank_y[j] - 1)
        if best:
            candidate = best + reward[j]
            if candidate > dp[j]:
                dp[j], ways[j] = candidate, count
            elif candidate == dp[j]:
                ways[j] = (ways[j] + count) % MOD
    for pos in touched:
        bit_score[pos] = bit_ways[pos] = 0
    touched.clear()
    cdq(middle, right)

cdq(0, n)
best = max(dp)
count = sum(ways[i] for i in range(n) if dp[i] == best) % MOD
print(best, count)

if __name__ == "__main__":
    main()

```

05. Dynamic Forest Ledger

Restatement and why HLD is not enough

The forest topology changes through valid links and cuts. We need affine updates and sums on arbitrary connected paths. Heavy–light decomposition assumes one fixed tree, so rebuilding it after structural changes is too slow.

A link–cut tree represents each preferred path by an auxiliary splay tree. Its central operation is

```
make_root(u); access(v)
```

after which the auxiliary subtree rooted at v contains exactly the represented tree path $u \rightarrow v$. A path update or query therefore becomes one lazy splay-tree operation.

The information in one auxiliary node

Store the vertex value, auxiliary-subtree size, and auxiliary-subtree sum. For a pending affine map $f(x) = m \cdot x + b$, applying it to an auxiliary subtree of size s changes

```

value -> m*value+b
sum   -> m*sum+b*s.

```

If an old tag is $g(x)=a*x+c$ and the new update is f , descendants must later receive $f(g(x))$, so tag composition is

```
lazy_mul = m*a
lazy_add = m*c+b.
```

All arithmetic is modulo MOD. Path reversal swaps auxiliary children and toggles a flag. It commutes with a uniform affine update because neither the sum nor the per-vertex map depends on path order.

Structural operations

- `access(x)` repeatedly splays ancestors and replaces their preferred right child, finishing with x at the auxiliary root.
- `make_root(x)` accesses x and reverses its exposed root path.
- `link(u,v)` makes u a represented root, then gives it represented parent v ; the guarantee prevents a cycle.
- `cut(u,v)` exposes the guaranteed edge as a two-vertex preferred path and disconnects the left child of v .

Before rotations, push all reversal and affine tags from the auxiliary root down to the rotated node. Otherwise child orientation or aggregates used by a rotation may be stale.

Sample trace

After linking 1-2-3, exposing 1--3 yields sum $1+2+3=6$. Applying $2x+1$ changes the three values to 3,5,7; the aggregate formula obtains $2*6+1*3=15$ without descending. Linking vertex 4, then cutting 2-3, does not alter values. Path 3--4 therefore sums to $7+4=11$.

Algorithm

```
initialize one link-cut node per isolated vertex
for each operation:
    LINK:  make_root(u); parent[u]=v
    CUT:   make_root(u); access(v); detach the exposed edge
    AFFINE: root=expose_path(u,v); apply composed affine tag to root
    SUM:   root=expose_path(u,v); print root.sum
```

Correctness proof

Lemma 1. After `make_root(u)`; `access(v)`, the auxiliary subtree at v contains exactly the represented path $u\rightsquigarrow v$, in path order.

Proof. `make_root` reverses the represented root path so u becomes the represented root. `access(v)` then changes preferred edges along the unique root-to- v path and makes that path one auxiliary splay tree. No off-path represented edge is selected as an auxiliary child.

Lemma 2. `apply_affine` updates every value and the sum of an exposed path correctly, and its stored lazy tag represents exactly the pending composition for descendants.

Proof. Distributing $m*x+b$ over s values gives $m*sum+b*s$. Function composition gives the stated multiplier and addition formulas. Induction on the number of pending updates proves the tag claim.

Lemma 3. Link, cut, rotations, and reversals preserve represented-forest connectivity and all auxiliary aggregates.

Proof. The validity guarantees make link add one inter-component edge and cut remove the exposed existing edge. Splay rotations change only the auxiliary representation, not represented parent paths. Pushing tags before a rotation makes child orientation and values current; pulling the two rotated nodes recomputes their exact size and sum. Reversal changes path orientation but not its vertices or sum.

Theorem. Every operation has the specified effect and every SUM output is correct.

Proof. Lemma 3 maintains the forest and structure. Lemma 1 exposes precisely the requested path. Lemma 2 proves affine updates and aggregates on that path.

Complexity, wrong answers, and tests

Link-cut operations take $O(\log n)$ amortized time. Total time is $O((n+q) \log n)$ and memory is $O(n)$.

The most common error is composing affine tags in the wrong order. Other failures include not pushing all ancestors before splaying, forgetting that `access` changes preferred rather than represented edges, and cutting without first making one endpoint the root. Test multiplier zero, repeated reversals, single-vertex paths, alternating link/cut of the same edge, affine updates before a cut, and updates modulo MOD near the modulus.

C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

constexpr int64 MOD = 1'000'000'007;

struct LinkCutTree {
    int n;
    vector<array<int, 2>> child;
    vector<int> parent, reversed, size;
    vector<int64> value, sum, lazy_mul, lazy_add;

    explicit LinkCutTree(const vector<int64> &initial)
        : n(initial.size()), child(n + 1), parent(n + 1), reversed(n + 1),
          size(n + 1, 1), value(n + 1), sum(n + 1), lazy_mul(n + 1, 1),
          lazy_add(n + 1) {
        size[0] = 0;
        for (int i = 1; i <= n; ++i) value[i] = sum[i] = initial[i - 1];
    }

    bool is_auxiliary_root(int x) const {
        int p = parent[x];
        return p == 0 || (child[p][0] != x && child[p][1] != x);
    }

    void pull(int x) {
        int left = child[x][0], right = child[x][1];
```

```

    size[x] = 1 + size[left] + size[right];
    sum[x] = (sum[left] + value[x] + sum[right]) % MOD;
}

void apply_reverse(int x) {
    if (x == 0) return;
    swap(child[x][0], child[x][1]);
    reversed[x] ^= 1;
}

void apply_affine(int x, int64 multiplier, int64 addition) {
    if (x == 0) return;
    value[x] = (multiplier * value[x] + addition) % MOD;
    sum[x] = (multiplier * sum[x] + addition * size[x]) % MOD;
    lazy_mul[x] = multiplier * lazy_mul[x] % MOD;
    lazy_add[x] = (multiplier * lazy_add[x] + addition) % MOD;
}

void push(int x) {
    if (reversed[x]) {
        apply_reverse(child[x][0]);
        apply_reverse(child[x][1]);
        reversed[x] = 0;
    }
    if (lazy_mul[x] != 1 || lazy_add[x] != 0) {
        apply_affine(child[x][0], lazy_mul[x], lazy_add[x]);
        apply_affine(child[x][1], lazy_mul[x], lazy_add[x]);
        lazy_mul[x] = 1;
        lazy_add[x] = 0;
    }
}

void rotate(int x) {
    int p = parent[x], g = parent[p];
    int side = child[p][1] == x;
    int middle = child[x][side ^ 1];
    if (!is_auxiliary_root(p)) child[g][child[g][1] == p] = x;
    parent[x] = g;
    child[x][side ^ 1] = p;
    parent[p] = x;
    child[p][side] = middle;
    if (middle) parent[middle] = p;
    pull(p);
    pull(x);
}

void splay(int x) {
    vector<int> ancestors{x};
    for (int y = x; !is_auxiliary_root(y);) {
        y = parent[y];
        ancestors.push_back(y);
    }
}

```

```

    }
    while (!ancestors.empty()) {
        push(ancestors.back());
        ancestors.pop_back();
    }
    while (!is_auxiliary_root(x)) {
        int p = parent[x], g = parent[p];
        if (!is_auxiliary_root(p))
            rotate((child[p][1] == x) == (child[g][1] == p) ? p : x);
        rotate(x);
    }
}

void access(int x) {
    int previous = 0;
    for (int current = x; current; current = parent[current]) {
        splay(current);
        child[current][1] = previous;
        pull(current);
        previous = current;
    }
    splay(x);
}

void make_root(int x) {
    access(x);
    apply_reverse(x);
}

void link(int u, int v) {
    make_root(u);
    parent[u] = v;
}

void cut(int u, int v) {
    make_root(u);
    access(v);
    child[v][0] = 0;
    parent[u] = 0;
    pull(v);
}

int expose_path(int u, int v) {
    make_root(u);
    access(v);
    return v;
}

};

int main() {
    ios::sync_with_stdio(false);

```

```

cin.tie(nullptr);

int n, q;
cin >> n >> q;
vector<int64> initial(n);
for (int64 &x : initial) cin >> x;
LinkCutTree tree(initial);
while (q--) {
    string operation;
    int u, v;
    cin >> operation >> u >> v;
    if (operation == "LINK") {
        tree.link(u, v);
    } else if (operation == "CUT") {
        tree.cut(u, v);
    } else if (operation == "AFFINE") {
        int64 multiplier, addition;
        cin >> multiplier >> addition;
        tree.apply_affine(tree.expose_path(u, v), multiplier, addition);
    } else {
        cout << tree.sum[tree.expose_path(u, v)] << '\n';
    }
}
}

```

Python

```

import sys

MOD = 1_000_000_007

class LinkCutTree:
    def __init__(self, values):
        n = len(values)
        self.child = [[0, 0] for _ in range(n + 1)]
        self.parent = [0] * (n + 1)
        self.reversed = [0] * (n + 1)
        self.size = [0] + [1] * n
        self.value = [0] + values
        self.total = [0] + values.copy()
        self.lazy_mul = [1] * (n + 1)
        self.lazy_add = [0] * (n + 1)

    def is_auxiliary_root(self, x):
        p = self.parent[x]
        return not p or (self.child[p][0] != x and self.child[p][1] != x)

    def pull(self, x):
        left, right = self.child[x]

```



```

self.size[x] = 1 + self.size[left] + self.size[right]
self.total[x] = (self.total[left] + self.value[x] + self.total[right]) % MOD

def apply_reverse(self, x):
    if x:
        self.child[x].reverse()
        self.reversed[x] ^= 1

def apply_affine(self, x, multiplier, addition):
    if not x:
        return
    self.value[x] = (multiplier * self.value[x] + addition) % MOD
    self.total[x] = (multiplier * self.total[x] + addition * self.size[x]) % MOD
    self.lazy_mul[x] = multiplier * self.lazy_mul[x] % MOD
    self.lazy_add[x] = (multiplier * self.lazy_add[x] + addition) % MOD

def push(self, x):
    if self.reversed[x]:
        self.apply_reverse(self.child[x][0])
        self.apply_reverse(self.child[x][1])
        self.reversed[x] = 0
    if self.lazy_mul[x] != 1 or self.lazy_add[x]:
        multiplier, addition = self.lazy_mul[x], self.lazy_add[x]
        self.apply_affine(self.child[x][0], multiplier, addition)
        self.apply_affine(self.child[x][1], multiplier, addition)
        self.lazy_mul[x] = 1
        self.lazy_add[x] = 0

def rotate(self, x):
    p = self.parent[x]
    g = self.parent[p]
    side = self.child[p][1] == x
    middle = self.child[x][side ^ 1]
    if not self.is_auxiliary_root(p):
        self.child[g][self.child[g][1] == p] = x
    self.parent[x] = g
    self.child[x][side ^ 1] = p
    self.parent[p] = x
    self.child[p][side] = middle
    if middle:
        self.parent[middle] = p
    self.pull(p)
    self.pull(x)

def splay(self, x):
    ancestors = [x]
    y = x
    while not self.is_auxiliary_root(y):
        y = self.parent[y]
        ancestors.append(y)
    for y in reversed(ancestors):

```

```

        self.push(y)
    while not self.is_auxiliary_root(x):
        p = self.parent[x]
        g = self.parent[p]
        if not self.is_auxiliary_root(p):
            self.rotate(p if ((self.child[p][1] == x)
                             == (self.child[g][1] == p)) else x)

        self.rotate(x)

def access(self, x):
    previous = 0
    current = x
    while current:
        self.splay(current)
        self.child[current][1] = previous
        self.pull(current)
        previous = current
        current = self.parent[current]
    self.splay(x)

def make_root(self, x):
    self.access(x)
    self.apply_reverse(x)

def link(self, u, v):
    self.make_root(u)
    self.parent[u] = v

def cut(self, u, v):
    self.make_root(u)
    self.access(v)
    self.child[v][0] = 0
    self.parent[u] = 0
    self.pull(v)

def expose_path(self, u, v):
    self.make_root(u)
    self.access(v)
    return v

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    n, q = map(int, data[:2])
    tree = LinkCutTree(list(map(int, data[2:2 + n])))
    index = 2 + n
    output = []
    for _ in range(q):
        operation = data[index]

```

```

    u, v = int(data[index + 1]), int(data[index + 2])
    index += 3
    if operation == b"LINK":
        tree.link(u, v)
    elif operation == b"CUT":
        tree.cut(u, v)
    elif operation == b"AFFINE":
        multiplier, addition = int(data[index]), int(data[index + 1])
        index += 2
        tree.apply_affine(tree.expose_path(u, v), multiplier, addition)
    else:
        output.append(str(tree.total[tree.expose_path(u, v)]))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

06. Colored Distance Census

Restatement and the quadratic barrier

For every distance, count same-color unordered vertex pairs. Running BFS from every vertex is quadratic. A centroid decomposition can assign every pair to one balanced separator, but processing every pair crossing that separator is still quadratic. Polynomial convolution performs all distance-sum pairings at once.

Fix a centroid c . For each color, form a polynomial

```
P[d] = number of vertices of that color at distance d from c
```

inside the current component. The coefficient of degree d in $P*P$ counts ordered same-color pairs whose two distances from c sum to d . This equals their tree distance only if their path passes through c .

For every component created by removing c , form its own polynomial Q and subtract $Q*Q$. The removed ordered pairs have both endpoints in one branch, exactly the pairs whose path does not pass through c . Do this independently for both colors.

Then block the centroid and recurse. NTT computes every square in near-linear time in its polynomial length.

A small trace

Suppose a centroid has one color-zero vertex at distances $0, 1, 2$ in three different branches. The global square contributes ordered pairs at all sums. A branch square removes pairs whose two vertices came from that same branch. The two orientations of each remaining distinct pair survive. A pair removed here remains together in one recursive component and will be counted when a later centroid first separates it.

The term (c, c) appears at distance zero once at the level where c is the centroid. Since distinct pairs are required, distance zero is set to zero explicitly. Every other surviving unordered pair occurs in two orientations, so multiply positive-distance coefficients by the modular inverse of two.

Algorithm

```
answer = zero polynomial

decompose(component):
    find a centroid c
    initialize P[color(c)][0] with one
    for every unblocked neighbor branch B of c:
        collect distances in Q[0] and Q[1]
        answer -= Q[0]^2 + Q[1]^2
        add Q[color] coefficient-wise into P[color]
    answer += P[0]^2 + P[1]^2
    block c
    recursively decompose every remaining component

answer[0] = 0
divide answer[d] by two for every d>0
```

Correctness proof

Lemma 1. At centroid c , after subtracting branch squares, the coefficient at degree d equals the number of ordered same-color pairs in the component whose path passes through c and has length d .

Proof. The global square enumerates every ordered same-color pair by the sum of its endpoint distances from c . Endpoints in different branches, or with one endpoint equal to c , have a path through c , so this sum is their path length. Endpoints in one branch are enumerated by exactly that branch's square and are subtracted. These are precisely the pairs whose path avoids c .

Lemma 2. Every ordered pair of distinct vertices is contributed at exactly one centroid-decomposition node.

Proof. Follow the recursive component containing both endpoints. Eventually its chosen centroid is one endpoint or separates them into different branches; at that first node their path passes through the centroid, so Lemma 1 counts them. At every earlier node they lie in one branch and are subtracted. After the separating node they belong to different recursive components and can never be considered together again.

Lemma 3. For positive distance, dividing by two converts the accumulated ordered count into the requested unordered count.

Proof. Distinct endpoints generate exactly the two orientations (u,v) and (v,u) , with identical color and distance. There are no other representations of the same unordered pair.

Theorem. The printed coefficient for every distance is the requested same-color unordered-pair count modulo 998244353.

Proof. Lemmas 1–2 count every ordered eligible pair once at its unique separator. Lemma 3 gives unordered pairs; explicitly clearing coefficient zero removes self-pairs.

Complexity

A centroid level contains components of total size n . Their distance polynomials have total length $O(n)$, while NTT introduces logarithmic factors. Across $O(\log n)$ centroid levels, the implementation runs in $O(n \log^2 n)$ time. Component sizes shrink geometrically along a recursive chain, so graph, decomposition, and simultaneously live polynomial storage are $O(n)$.

Implementation notes, wrong answers, and tests

- Build separate polynomials for the two colors; squaring their sum would also count different-color pairs.
- Subtract every child square before blocking the centroid.
- Normalize negative modular coefficients.
- Distance arrays begin at index one for a neighbor branch and index zero only for the centroid.
- A naive convolution threshold is worthwhile for numerous tiny components.
- Find centroids iteratively inside a component so an original chain does not overflow the DFS stack.

Test a single vertex, one color only, alternating-color chains, stars, balanced trees, and pairs at the maximum possible distance. The independent oracle runs BFS from every source and enumerates endpoint pairs directly.

C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

constexpr int MOD = 998244353;
constexpr int ROOT = 3;

int mod_power(int base, int exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1) result = result * base % MOD;
        base = static_cast<int64>(base) * base % MOD;
        exponent >>= 1;
    }
    return result;
}

void ntt(vector<int> &a, bool inverse) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        while (j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int length = 2; length <= n; length <= 1) {
        int step = mod_power(ROOT, (MOD - 1) / length);
        if (inverse) step = mod_power(step, MOD - 2);
        for (int start = 0; start < n; start += length) {
            int64 root = 1;
            for (int offset = 0; offset < length / 2; ++offset) {
                int u = a[start + offset];
                int v = root * a[start + offset + length / 2] % MOD;
```

```

        a[start + offset] = u + v;
        if (a[start + offset] >= MOD) a[start + offset] -= MOD;
        a[start + offset + length / 2] = u - v;
        if (a[start + offset + length / 2] < 0)
            a[start + offset + length / 2] += MOD;
        root = root * step % MOD;
    }
}

if (inverse) {
    int inverse_n = mod_power(n, MOD - 2);
    for (int &x : a) x = static_cast<int64>(x) * inverse_n % MOD;
}

vector<int> convolution(const vector<int> &a, const vector<int> &b) {
    if (a.empty() || b.empty()) return {};
    int total = a.size() + b.size() - 1;
    if (min(a.size(), b.size()) < 40) {
        vector<int> result(total);
        for (int i = 0; i < static_cast<int>(a.size()); ++i)
            for (int j = 0; j < static_cast<int>(b.size()); ++j)
                result[i + j] = (result[i + j]
                    + static_cast<int64>(a[i]) * b[j]) % MOD;
        return result;
    }
    int size = 1;
    while (size < total) size <= 1;
    vector<int> first(a.begin(), a.end()), second(b.begin(), b.end());
    first.resize(size);
    second.resize(size);
    ntt(first, false);
    ntt(second, false);
    for (int i = 0; i < size; ++i)
        first[i] = static_cast<int64>(first[i]) * second[i] % MOD;
    ntt(first, true);
    first.resize(total);
    return first;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> color(n);
    for (int &c : color) cin >> c;
    vector<vector<int>> graph(n);
    for (int i = 1; i < n; ++i) {
        int u, v;

```

```

    cin >> u >> v;
    --u;
    --v;
    graph[u].push_back(v);
    graph[v].push_back(u);
}

vector<bool> blocked(n);
vector<int> parent(n), subtree_size(n), answer(n);

auto find_centroid = [&](int start) {
    vector<int> nodes{start};
    parent[start] = -1;
    for (size_t at = 0; at < nodes.size(); ++at) {
        int u = nodes[at];
        for (int v : graph[u]) {
            if (blocked[v] || v == parent[u]) continue;
            parent[v] = u;
            nodes.push_back(v);
        }
    }
    for (auto it = nodes.rbegin(); it != nodes.rend(); ++it) {
        int u = *it;
        subtree_size[u] = 1;
        for (int v : graph[u])
            if (!blocked[v] && parent[v] == u)
                subtree_size[u] += subtree_size[v];
    }
    int total = nodes.size();
    int centroid = start, best = total + 1;
    for (int u : nodes) {
        int largest = total - subtree_size[u];
        for (int v : graph[u])
            if (!blocked[v] && parent[v] == u)
                largest = max(largest, subtree_size[v]);
        if (largest < best) {
            best = largest;
            centroid = u;
        }
    }
    return centroid;
};

auto add_square = [&](const vector<int> &polynomial, int sign) {
    vector<int> product = convolution(polynomial, polynomial);
    int limit = min(n, static_cast<int>(product.size()));
    for (int distance = 0; distance < limit; ++distance) {
        answer[distance] += sign * product[distance];
        if (answer[distance] >= MOD) answer[distance] -= MOD;
        if (answer[distance] < 0) answer[distance] += MOD;
    }
};

```

```

};

auto decompose = [&](auto &&self, int start) -> void {
    int centroid = find_centroid(start);
    array<vector<int>, 2> all;
    all[color[centroid]].resize(1, 1);

    for (int neighbor : graph[centroid]) {
        if (blocked[neighbor]) continue;
        array<vector<int>, 2> branch;
        vector<tuple<int, int, int>> stack{{neighbor, centroid, 1}};
        while (!stack.empty()) {
            auto [u, p, distance] = stack.back();
            stack.pop_back();
            if (static_cast<int>(branch[color[u]].size()) <= distance)
                branch[color[u]].resize(distance + 1);
            ++branch[color[u]][distance];
            for (int v : graph[u])
                if (!blocked[v] && v != p)
                    stack.push_back({v, u, distance + 1});
        }
        for (int c = 0; c < 2; ++c) {
            add_square(branch[c], -1);
            if (all[c].size() < branch[c].size()) all[c].resize(branch[c].size());
            for (int d = 0; d < static_cast<int>(branch[c].size()); ++d) {
                all[c][d] += branch[c][d];
                if (all[c][d] >= MOD) all[c][d] -= MOD;
            }
        }
    }
    add_square(all[0], 1);
    add_square(all[1], 1);
    blocked[centroid] = true;
    for (int neighbor : graph[centroid])
        if (!blocked[neighbor]) self(self, neighbor);
};

decompose(decompose, 0);
answer[0] = 0;
int inverse_two = (MOD + 1) / 2;
for (int distance = 1; distance < n; ++distance)
    answer[distance] = static_cast<int64>(answer[distance]) * inverse_two % MOD;
for (int distance = 0; distance < n; ++distance)
    cout << answer[distance] << " \n"[distance + 1 == n];
}

```

Python

```
import sys
```



```

MOD = 998244353
ROOT = 3

def ntt(values, inverse):
    n = len(values)
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            values[i], values[j] = values[j], values[i]
    length = 2
    while length <= n:
        step = pow(ROOT, (MOD - 1) // length, MOD)
        if inverse:
            step = pow(step, MOD - 2, MOD)
        half = length // 2
        for start in range(0, n, length):
            root = 1
            for offset in range(half):
                u = values[start + offset]
                v = values[start + offset + half] * root % MOD
                values[start + offset] = (u + v) % MOD
                values[start + offset + half] = (u - v) % MOD
                root = root * step % MOD
            length *= 2
    if inverse:
        inverse_n = pow(n, MOD - 2, MOD)
        for i in range(n):
            values[i] = values[i] * inverse_n % MOD

def convolution(first, second):
    if not first or not second:
        return []
    total = len(first) + len(second) - 1
    if min(len(first), len(second)) < 35:
        result = [0] * total
        for i, x in enumerate(first):
            for j, y in enumerate(second):
                result[i + j] = (result[i + j] + x * y) % MOD
        return result
    size = 1
    while size < total:
        size *= 2
    a = first + [0] * (size - len(first))
    b = second + [0] * (size - len(second))

```

```

ntt(a, False)
ntt(b, False)
for i in range(size):
    a[i] = a[i] * b[i] % MOD
ntt(a, True)
return a[:total]

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    colors = data[1:1 + n]
    graph = [[] for _ in range(n)]
    index = 1 + n
    for _ in range(n - 1):
        u, v = data[index] - 1, data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    blocked = [False] * n
    parent = [-1] * n
    subtree = [0] * n
    answer = [0] * n

    def find_centroid(start):
        nodes = [start]
        parent[start] = -1
        for u in nodes:
            for v in graph[u]:
                if not blocked[v] and v != parent[u]:
                    parent[v] = u
                    nodes.append(v)
        for u in reversed(nodes):
            size = 1
            for v in graph[u]:
                if not blocked[v] and parent[v] == u:
                    size += subtree[v]
            subtree[u] = size
        total = len(nodes)
        centroid = start
        best = total + 1
        for u in nodes:
            largest = total - subtree[u]
            for v in graph[u]:
                if not blocked[v] and parent[v] == u:
                    largest = max(largest, subtree[v])
            if largest < best:
                best = largest

```

```

        centroid = u
    return centroid

def add_square(polynomial, sign):
    product = convolution(polynomial, polynomial)
    for distance, contribution in enumerate(product[:n]):
        answer[distance] = (answer[distance] + sign * contribution) % MOD

sys.setrecursionlimit(1_000_000)

def decompose(start):
    centroid = find_centroid(start)
    all_colors = [[], []]
    all_colors[colors[centroid]] = [1]
    for neighbor in graph[centroid]:
        if blocked[neighbor]:
            continue
        branch = [[], []]
        stack = [(neighbor, centroid, 1)]
        while stack:
            u, p, distance = stack.pop()
            bucket = branch[colors[u]]
            if len(bucket) <= distance:
                bucket.extend([0] * (distance + 1 - len(bucket)))
            bucket[distance] += 1
            for v in graph[u]:
                if not blocked[v] and v != p:
                    stack.append((v, u, distance + 1))
        for color in range(2):
            add_square(branch[color], -1)
            if len(all_colors[color]) < len(branch[color]):
                all_colors[color].extend(
                    [0] * (len(branch[color]) - len(all_colors[color])))
            for distance, count in enumerate(branch[color]):
                all_colors[color][distance] += count
        add_square(all_colors[0], 1)
        add_square(all_colors[1], 1)
        blocked[centroid] = True
        for neighbor in graph[centroid]:
            if not blocked[neighbor]:
                decompose(neighbor)

decompose(0)
answer[0] = 0
inverse_two = (MOD + 1) // 2
for distance in range(1, n):
    answer[distance] = answer[distance] * inverse_two % MOD
print(*answer)

```

```

if __name__ == "__main__":

```

```
main()
```

07. Virtual Tree Firewall

Restatement and compression target

For each small terminal set, cut minimum-cost edges so no component contains two terminals. Processing the full n -vertex tree per query ignores the bound on the total number of terminals.

The relevant branching structure is formed by the terminals and their LCAs. Sort terminals by Euler entry time, add LCAs of adjacent sorted terminals, sort again, and connect each node to its nearest ancestor in this set. This virtual tree has fewer than $2k$ vertices.

A virtual edge from ancestor a to descendant v represents an original path whose internal vertices contain no terminal or virtual branching point. If we decide to disconnect the two virtual endpoints, removing more than one edge is never useful because costs are positive. Its effective cut choice is therefore

```
(minimum original edge cost on path  $a \rightarrow v$ , 1 removed edge).
```

Binary lifting stores both ancestors and minimum edge costs, so this compressed edge value is available in $O(\log n)$.

The two DP states

Root the virtual tree. For a processed virtual subtree, store the best lexicographic pair $(cost, edge_count)$ under two possibilities:

- $dp0$: the component still connected to the parent contains no terminal;
- $dp1$: that component contains exactly one terminal.

A marked node starts in state one; an unmarked node starts in state zero. For a child joined by effective edge $(w, 1)$, define

```
separated = min(child.dp0,  
                child.dp1 + (w, 1)).
```

The first option keeps a terminal-free child attached. The second cuts off a child containing one terminal. Then

```
new_dp0 = dp0 + separated  
new_dp1 = min(dp1 + separated,  
              dp0 + child.dp1).
```

The second way to obtain state one keeps the child's one terminal and requires the current partial component to contain none.

Sample reasoning

For terminals 3 and 5, their path contains edge costs 2,3,4; its virtual edge can be severed for cost two with one cut. With terminals 1,3,5, cutting edge 1-2 for five would still leave 3 and 5 together and require another cut. The DP instead separates the two descendant branches for costs two and three, giving (5,2).

Correctness proof

Lemma 1. The virtual tree preserves the ancestor and branching relations among all terminals, and has fewer than $2k$ vertices.

Proof. Adding LCAs of adjacent Euler-ordered terminals closes the set under all LCAs needed by the induced subtree. The ancestor stack connects every node to its nearest included ancestor. A standard Euler-order argument charges each new LCA to a gap between adjacent terminals, giving at most $k-1$ additional vertices.

Lemma 2. Replacing a virtual edge by its minimum original edge cost preserves every optimal answer and the secondary edge count.

Proof. The represented path contains no relevant branch or terminal internally. Keeping it preserves connectivity between its endpoints. To break that connectivity, one removed edge suffices, and positivity makes additional removals strictly worse in cost. The cheapest one-edge choice is precisely the minimum edge on the path.

Lemma 3. `dp0` and `dp1` are exact after every processed child.

Proof. Induct on children. By the validity requirement, the upward component may contain at most one terminal. A child with no upward terminal may remain attached. A child with one may either be cut off, or be kept only when the current upward component has none. The transitions enumerate exactly these cases and lexicographically minimize their additive objectives.

Theorem. The minimum of the two root states is the requested optimum.

Proof. The root has no parent, so either zero or one terminal in its remaining component is valid. Lemmas 1–2 preserve the original feasible cuts, and Lemma 3 optimizes all of them on the compressed tree.

Complexity and tests

Preprocessing binary lifting takes $O(n \log n)$. A query with k terminals builds and solves its virtual tree in $O(k \log n)$ time and $O(k)$ temporary memory. The total query time is therefore proportional to the promised sum of terminal counts times $\log n$.

Do not use the sum of a compressed path: cutting needs its minimum edge. Keep cost and number of edges together in every comparison. Mark terminals with a query token rather than clearing an n -element array. Tests should include one terminal, ancestor-descendant terminals, an LCA that is marked, equal minimum edges, stars, chains, and a query containing every vertex.

C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

constexpr int64 INF = 4'000'000'000'000'000'000LL;
using State = pair<int64, int>;

State add_state(State a, State b) {
    if (a.first >= INF || b.first >= INF) return {INF, INT_MAX / 2};
    return {a.first + b.first, a.second + b.second};
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, query_count;
    cin >> n >> query_count;
    vector<vector<pair<int, int64>>> graph(n);
    for (int i = 1; i < n; ++i) {
        int u, v;
        int64 cost;
        cin >> u >> v >> cost;
        --u;
        --v;
        graph[u].push_back({v, cost});
        graph[v].push_back({u, cost});
    }

    int log = max(1, static_cast<int>(bit_width(static_cast<unsigned>(n))));
    vector<vector<int>> up(log, vector<int>(n));
    vector<vector<int64>> minimum(log, vector<int64>(n, INF));
    vector<int> depth(n), tin(n), tout(n);
    struct Event { int u, parent; int64 weight; bool exit; };
    vector<Event> events{{0, 0, INF, false}};
    int timer = 0;
    while (!events.empty()) {
        Event event = events.back();
        events.pop_back();
        int u = event.u;
        if (event.exit) {
            tout[u] = timer;
            continue;
        }
        tin[u] = timer++;
        up[0][u] = event.parent;
        minimum[0][u] = event.weight;
        if (u != 0) depth[u] = depth[event.parent] + 1;
        for (int bit = 1; bit < log; ++bit) {
            up[bit][u] = up[bit - 1][up[bit - 1][u]];
            minimum[bit][u] = min(minimum[bit - 1][u],
                                   minimum[bit - 1][up[bit - 1][u]]);
        }
        events.push_back({u, event.parent, event.weight, true});
        for (auto it = graph[u].rbegin(); it != graph[u].rend(); ++it)
            if (it->first != event.parent)
                events.push_back({it->first, u, it->second, false});
    }

    auto is_ancestor = [&](int u, int v) {
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    };
    auto lca = [&](int u, int v) {

```

```

    if (is_ancestor(u, v)) return u;
    if (is_ancestor(v, u)) return v;
    for (int bit = log - 1; bit >= 0; --bit)
        if (!is_ancestor(up[bit][u], v)) u = up[bit][u];
    return up[0][u];
};

auto path_minimum = [&](int ancestor, int vertex) {
    int difference = depth[vertex] - depth[ancestor];
    int64 result = INF;
    for (int bit = 0; bit < log; ++bit) {
        if (difference >> bit & 1) {
            result = min(result, minimum[bit][vertex]);
            vertex = up[bit][vertex];
        }
    }
    return result;
};

vector<int> marked(n);
int token = 0;
while (query_count--) {
    int k;
    cin >> k;
    vector<int> terminals(k);
    ++token;
    for (int &v : terminals) {
        cin >> v;
        --v;
        marked[v] = token;
    }
    if (k <= 1) {
        cout << "0 0\n";
        continue;
    }
    sort(terminals.begin(), terminals.end(),
        [&](int u, int v) { return tin[u] < tin[v]; });
    vector<int> nodes = terminals;
    for (int i = 1; i < k; ++i) nodes.push_back(lca(terminals[i - 1], terminals[i]));
    sort(nodes.begin(), nodes.end(), [&](int u, int v) { return tin[u] < tin[v]; });
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());

    vector<vector<pair<int, int64>>> virtual_children(nodes.size());
    vector<int> stack;
    for (int i = 0; i < static_cast<int>(nodes.size()); ++i) {
        while (!stack.empty() && !is_ancestor(nodes[stack.back()], nodes[i]))
            stack.pop_back();
        if (!stack.empty()) {
            int parent_index = stack.back();
            virtual_children[parent_index].push_back(
                {i, path_minimum(nodes[parent_index], nodes[i])});
        }
    }
}

```

```

        stack.push_back(i);
    }

    vector<State> dp_zero(nodes.size()), dp_one(nodes.size());
    for (int i = static_cast<int>(nodes.size()) - 1; i >= 0; --i) {
        State zero = marked[nodes[i]] == token ? State{INF, INT_MAX / 2}
            : State{0, 0};
        State one = marked[nodes[i]] == token ? State{0, 0}
            : State{INF, INT_MAX / 2};
        for (auto [child, edge_cost] : virtual_children[i]) {
            State separated = min(dp_zero[child],
                add_state(dp_one[child], {edge_cost, 1}));
            State new_zero = add_state(zero, separated);
            State new_one = min(add_state(one, separated),
                add_state(zero, dp_one[child]));
            zero = new_zero;
            one = new_one;
        }
        dp_zero[i] = zero;
        dp_one[i] = one;
    }
    State answer = min(dp_zero[0], dp_one[0]);
    cout << answer.first << ' ' << answer.second << '\n';
}
}

```

Python

```

import sys

INF = 10**40

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    n, query_count = data[index], data[index + 1]
    index += 2
    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v, cost = data[index] - 1, data[index + 1] - 1, data[index + 2]
        index += 3
        graph[u].append((v, cost))
        graph[v].append((u, cost))

    log = max(1, n.bit_length())
    up = [[0] * n for _ in range(log)]
    minimum = [[INF] * n for _ in range(log)]

```



```

depth = [0] * n
tin = [0] * n
tout = [0] * n
timer = 0
events = [(0, 0, INF, 0)]
while events:
    u, parent, weight, phase = events.pop()
    if phase:
        tout[u] = timer
        continue
    tin[u] = timer
    timer += 1
    up[0][u] = parent
    minimum[0][u] = weight
    if u:
        depth[u] = depth[parent] + 1
    for bit in range(1, log):
        middle = up[bit - 1][u]
        up[bit][u] = up[bit - 1][middle]
        minimum[bit][u] = min(minimum[bit - 1][u], minimum[bit - 1][middle])
    events.append((u, parent, weight, 1))
    for v, cost in reversed(graph[u]):
        if v != parent:
            events.append((v, u, cost, 0))

def is_ancestor(u, v):
    return tin[u] <= tin[v] and tout[v] <= tout[u]

def lca(u, v):
    if is_ancestor(u, v):
        return u
    if is_ancestor(v, u):
        return v
    for bit in range(log - 1, -1, -1):
        if not is_ancestor(up[bit][u], v):
            u = up[bit][u]
    return up[0][u]

def path_minimum(ancestor, vertex):
    difference = depth[vertex] - depth[ancestor]
    result = INF
    bit = 0
    while difference:
        if difference & 1:
            result = min(result, minimum[bit][vertex])
            vertex = up[bit][vertex]
        difference >>= 1
        bit += 1
    return result

marked = [0] * n

```

```

output = []
for token in range(1, query_count + 1):
    k = data[index]
    index += 1
    terminals = [value - 1 for value in data[index:index + k]]
    index += k
    for vertex in terminals:
        marked[vertex] = token
    if k <= 1:
        output.append("0 0")
        continue
    terminals.sort(key=terminals.__getitem__)
    nodes = terminals + [lca(terminals[i - 1], terminals[i]) for i in range(1, k)]
    nodes = sorted(set(nodes), key=nodes.__getitem__)
    children = [[] for _ in nodes]
    stack = []
    for i, vertex in enumerate(nodes):
        while stack and not is_ancestor(nodes[stack[-1]], vertex):
            stack.pop()
        if stack:
            parent_index = stack[-1]
            children[parent_index].append(
                (i, path_minimum(nodes[parent_index], vertex)))
        stack.append(i)

    dp_zero = [None] * len(nodes)
    dp_one = [None] * len(nodes)
    for i in range(len(nodes) - 1, -1, -1):
        if marked[nodes[i]] == token:
            zero, one = (INF, INF), (0, 0)
        else:
            zero, one = (0, 0), (INF, INF)
        for child, edge_cost in children[i]:
            cut_child = (dp_one[child][0] + edge_cost,
                        dp_one[child][1] + 1)
            separated = min(dp_zero[child], cut_child)
            new_zero = (zero[0] + separated[0], zero[1] + separated[1])
            new_one = min((one[0] + separated[0], one[1] + separated[1]),
                          (zero[0] + dp_one[child][0],
                           zero[1] + dp_one[child][1]))
            zero, one = new_zero, new_one
        dp_zero[i], dp_one[i] = zero, one
    answer = min(dp_zero[0], dp_one[0])
    output.append(f"{answer[0]} {answer[1]}")
print("\n".join(output))

```

```

if __name__ == "__main__":
    main()

```

08. Subtree Order Laboratory

Restatement and two dimensions

Point values change, and a query asks for a subtree order statistic. An Euler tour maps every rooted subtree to one contiguous position interval $[\text{tin}[v], \text{tout}[v])$. The remaining problem is dynamic range k -th queries on an array.

The input is online semantically but available offline physically: answers do not influence later operations. Read all updates first and collect every value that each Euler position may ever hold. This lets us preallocate a Fenwick tree over positions whose every outer cell contains another compressed Fenwick tree over exactly its possible values.

For an Euler position p , a point update visits the usual outer indices

```
p+1, p+1+lowbit(p+1), ...
```

and updates the compressed value in each inner Fenwick tree. A prefix query `count(position,value)` walks outer indices downward and asks each inner tree for values at most the threshold. Subtracting two prefixes counts such values inside one subtree.

Finally binary-search the globally sorted values for the first threshold whose subtree count is at least k .

Trace

The sample subtree of vertex 2 initially occupies one Euler interval and has values $\{1,3,7\}$. The first threshold with count at least two is 3. Updating vertex 4 removes one occurrence of 3 from every outer Fenwick bucket covering its Euler position and inserts 10 into the same buckets. The subtree multiset becomes $\{1,7,10\}$, so rank two is 7.

Algorithm

```
root the tree and compute tin, subtree_size, tout
read all operations
for every vertex, collect its initial value and all future assigned values
for every possible (Euler position,value):
    append value to every outer Fenwick bucket covering the position
sort/unique each bucket and allocate its inner Fenwick counts

insert all initial values
replay operations:
    U v x: remove old value at tin[v], insert x
    K v k:
        binary-search global values x using
        prefix_count(tout[v],x)-prefix_count(tin[v],x)
```

Correctness proof

Lemma 1. The Euler interval $[\text{tin}[v], \text{tout}[v])$ contains exactly the vertices in the rooted subtree of v .

Proof. DFS enters v before any descendant and completes every descendant before leaving its subtree. No vertex outside the subtree is entered between these events.

Lemma 2. `prefix_count(p, x)` equals the number of current Euler positions strictly below p whose values are at most x .

Proof. The outer Fenwick prefix decomposes $[0, p)$ into disjoint canonical position ranges. Every point update changes exactly the outer nodes whose ranges contain its position. In each selected outer node, its inner Fenwick prefix counts exactly stored values at most x . Summing the disjoint ranges proves the claim.

Lemma 3. The binary search returns the k -th smallest subtree value.

Proof. By Lemmas 1–2, the tested count is the number of subtree values at most the threshold. This count is monotone over sorted candidate values. The first value whose count reaches k is precisely the k -th order statistic, including duplicates.

Theorem. Every update and query is processed according to the statement.

Proof. Updates remove the unique old contribution and insert the new one in all canonical buckets. Lemma 3 proves every subsequent query.

Complexity and implementation traps

Let V be the number of distinct values appearing anywhere. Preallocation and all updates use $O(\log^2 n)$ work per possible vertex value or operation. A threshold count is $O(\log^2 n)$, so a k -th query is $O(\log V * \log^2 n)$. Memory is $O((n+q)\log n)$.

Offline collection does not apply updates early; it only determines inner coordinate systems. Deduplicate possible values per vertex before distributing them. Inner updates use `lower_bound`, while threshold queries use `upper_bound`. Preserve duplicate current values as counts. Test repeated assignments, all-equal values, negative values, leaf subtrees, root queries, chains, and updates to the current k -th value.

C++

```
#include <bits/stdc++.h>
using namespace std;

struct Operation {
    char type;
    int vertex;
    int argument;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
```

```

vector<int> current(n), all_values;
vector<vector<int>> possible(n);
for (int v = 0; v < n; ++v) {
    cin >> current[v];
    possible[v].push_back(current[v]);
    all_values.push_back(current[v]);
}
vector<vector<int>> graph(n);
for (int i = 1; i < n; ++i) {
    int u, v;
    cin >> u >> v;
    --u;
    --v;
    graph[u].push_back(v);
    graph[v].push_back(u);
}

vector<int> parent(n, -1), order, stack{0};
while (!stack.empty()) {
    int u = stack.back();
    stack.pop_back();
    order.push_back(u);
    for (auto it = graph[u].rbegin(); it != graph[u].rend(); ++it) {
        int v = *it;
        if (v == parent[u]) continue;
        parent[v] = u;
        stack.push_back(v);
    }
}

vector<int> tin(n), subtree_size(n, 1), tout(n);
for (int i = 0; i < n; ++i) tin[order[i]] = i;
for (int i = n - 1; i > 0; --i)
    subtree_size[parent[order[i]]] += subtree_size[order[i]];
for (int v = 0; v < n; ++v) tout[v] = tin[v] + subtree_size[v];

vector<Operation> operations;
operations.reserve(q);
for (int i = 0; i < q; ++i) {
    char type;
    int vertex, argument;
    cin >> type >> vertex >> argument;
    --vertex;
    operations.push_back({type, vertex, argument});
    if (type == 'U') {
        possible[vertex].push_back(argument);
        all_values.push_back(argument);
    }
}

sort(all_values.begin(), all_values.end());
all_values.erase(unique(all_values.begin(), all_values.end()), all_values.end());

```

```

vector<vector<int>> candidates(n + 1);
for (int v = 0; v < n; ++v) {
    sort(possible[v].begin(), possible[v].end());
    possible[v].erase(unique(possible[v].begin(), possible[v].end()),
                      possible[v].end());
    for (int value : possible[v])
        for (int outer = tin[v] + 1; outer <= n; outer += outer & -outer)
            candidates[outer].push_back(value);
}

vector<vector<int>> counts(n + 1);
for (int outer = 1; outer <= n; ++outer) {
    auto &bucket = candidates[outer];
    sort(bucket.begin(), bucket.end());
    bucket.erase(unique(bucket.begin(), bucket.end()), bucket.end());
    counts[outer].assign(bucket.size() + 1, 0);
}

auto modify = [&](int vertex, int value, int delta) {
    for (int outer = tin[vertex] + 1; outer <= n; outer += outer & -outer) {
        int inner = lower_bound(candidates[outer].begin(), candidates[outer].end(),
                                value) - candidates[outer].begin() + 1;
        for (; inner < static_cast<int>(counts[outer].size()); inner += inner & -inner)
            counts[outer][inner] += delta;
    }
};

auto prefix_count = [&](int position, int value) {
    int result = 0;
    for (int outer = position; outer > 0; outer -= outer & -outer) {
        int inner = upper_bound(candidates[outer].begin(), candidates[outer].end(),
                                value) - candidates[outer].begin();
        for (; inner > 0; inner -= inner & -inner)
            result += counts[outer][inner];
    }
    return result;
};

for (int v = 0; v < n; ++v) modify(v, current[v], 1);

for (Operation operation : operations) {
    int v = operation.vertex;
    if (operation.type == 'U') {
        modify(v, current[v], -1);
        current[v] = operation.argument;
        modify(v, current[v], 1);
    } else {
        int k = operation.argument;
        int left = 0, right = all_values.size() - 1;
        while (left < right) {
            int middle = (left + right) / 2;
            int count = prefix_count(tout[v], all_values[middle])
                        - prefix_count(tin[v], all_values[middle]);
            if (count >= k) right = middle;
        }
    }
}

```

```

        else left = middle + 1;
    }
    cout << all_values[left] << '\n';
}
}
}

```

Python

```

import bisect
import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    index = 0
    n, q = int(data[index]), int(data[index + 1])
    index += 2
    current = list(map(int, data[index:index + n]))
    index += n
    possible = [[value] for value in current]
    all_values = current.copy()
    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v = int(data[index]) - 1, int(data[index + 1]) - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    parent = [-1] * n
    order = []
    stack = [0]
    while stack:
        u = stack.pop()
        order.append(u)
        for v in reversed(graph[u]):
            if v != parent[u]:
                parent[v] = u
                stack.append(v)

    tin = [0] * n
    subtree_size = [1] * n
    for position, vertex in enumerate(order):
        tin[vertex] = position
    for vertex in reversed(order[1:]):
        subtree_size[parent[vertex]] += subtree_size[vertex]
    tout = [tin[v] + subtree_size[v] for v in range(n)]

    operations = []

```

```

for _ in range(q):
    kind = data[index]
    vertex, argument = int(data[index + 1]) - 1, int(data[index + 2])
    index += 3
    operations.append((kind, vertex, argument))
    if kind == b"U":
        possible[vertex].append(argument)
        all_values.append(argument)
all_values = sorted(set(all_values))

candidates = [[] for _ in range(n + 1)]
for vertex in range(n):
    for value in set(possible[vertex]):
        outer = tin[vertex] + 1
        while outer <= n:
            candidates[outer].append(value)
            outer += outer & -outer
counts = [None] * (n + 1)
for outer in range(1, n + 1):
    candidates[outer] = sorted(set(candidates[outer]))
    counts[outer] = [0] * (len(candidates[outer]) + 1)

def modify(vertex, value, delta):
    outer = tin[vertex] + 1
    while outer <= n:
        inner = bisect.bisect_left(candidates[outer], value) + 1
        bucket = counts[outer]
        while inner < len(bucket):
            bucket[inner] += delta
            inner += inner & -inner
        outer += outer & -outer

def prefix_count(position, value):
    result = 0
    outer = position
    while outer:
        inner = bisect.bisect_right(candidates[outer], value)
        bucket = counts[outer]
        while inner:
            result += bucket[inner]
            inner -= inner & -inner
        outer -= outer & -outer
    return result

for vertex, value in enumerate(current):
    modify(vertex, value, 1)
output = []
for kind, vertex, argument in operations:
    if kind == b"U":
        modify(vertex, current[vertex], -1)
        current[vertex] = argument

```



```

        modify(vertex, current[vertex], 1)
    else:
        k = argument
        left, right = 0, len(all_values) - 1
        while left < right:
            middle = (left + right) // 2
            count = (prefix_count(tout[vertex], all_values[middle])
                    - prefix_count(tin[vertex], all_values[middle]))
            if count >= k:
                right = middle
            else:
                left = middle + 1
        output.append(str(all_values[left]))
    print("\n".join(output))

if __name__ == "__main__":
    main()

```

09. Ancestral Pattern Index

Restatement and path orientation

Each query compares two strings read along directed tree paths. HLD decomposes an undirected path into $O(\log n)$ intervals of one base array, but direction matters: climbing toward the LCA reads an interval backward, while descending from the LCA reads intervals forward.

For $S(u, v)$, let $w = \text{LCA}(u, v)$. Append the heavy-chain intervals from u up through w immediately, each marked reversed. Collect intervals from v toward w , each marked forward, and then reverse their list. Exclude w from the descending half because the upward half already contains it. The resulting descriptor list spells the path exactly and has $O(\log n)$ segments.

Hashing arbitrary directed pieces

Build polynomial-hash prefixes for the HLD base array and for its reversal, using two prime moduli. A forward subinterval uses the first prefix. A reversed subinterval maps its endpoints into the reversed base array. Therefore any substring of any descriptor is available in $O(1)$.

Walk the two descriptor lists simultaneously. Compare the largest chunk that fits in both current segments. Equal hashes skip the entire chunk. On a mismatch, binary-search inside that chunk for its LCP, inspect the next actual characters, and determine lexicographic order. If all shared characters match, the shorter path string is smaller.

Sample trace

$S(4, 3)$ is formed by reversed interval $4 \rightarrow 2 \rightarrow 1$ followed by forward interval 3, spelling **abac**. $S(1, 5)$ spells **abb**. Their first two characters match; the next characters are **a** and **b**, so the result is $2 - 1$. The path descriptions may have different HLD boundaries, but the two-pointer comparison splits chunks at whichever boundary comes first.

Correctness proof

Lemma 1. `path_segments(u,v)` concatenates to exactly $S(u,v)$.

Proof. Every upward heavy-chain interval is read from its deeper endpoint to its head, so reversal gives path order. These intervals are appended in the order encountered while climbing. Descending intervals are discovered from v upward, hence reversing their list gives LCA-to- v order; each interval's internal order is already top to bottom. The LCA is included once and all path vertices are covered without gaps.

Lemma 2. The hash returned for a descriptor substring equals the polynomial hash of its spelled characters.

Proof. For a forward descriptor this is the standard prefix-hash identity. For a reversed descriptor, the index mapping selects exactly the corresponding forward interval of the globally reversed base array. Both moduli use the same character order and length.

Lemma 3. The comparison loop finds the true LCP and lexicographic result, assuming no double-hash collision.

Proof. Equal chunks are skipped only after Lemma 2 establishes equal hashes. At the first unequal chunk, binary search finds its longest equal prefix and the next characters decide order. If no unequal chunk exists, every character of the shorter string matches and length decides exactly as in the definition of lexicographic order.

Theorem. Subject to the standard negligible double-hash collision probability, every output is correct.

Proof. Lemma 1 supplies the requested strings, Lemma 2 supports constant-time piece comparisons, and Lemma 3 implements LCP and lexicographic comparison.

Complexity and implementation traps

HLD and hashes take $O(n)$ preprocessing after the tree DFS. A path has $O(\log n)$ descriptors. Equal chunks are consumed once, while one mismatching chunk adds an $O(\log n)$ binary search, so each query is $O(\log n)$ hash comparisons and $O(\log n)$ descriptor construction; the conservative bound is $O(\log^2 n)$.

Keep the LCA exactly once. Reverse the order of downward descriptors, not their characters. Hash substrings must be normalized by length with precomputed powers. Use two moduli (or independently randomized wide hashes); one small hash is inappropriate for adversarial strings. Test single vertices, one path being a prefix, opposite directions on the same path, mismatches at LCAs and chain boundaries, stars, chains, and all-equal labels.

C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

constexpr int MOD1 = 1'000'000'007;
constexpr int MOD2 = 1'000'000'009;
constexpr int BASE = 911382323;

struct Segment {
    int left, right;
    bool reversed;
};
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    string labels;
    cin >> n >> q >> labels;
    vector<vector<int>> graph(n);
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<int> parent(n, -1), depth(n), order{0};
    for (size_t at = 0; at < order.size(); ++at) {
        int u = order[at];
        for (int v : graph[u]) {
            if (v == parent[u]) continue;
            parent[v] = u;
            depth[v] = depth[u] + 1;
            order.push_back(v);
        }
    }

    vector<int> size(n, 1), heavy(n, -1);
    for (int at = n - 1; at > 0; --at) {
        int u = order[at], p = parent[u];
        size[p] += size[u];
        if (heavy[p] == -1 || size[u] > size[heavy[p]]) heavy[p] = u;
    }

    vector<int> head(n), position(n), vertex_at(n);
    vector<pair<int, int>> tasks{{0, 0}};
    int timer = 0;
    while (!tasks.empty()) {
        auto [start, chain_head] = tasks.back();
        tasks.pop_back();
        for (int u = start; u != -1; u = heavy[u]) {
            head[u] = chain_head;
            position[u] = timer;
            vertex_at[timer++] = u;
            for (int v : graph[u])
                if (parent[v] == u && v != heavy[u]) tasks.push_back({v, v});
        }
    }

    auto lca = [&](int u, int v) {
        while (head[u] != head[v]) {
            if (depth[head[u]] > depth[head[v]]) u = parent[head[u]];

```

```

        else v = parent[head[v]];
    }
    return depth[u] < depth[v] ? u : v;
};

auto path_segments = [&](int u, int v) {
    int ancestor = lca(u, v);
    vector<Segment> result, down;
    while (head[u] != head[ancestor]) {
        result.push_back({position[head[u]], position[u] + 1, true});
        u = parent[head[u]];
    }
    result.push_back({position[ancestor], position[u] + 1, true});
    while (head[v] != head[ancestor]) {
        down.push_back({position[head[v]], position[v] + 1, false});
        v = parent[head[v]];
    }
    if (position[ancestor] + 1 <= position[v])
        down.push_back({position[ancestor] + 1, position[v] + 1, false});
    reverse(down.begin(), down.end());
    result.insert(result.end(), down.begin(), down.end());
    return result;
};

vector<int> base(n), reversed_base(n);
for (int i = 0; i < n; ++i) base[i] = labels[vertex_at[i]] - 'a' + 1;
for (int i = 0; i < n; ++i) reversed_base[i] = base[n - 1 - i];
array<vector<int>, 2> powers, prefix, reverse_prefix;
int moduli[2] = {MOD1, MOD2};
for (int h = 0; h < 2; ++h) {
    int mod = moduli[h];
    powers[h].resize(n + 1, 1);
    prefix[h].resize(n + 1);
    reverse_prefix[h].resize(n + 1);
    for (int i = 0; i < n; ++i) {
        powers[h][i + 1] = static_cast<int64>(powers[h][i]) * BASE % mod;
        prefix[h][i + 1] = (static_cast<int64>(prefix[h][i]) * BASE + base[i]) % mod;
        reverse_prefix[h][i + 1] =
            (static_cast<int64>(reverse_prefix[h][i]) * BASE + reversed_base[i]) % mod;
    }
}

auto range_hash = [&](const array<vector<int>, 2> &source, int left, int right) {
    int parts[2];
    for (int h = 0; h < 2; ++h) {
        int mod = moduli[h];
        parts[h] = (source[h][right]
            - static_cast<int64>(source[h][left]) * powers[h][right - left] % mod
            + mod) % mod;
    }
    return pair{parts[0], parts[1]};
};

auto segment_hash = [&](const Segment &segment, int offset, int length) {

```

```

    if (!segment.reversed)
        return range_hash(prefix, segment.left + offset,
                           segment.left + offset + length);
    int original_left = segment.right - offset - length;
    int original_right = segment.right - offset;
    return range_hash(reverse_prefix, n - original_right, n - original_left);
};

auto segment_character = [&](const Segment &segment, int offset) {
    int base_position = segment.reversed ? segment.right - 1 - offset
        : segment.left + offset;

    return base[base_position];
};

while (q--) {
    int u, v, x, y;
    cin >> u >> v >> x >> y;
    --u; --v; --x; --y;
    vector<Segment> first = path_segments(u, v);
    vector<Segment> second = path_segments(x, y);
    int i = 0, j = 0, offset_first = 0, offset_second = 0, common = 0;
    int comparison = 0;
    while (i < static_cast<int>(first.size()) &&
           j < static_cast<int>(second.size())) {
        int remaining_first = first[i].right - first[i].left - offset_first;
        int remaining_second = second[j].right - second[j].left - offset_second;
        int take = min(remaining_first, remaining_second);
        if (segment_hash(first[i], offset_first, take) ==
            segment_hash(second[j], offset_second, take)) {
            common += take;
            offset_first += take;
            offset_second += take;
            if (offset_first == first[i].right - first[i].left) {
                ++i;
                offset_first = 0;
            }
            if (offset_second == second[j].right - second[j].left) {
                ++j;
                offset_second = 0;
            }
            continue;
        }
        int low = 0, high = take;
        while (low < high) {
            int middle = (low + high + 1) / 2;
            if (segment_hash(first[i], offset_first, middle) ==
                segment_hash(second[j], offset_second, middle)) low = middle;
            else high = middle - 1;
        }
        common += low;
        int a = segment_character(first[i], offset_first + low);
        int b = segment_character(second[j], offset_second + low);
    }
}

```

```

        comparison = a < b ? -1 : 1;
        break;
    }
    if (comparison == 0) {
        bool first_finished = i == static_cast<int>(first.size());
        bool second_finished = j == static_cast<int>(second.size());
        if (first_finished != second_finished) comparison = first_finished ? -1 : 1;
    }
    cout << common << ' ' << comparison << '\n';
}
}

```

Python

```

import sys

MOD1 = 1_000_000_007
MOD2 = 1_000_000_009
BASE = 911382323

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    index = 0
    n, q = int(data[index]), int(data[index + 1])
    labels = data[index + 2]
    index += 3
    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v = int(data[index]) - 1, int(data[index + 1]) - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    parent = [-1] * n
    depth = [0] * n
    order = [0]
    for u in order:
        for v in graph[u]:
            if v != parent[u]:
                parent[v] = u
                depth[v] = depth[u] + 1
                order.append(v)

    size = [1] * n
    heavy = [-1] * n
    for u in reversed(order[1:]):
        p = parent[u]
        size[p] += size[u]

```

```

        if heavy[p] == -1 or size[u] > size[heavy[p]]:
            heavy[p] = u

head = [0] * n
position = [0] * n
vertex_at = [0] * n
tasks = [(0, 0)]
timer = 0
while tasks:
    u, chain_head = tasks.pop()
    while u != -1:
        head[u] = chain_head
        position[u] = timer
        vertex_at[timer] = u
        timer += 1
        for v in graph[u]:
            if parent[v] == u and v != heavy[u]:
                tasks.append((v, v))
        u = heavy[u]

def lca(u, v):
    while head[u] != head[v]:
        if depth[head[u]] > depth[head[v]]:
            u = parent[head[u]]
        else:
            v = parent[head[v]]
    return u if depth[u] < depth[v] else v

def path_segments(u, v):
    ancestor = lca(u, v)
    result = []
    down = []
    while head[u] != head[ancestor]:
        result.append((position[head[u]], position[u] + 1, 1))
        u = parent[head[u]]
    result.append((position[ancestor], position[u] + 1, 1))
    while head[v] != head[ancestor]:
        down.append((position[head[v]], position[v] + 1, 0))
        v = parent[head[v]]
    if position[ancestor] + 1 <= position[v]:
        down.append((position[ancestor] + 1, position[v] + 1, 0))
    result.extend(reversed(down))
    return result

base = [labels[vertex_at[i]] - 96 for i in range(n)]
reversed_base = base[::-1]
powers = []
prefixes = []
reverse_prefixes = []
for mod in (MOD1, MOD2):
    power = [1] * (n + 1)

```

```

prefix = [0] * (n + 1)
reverse_prefix = [0] * (n + 1)
for i in range(n):
    power[i + 1] = power[i] * BASE % mod
    prefix[i + 1] = (prefix[i] * BASE + base[i]) % mod
    reverse_prefix[i + 1] = (reverse_prefix[i] * BASE + reversed_base[i]) % mod
powers.append(power)
prefixes.append(prefix)
reverse_prefixes.append(reverse_prefix)

def range_hash(source, left, right):
    length = right - left
    return tuple((source[h][right] - source[h][left] * powers[h][length]) % mod
                  for h, mod in enumerate((MOD1, MOD2)))

def segment_hash(segment, offset, length):
    left, right, reversed_flag = segment
    if not reversed_flag:
        return range_hash(prefixes, left + offset, left + offset + length)
    original_left = right - offset - length
    original_right = right - offset
    return range_hash(reverse_prefixes, n - original_right, n - original_left)

def character(segment, offset):
    left, right, reversed_flag = segment
    return base[right - 1 - offset if reversed_flag else left + offset]

output = []
for _ in range(q):
    u, v, x, y = (int(data[index]) - 1, int(data[index + 1]) - 1,
                  int(data[index + 2]) - 1, int(data[index + 3]) - 1)
    index += 4
    first = path_segments(u, v)
    second = path_segments(x, y)
    i = j = offset_first = offset_second = common = 0
    comparison = 0
    while i < len(first) and j < len(second):
        remaining_first = first[i][1] - first[i][0] - offset_first
        remaining_second = second[j][1] - second[j][0] - offset_second
        take = min(remaining_first, remaining_second)
        if (segment_hash(first[i], offset_first, take)
            == segment_hash(second[j], offset_second, take)):
            common += take
            offset_first += take
            offset_second += take
            if offset_first == first[i][1] - first[i][0]:
                i += 1
                offset_first = 0
            if offset_second == second[j][1] - second[j][0]:
                j += 1
                offset_second = 0

```



```

        continue
    low, high = 0, take
    while low < high:
        middle = (low + high + 1) // 2
        if (segment_hash(first[i], offset_first, middle)
            == segment_hash(second[j], offset_second, middle)):
            low = middle
        else:
            high = middle - 1
    common += low
    a = character(first[i], offset_first + low)
    b = character(second[j], offset_second + low)
    comparison = -1 if a < b else 1
    break
if comparison == 0 and (i == len(first)) != (j == len(second)):
    comparison = -1 if i == len(first) else 1
output.append(f"{common} {comparison}")
print("\n".join(output))

if __name__ == "__main__":
    main()

```

10. Kruskal Time Machine

Restatement and threshold components

For a threshold W , edges of weight at most W define connected components. Inside the component of one vertex, answer a k -th-value query. Rebuilding a DSU and an order-statistics structure for every query is too slow.

Kruskal's reconstruction forest records the complete history of component merges. Begin with each original vertex as a leaf. Process edges by increasing weight. Whenever an edge joins two different DSU components, create a new internal node whose children are their current reconstruction roots and whose label is the edge weight. This new node becomes the DSU representative.

Along every leaf-to-root path, merge labels are nondecreasing. For query (v, W) , climb from leaf v to its highest ancestor whose label is at most W . The original leaves in that ancestor's subtree are exactly the threshold component.

Adding order statistics

DFS the reconstruction forest and assign Euler positions only to original leaves. Every reconstruction subtree then occupies one contiguous leaf interval. Build persistent value-frequency segment trees over prefixes of this leaf order. The multiset for interval $[l, r]$ is the difference between prefix roots $R[r]$ and $R[l]$; descending their count difference returns its k -th value.

Sample trace

At threshold two, only edge 3-4 is active, so the ancestor reached from leaf 3 contains leaves {3,4} and rank two is 6. At threshold seven, Kruskal nodes of weights two, four, and seven have merged vertices {1,2,3,4}. Their values sorted are {1,3,6,8}, giving rank two equal to 3.

Algorithm

```
sort graph edges by weight
create the Kruskal reconstruction forest with DSU
binary-lift parent pointers in that forest
DFS every reconstruction root, recording each node's leaf interval
build persistent value-frequency roots over original leaves in Euler order

for query (v,W,k):
    binary-lift v to the highest ancestor with merge_weight <= W
    obtain its interval [l,r)
    descend persistent roots prefix[r] - prefix[l] to rank k
```

Correctness proof

Lemma 1. For every original vertex v and threshold W , the highest ancestor of v labeled at most W has exactly the leaves in v 's component using edges of weight at most W .

Proof. Kruskal creates an internal node exactly when an edge first merges two previous components, labeling that union by the edge's weight. Thus every ancestor at label at most W represents unions achievable with permitted edges. The next ancestor, if any, requires its larger label and is not yet connected. Equal-weight successive unions remain climbable at the same threshold, so the highest such ancestor is the complete component.

Lemma 2. The original leaves below any reconstruction node form exactly its recorded Euler interval.

Proof. DFS processes an entire child subtree before the next child and emits positions only at original leaves. Therefore all and only descendant leaves are emitted contiguously between entry and exit counters.

Lemma 3. Persistent-root difference $R[r] - R[l]$ gives exact value frequencies for leaves in interval $[l, r)$.

Proof. $R[t]$ inserts exactly the first t leaf values. Coordinate-wise subtraction cancels the first 1 and retains insertions 1 through $r-1$.

Theorem. Every answer is the requested k -th smallest value.

Proof. Lemma 1 identifies the threshold component, Lemma 2 maps it to one leaf interval, and Lemma 3 supplies its exact multiset. Standard count descent returns rank k , including duplicates.

Complexity and implementation traps

Sorting costs $O(m \log m)$. Reconstruction, Euler traversal, binary lifting, and persistent prefixes require $O((n+m) \alpha(n) + n \log n)$ after sorting; only successful unions create nodes, so there are fewer than $2n$. Each query takes $O(\log n)$ time. Memory is $O(n \log n + m)$.

Create no reconstruction node for an edge already inside one DSU component. Process all equal weights normally; binary lifting must be allowed to cross several equal-labeled ancestors. In disconnected graphs, preprocess every reconstruction root. Euler positions belong only to original leaves, not internal merge nodes. Test no edges,

parallel edges, all equal weights, duplicate values, negative weights, isolated vertices, and thresholds between consecutive edge weights.

C++

```
#include <bits/stdc++.h>
using namespace std;

struct Edge { int u, v, weight; };
struct PersistentNode { int left = 0, right = 0, count = 0; };

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    cin >> n >> m >> q;
    vector<int> value(n), coordinates;
    for (int &x : value) cin >> x;
    coordinates = value;
    sort(coordinates.begin(), coordinates.end());
    coordinates.erase(unique(coordinates.begin(), coordinates.end()), coordinates.end());

    vector<Edge> edges(m);
    for (Edge &edge : edges) {
        cin >> edge.u >> edge.v >> edge.weight;
        --edge.u;
        --edge.v;
    }
    sort(edges.begin(), edges.end(), [](const Edge &a, const Edge &b) {
        return a.weight < b.weight;
    });

    int capacity = 2 * n + 1;
    vector<int> dsu(capacity), tree_parent(capacity, -1), merge_weight(capacity, INT_MIN);
    vector<array<int, 2>> children(capacity, array<int, 2>{-1, -1});
    iota(dsu.begin(), dsu.end(), 0);
    auto find = [&](auto &&self, int x) -> int {
        return dsu[x] == x ? x : dsu[x] = self(self, dsu[x]);
    };
    int nodes = n;
    for (Edge edge : edges) {
        int first = find(find, edge.u);
        int second = find(find, edge.v);
        if (first == second) continue;
        int parent = nodes++;
        merge_weight[parent] = edge.weight;
        children[parent] = {first, second};
        tree_parent[first] = tree_parent[second] = parent;
        dsu[first] = dsu[second] = parent;
    }
}
```

```

    dsu[parent] = parent;
}

vector<int> roots;
for (int node = 0; node < nodes; ++node)
    if (tree_parent[node] == -1) roots.push_back(node);
vector<int> left(nodes), right(nodes), leaf_order;
leaf_order.reserve(n);
int timer = 0;
for (int root : roots) {
    vector<pair<int, bool>> stack{{root, false}};
    while (!stack.empty()) {
        auto [node, exit] = stack.back();
        stack.pop_back();
        if (exit) {
            right[node] = timer;
            continue;
        }
        left[node] = timer;
        stack.push_back({node, true});
        if (node < n) {
            leaf_order.push_back(node);
            ++timer;
        } else {
            stack.push_back({children[node][1], false});
            stack.push_back({children[node][0], false});
        }
    }
}

int log = max(1, static_cast<int>(bit_width(static_cast<unsigned>(nodes))));
vector<vector<int>> up(log, vector<int>(nodes));
for (int node = 0; node < nodes; ++node)
    up[0][node] = tree_parent[node] == -1 ? node : tree_parent[node];
for (int bit = 1; bit < log; ++bit)
    for (int node = 0; node < nodes; ++node)
        up[bit][node] = up[bit - 1][up[bit - 1][node]];

vector<PersistentNode> segment(1);
segment.reserve(static_cast<size_t>(n) * (bit_width(static_cast<unsigned>(n)) + 2));
auto insert = [&](auto &&self, int previous, int low, int high, int position) -> int {
    int current = segment.size();
    segment.push_back(segment[previous]);
    ++segment[current].count;
    if (high - low == 1) return current;
    int middle = (low + high) / 2;
    if (position < middle)
        segment[current].left = self(self, segment[previous].left, low, middle, position);
    else
        segment[current].right = self(self, segment[previous].right, middle, high, position);
    return current;
};

```

```

};
vector<int> prefix_root(n + 1);
for (int i = 0; i < n; ++i) {
    int compressed = lower_bound(coordinates.begin(), coordinates.end(),
                                value[leaf_order[i]]) - coordinates.begin();
    prefix_root[i + 1] = insert(insert, prefix_root[i], 0, coordinates.size(), compressed);
}

while (q-- > 0) {
    int vertex, threshold, k;
    cin >> vertex >> threshold >> k;
    int component = --vertex;
    for (int bit = log - 1; bit >= 0; --bit) {
        int ancestor = up[bit][component];
        if (ancestor != component && merge_weight[ancestor] <= threshold)
            component = ancestor;
    }
    int a = prefix_root[right[component]], b = prefix_root[left[component]];
    int low = 0, high = coordinates.size();
    while (high - low > 1) {
        int count_left = segment[segment[a].left].count
                        - segment[segment[b].left].count;
        int middle = (low + high) / 2;
        if (k <= count_left) {
            a = segment[a].left;
            b = segment[b].left;
            high = middle;
        } else {
            k -= count_left;
            a = segment[a].right;
            b = segment[b].right;
            low = middle;
        }
    }
    cout << coordinates[low] << '\n';
}
}

```

Python

```

import bisect
import sys
from array import array

def main() -> None:
    data = array("q", map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0

```

```

n, m, q = data[index:index + 3]
index += 3
values = data[index:index + n]
index += n
coordinates = sorted(set(values))
edges = []
for _ in range(m):
    u, v, weight = data[index] - 1, data[index + 1] - 1, data[index + 2]
    index += 3
    edges.append((weight, u, v))
edges.sort()

capacity = 2 * n + 1
dsu = array("i", range(capacity))
tree_parent = array("i", [-1]) * capacity
merge_weight = array("q", [-(1 << 60)]) * capacity
child_left = array("i", [-1]) * capacity
child_right = array("i", [-1]) * capacity

def find(x):
    root = x
    while dsu[root] != root:
        root = dsu[root]
    while dsu[x] != x:
        x, dsu[x] = dsu[x], root
    return root

nodes = n
for weight, u, v in edges:
    first, second = find(u), find(v)
    if first == second:
        continue
    parent = nodes
    nodes += 1
    merge_weight[parent] = weight
    child_left[parent] = first
    child_right[parent] = second
    tree_parent[first] = tree_parent[second] = parent
    dsu[first] = dsu[second] = parent
    dsu[parent] = parent

roots = [node for node in range(nodes) if tree_parent[node] == -1]
left = array("i", [0]) * nodes
right = array("i", [0]) * nodes
leaf_order = []
timer = 0
for root in roots:
    stack = [(root, 0)]
    while stack:
        node, phase = stack.pop()
        if phase:

```

```

        right[node] = timer
        continue
    left[node] = timer
    stack.append((node, 1))
    if node < n:
        leaf_order.append(node)
        timer += 1
    else:
        first, second = child_left[node], child_right[node]
        stack.append((second, 0))
        stack.append((first, 0))

log = max(1, nodes.bit_length())
up = [array("i", (
    node if tree_parent[node] == -1 else tree_parent[node]
    for node in range(nodes)
))]
for bit in range(1, log):
    previous = up[bit - 1]
    up.append(array("i", (previous[previous[node]] for node in range(nodes))))

segment_left = array("i", [0])
segment_right = array("i", [0])
segment_count = array("i", [0])

def clone(node):
    segment_left.append(segment_left[node])
    segment_right.append(segment_right[node])
    segment_count.append(segment_count[node])
    return len(segment_count) - 1

def insert(previous, position):
    root = clone(previous)
    old, current = previous, root
    low, high = 0, len(coordinates)
    while True:
        segment_count[current] += 1
        if high - low == 1:
            return root
        middle = (low + high) // 2
        if position < middle:
            old_child = segment_left[old]
            new_child = clone(old_child)
            segment_left[current] = new_child
            old, current, high = old_child, new_child, middle
        else:
            old_child = segment_right[old]
            new_child = clone(old_child)
            segment_right[current] = new_child
            old, current, low = old_child, new_child, middle

```

```

prefix_root = array("i", [0])
for vertex in leaf_order:
    compressed = bisect.bisect_left(coordinates, values[vertex])
    prefix_root.append(insert(prefix_root[-1], compressed))

output = []
for _ in range(q):
    vertex, threshold, k = data[index] - 1, data[index + 1], data[index + 2]
    index += 3
    component = vertex
    for bit in range(log - 1, -1, -1):
        ancestor = up[bit][component]
        if ancestor != component and merge_weight[ancestor] <= threshold:
            component = ancestor
    a, b = prefix_root[right[component]], prefix_root[left[component]]
    low, high = 0, len(coordinates)
    while high - low > 1:
        count_left = (segment_count[segment_left[a]]
                      - segment_count[segment_left[b]])
        middle = (low + high) // 2
        if k <= count_left:
            a, b = segment_left[a], segment_left[b]
            high = middle
        else:
            k -= count_left
            a, b = segment_right[a], segment_right[b]
            low = middle
    output.append(str(coordinates[low]))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

Round I stress strategy

The first ten problems deliberately use different independent oracles: explicit path sorting, assignment enumeration, ancestor-line scanning, subsequence enumeration, BFS forest paths, all-pairs distances, edge-subset cuts, subtree sorting, literal path strings, and threshold DSU rebuilding. None shares the optimized structure it validates. Carry forward the recurring principle: expose the lifetime or separator first, then store only information that can cross it.

11. Bounded Convex Shipping

Restatement and the two reductions

Choose an integer circulation within lower and upper bounds. Edge cost is discrete convex. Two transformations make this an ordinary min-cost-flow instance.

First commit every lower bound. For edge $u \rightarrow v$ with lower 1, add its cost to the answer and record balance changes

```
balance[u] -= 1
balance[v] += 1.
```

A vertex with positive balance has excess lower-bound inflow and must send that amount through additional original capacity. Connect the super-source to it. A negative-balance vertex needs additional inflow; connect it to the super-sink. Sending the total positive balance from super-source to super-sink is equivalent to restoring conservation.

Second, expand convex residual capacity into marginal units. For

```
f(x) = c*x + d*x*(x-1)/2,
```

increasing flow from x to $x+1$ costs $c+d*x$. Thus residual choices above lower bound 1 become unit-capacity parallel edges with costs

```
c+d*1, c+d*(1+1), ..., c+d*(upper-1).
```

They are nondecreasing. Min-cost flow always uses a cheaper marginal before a more expensive parallel marginal, so selected units correspond to one valid integer flow amount.

Sample trace

The lower bounds send one unit $1 \rightarrow 2 \rightarrow 3$, costing 2+1. Vertex 3 has one unit of excess inflow and vertex 1 lacks one. The only repair sends one unit on $3 \rightarrow 1$ for cost three. Conservation is restored and the total is six.

Algorithm

```
for every edge:
    commit lower flow and its base cost
    update endpoint balances
    add one residual edge for every convex marginal
add super-source/sink balance edges
run min-cost max-flow for total positive balance
if that amount cannot be sent: IMPOSSIBLE
otherwise: base cost + residual min cost
```

Potentials and Dijkstra are valid initially because every forward marginal and super edge has nonnegative cost. Residual reverse edges may be negative, but updated potentials keep every reachable residual reduced cost nonnegative.

Correctness proof

Lemma 1. Feasible flows in the transformed super-source network that send all required flow correspond one-to-one with feasible bounded circulations after adding the committed lower bounds.

Proof. Committing lower bounds leaves exactly capacities **upper-lower**. Super edges encode the opposite of each lower-flow imbalance. A full super-source-to-sink flow makes additional original inflow equal additional outflow after those demands are removed, hence restores conservation. Conversely every feasible circulation's additional flow satisfies these balances and extends to a full super flow.

Lemma 2. For each original edge, the minimum marginal-edge cost of choosing **t** residual units is **f(lower+t)-f(lower)**.

Proof. Marginal costs are nondecreasing, so the cheapest **t** parallel units are the first **t**. Their telescoping sum is exactly the stated difference.

Lemma 3. Minimum-cost max-flow finds the least additional convex cost among all feasible repairs.

Proof. Lemma 1 gives the feasible-set correspondence. Lemma 2 makes the minimum parallel-edge representation of every residual amount equal its convex cost. Successive shortest augmenting paths with feasible potentials are the standard optimal min-cost-flow algorithm.

Theorem. The algorithm reports IMPOSSIBLE exactly when no bounded circulation exists; otherwise it reports the minimum total cost.

Proof. Failure to send all balance demand contradicts Lemma 1. In the feasible case, add the fixed lower-bound base cost to the optimum from Lemma 3.

Complexity and traps

Let **C=sum(upper-lower)**, at most 6000, and let **F** be the required balancing flow. The expanded residual graph has $O(n+C)$ vertices/edges in the relevant count, and the unit marginals can cause up to $F \leq C$ augmentations. The bound is $O(F (n+C) \log n)$ with Dijkstra and $O(n+C)$ residual storage.

The balance direction is easy to reverse: positive **in_lower-out_lower** must receive a super-source edge so it can send additional original flow outward. Use the marginal at the already committed amount 1, not at zero. Preserve reverse residual edges so later paths can reroute earlier choices. The nonnegative-marginal constraint is essential: otherwise an independent negative circulation could improve the objective without helping super flow. Test fixed edges, zero residual capacity, zero marginal costs, parallel edges, an impossible imbalance, and several marginals that must be rerouted.

C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

struct Edge {
    int to, reverse;
    int64 capacity, cost;
};

int main() {
```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);

int n, m;
cin >> n >> m;
int source = n, sink = n + 1;
vector<vector<Edge>> graph(n + 2);
auto add_edge = [&](int u, int v, int64 capacity, int64 cost) {
    Edge forward{v, static_cast<int>(graph[v].size()), capacity, cost};
    Edge backward{u, static_cast<int>(graph[u].size()), 0, -cost};
    graph[u].push_back(forward);
    graph[v].push_back(backward);
};

vector<int64> balance(n);
int64 base_cost = 0;
for (int edge = 0; edge < m; ++edge) {
    int u, v;
    int64 lower, upper, c, d;
    cin >> u >> v >> lower >> upper >> c >> d;
    --u;
    --v;
    balance[u] -= lower;
    balance[v] += lower;
    base_cost += c * lower + d * lower * (lower - 1) / 2;
    for (int64 amount = lower; amount < upper; ++amount)
        add_edge(u, v, 1, c + d * amount);
}

int64 required = 0;
for (int vertex = 0; vertex < n; ++vertex) {
    if (balance[vertex] > 0) {
        add_edge(source, vertex, balance[vertex], 0);
        required += balance[vertex];
    } else if (balance[vertex] < 0) {
        add_edge(vertex, sink, -balance[vertex], 0);
    }
}

const int64 INF = 4'000'000'000'000'000'000LL;
vector<int64> potential(n + 2), distance(n + 2);
vector<int> previous_vertex(n + 2), previous_edge(n + 2);
int64 sent = 0, extra_cost = 0;
while (sent < required) {
    fill(distance.begin(), distance.end(), INF);
    distance[source] = 0;
    priority_queue<pair<int64, int>, vector<pair<int64, int>>, greater<>> queue;
    queue.push({0, source});
    while (!queue.empty()) {
        auto [current_distance, u] = queue.top();
        queue.pop();
        if (current_distance != distance[u]) continue;

```

```

        for (int index = 0; index < static_cast<int>(graph[u].size()); ++index) {
            const Edge &edge = graph[u][index];
            if (edge.capacity == 0) continue;
            int64 candidate = current_distance + edge.cost
                             + potential[u] - potential[edge.to];
            if (candidate < distance[edge.to]) {
                distance[edge.to] = candidate;
                previous_vertex[edge.to] = u;
                previous_edge[edge.to] = index;
                queue.push({candidate, edge.to});
            }
        }
    }
    if (distance[sink] == INF) break;
    for (int vertex = 0; vertex < n + 2; ++vertex)
        if (distance[vertex] != INF) potential[vertex] += distance[vertex];
    int64 pushed = required - sent;
    for (int v = sink; v != source; v = previous_vertex[v])
        pushed = min(pushed, graph[previous_vertex[v]][previous_edge[v]].capacity);
    for (int v = sink; v != source; v = previous_vertex[v]) {
        int u = previous_vertex[v], index = previous_edge[v];
        Edge &edge = graph[u][index];
        extra_cost += pushed * edge.cost;
        edge.capacity -= pushed;
        graph[v][edge.reverse].capacity += pushed;
    }
    sent += pushed;
}
if (sent != required) cout << "IMPOSSIBLE\n";
else cout << base_cost + extra_cost << '\n';
}

```

Python

```

import heapq
import sys

INF = 10**40

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    n, m = data[index], data[index + 1]
    index += 2
    source, sink = n, n + 1
    graph = [[] for _ in range(n + 2)]

```

```

def add_edge(u, v, capacity, cost):
    graph[u].append([v, len(graph[v]), capacity, cost])
    graph[v].append([u, len(graph[u]) - 1, 0, -cost])

balance = [0] * n
base_cost = 0
for _ in range(m):
    u, v, lower, upper, c, d = data[index:index + 6]
    index += 6
    u -= 1
    v -= 1
    balance[u] -= lower
    balance[v] += lower
    base_cost += c * lower + d * lower * (lower - 1) // 2
    for amount in range(lower, upper):
        add_edge(u, v, 1, c + d * amount)
required = 0
for vertex, value in enumerate(balance):
    if value > 0:
        add_edge(source, vertex, value, 0)
        required += value
    elif value < 0:
        add_edge(vertex, sink, -value, 0)

potential = [0] * (n + 2)
sent = extra_cost = 0
while sent < required:
    distance = [INF] * (n + 2)
    previous = [None] * (n + 2)
    distance[source] = 0
    queue = [(0, source)]
    while queue:
        current_distance, u = heapq.heappop(queue)
        if current_distance != distance[u]:
            continue
        for edge_index, edge in enumerate(graph[u]):
            v, _, capacity, cost = edge
            if not capacity:
                continue
            candidate = current_distance + cost + potential[u] - potential[v]
            if candidate < distance[v]:
                distance[v] = candidate
                previous[v] = (u, edge_index)
                heapq.heappush(queue, (candidate, v))
    if distance[sink] == INF:
        break
    for vertex in range(n + 2):
        if distance[vertex] != INF:
            potential[vertex] += distance[vertex]
    pushed = required - sent
    vertex = sink

```

```

while vertex != source:
    u, edge_index = previous[vertex]
    pushed = min(pushed, graph[u][edge_index][2])
    vertex = u
vertex = sink
while vertex != source:
    u, edge_index = previous[vertex]
    edge = graph[u][edge_index]
    extra_cost += pushed * edge[3]
    edge[2] -= pushed
    graph[vertex][edge[1]][2] += pushed
    vertex = u
    sent += pushed
print("IMPOSSIBLE" if sent != required else base_cost + extra_cost)

if __name__ == "__main__":
    main()

```

12. Quota Project Portfolio

Why the promise appears

Maximum-weight closure is a min-cut problem, but adding an arbitrary exact cardinality constraint does not automatically remain solvable by a Lagrangian penalty. Some cardinalities may lie below the supported concave envelope and never be optimal for any penalty. The statement therefore promises that the requested K is attained by every optimum for some integer penalty. This is a mathematical precondition, not a convenience for the implementation.

For penalty `lambda`, replace every profit by

```
adjusted[u] = p[u] - lambda.
```

Maximize adjusted profit over dependency-closed sets. Raising `lambda` makes large sets less attractive, and canonical optimal closures are nested, so their size is nonincreasing. Binary-search the first integer penalty whose canonical optimum has size at most K . The promise forces its size to be exactly K .

Closure as a cut

For positive adjusted weight w , add source edge `source` $\rightarrow$ `u` of capacity w . For negative weight, add `u` $\rightarrow$ `sink` of capacity $-w$. For dependency `u requires v`, add an infinite-capacity edge `u` $\rightarrow$ `v`.

A finite cut cannot put `u` on the source side and `v` on the sink side, so its source-side projects are closed. The usual maximum-closure identity says

```
maximum adjusted profit = sum of positive adjusted weights - minimum cut.
```

After maximum flow, vertices reachable from the source in the residual graph form the inclusion-minimal optimal closure. Choosing the same canonical side at every penalty gives the needed monotone size function.

Sample trace

Project 1 requires 2. For penalties five and six, closure $\{1,2\}$ has adjusted value larger than the empty set, $\{2\}$, $\{3\}$, $\{2,3\}$, and the full set. Its size is two and original profit is $8+3=11$. Because every exact-two candidate pays the same $2*\lambda$ penalty, it is also the best original-profit closure of size two.

Algorithm

```
solve(lambda):
    build the maximum-closure cut network with weights  $p[u]-\lambda$ 
    run max flow
    return residual-source-side size and original profit

binary-search lambda in  $[-10^9-1, 10^9+1]$ 
for the first solve(lambda).size  $\leq K$ 
run solve once more and print its original profit
```

At the low endpoint every adjusted profit is positive, so selecting all projects is uniquely optimal. At the high endpoint every adjusted profit is negative, so the empty set is uniquely optimal.

Correctness proof

Lemma 1. `solve(lambda)` returns a dependency-closed set maximizing $\sum(p[u]-\lambda)$.

Proof. Infinite dependency arcs exclude exactly non-closed source sides. For every finite cut, its capacity equals the total positive adjusted weight minus the adjusted profit of its source-side closure. Minimizing the former therefore maximizes the latter. Residual reachability identifies one minimum cut.

Lemma 2. The size returned by `solve(lambda)` is nonincreasing as λ increases.

Proof. Maximum-closure cuts form a lattice. Choosing the inclusion-minimal minimum source side by residual reachability yields the standard nested family under a uniform increase of every vertex penalty. Equivalently, compare optimality inequalities at two penalties and use the submodularity of cut capacity; their minimal minimizers cannot gain a vertex at the larger penalty.

Lemma 3. Binary search returns a penalty whose closure size is exactly K .

Proof. Lemma 2 justifies searching the first size at most K . By the promise, some integer penalty has all optimum closures of size K , including the canonical one. A first value below K could never later rise to K , so the first value at most K must equal K .

Lemma 4. A penalized optimum of size K maximizes original profit among all closed sets of size K .

Proof. Every size- K set subtracts the same quantity $\lambda*K$. Thus their penalized ordering is exactly their original-profit ordering.

Theorem. The printed value is the requested exact-quota maximum profit.

Proof. Combine Lemmas 1, 3, and 4.

Complexity and traps

The binary search performs at most 31 max-flow computations on $n+2$ vertices and $O(n+m)$ edges. With the supplied Dinic implementation, a conservative bound is $O(\log P * V^2 * E)$, where P is the profit range; actual instances are much smaller under the stated limits. Each network uses $O(n+m)$ memory.

Do not claim that Lagrangian relaxation solves unrestricted exact-cardinality closure; the attainment promise is indispensable. Use an infinite capacity larger than every finite cut. Recover the residual source side after max flow, and use the same canonical recovery for monotonicity. Test $K=0$, $K=n$, dependency cycles, zero adjusted weights, multiple optimal sets of the same cardinality, and penalties at the two search endpoints.

C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

constexpr int64 INF = 4'000'000'000'000'000'000LL;

struct Dinic {
    struct Edge { int to, reverse; int64 capacity; };
    int n;
    vector<vector<Edge>> graph;
    vector<int> level, next_edge;

    explicit Dinic(int n) : n(n), graph(n), level(n), next_edge(n) {}

    void add_edge(int u, int v, int64 capacity) {
        graph[u].push_back({v, static_cast<int>(graph[v].size()), capacity});
        graph[v].push_back({u, static_cast<int>(graph[u].size()) - 1, 0});
    }

    bool bfs(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> queue;
        level[source] = 0;
        queue.push(source);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (const Edge &edge : graph[u])
                if (edge.capacity > 0 && level[edge.to] == -1) {
                    level[edge.to] = level[u] + 1;
                    queue.push(edge.to);
                }
        }
        return level[sink] != -1;
    }

    int64 dfs(int u, int sink, int64 pushed) {
        if (u == sink) return pushed;

```



```

        for (int &index = next_edge[u]; index < static_cast<int>(graph[u].size()); ++index) {
            Edge &edge = graph[u][index];
            if (edge.capacity == 0 || level[edge.to] != level[u] + 1) continue;
            int64 sent = dfs(edge.to, sink, min(pushed, edge.capacity));
            if (sent == 0) continue;
            edge.capacity -= sent;
            graph[edge.to][edge.reverse].capacity += sent;
            return sent;
        }
        return 0;
    }

int64 max_flow(int source, int sink) {
    int64 result = 0;
    while (bfs(source, sink)) {
        fill(next_edge.begin(), next_edge.end(), 0);
        while (int64 pushed = dfs(source, sink, INF)) result += pushed;
    }
    return result;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, target;
    cin >> n >> m >> target;
    vector<int64> profit(n);
    for (int64 &x : profit) cin >> x;
    vector<pair<int, int>> dependencies(m);
    for (auto &[u, v] : dependencies) {
        cin >> u >> v;
        --u;
        --v;
    }

    auto solve = [&](int64 penalty) {
        int source = n, sink = n + 1;
        Dinic flow(n + 2);
        for (int vertex = 0; vertex < n; ++vertex) {
            int64 adjusted = profit[vertex] - penalty;
            if (adjusted > 0) flow.add_edge(source, vertex, adjusted);
            else if (adjusted < 0) flow.add_edge(vertex, sink, -adjusted);
        }
        for (auto [u, v] : dependencies) flow.add_edge(u, v, INF);
        flow.max_flow(source, sink);
        vector<bool> reachable(n + 2);
        queue<int> queue;
        reachable[source] = true;
        queue.push(source);
    };

```

```

while (!queue.empty()) {
    int u = queue.front();
    queue.pop();
    for (const auto &edge : flow.graph[u])
        if (edge.capacity > 0 && !reachable[edge.to]) {
            reachable[edge.to] = true;
            queue.push(edge.to);
        }
}

int size = 0;
int64 original_profit = 0;
for (int vertex = 0; vertex < n; ++vertex)
    if (reachable[vertex]) {
        ++size;
        original_profit += profit[vertex];
    }
return pair{size, original_profit};
};

int64 low = -1'000'000'001LL, high = 1'000'000'001LL;
while (low < high) {
    int64 middle = low + (high - low) / 2;
    if (solve(middle).first <= target) high = middle;
    else low = middle + 1;
}
auto [size, answer] = solve(low);
if (size != target) {
    // The input promise guarantees this branch is unreachable.
    return 1;
}
cout << answer << '\n';
}

```

Python

```

import collections
import sys

INF = 10**30

class Dinic:
    def __init__(self, n):
        self.graph = [[] for _ in range(n)]
        self.level = [-1] * n
        self.next_edge = [0] * n

    def add_edge(self, u, v, capacity):
        self.graph[u].append([v, len(self.graph[v]), capacity])
        self.graph[v].append([u, len(self.graph[u]) - 1, 0])

```

```

def max_flow(self, source, sink):
    total = 0
    n = len(self.graph)
    while True:
        self.level[:] = [-1] * n
        self.level[source] = 0
        queue = collections.deque([source])
        while queue:
            u = queue.popleft()
            for v, _, capacity in self.graph[u]:
                if capacity and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        if self.level[sink] == -1:
            return total
        self.next_edge[:] = [0] * n

    def send(u, pushed):
        if u == sink:
            return pushed
        while self.next_edge[u] < len(self.graph[u]):
            edge_index = self.next_edge[u]
            edge = self.graph[u][edge_index]
            v, reverse, capacity = edge
            if capacity and self.level[v] == self.level[u] + 1:
                sent = send(v, min(pushed, capacity))
                if sent:
                    edge[2] -= sent
                    self.graph[v][reverse][2] += sent
                    return sent
            self.next_edge[u] += 1
        return 0

    while True:
        pushed = send(source, INF)
        if not pushed:
            break
        total += pushed

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    n, m, target = data[index:index + 3]
    index += 3
    profits = data[index:index + n]
    index += n
    dependencies = []
    for _ in range(m):

```

```

dependencies.append((data[index] - 1, data[index + 1] - 1))
index += 2

def solve(penalty):
    source, sink = n, n + 1
    flow = Dinic(n + 2)
    for vertex, profit in enumerate(profits):
        adjusted = profit - penalty
        if adjusted > 0:
            flow.add_edge(source, vertex, adjusted)
        elif adjusted < 0:
            flow.add_edge(vertex, sink, -adjusted)
    for u, v in dependencies:
        flow.add_edge(u, v, INF)
    flow.max_flow(source, sink)
    reachable = [False] * (n + 2)
    reachable[source] = True
    queue = [source]
    for u in queue:
        for v, _, capacity in flow.graph[u]:
            if capacity and not reachable[v]:
                reachable[v] = True
                queue.append(v)

    size = 0
    original_profit = 0
    for vertex in range(n):
        if reachable[vertex]:
            size += 1
            original_profit += profits[vertex]
    return size, original_profit

low, high = -1_000_000_001, 1_000_000_001
while low < high:
    middle = (low + high) // 2
    if solve(middle)[0] <= target:
        high = middle
    else:
        low = middle + 1
size, answer = solve(low)
if size != target:
    raise RuntimeError("input violates the promised attainable cardinality")
print(answer)

if __name__ == "__main__":
    sys.setrecursionlimit(1_000_000)
    main()

```

13. Rainbow Bottleneck Forest

Two independent restrictions

For a fixed weight limit B , discard every edge heavier than B . We only need to decide whether K remaining edges can satisfy both restrictions.

The cycle-free edge sets form the **graphic matroid**. The sets containing at most one edge of every color form a **partition matroid**. We therefore seek a common independent set of size K . Neither ordinary Kruskal nor choosing the lightest edge of each color is enough: accepting an edge may consume a color needed to escape a future graphic cycle.

Feasibility is monotone in B , so after building a common-independent-set oracle, binary-search the distinct input weights.

The exchange graph

Let I be the current rainbow forest. Its exchange graph has one vertex for every allowed edge. Vertices alternate between edges outside and inside I .

- An outside edge y is a **source** if $I + y$ is still a forest.
- An outside edge y is a **target** if its color is unused by I .
- Add $x \rightarrow y$, with x inside and y outside, when $I - x + y$ is a forest.
- Add $y \rightarrow x$ when $I - x + y$ still has distinct colors.

A source-to-target path describes an alternating sequence of insertions and deletions. Toggle every edge on a shortest such path. It starts and ends outside I , so the size grows by one; the matroid-intersection augmenting-path theorem guarantees that both independence conditions survive.

Repeat until the set has size K , or until no augmenting path exists. In the latter case, the current common independent set has maximum possible size.

Implementing both exchange oracles

Because I is a forest, the graphic exchanges are especially concrete. For an outside edge $y=(u,v)$:

- if u and v lie in different components, y is a source;
- otherwise they have one unique path in I , and exactly the selected edges on that path may be removed to make room for y . Add $x \rightarrow y$ for each of those path edges x .

A BFS in the current forest recovers the path. Repeating it for every outside edge is fast enough for the limits.

For colors, keep `color_edge[c]`, the selected edge of color c , or -1 . If y has an unused color, it is a target. Otherwise only the selected edge of the same color can be replaced, so add $y \rightarrow \text{color_edge}[\text{color}[y]]$. When the color is unused, exchanges to other selected edges are also legal, but unnecessary: reaching y has already completed an augmenting path.

Sample trace

At limit five, the available sample edges have colors $1,1,4$; at most two distinct colors can be selected, so size three is impossible. At limit six, edges $(1,2)$, $(1,3)$, and $(2,4)$ have colors $1,3,4$ and form a tree. Thus the first feasible bottleneck is six.

Correctness proof

Lemma 1. The sources and arcs directed from selected to unselected edges encode exactly the legal graphic additions and one-edge exchanges.

Proof. Adding (u, v) to a forest is legal exactly when its endpoints are in different components. Otherwise it creates one cycle: the outside edge plus the unique forest path from u to v . Removing a selected edge restores a forest exactly when that edge lies on this cycle, equivalently on the path.

Lemma 2. The targets and arcs directed from unselected to selected edges encode every color exchange needed by an augmenting path.

Proof. An outside edge is directly addable exactly when its color is unused. If the color is used, partition independence can be restored only by deleting the unique selected edge with that color. If it is unused, the vertex is already a target and the augmenting path can end there.

Lemma 3. If the exchange graph has a source-to-target path, toggling the edges of a shortest one produces a rainbow forest of size $|I|+1$.

Proof. The path alternates between outside and selected edges and has two outside endpoints, so toggling increases the cardinality by one. Every selected-to-unselected step is a valid graphic exchange by Lemma 1; every unselected-to-selected step is a valid partition exchange by Lemma 2. For a shortest path, if a later insertion invalidated an earlier exchange, the matroid circuit-exchange property would supply an arc skipping the conflicting segment, contradicting shortestness. Hence the simultaneous toggle is independent in both matroids. This is the augmenting-path lemma for two matroids, specialized here to graphic circuits and color classes.

Lemma 4. If no source-to-target path exists, no larger rainbow forest exists among the allowed edges.

Proof. Suppose a larger common independent set J existed. Compare $J-I$ with $I-J$. The matroid exchange property for the graphic matroid links an edge addable to I through legal graphic exchanges; the corresponding property for the partition matroid links the alternating exchanges onward. Because J has more edges, this alternating chain must end at an edge directly addable in the partition matroid. It is therefore a source-to-target path, a contradiction. This is the maximality half of the matroid-intersection augmenting-path theorem.

Lemma 5. The feasibility predicate is monotone in the weight limit.

Proof. Raising the limit only adds available edges. Every previously valid rainbow forest remains valid.

Theorem. The algorithm prints the minimum possible maximum edge weight, or -1 exactly when no requested forest exists.

Proof. Lemmas 3 and 4 show that the oracle accepts exactly when an allowed common independent set has size at least K . By Lemma 5, binary search finds the first distinct edge weight accepted by the oracle. This is precisely the minimum bottleneck. Rejection at the largest input weight means that even the whole graph has no valid size- K set.

Complexity and traps

Let $R \leq \min(K, n-1)$ be the attained common rank. During one augmentation, finding all forest paths and building the exchange graph costs $O(m(n+R))$; its BFS costs no more. One feasibility call therefore costs $O(R \cdot m \cdot (n+R))$ time and $O(n+m \cdot R)$ memory. Binary search adds a factor $O(\log m)$.

Do not direct both exchange types the same way. Do not treat an arbitrary maximal rainbow forest as maximum: the entire purpose of alternating paths is to repair earlier choices. Search only distinct input weights, handle parallel edges normally, and test one color repeated everywhere, a graph with enough colors but insufficient graphic rank, bottleneck ties, and an augmenting path requiring more than one exchange.

C++

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int u, v, color;
    long long weight;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, need;
    cin >> n >> m >> need;
    vector<Edge> edges(m);
    vector<int> colors;
    vector<long long> weights;
    for (auto &[u, v, color, weight] : edges) {
        cin >> u >> v >> color >> weight;
        --u;
        --v;
        colors.push_back(color);
        weights.push_back(weight);
    }
    sort(colors.begin(), colors.end());
    colors.erase(unique(colors.begin(), colors.end()), colors.end());
    for (auto &edge : edges) {
        edge.color = lower_bound(colors.begin(), colors.end(), edge.color)
            - colors.begin();
    }
    sort(weights.begin(), weights.end());
    weights.erase(unique(weights.begin(), weights.end()), weights.end());

    auto feasible = [&](long long limit) {
        vector<char> allowed(m), chosen(m);
        for (int i = 0; i < m; ++i) allowed[i] = edges[i].weight <= limit;

        for (int size = 0; size < need; ++size) {
            vector<vector<pair<int, int>>> forest(n);
            vector<int> color_edge(colors.size(), -1);
            for (int i = 0; i < m; ++i) if (chosen[i]) {
                forest[edges[i].u].push_back({edges[i].v, i});
                forest[edges[i].v].push_back({edges[i].u, i});
                color_edge[edges[i].color] = i;
            }

            vector<vector<int>> next(m);
            vector<char> source(m), target(m);
            for (int y = 0; y < m; ++y) if (allowed[y] && !chosen[y]) {
```

```

vector<int> parent_vertex(n, -1), parent_edge(n, -1);
queue<int> vertices;
parent_vertex[edges[y].u] = edges[y].u;
vertices.push(edges[y].u);
while (!vertices.empty() && parent_vertex[edges[y].v] == -1) {
    int u = vertices.front();
    vertices.pop();
    for (auto [v, index] : forest[u]) {
        if (parent_vertex[v] != -1) continue;
        parent_vertex[v] = u;
        parent_edge[v] = index;
        vertices.push(v);
    }
}

if (parent_vertex[edges[y].v] == -1) {
    source[y] = true;
} else {
    for (int v = edges[y].v; v != edges[y].u;
        v = parent_vertex[v]) {
        next[parent_edge[v]].push_back(y);
    }
}

int same = color_edge[edges[y].color];
if (same == -1) target[y] = true;
else next[y].push_back(same);
}

queue<int> queue_nodes;
vector<int> parent(m, -2);
for (int i = 0; i < m; ++i) if (source[i]) {
    parent[i] = -1;
    queue_nodes.push(i);
}
int finish = -1;
while (!queue_nodes.empty() && finish == -1) {
    int u = queue_nodes.front();
    queue_nodes.pop();
    if (target[u]) {
        finish = u;
        break;
    }
    for (int v : next[u]) if (parent[v] == -2) {
        parent[v] = u;
        queue_nodes.push(v);
    }
}
if (finish == -1) return false;
for (int at = finish; at != -1; at = parent[at]) chosen[at] ^= 1;
}
return true;

```



```

};

if (weights.empty() || !feasible(weights.back())) {
    cout << -1 << '\n';
    return 0;
}
int low = 0, high = (int)weights.size() - 1;
while (low < high) {
    int middle = (low + high) / 2;
    if (feasible(weights[middle])) high = middle;
    else low = middle + 1;
}
cout << weights[low] << '\n';
}

```

Python

```

import sys
from collections import deque

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, need = data[:3]
    raw = [data[i:i + 4] for i in range(3, len(data), 4)]
    color_values = {edge[2] for edge in raw}
    color_id = {color: i for i, color in enumerate(color_values)}
    edges = [(u - 1, v - 1, color_id[color], weight)
              for u, v, color, weight in raw]
    weights = sorted({edge[3] for edge in edges})

    def feasible(limit: int) -> bool:
        allowed = [weight <= limit for _, _, _, weight in edges]
        chosen = [False] * m
        for _ in range(need):
            forest = [[] for _ in range(n)]
            color_edge = [-1] * len(color_values)
            for i, (u, v, color, _) in enumerate(edges):
                if chosen[i]:
                    forest[u].append((v, i))
                    forest[v].append((u, i))
                    color_edge[color] = i

            exchange = [[] for _ in range(m)]
            source = [False] * m
            target = [False] * m
            for y, (start, goal, color, _) in enumerate(edges):
                if not allowed[y] or chosen[y]:

```

```

        continue
    parent_vertex = [-1] * n
    parent_edge = [-1] * n
    parent_vertex[start] = start
    queue = deque([start])
    while queue and parent_vertex[goal] == -1:
        u = queue.popleft()
        for v, index in forest[u]:
            if parent_vertex[v] == -1:
                parent_vertex[v] = u
                parent_edge[v] = index
                queue.append(v)
    if parent_vertex[goal] == -1:
        source[y] = True
    else:
        v = goal
        while v != start:
            exchange[parent_edge[v]].append(y)
            v = parent_vertex[v]

    same = color_edge[color]
    if same == -1:
        target[y] = True
    else:
        exchange[y].append(same)

parent = [-2] * m
queue = deque()
for i in range(m):
    if source[i]:
        parent[i] = -1
        queue.append(i)
finish = -1
while queue and finish == -1:
    u = queue.popleft()
    if target[u]:
        finish = u
        break
    for v in exchange[u]:
        if parent[v] == -2:
            parent[v] = u
            queue.append(v)
if finish == -1:
    return False
while finish != -1:
    chosen[finish] = not chosen[finish]
    finish = parent[finish]
return True

if not weights or not feasible(weights[-1]):
    print(-1)

```

```

    return
    low, high = 0, len(weights) - 1
    while low < high:
        middle = (low + high) // 2
        if feasible(weights[middle]):
            high = middle
        else:
            low = middle + 1
    print(weights[low])

if __name__ == "__main__":
    main()

```

14. Rooted Broadcast Choice

Remove the unknown root

If the broadcaster were fixed, the chosen links would be a minimum directed spanning arborescence rooted there: every other vertex has one incoming edge, and all vertices are reachable from the root. Trying all roots would multiply an already nontrivial algorithm by n .

Add a new vertex s . For every station v , add

```
s -> v with cost PENALTY + activation[v].
```

Now find one minimum arborescence rooted at s . A solution using exactly one super-root edge chooses one broadcaster; deleting s leaves precisely the required broadcast tree.

Choose

```
PENALTY = (n+1) * 109 + 1.
```

Every arborescence has n edges entering real vertices, and the sum of their real activation/link parts is at most $n \cdot 10^9$, strictly less than the penalty. Consequently the encoded objective first minimizes the number of super-root edges and only then the real cost. If its quotient by $PENALTY$ is one, its remainder is the answer. If the quotient is larger, no single broadcaster can reach every station.

Directed minimum arborescence

Chu–Liu/Edmonds repeatedly performs the following operation on the current contracted graph.

1. For every non-root vertex v , choose its cheapest incoming edge and add that cost to the answer.
2. If some non-root vertex has no incoming edge, no arborescence exists.
3. If the chosen incoming edges contain no directed cycle, they are the minimum arborescence.
4. Contract every chosen directed cycle and continue.

Why is contraction enough? In a chosen cycle, every vertex already has its cheapest incoming edge, but a rooted arborescence cannot keep the whole cycle. Some external edge $u \rightarrow v$ must enter it and replace the chosen edge entering v . We already prepaid $\text{incoming}[v]$, so this option has additional cost

```
cost(u -> v) - incoming[v].
```

Use that reduced cost for every edge crossing between contracted components. Internal edges disappear as self-loops. When the contracted optimum is added to the prepaid costs, entering a cycle automatically pays exactly the cost of the edge it substitutes.

Several vertex-disjoint cycles may be contracted in the same phase. Vertices outside every cycle receive singleton component identifiers. Continue until a phase has no cycle.

Sample trace

Choosing station 2 costs three. Links 2->3, 3->4, and 4->1 cost two, two, and one, for a total of eight. Every station is reached. The encoded arborescence uses one super edge, and no real solution can cost less.

Correctness proof

Lemma 1. Arborescences rooted at the super-root and using exactly one super edge correspond bijectively to valid broadcast choices, preserving real cost.

Proof. The unique super edge selects a station r . Every other real vertex has exactly one incoming real edge. Reachability from the super-root becomes reachability from r after deleting the super-root. Conversely, attach the super-root to the broadcaster of any valid solution.

Lemma 2. One contraction phase preserves the minimum remaining cost after adding the selected minimum incoming costs.

Proof. If the selected edges are acyclic, they give every non-root vertex one incoming edge and are reachable from the root; choosing any dearer incoming edge cannot improve the total. If they contain a cycle C , every arborescence must replace at least one selected incoming edge of C by an edge entering C . Replacing the edge entering v has extra cost $\text{cost}(u,v) - \text{incoming}[v]$, exactly the contracted edge cost. Conversely, expand any chosen contracted entering edge by restoring all prepaid cycle edges except the one entering its endpoint. This gives a cost-preserving correspondence. Disjoint cycles can be treated independently.

Lemma 3. The directed-MST procedure returns a minimum arborescence rooted at the supplied root.

Proof. Every contraction strictly reduces the vertex count. Apply Lemma 2 at each phase; the final acyclic phase is optimal, and expanding the contractions preserves that cost back to the original graph.

Lemma 4. The penalty makes the encoded objective lexicographically minimize first the number of super edges, then real cost.

Proof. Two solutions whose super-edge counts differ by at least one differ in penalty contribution by at least PENALTY . Their real-cost difference has absolute value below PENALTY , so it cannot reverse that ordering. Equal counts compare only by real cost.

Theorem. The algorithm prints the minimum valid broadcast cost, or -1 exactly when no broadcaster can reach all stations.

Proof. Super edges to every real vertex guarantee that some encoded arborescence exists. By Lemmas 3 and 4, it uses the minimum possible number of super edges. That number is one exactly when a valid broadcast tree exists by Lemma 1. In that case, Lemmas 1 and 4 show that the remainder is the minimum real cost; otherwise the algorithm prints -1 .

Complexity and traps

Each phase scans all current edges. A phase containing a cycle reduces the number of vertices, so the total time is $O(VE)$ and memory is $O(V+E)$, with $V=n+1$ and $E=m+n$ after adding the super-root.

Directed arborescences use one **incoming** edge per non-root vertex; reversing this convention silently solves a different problem. Subtract the prepaid cost of the original target before replacing its component identifier. Ignore self-loops when selecting incoming edges. Use 64-bit integers: the encoded cost is much larger than the real answer. Test $n=1$, parallel links, a cheap directed cycle, a graph needing two super edges, and a contraction whose best entering edge targets a non-obvious cycle vertex.

C++

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int from, to;
    long long cost;
};

long long directed_mst(int vertices, int root, vector<Edge> edges) {
    const long long INF = (1LL << 62);
    long long answer = 0;
    while (true) {
        vector<long long> incoming(vertices, INF);
        vector<int> parent(vertices, -1);
        for (const auto &edge : edges) {
            if (edge.from != edge.to && edge.cost < incoming[edge.to]) {
                incoming[edge.to] = edge.cost;
                parent[edge.to] = edge.from;
            }
        }
        incoming[root] = 0;
        for (int v = 0; v < vertices; ++v) {
            if (incoming[v] == INF) return -1;
            answer += incoming[v];
        }

        int cycles = 0;
        vector<int> component(vertices, -1), seen(vertices, -1);
        for (int start = 0; start < vertices; ++start) {
            int v = start;
            while (seen[v] != start && component[v] == -1 && v != root) {
                seen[v] = start;
                v = parent[v];
            }
            if (v != root && component[v] == -1) {
                for (int u = parent[v]; u != v; u = parent[u]) {
                    component[u] = cycles;
                }
            }
        }
    }
}
```

```

        component[v] = cycles++;
    }
}
if (cycles == 0) return answer;

for (int v = 0; v < vertices; ++v) {
    if (component[v] == -1) component[v] = cycles++;
}
vector<Edge> contracted;
contracted.reserve(edges.size());
for (const auto &edge : edges) {
    int from = component[edge.from];
    int to = component[edge.to];
    long long cost = edge.cost;
    if (from != to) cost -= incoming[edge.to];
    contracted.push_back({from, to, cost});
}
root = component[root];
vertices = cycles;
edges.swap(contracted);
}
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<long long> activation(n);
    for (long long &cost : activation) cin >> cost;

    const long long PENALTY = (long long)(n + 1) * 1'000'000'000LL + 1;
    vector<Edge> edges;
    edges.reserve(m + n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long cost;
        cin >> u >> v >> cost;
        edges.push_back({u - 1, v - 1, cost});
    }
    int super_root = n;
    for (int v = 0; v < n; ++v) {
        edges.push_back({super_root, v, PENALTY + activation[v]});
    }

    long long encoded = directed_mst(n + 1, super_root, edges);
    if (encoded < 0 || encoded / PENALTY != 1) cout << -1 << '\n';
    else cout << encoded % PENALTY << '\n';
}

```

Python

```
import sys

def directed_mst(vertices: int, root: int,
                 edges: list[tuple[int, int, int]]) -> int | None:
    answer = 0
    infinity = 10**40
    while True:
        incoming = [infinity] * vertices
        parent = [-1] * vertices
        for u, v, cost in edges:
            if u != v and cost < incoming[v]:
                incoming[v] = cost
                parent[v] = u
        incoming[root] = 0
        if any(cost == infinity for cost in incoming):
            return None
        answer += sum(incoming)

        component = [-1] * vertices
        seen = [-1] * vertices
        cycles = 0
        for start in range(vertices):
            v = start
            while seen[v] != start and component[v] == -1 and v != root:
                seen[v] = start
                v = parent[v]
            if v != root and component[v] == -1:
                u = parent[v]
                while u != v:
                    component[u] = cycles
                    u = parent[u]
                component[v] = cycles
                cycles += 1
        if cycles == 0:
            return answer

        for v in range(vertices):
            if component[v] == -1:
                component[v] = cycles
                cycles += 1
        contracted = []
        for u, v, cost in edges:
            new_u, new_v = component[u], component[v]
            if new_u != new_v:
                cost -= incoming[v]
            contracted.append((new_u, new_v, cost))
        root = component[root]
        vertices = cycles
```

```

edges = contracted

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m = data[:2]
    activation = data[2:2 + n]
    edges = []
    index = 2 + n
    for _ in range(m):
        u, v, cost = data[index:index + 3]
        index += 3
        edges.append((u - 1, v - 1, cost))

    penalty = (n + 1) * 10**9 + 1
    super_root = n
    edges.extend((super_root, v, penalty + activation[v]) for v in range(n))
    encoded = directed_mst(n + 1, super_root, edges)
    if encoded is None or encoded // penalty != 1:
        print(-1)
    else:
        print(encoded % penalty)

if __name__ == "__main__":
    main()

```

15. Pairing Under Thresholds

Turn a bottleneck into a rank query

Fix a difficulty threshold D and retain only edges of difficulty at most D . A query k is feasible exactly when this graph has a matching of size at least k . Feasibility is monotone as D grows, so every answer is the first feasible distinct input difficulty.

The graph is not bipartite. In particular, odd cycles are legal input, so the usual two-part augmenting-path algorithm is insufficient. We need maximum cardinality matching in a general graph: Edmonds' blossom algorithm.

Alternating forests and blossoms

Starting from every currently unmatched root, search an alternating forest. Even-level vertices may use unmatched edges; after reaching an odd-level vertex, immediately cross its matched edge back to an even level.

Three events matter while scanning an edge from an even vertex v :

1. If it reaches an unseen vertex u , record $\text{parent}[u]=v$. If u is unmatched, an augmenting path is complete. Otherwise enqueue $\text{match}[u]$ as the next even vertex.

2. If it joins two suitable even vertices in the same alternating structure, their tree paths plus the edge form an odd alternating cycle: a blossom.
3. Edges internal to an already contracted base, and the current matched edge, carry no new information.

An alternating path can enter and leave an odd blossom while preserving its alternation: between any entry point and the blossom base, exactly one of the two directions around the odd cycle has the needed parity. Therefore contract the entire blossom to its lowest common alternating ancestor. The search may continue as though it were one even vertex. `mark_path` stores enough parents to expand a later augmenting path correctly.

When an unmatched endpoint is reached, follow parent and matched edges back to the root and flip their matched status. Each augmentation increases the matching size by one. Berge's lemma says a matching is maximum exactly when no augmenting path exists; blossom contraction ensures the search does not miss paths hidden behind odd cycles.

Share threshold searches offline

There can be 200,000 queries but only at most $\text{floor}(n/2)$ distinct requested matching sizes. Keep binary-search bounds `low[k]` and `high[k]` only for those distinct values, where `high=len(thresholds)` is an impossible sentinel.

In each parallel-binary-search round:

```
group every unfinished k by middle = (low[k]+high[k])/2
for each distinct middle:
    compute rank(middle) once with blossom
    for every k in the group:
        if rank >= k: high[k] = middle
        else:         low[k] = middle+1
```

Memoize every computed rank by threshold index. Repeated queries, equal midpoints in one round, and midpoints revisited in later rounds then all reuse the same expensive matching computation.

Sample trace

At threshold one, edge 2-3 gives rank one. At threshold three, 2-3 and 4-5 give rank two. Up through threshold six, the graph is two disjoint odd triangles, whose total matching rank is only two. Threshold seven adds 2-4; then 1-3, 2-4, and 5-6 form a perfect matching of size three.

Correctness proof

Lemma 1. Contracting an odd alternating blossom preserves whether an augmenting path exists.

Proof. Contract any augmenting path's maximal segment inside the blossom to the blossom base to obtain a path in the contracted graph. Conversely, when a contracted path touches the blossom, the odd cycle offers two routes between its contact vertex and base; their lengths have opposite parity. Exactly one continues the required matched/unmatched alternation. Expanding through that route yields an augmenting path in the original graph.

Lemma 2. `maximum_matching` returns the maximum matching size at its given threshold.

Proof. Every discovered augmenting path is flipped and increases the size by one. The alternating-forest search explores every possible augmenting path; by Lemma 1, blossom contractions preserve paths passing through

odd cycles. When all unmatched roots fail, no augmenting path exists. Berge's lemma then implies that the matching is maximum.

Lemma 3. For every k , feasibility is monotone in the threshold index.

Proof. Increasing the threshold only adds edges. Any old size- k matching remains available.

Lemma 4. Parallel binary search finishes with $\text{low}[k]$ equal to the first threshold whose matching rank is at least k , or the sentinel if none exists.

Proof. Initially the answer lies in the half-open search domain including the sentinel. By Lemma 2, $\text{rank}(\text{middle}) \geq k$ is the exact feasibility test, and by Lemma 3 it is monotone. Each update is therefore an ordinary lower-bound binary-search update. Grouping equal midpoints changes only how their common rank is computed, not any interval update.

Theorem. Every printed line is the minimum permitted difficulty for its requested number of pairs, or -1 exactly when impossible.

Proof. Apply Lemma 4 to the query's k . A real threshold index gives its minimum feasible difficulty. The sentinel means the maximum matching in the full graph is smaller than k , so no solution exists.

Complexity and traps

One blossom computation takes $O(n^3)$ time and $O(n^2)$ memory. If r is the number of distinct queried sizes, at most $r \cdot \text{ceil}(\log_2(m+1))$ threshold ranks are requested before memoization. Thus a conservative bound is $O(r \cdot n^3 \cdot \log m + q)$ time and $O(n^2 + m + q)$ memory, with $r \leq n/2$.

Do not replace blossom by bipartite matching unless the statement supplies a bipartition. During contraction, update every vertex whose current base is marked, not merely the cycle vertices first encountered. Reinitialize search parents and bases for each unmatched root, but retain the matching between augmentations. Binary-search distinct difficulties rather than the numeric range. Test triangles, two odd cycles connected by one late edge, parallel offers with different difficulties, repeated k , and an impossible maximum rank.

C++

```
#include <bits/stdc++.h>
using namespace std;

class Blossom {
    int n;
    vector<vector<int>> graph;
    vector<int> match, parent, base, queue_nodes;
    vector<char> used, in_blossom;

    int lca(int a, int b) {
        vector<char> seen(n, false);
        while (true) {
            a = base[a];
            seen[a] = true;
            if (match[a] == -1) break;
            a = parent[match[a]];
        }
        while (true) {
```

```

        b = base[b];
        if (seen[b]) return b;
        b = parent[match[b]];
    }
}

void mark_path(int v, int blossom_base, int child) {
    while (base[v] != blossom_base) {
        in_blossom[base[v]] = in_blossom[base[match[v]]] = true;
        parent[v] = child;
        child = match[v];
        v = parent[match[v]];
    }
}

bool augment_from(int root) {
    used.assign(n, false);
    parent.assign(n, -1);
    iota(base.begin(), base.end(), 0);
    queue_nodes.clear();
    queue_nodes.push_back(root);
    used[root] = true;
    for (int head = 0; head < (int)queue_nodes.size(); ++head) {
        int v = queue_nodes[head];
        for (int u : graph[v]) {
            if (base[v] == base[u] || match[v] == u) continue;
            if (u == root || (match[u] != -1 && parent[match[u]] != -1)) {
                int blossom_base = lca(v, u);
                in_blossom.assign(n, false);
                mark_path(v, blossom_base, u);
                mark_path(u, blossom_base, v);
                for (int x = 0; x < n; ++x) if (in_blossom[base[x]]) {
                    base[x] = blossom_base;
                    if (!used[x]) {
                        used[x] = true;
                        queue_nodes.push_back(x);
                    }
                }
            }
            else if (parent[u] == -1) {
                parent[u] = v;
                if (match[u] == -1) {
                    int x = u;
                    while (x != -1) {
                        int previous = parent[x];
                        int next = previous == -1 ? -1 : match[previous];
                        match[x] = previous;
                        if (previous != -1) match[previous] = x;
                        x = next;
                    }
                }
                return true;
            }
        }
    }
}

```

```

        u = match[u];
        used[u] = true;
        queue_nodes.push_back(u);
    }
}
return false;
}

public:
    explicit Blossom(vector<vector<int>> adjacency)
        : n(adjacency.size()), graph(move(adjacency)), match(n, -1),
          parent(n), base(n), used(n), in_blossom(n) {}

    int maximum_matching() {
        int size = 0;
        for (int v = 0; v < n; ++v) {
            if (match[v] == -1 && augment_from(v)) ++size;
        }
        return size;
    }
};

struct Edge {
    int u, v;
    long long difficulty;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    cin >> n >> m >> q;
    vector<Edge> edges(m);
    vector<long long> thresholds;
    for (auto &[u, v, difficulty] : edges) {
        cin >> u >> v >> difficulty;
        --u;
        --v;
        thresholds.push_back(difficulty);
    }
    vector<int> queries(q);
    vector<char> needed(n / 2 + 1, false);
    for (int &k : queries) {
        cin >> k;
        needed[k] = true;
    }
    sort(thresholds.begin(), thresholds.end());
    thresholds.erase(unique(thresholds.begin(), thresholds.end()), thresholds.end());
    if (thresholds.empty()) {

```

```

        while (q-->) cout << -1 << '\n';
        return 0;
    }

    vector<int> low(n / 2 + 1, 0), high(n / 2 + 1, thresholds.size());
    unordered_map<int, int> rank_cache;
    auto rank_at = [&](int index) {
        auto found = rank_cache.find(index);
        if (found != rank_cache.end()) return found->second;
        vector<vector<int>> graph(n);
        for (const auto &edge : edges) if (edge.difficulty <= thresholds[index]) {
            graph[edge.u].push_back(edge.v);
            graph[edge.v].push_back(edge.u);
        }
        int rank = Blossom(move(graph)).maximum_matching();
        rank_cache[index] = rank;
        return rank;
    };

    while (true) {
        map<int, vector<int>> groups;
        for (int k = 1; k <= n / 2; ++k) {
            if (needed[k] && low[k] < high[k]) {
                int middle = (low[k] + high[k]) / 2;
                groups[middle].push_back(k);
            }
        }
        if (groups.empty()) break;
        for (const auto &[middle, sizes] : groups) {
            int rank = rank_at(middle);
            for (int k : sizes) {
                if (rank >= k) high[k] = middle;
                else low[k] = middle + 1;
            }
        }
    }
    for (int k : queries) {
        if (low[k] == (int)thresholds.size()) cout << -1 << '\n';
        else cout << thresholds[low[k]] << '\n';
    }
}

```

Python

```

import sys
from collections import defaultdict, deque

def maximum_matching(graph: list[list[int]]) -> int:
    n = len(graph)

```

```

match = [-1] * n
parent = [-1] * n
base = list(range(n))
used = [False] * n
in_blossom = [False] * n

def lca(a: int, b: int) -> int:
    seen = [False] * n
    while True:
        a = base[a]
        seen[a] = True
        if match[a] == -1:
            break
        a = parent[match[a]]
    while True:
        b = base[b]
        if seen[b]:
            return b
        b = parent[match[b]]

def mark_path(v: int, blossom_base: int, child: int) -> None:
    while base[v] != blossom_base:
        in_blossom[base[v]] = True
        in_blossom[base[match[v]]] = True
        parent[v] = child
        child = match[v]
        v = parent[match[v]]

def augment_from(root: int) -> bool:
    nonlocal parent, base, used, in_blossom
    used = [False] * n
    parent = [-1] * n
    base = list(range(n))
    queue = deque([root])
    used[root] = True
    while queue:
        v = queue.popleft()
        for u in graph[v]:
            if base[v] == base[u] or match[v] == u:
                continue
            if u == root or (match[u] != -1 and parent[match[u]] != -1):
                blossom_base = lca(v, u)
                in_blossom = [False] * n
                mark_path(v, blossom_base, u)
                mark_path(u, blossom_base, v)
                for x in range(n):
                    if in_blossom[base[x]]:
                        base[x] = blossom_base
                    if not used[x]:
                        used[x] = True
                        queue.append(x)

```

```

        elif parent[u] == -1:
            parent[u] = v
            if match[u] == -1:
                x = u
                while x != -1:
                    previous = parent[x]
                    next_x = -1 if previous == -1 else match[previous]
                    match[x] = previous
                    if previous != -1:
                        match[previous] = x
                    x = next_x
                return True
            u = match[u]
            used[u] = True
            queue.append(u)
    return False

size = 0
for v in range(n):
    if match[v] == -1 and augment_from(v):
        size += 1
return size

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, q = data[:3]
    index = 3
    edges = []
    for _ in range(m):
        u, v, difficulty = data[index:index + 3]
        index += 3
        edges.append((u - 1, v - 1, difficulty))
    queries = data[index:index + q]
    thresholds = sorted({difficulty for _, _, difficulty in edges})
    if not thresholds:
        print("\n".join("-1" for _ in queries))
        return

    requested = set(queries)
    low = {k: 0 for k in requested}
    high = {k: len(thresholds) for k in requested}
    rank_cache = {}

    def rank_at(threshold_index: int) -> int:
        if threshold_index not in rank_cache:
            graph = [[] for _ in range(n)]
            limit = thresholds[threshold_index]
            for u, v, difficulty in edges:

```

```

        if difficulty <= limit:
            graph[u].append(v)
            graph[v].append(u)
        rank_cache[threshold_index] = maximum_matching(graph)
    return rank_cache[threshold_index]

while True:
    groups = defaultdict(list)
    for k in requested:
        if low[k] < high[k]:
            groups[(low[k] + high[k]) // 2].append(k)
    if not groups:
        break
    for middle, sizes in groups.items():
        rank = rank_at(middle)
        for k in sizes:
            if rank >= k:
                high[k] = middle
            else:
                low[k] = middle + 1

answers = ["-1" if low[k] == len(thresholds)
           else str(thresholds[low[k]]) for k in queries]
print("\n".join(answers))

if __name__ == "__main__":
    main()

```

16. All-Pairs Cut Statistics

Compress all pair cuts into one tree

Running a max flow for every unordered pair would require quadratically many flows. An undirected capacitated graph has a much stronger representation: a **Gomory–Hu tree** on the same vertices. Its defining property is

```
minimum_cut(u,v) = minimum edge weight on the tree path u...v.
```

Thus $n-1$ carefully selected max-flow computations represent all $n(n-1)/2$ pair values.

Constructing the Gomory–Hu tree

Maintain `parent[v]`, initially zero for every $v > 0$, and a cut value for the future tree edge from v to its parent. Process $s=1, 2, \dots, n-1$:

1. Let $t = \text{parent}[s]$ and compute a minimum s – t cut.
2. In the final residual graph, let S be the vertices reachable from s .
3. Every later vertex v with $\text{parent}[v] = t$ and v in S is reparented to s .
4. Usually store the flow value on s – $\text{parent}[s]$.

5. If $\text{parent}[t]$ also lies in S , rotate the two parent links: s takes t 's former parent, t becomes a child of s , s inherits t 's old cut value, and the new flow value is stored on $t-s$.

The reachable residual side is a minimum-cut shore. The reparenting preserves which already represented cuts separate each group. The rotation handles the case where the new shore crosses upward through t 's existing tree edge; omitting it is a common bug that may survive tests without nested cut shores.

The Gomory–Hu invariant says that after the last iteration, the minimum edge on every tree path equals the corresponding graph minimum cut. Disconnected graphs need no special case: their separating cuts have value zero, producing zero-weight tree edges.

From path minima to pair counts

For a query threshold X , a pair qualifies exactly when every edge on its Gomory–Hu path has weight at least X . Equivalently, the endpoints are in the same component after retaining only tree edges of weight at least X .

Sort tree edges and queries in descending order. A DSU initially has n singleton components. Before answering X , insert every tree edge whose weight is at least X . When components of sizes a and b merge, exactly $a*b$ formerly separated unordered pairs become connected. Maintain their running total and assign it to the query.

Sample trace

The sample graph is already a tree with edge capacities five, two, and four, so it is also a valid cut-equivalent tree. At threshold three, insert weights five and four. The first union creates one connected pair; the second joins a singleton to that size-two component and creates two more, for three qualifying pairs total. At threshold zero, the weight-two edge is also present and all six unordered pairs qualify.

Correctness proof

Lemma 1. The construction produces a tree in which the minimum edge on the path between any u, v equals their minimum cut in the original graph.

Proof. This is the Gomory–Hu invariant. At iteration s , the residual shore is a minimum cut separating s from its current parent. Reparenting vertices on the same shore makes the new tree edge represent that cut without changing separations represented by earlier edges. If the shore contains the parent's parent, the rotation preserves the older cut on the upper edge and places the new cut on the crossed lower edge. The uncrossing property of undirected minimum cuts guarantees these shores can be represented laminarily by those updates. Induction over processed vertices gives the path-minimum property for all pairs after $n-1$ iterations.

Lemma 2. For a fixed threshold X , two vertices qualify exactly when DSU connects them after all tree edges of weight at least X are inserted.

Proof. By Lemma 1, their graph minimum cut is at least X exactly when the minimum edge on their tree path is at least X . This holds exactly when every path edge has been inserted, which is equivalent to connectivity in the retained tree forest.

Lemma 3. Merging DSU components of sizes a and b increases the number of connected unordered pairs by exactly $a*b$.

Proof. All pairs internal to either component were already connected. The new edge connects precisely the pairs with one endpoint in each component, of which there are $a*b$.

Theorem. The algorithm outputs the requested count for every threshold.

Proof. Descending order ensures that immediately before a query X , DSU contains exactly the tree edges of weight at least X . Lemma 2 identifies its connected pairs with the qualifying graph pairs, while Lemma 3 proves that the maintained total counts those pairs exactly.

Complexity and traps

The construction performs $n-1$ max-flow computations. With general-capacity Dinic and its conservative $O(V^2 E)$ bound per flow, this is $O(n^3 m)$ time; the DSU phase costs $O((n+q) \log(n+q))$ for sorting and nearly linear time afterward. Memory is $O(n+m+q)$ beyond each residual network.

An undirected capacity is not a one-way arc: the residual representation must permit flow in both directions. Rebuild capacities for every Gomory–Hu flow. Take the source shore after max flow, not the BFS levels from an earlier phase. Preserve the parent-rotation case and use 64-bit values for cut sums and pair counts. For $X=0$, even vertices from different original connected components qualify because their minimum cut is zero. Test disconnected graphs, parallel edges, equal tree weights, nested cut shores, $n=1$, and thresholds above every cut.

C++

```
#include <bits/stdc++.h>
using namespace std;

struct InputEdge {
    int u, v;
    long long capacity;
};

class Dinic {
    struct Edge {
        int to, reverse;
        long long capacity;
    };
    int n;
    vector<vector<Edge>> graph;
    vector<int> level, next_edge;

    long long send(int u, int sink, long long pushed) {
        if (u == sink) return pushed;
        for (int &i = next_edge[u]; i < (int)graph[u].size(); ++i) {
            Edge &edge = graph[u][i];
            if (edge.capacity == 0 || level[edge.to] != level[u] + 1) continue;
            long long flow = send(edge.to, sink, min(pushed, edge.capacity));
            if (flow == 0) continue;
            edge.capacity -= flow;
            graph[edge.to][edge.reverse].capacity += flow;
            return flow;
        }
        return 0;
    }

public:
```

```

explicit Dinic(int vertices) : n(vertices), graph(n), level(n), next_edge(n) {}

void add_undirected(int u, int v, long long capacity) {
    int ru = graph[v].size(), rv = graph[u].size();
    graph[u].push_back({v, ru, capacity});
    graph[v].push_back({u, rv, capacity});
}

long long max_flow(int source, int sink) {
    long long answer = 0;
    while (true) {
        fill(level.begin(), level.end(), -1);
        queue<int> queue_nodes;
        level[source] = 0;
        queue_nodes.push(source);
        while (!queue_nodes.empty()) {
            int u = queue_nodes.front();
            queue_nodes.pop();
            for (const Edge &edge : graph[u]) {
                if (edge.capacity > 0 && level[edge.to] == -1) {
                    level[edge.to] = level[u] + 1;
                    queue_nodes.push(edge.to);
                }
            }
        }
        if (level[sink] == -1) break;
        fill(next_edge.begin(), next_edge.end(), 0);
        while (long long flow = send(source, sink, (1LL << 62))) {
            answer += flow;
        }
    }
    return answer;
}

vector<char> source_side(int source) const {
    vector<char> seen(n, false);
    queue<int> queue_nodes;
    seen[source] = true;
    queue_nodes.push(source);
    while (!queue_nodes.empty()) {
        int u = queue_nodes.front();
        queue_nodes.pop();
        for (const Edge &edge : graph[u]) {
            if (edge.capacity > 0 && !seen[edge.to]) {
                seen[edge.to] = true;
                queue_nodes.push(edge.to);
            }
        }
    }
    return seen;
}

```

```

};

class DSU {
    vector<int> parent, size;
public:
    explicit DSU(int n) : parent(n), size(n, 1) { iota(parent.begin(), parent.end(), 0); }
    int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }
    long long unite_pairs(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return 0;
        if (size[a] < size[b]) swap(a, b);
        long long added = 1LL * size[a] * size[b];
        parent[b] = a;
        size[a] += size[b];
        return added;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    cin >> n >> m >> q;
    vector<InputEdge> input_edges(m);
    for (auto &[u, v, capacity] : input_edges) {
        cin >> u >> v >> capacity;
        --u; --v;
    }

    vector<int> parent(n, 0);
    vector<long long> cut_value(n, 0);
    for (int s = 1; s < n; ++s) {
        int t = parent[s];
        Dinic flow(n);
        for (const auto &edge : input_edges) {
            flow.add_undirected(edge.u, edge.v, edge.capacity);
        }
        long long value = flow.max_flow(s, t);
        vector<char> side = flow.source_side(s);
        for (int v = s + 1; v < n; ++v) {
            if (parent[v] == t && side[v]) parent[v] = s;
        }
        if (side[parent[t]]) {
            parent[s] = parent[t];
            parent[t] = s;
            cut_value[s] = cut_value[t];
            cut_value[t] = value;
        } else {
            cut_value[s] = value;
        }
    }
}

```

```

}

vector<tuple<long long, int, int>> tree_edges;
for (int v = 1; v < n; ++v) {
    tree_edges.push_back({cut_value[v], v, parent[v]});
}
sort(tree_edges.begin(), tree_edges.end(), greater<>());
vector<pair<long long, int>> queries(q);
for (int i = 0; i < q; ++i) {
    cin >> queries[i].first;
    queries[i].second = i;
}
sort(queries.begin(), queries.end(), greater<>());
vector<long long> answers(q);
DSU dsu(n);
long long pairs = 0;
int edge_index = 0;
for (auto [threshold, query_index] : queries) {
    while (edge_index < (int)tree_edges.size()
           && get<0>(tree_edges[edge_index]) >= threshold) {
        auto [weight, u, v] = tree_edges[edge_index++];
        pairs += dsu.unite_pairs(u, v);
    }
    answers[query_index] = pairs;
}
for (long long answer : answers) cout << answer << '\n';
}

```

Python

```

import sys
from collections import deque

class Dinic:
    def __init__(self, n: int) -> None:
        self.graph = [[] for _ in range(n)]

    def add_undirected(self, u: int, v: int, capacity: int) -> None:
        forward = [v, len(self.graph[v]), capacity]
        backward = [u, len(self.graph[u]), capacity]
        self.graph[u].append(forward)
        self.graph[v].append(backward)

    def max_flow(self, source: int, sink: int) -> int:
        answer = 0
        n = len(self.graph)
        while True:
            level = [-1] * n
            level[source] = 0

```

```

        queue = deque([source])
        while queue:
            u = queue.popleft()
            for v, _, capacity in self.graph[u]:
                if capacity and level[v] == -1:
                    level[v] = level[u] + 1
                    queue.append(v)
        if level[sink] == -1:
            break
        next_edge = [0] * n

    def send(u: int, pushed: int) -> int:
        if u == sink:
            return pushed
        while next_edge[u] < len(self.graph[u]):
            edge = self.graph[u][next_edge[u]]
            v, reverse, capacity = edge
            if capacity and level[v] == level[u] + 1:
                flow = send(v, min(pushed, capacity))
                if flow:
                    edge[2] -= flow
                    self.graph[v][reverse][2] += flow
                    return flow
            next_edge[u] += 1
        return 0

    while True:
        pushed = send(source, 10**30)
        if not pushed:
            break
        answer += pushed
    return answer

def source_side(self, source: int) -> list[bool]:
    seen = [False] * len(self.graph)
    seen[source] = True
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v, _, capacity in self.graph[u]:
            if capacity and not seen[v]:
                seen[v] = True
                queue.append(v)
    return seen

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, q = data[:3]

```

```

index = 3
edges = []
for _ in range(m):
    u, v, capacity = data[index:index + 3]
    index += 3
    edges.append((u - 1, v - 1, capacity))
queries = data[index:index + q]

parent = [0] * n
cut_value = [0] * n
for s in range(1, n):
    t = parent[s]
    flow = Dinic(n)
    for u, v, capacity in edges:
        flow.add_undirected(u, v, capacity)
    value = flow.max_flow(s, t)
    side = flow.source_side(s)
    for v in range(s + 1, n):
        if parent[v] == t and side[v]:
            parent[v] = s
    if side[parent[t]]:
        parent[s] = parent[t]
        parent[t] = s
        cut_value[s] = cut_value[t]
        cut_value[t] = value
    else:
        cut_value[s] = value

tree_edges = sorted(((cut_value[v], v, parent[v]) for v in range(1, n)),
                    reverse=True)
ordered_queries = sorted(((threshold, i) for i, threshold in enumerate(queries)),
                        reverse=True)
dsu_parent = list(range(n))
dsu_size = [1] * n

def find(x: int) -> int:
    while dsu_parent[x] != x:
        dsu_parent[x] = dsu_parent[dsu_parent[x]]
        x = dsu_parent[x]
    return x

answers = [0] * q
pairs = 0
edge_index = 0
for threshold, query_index in ordered_queries:
    while edge_index < len(tree_edges) and tree_edges[edge_index][0] >= threshold:
        _, u, v = tree_edges[edge_index]
        edge_index += 1
        u, v = find(u), find(v)
        if u != v:
            if dsu_size[u] < dsu_size[v]:

```

```

        u, v = v, u
        pairs += dsu_size[u] * dsu_size[v]
        dsu_parent[v] = u
        dsu_size[u] += dsu_size[v]
        answers[query_index] = pairs
    print("\n".join(map(str, answers)))

if __name__ == "__main__":
    main()

```

17. Isotonic Tree Labels

Choose the right boundary function

For a vertex v , define $F_v(p)$ as the minimum cost in v 's subtree when the label of v must be at least p . This is exactly the information a parent passes across the subtree boundary.

The useful slope-trick invariant is

$$F_v(p) = C_v + \text{sum over } b \text{ in } H_v \text{ of } \max(0, p-b),$$

where H_v is a min-heap of integer breakpoints. The constant C_v is the unconstrained minimum subtree cost. Every hinge is flat before its breakpoint and has slope one afterward, so the heap describes the whole convex, nondecreasing function without storing values for every possible label.

Derive the heap transition

Suppose the label chosen at v is exactly y . Before applying its lower bound p , the cost is

$$G_v(y) = |y - a[v]| + \text{sum } F_{\text{child}}(y).$$

Merge all child breakpoint heaps. The absolute-value term has slope -1 before $a[v]$ and $+1$ after it, which is a slope jump of two. Therefore insert two copies of $a[v]$ into the merged heap.

Far to the left, G_v has slope -1 ; each sorted breakpoint raises the slope by one. Let b be the smallest breakpoint. Removing this first slope jump clips the negative part when we form

$$F_v(p) = \min \text{ over } y \geq p \text{ of } G_v(y).$$

Thus the complete structural transition is:

```

merge every child heap
push a[v] twice
b = pop minimum

```

What happens to the constant? Because the heap contains two copies of $a[v]$, we have $b \leq a[v]$. Because b is the smallest merged breakpoint, every child hinge is still zero at b . Hence

$$\begin{aligned}
C_v &= \sum C_{\text{child}} + |b - a[v]| \\
&= \sum C_{\text{child}} + a[v] - b.
\end{aligned}$$

We may accumulate $a[v] - b$ globally while processing vertices bottom-up. At the root there is no parent lower bound, so C_{root} itself is the answer.

Merge heaps efficiently

Each subtree heap contains exactly one breakpoint per subtree vertex: child sizes add, then pushing twice and popping once increases the total by one. Always keep the largest child heap and insert every element of smaller child heaps into it. Whenever a breakpoint moves, the size of its containing heap at least doubles. It therefore moves only $O(\log n)$ times.

Root the tree iteratively and process the reversed traversal order to avoid recursion-depth problems on a chain.

Sample trace

Each leaf with observation zero inserts 0,0, removes one zero, and leaves heap [0] with cost zero. At the root, merge the two leaf heaps, insert 10,10, and remove the minimum zero. The root contributes $10 - 0 = 10$, which is the optimum: label all three vertices zero.

Correctness proof

Lemma 1. If every child function has the hinge representation, the stated merge, double insertion, and minimum removal produce the hinge representation of F_v .

Proof. Summing child functions merges their constants and breakpoint multisets. Adding $|y - a[v]|$ contributes initial slope -1 and a jump of two at $a[v]$, represented by two copies. Taking the suffix minimum $\min_{\{y \geq p\}} G_v(y)$ replaces every negative slope by zero. Since the initial slope is exactly -1 , this removes exactly the first unit breakpoint, the minimum heap element. All remaining slope jumps are unchanged.

Lemma 2. The constant added at vertex v is $a[v] - b$, where b is the removed minimum.

Proof. The suffix-minimum function is flat at the unconstrained minimum of G_v , first reached at b . No child hinge is positive there because b is the smallest merged breakpoint. Also $b \leq a[v]$ because two copies of $a[v]$ were inserted. Therefore $G_v(b)$ exceeds the sum of child constants only by $|b - a[v]| = a[v] - b$.

Lemma 3. After processing v , its heap and accumulated subtree constant represent the exact optimum for every parent lower bound p .

Proof. Induct on subtree size. Leaves satisfy Lemmas 1 and 2 with no child heaps. For an internal vertex, the induction hypothesis makes every child representation exact; Lemmas 1 and 2 then derive the exact representation at v .

Theorem. The algorithm prints the minimum total deviation over all isotonic integer labelings.

Proof. Lemma 3 applies to the root. With no parent, its label has no lower bound, so the required value is the unconstrained constant C_{root} , exactly the accumulated answer. All breakpoints are observed integers, so an integer minimizer exists.

Complexity and traps

Small-to-large movement makes each of the n breakpoints move $O(\log n)$ times, and each heap insertion costs $O(\log n)$. The bound is $O(n \log^2 n)$ time and $O(n)$ memory.

The orientation matters: for $\text{parent} \leq \text{child}$, use a **min-heap** and remove the minimum. Reversing the inequality reverses the slope picture. Insert the observation twice, not once. Add the removed-breakpoint difference before discarding it. Do not attempt to store one DP value per possible label; values range over two billion integers. Test a decreasing chain, an increasing chain, a star whose children pull the root downward, equal observations, negative values, and $n=1$.

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> value(n);
    for (long long &x : value) cin >> x;
    vector<vector<int>> graph(n);
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<int> parent(n, -1), order{0};
    for (int index = 0; index < n; ++index) {
        int u = order[index];
        for (int v : graph[u]) if (v != parent[u]) {
            parent[v] = u;
            order.push_back(v);
        }
    }

    using MinHeap = priority_queue<long long, vector<long long>, greater<long long>>;
    vector<unique_ptr<MinHeap>> heap(n);
    long long answer = 0;
    for (int index = n - 1; index >= 0; --index) {
        int u = order[index];
        heap[u] = make_unique<MinHeap>();
        for (int v : graph[u]) if (parent[v] == u) {
            if (heap[u]->size() < heap[v]->size()) heap[u].swap(heap[v]);
            while (!heap[v]->empty()) {
```

```

        heap[u]->push(heap[v]->top());
        heap[v]->pop();
    }
    heap[v].reset();
}
heap[u]->push(value[u]);
heap[u]->push(value[u]);
long long first_breakpoint = heap[u]->top();
heap[u]->pop();
answer += value[u] - first_breakpoint;
}
cout << answer << '\n';
}

```

Python

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    values = data[1:1 + n]
    graph = [[] for _ in range(n)]
    index = 1 + n
    for _ in range(n - 1):
        u, v = data[index] - 1, data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    parent = [-1] * n
    order = [0]
    for u in order:
        for v in graph[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)

    heaps = [None] * n
    answer = 0
    for u in reversed(order):
        current = []
        for v in graph[u]:
            if parent[v] != u:
                continue
            child = heaps[v]

```

```

        if len(current) < len(child):
            current, child = child, current
        for breakpoint in child:
            heapq.heappush(current, breakpoint)
        heaps[v] = None
        heapq.heappush(current, values[u])
        heapq.heappush(current, values[u])
        first_breakpoint = heapq.heappop(current)
        answer += values[u] - first_breakpoint
        heaps[u] = current
    print(answer)

if __name__ == "__main__":
    main()

```

18. Circulation Repair Queries

Turn changes into a residual path

The supplied flow already obeys every old bound and conserves flow. Build its residual graph. An old edge $x \rightarrow y$ contributes $x \rightarrow y$ when its flow can be increased, and $y \rightarrow x$ when its flow can be decreased.

A proposed mandatory unit on $u \rightarrow v$ removes one unit of balance from u and adds one at v . The old edges repair this exactly when one residual unit can travel from v back to u . Capacities are integral, so existence of a path is both necessary and sufficient; the actual path is also a certificate.

All queries use the same original circulation. Compute the transitive closure of the residual graph once, using bitsets in Python or a Boolean Floyd–Warshall loop in C++. Then query (u, v) is YES precisely when v reaches u .

Correctness proof

Lemma 1. Every legal change to the old flow is an integral flow in the residual graph, and conversely every integral residual flow produces legal old edge values.

Proof. A forward residual unit increases an edge below its upper bound; a reverse unit decreases an edge above its lower bound. Cancelling opposite directions gives the converse mapping without crossing either bound.

Lemma 2. The enlarged instance is feasible iff the residual graph contains a path from v to u .

Proof. Sending one unit on the new $u \rightarrow v$ edge creates surplus at v and deficit at u . A residual $v \rightarrow u$ path repairs both and conserves every internal vertex. Conversely, any feasible repair decomposes into cycles and a unit $v \rightarrow u$ residual flow; its positive support contains such a path.

Theorem. The algorithm answers every query correctly.

Proof. Transitive closure records exactly the paths characterized by Lemma 2.

Complexity and traps

Closure costs $O(n^3)$ time and $O(n^2)$ bits/Booleans; each query is $O(1)$. Do not rebuild the network per query. A pinned edge contributes no residual arc, and the direction tested is $v \rightarrow u$, opposite the proposed edge.

C++

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);
    int n,m,q;if(!(cin>>n>>m>>q))return 0; vector<bitset<700>> r(n);
    for(int i=0,u,v;i<m;i++){long long l,h,f;cin>>u>>v>>l>>h>>f;--u;--v;if(f<h)r[u].set(v);if(f>l)r[v].set(u);
    for(int i=0;i<n;i++)r[i].set(i);
    for(int k=0;k<n;k++)for(int i=0;i<n;i++)if(r[i].test(k))r[i]|=r[k];
    while(q--){int u,v;cin>>u>>v;--u;--v;cout<<(r[v].test(u)?"YES\n":"NO\n");}
}
```

Python

```
import sys
def main():
    it=iter(map(int,sys.stdin.buffer.read().split())); n=next(it);m=next(it);q=next(it); reach=[1<<u for u in range(n)]
    for _ in range(m):
        u=next(it)-1;v=next(it)-1;l=next(it);h=next(it);f=next(it)
        if f<h: reach[u]|=1<<v
        if f>l: reach[v]|=1<<u
    for k in range(n):
        bit=1<<k; add=reach[k]
        for i in range(n):
            if reach[i]&bit: reach[i]|=add
    out=[]
    for _ in range(q):
        u=next(it)-1;v=next(it)-1;out.append("YES" if reach[v]>>u&1 else "NO")
    print("\n".join(out))
if __name__=="__main__":main()
```

19. Convex Resource Schedule

Penalize the number of batches

Let $S[i]$ be the prefix sum. With a penalty λ per batch, the DP is

```
dp[i] = min over j<i of dp[j] + (S[i]-S[j])^2 + lambda.
```

Expand the square:

```
dp[i] = S[i]^2 + lambda
      + min_j ((-2*S[j])*S[i] + dp[j] + S[j]^2).
```

Thus every completed prefix inserts one line, and every next prefix makes one minimum query. A discrete Li Chao tree over the nondecreasing prefix sums handles equal sums without division or floating point.

Store (penalized_cost, batch_count). Run once preferring the fewest batches on cost ties and once preferring the most. As `lambda` increases, optimal batch counts never increase. Binary-search the largest integer penalty whose maximum-tie optimum still uses at least `K` batches. Convexity of squared batch sums (the Monge quadrangle inequality) guarantees that at this breakpoint the tie interval contains `K`. If its penalized value is `V`, the answer is `V-lambda*K`.

Correctness proof

Lemma 1. A Li Chao query at `S[i]` returns the minimum penalized transition into prefix `i`, with the requested count tie rule.

Proof. Expansion maps every predecessor to exactly one affine function. Li Chao insertion preserves the best line at every stored coordinate; comparing the count after equal values extends the same total order to ties.

Lemma 2. Optimal batch count is nonincreasing in `lambda`.

Proof. For penalties `a<b`, compare optima with counts `x,y`. Adding their two optimality inequalities cancels original costs and gives $(b-a)(y-x) \leq 0$, hence $y \leq x$.

Lemma 3. At the located breakpoint, `K` lies between the minimum and maximum optimal counts, and `V-lambda*K` is the exact `K`-batch optimum.

Proof. The interval-cost matrix is Monge because prefix sums are ordered and the square is convex. Therefore optimum costs as a function of batch count are discrete convex: adjacent differences are monotone integers. Its supporting line of slope `-lambda` touches a consecutive interval of counts. The binary search selects the support where that interval crosses `K`. Every touched count has original value `V-lambda*count`, giving the formula.

Theorem. The algorithm prints the minimum cost among exactly `K` batches.

Proof. Lemma 1 solves each penalized DP; Lemmas 2 and 3 locate and recover the required cardinality.

Complexity and traps

One run takes $O(n \log n)$ time and $O(n)$ memory. There are $O(\log(S[n]^2+1))$ runs. Treat `K=1` and `K=n` directly. Li Chao children may be absent during a query; stop at the best line already seen. Counts are part of comparisons only after costs tie.

C++

```
#include <bits/stdc++.h>
using namespace std; using i128=__int128_t; const i128 INF=(i128(1)<<120);
struct L{long long m; i128 b; int c;};
pair<i128,int> run(const vector<long long>&x,long long lam,bool mx){int n=x.size()-1;vector<optional<L>>
  auto better=[&](const L&a,const L&b,long long z){i128 u=i128(a.m)*z+a.b,v=i128(b.m)*z+b.b;return u<v?u:v;};
  function<void(L,int,int,int)>add=[&](L z,int p,int l,int r){if(!t[p]){t[p]=z;return;}int md=(l+r)/2;
  function<L(int,int,int,int)>qry=[&](int at,int p,int l,int r){L z=*t[p];if(l==r)return z;int md=(l+r)/2;
    if(qry(at,md,l,md).m<z.m)z=qry(at,md,l,md);if(qry(at,md+1,r,md+1).m<z.m)z=qry(at,md+1,r,md+1);return z;};
  if(mx)for(int i=0;i<=n;i++)add(L{x[i],0,i,0});return {t[n].m,t[n].c};}
```

```

add({0,0,0},1,0,n);i128 val=0;int cnt=0;for(int i=1;i<=n;i++){L z=qry(i,1,0,n);val=i128(z.m)*x[i]+z.m;
void print128(i128 x){if(x==0){cout<<0;return;}string s;while(x){s+=char('0'+x%10);x/=10;}reverse(s.begin(),s.end());
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,k;if(!(cin>>n>>k))return 0;vector<long long> x(n);

```

Python

```

import sys
INF=10**80
def run(xs,lam,prefer_max):
    n=len(xs)-1; tree=[None]*(4*(n+1))
    def better(a,b,x):
        if b is None:return True
        va=a[0]*x+a[1];vb=b[0]*x+b[1]
        return va<vb or (va==vb and ((a[2]>b[2]) if prefer_max else (a[2]<b[2])))
    def add(line,p=1,l=0,r=n):
        mid=(l+r)//2
        if tree[p] is None:tree[p]=line;return
        left=better(line,tree[p],xs[l]);middle=better(line,tree[p],xs[mid])
        if middle:tree[p],line=line,tree[p]
        if l==r:return
        if left!=middle:add(line,p*2,l,mid)
        elif better(line,tree[p],xs[r])!=better(line,tree[p],xs[mid]):add(line,p*2+1,mid+1,r)
    def query(idx,p=1,l=0,r=n):
        line=tree[p];best=line
        if l==r:return best
        mid=(l+r)//2;child=p*2 if idx<=mid else p*2+1
        if tree[child] is None:return best
        other=query(idx,child,l,mid) if idx<=mid else query(idx,child,mid+1,r)
        return other if better(other,best,xs[idx]) else best
    add((0,0,0)); val=cnt=0
    for i in range(1,n+1):
        z=query(i);val=z[0]*xs[i]+z[1]+xs[i]*xs[i]+lam;cnt=z[2]+1;add((-2*xs[i],val+xs[i]*xs[i],cnt))
    return val,cnt
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n,k=d[:2];xs=[0]
    for a in d[2:]:xs.append(xs[-1]+a)
    if k==1:print(xs[-1]*xs[-1]);return
    if k==n:print(sum(a*a for a in d[2:]));return
    B=xs[-1]*xs[-1]+1;lo=-B;hi=B
    while lo<hi:
        mid=(lo+hi+1)//2
        if run(xs,mid,True)[1]>=k:lo=mid
        else:hi=mid-1
    value,maxc=run(xs,lo,True);_,minc=run(xs,lo,False)
    if not minc<=k<=maxc:raise RuntimeError("convexity invariant failed")
    print(value-lo*k)
if __name__=="__main__":main()

```

20. Laminar Assignment

Share capacity through the laminar tree

Expanding an offer to all descendant leaves can create quadratic many edges. Keep the tree itself in the flow network:

```
source -> worker          capacity 1, cost 0
worker -> offered tree node capacity 1, offer cost
parent -> child           capacity w, cost 0
leaf -> sink              capacity 1, cost 0
```

A unit entering a tree node can descend to exactly a leaf in its subtree. The unit capacity into the sink makes leaves distinct. Send w units with successive shortest augmenting paths and reduced-cost potentials.

Correctness proof

Lemma 1. Every integral source–sink path represents one legal worker assignment with the path’s offer cost.

Proof. The path chooses one worker, one of that worker’s offers, then follows only parent-to-child arcs to a descendant leaf. All later arcs cost zero.

Lemma 2. Integral flows of value w correspond cost-for-cost to complete assignments to distinct leaves.

Proof. Unit source edges make each worker occur once. Unit leaf-to-sink edges prevent repeated leaves. Flow decomposition and Lemma 1 give an assignment. Conversely, route every chosen worker through its offer and the unique tree path to its leaf; shared tree arcs have capacity w .

Lemma 3. Successive shortest augmenting paths produce a minimum-cost flow of each attained value.

Proof. Potentials preserve path costs while making every residual reduced cost nonnegative. Augmenting on a shortest residual path maintains absence of a negative-cost residual cycle, the optimality certificate for min-cost flow.

Theorem. The algorithm prints the minimum assignment cost, or IMPOSSIBLE exactly when none exists.

Proof. If value w is reached, Lemmas 2 and 3 give the optimum assignment. If the sink is unreachable earlier, no augmenting path exists, so no flow—and by Lemma 2 no complete assignment—can have larger value.

Complexity and traps

With $V=n+w+2$ and $E=O(n+w+\text{offers})$, the bound is $O(w \cdot E \log V)$ time and $O(V+E)$ memory. Offers enter a tree node, so tree arcs must point from parent to child. Duplicate offers are harmless. Count leaves from the given rooted tree, including the root when $n=1$.

C++

```
#include <bits/stdc++.h>
using namespace std; using ll=long long; struct E{int v, cap, rev; ll c;}; int main(){ios::sync_with_stdio
```


Python

```
import sys,heapq
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));it=iter(d);n=next(it);w=next(it);par=[-1]+[next(it)
    for _ in range(w):offers.append([(next(it)-1,next(it)) for _ in range(next(it))])
    N=1+w+n+1;S=0;T=N-1;g=[[] for _ in range(N)]
    def add(u,v,cap,c):g[u].append([v,cap,c,len(g[v])]);g[v].append([u,0,-c,len(g[u])-1])
    for i,o in enumerate(offers):
        add(S,1+i,1,0)
        for v,c in o:add(1+i,1+w+v,1,c)
    child=[0]*n
    for v in range(1,n):add(1+w+par[v],1+w+v,w,0);child[par[v]]+=1
    for v in range(n):
        if not child[v]:add(1+w+v,T,1,0)
    pot=[0]*N;cost=flow=0;INF=10**30
    while flow<w:
        dist=[INF]*N;dist[S]=0;prev=[None]*N;pq=[(0,S)]
        while pq:
            du,u=heapq.heappop(pq)
            if du!=dist[u]:continue
            for ei,e in enumerate(g[u]):
                if e[1] and du+e[2]+pot[u]-pot[e[0]]<dist[e[0]]:dist[e[0]]=du+e[2]+pot[u]-pot[e[0]];prev[e[0]]=u
            if dist[T]==INF:print('IMPOSSIBLE');return
            for v in range(N):
                if dist[v]<INF:pot[v]+=dist[v]
            v=T
            while v!=S:u,ei=prev[v];e=g[u][ei];e[1]-=1;g[v][e[3]][1]+=1;cost+=e[2];v=u
            flow+=1
        print(cost)
if __name__=='__main__':main()
```

21. Persistent Text Occurrences

Two independent orders

All suffixes beginning with pattern p form one contiguous suffix-array interval $[L,R)$, found by two binary searches. Inside that interval, suffixes are lexicographically ordered, but the query wants starting-position order.

Build persistent segment-tree root $root[i]$ from the first i suffix-array entries, inserting each suffix's starting position. Subtracting $root[L]$ from $root[R]$ leaves exactly the occurrence positions of p . Counts guide a standard k -th descent; insufficient root count means -1 .

Correctness. Suffix-array contiguity puts exactly the matching suffixes in $[L,R)$. Persistence inserts each such suffix once, so root subtraction gives their position multiset exactly. At every segment-tree level the left count is the number of remaining positions in that half; rank subtraction therefore ends at the k -th smallest position.

Building the suffix array and roots takes $O(n \log n)$ time and memory. A query takes $O(|p| \log n + \log n)$ with direct comparisons. Watch the upper-bound comparison: suffix prefixes equal to p belong inside the interval.

C++

```
#include <bits/stdc++.h>
using namespace std; struct N{int l=0,r=0,z=0;};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(cin>>s))return 0;int n=s.size();
vector<N>tr(1);function<int(int,int,int,int)>add=[&](int old,int l,int r,int x){int u=tr.size();tr.push_back(N{0,0,0});
function<int(int,int,int,int,int)>kth=[&](int a,int b,int l,int r,int k){if(l==r)return l;int z=tr[u-1].z;
auto cmp=[&](int pos,const string&p){int z=s.compare(pos,p.size(),p);return z<0?-1:z>0?1:0;};int q
```

Python

```
import sys
def main():
    d=sys.stdin.buffer.read().split();s=d[0];n=len(s);q=int(d[1]);sa=list(range(n));r=list(s);k=1
    while True:
        sa.sort(key=lambda i:(r[i],r[i+k] if i+k<n else -1));nr=[0]*n
        for j in range(1,n):nr[sa[j]]=nr[sa[j-1]]+((r[sa[j-1]],r[sa[j-1]+k] if sa[j-1]+k<n else -1)<(r[sa[j]],r[sa[j]+k] if sa[j]+k<n else -1))
        r=nr
        if r[sa[-1]]==n-1:break
        k*=2
    left=[0];right=[0];cnt=[0];roots=[0]
    def add(old,l,h,x):
        u=len(cnt);left.append(left[old]);right.append(right[old]);cnt.append(cnt[old]+1)
        if l<h:
            m=(l+h)//2
            if x<=m:left[u]=add(left[old],l,m,x)
            else:right[u]=add(right[old],m+1,h,x)
        return u
    for x in sa:roots.append(add(roots[-1],0,n-1,x))
    def kth(a,b,l,h,z):
        while l<h:
            m=(l+h)//2;c=cnt[left[b]]-cnt[left[a]]
            if z<=c:a,b,h=left[a],left[b],m
            else:a,b,l,z=right[a],right[b],m+1,z-c
        return l
    out=[];at=2
    for _ in range(q):
        p=d[at];z=int(d[at+1]);at+=2;lo=0;hi=n
        while lo<hi:
            m=(lo+hi)//2
            if s[sa[m]:sa[m]+len(p)]<p:lo=m+1
            else:hi=m
        L=lo;lo=0;hi=n
        while lo<hi:
            m=(lo+hi)//2
            if s[sa[m]:sa[m]+len(p)]<=p:lo=m+1
            else:hi=m
        out.append(str(kth(roots[L],roots[lo],0,n-1,z)+1) if lo-L>=z else '-1')
```

```
print('\n'.join(out))
if __name__ == '__main__': main()
```

22. Multi-Archive Common Substrings

Cap every suffix-automaton state

Build a suffix automaton for the first archive. State v represents substring lengths $(\text{len}(\text{link}[v]), \text{len}[v])$. Scan another archive while maintaining the longest current automaton match. Record the best length ending in each visited state, then propagate states from longer to shorter:

```
best[link[v]] = max(best[link[v]], min(best[v], len(link[v])))
```

The cap is essential: the parent state represents no length above its own maximum. Maintain $\text{common}[v]$, the minimum $\text{best}[v]$ over all archives. Then state v contributes $\max(0, \text{common}[v] - \text{len}(\text{link}[v]))$ distinct common substrings, and the longest answer is $\max \text{common}[v]$.

Correctness. The online scan finds every match ending at each position. Suffix-link propagation transfers all suffixes of those matches and no length outside the parent interval. Thus $\text{best}[v]$ is the largest represented length present in one archive; taking minima makes it the largest present everywhere. Automaton state intervals partition the distinct substrings, proving both the sum and maximum formulas.

Time is linear in total archive length plus k times the automaton size; memory is linear in the first archive. Reset per-archive best , but never reset the accumulated minima.

C++

```
#include <bits/stdc++.h>
using namespace std; struct S{int link=-1, len=0; array<int, 26> to{}; S(){to.fill(-1);}}; int main(){ios::sync_with_stdio(false);
```

Python

```
import sys
def main():
    d=sys.stdin.buffer.read().split(); k=int(d[0]); text=d[1]; link=[-1]; ln=[0]; to=[{}]; last=0
    for c in text:
        cur=len(link); link.append(0); ln.append(ln[last]+1); to.append({}); p=last
        while p>=0 and c not in to[p]: to[p][c]=cur; p=link[p]
        if p<0: link[cur]=0
        else:
            q=to[p][c]
            if ln[p]+1==ln[q]: link[cur]=q
            else:
                z=len(link); link.append(link[q]); ln.append(ln[p]+1); to.append(to[q].copy())
                while p>=0 and to[p].get(c)==q: to[p][c]=z; p=link[p]
            link[q]=link[cur]=z
    last=cur
```

```

order=sorted(range(len(link)),key=ln.__getitem__);common=ln.copy()
for s in d[2:]:
    best=[0]*len(link);v=l=0
    for c in s:
        while v and c not in to[v]:v=link[v];l=min(l,ln[v])
        if c in to[v]:v=to[v][c];l+=1
        else:v=l=0
    best[v]=max(best[v],l)
    for x in reversed(order[1:]):p=link[x];best[p]=max(best[p],min(best[x],ln[p]))
    for i in range(1,len(link)):common[i]=min(common[i],best[i])
vals=[max(0,common[i]-ln[link[i]]) for i in range(1,len(link))];print(max(common),sum(vals))
if __name__=='__main__':main()

```

23. Palindromic Range Census

Turn discoveries into points

An Eertree creates one node exactly when a distinct palindrome first appears. When a node of length d is created while reading position e , its discovery interval is the fixed point ($\text{start}=e-d+1, \text{end}=e$).

Group these points by end , and group queries by right endpoint. Sweep r from left to right. Insert every discovery ending at r into a Fenwick tree at its start. For query $[l, r]$, all inserted points already satisfy $\text{end} \leq r$; the answer is the number whose start is at least l :

```

inserted_count - fenwick_prefix(l-1).

```

Correctness. Eertree node uniqueness gives exactly one point per distinct palindrome, at its leftmost ending occurrence. At sweep time r , precisely the points satisfying the query's end inequality have been inserted. Fenwick suffix counting enforces the start inequality, hence counts exactly the wholly contained discovery intervals.

Eertree construction is $O(n)$ and the sweep is $O((n+q) \log n)$, using $O(n+q)$ memory. This problem counts first-occurrence certificates, not every palindromic occurrence and not distinct palindromes of the query substring.

C++

```

#include <bits/stdc++.h>
using namespace std;struct N{int len,link,end;array<int,26>to{}};N(int l=0):len(l),link(0),end(0){to

```

Python

```

import sys
def main():
    d=sys.stdin.buffer.read().split();s=d[0];n=len(s);ln=[-1,0];link=[0,0];to=[{},{}];last=1;events=[[]]
    for i,c in enumerate(s):
        p=last
        while i-1-ln[p]<0 or s[i-1-ln[p]]!=c:p=link[p]

```

```

if c not in to[p]:
    u=len(ln);ln.append(ln[p]+2);link.append(1);to.append({})
    if ln[u]>1:
        q=link[p]
        while i-1-ln[q]<0 or s[i-1-ln[q]]!=c:q=link[q]
        link[u]=to[q][c]
        to[p][c]=u;events[i+1].append(i+2-ln[u])
    last=to[p][c]
q=int(d[1]);queries=[[ ] for _ in range(n+1)];at=2
for z in range(q):l=int(d[at]);r=int(d[at+1]);at+=2;queries[r].append((l,z))
bit=[0]*(n+1);ans=[0]*q;total=0
for r in range(1,n+1):
    for x in events[r]:
        total+=1;y=x
        while y<=n:bit[y]+=1;y+=y&-y
    for l,z in queries[r]:
        y=l-1;v=0
        while y:v+=bit[y];y-=y&-y
        ans[z]=total-v
print('\n'.join(map(str,ans)))
if __name__=='__main__':main()

```

24. Lexicographic Substring Laboratory

Map every text position to its suffix-array rank. A wavelet-style range order structure over that rank array returns the k -th and $(k+1)$ -th ranks among positions $[l,r]$; the suffix array maps them back to positions. Their LCP is the minimum LCP-array entry strictly after the smaller rank through the larger rank. A Cartesian tree of the LCP array supports this RMQ as an LCA; the references use the equivalent static RMQ representation.

Suffix ranks totally order exactly the queried suffixes, so range selection returns the requested neighbors. The standard suffix-array theorem says the LCP of ranks $a < b$ is $\min(\text{LCP}[a+1..b])$, proving the second output. Construction is $O(n \log n)$ here; each selection and LCP query is $O(\log n)$. Do not compute LCP between adjacent global suffixes—the neighbors are adjacent only after the position restriction.

C++

```

#include <bits/stdc++.h>
using namespace std;struct N{int l=0,r=0,z=0;};int main(){ios::sync_with_stdio(false);cin.tie(nullptr);

```

Python

```

import sys
def main():
    d=sys.stdin.buffer.read().split();s=d[0];n=len(s);sa=list(range(n));rank=list(s);k=1
    while True:
        sa.sort(key=lambda i:(rank[i],rank[i+k] if i+k<n else -1));nr=[0]*n
        for j in range(1,n):nr[sa[j]]=nr[sa[j-1]]+((rank[sa[j-1]],rank[sa[j-1]+k] if sa[j-1]+k<n else -1)<

```

```

rank=nr
if rank[sa[-1]]==n-1:break
k*=2
h=[0]*n;k=0
for i in range(n):
    x=rank[i]
    if x:
        j=sa[x-1]
        while i+k<n and j+k<n and s[i+k]==s[j+k]:k+=1
        h[x]=k;k=max(0,k-1)
size=1
while size<n:size*=2
seg=[n*2]*(2*size);seg[size:size+n]=h
for i in range(size-1,0,-1):seg[i]=min(seg[i*2],seg[i*2+1])
def lcp(a,b):
    if a==b:return n-sa[a]
    if a>b:a,b=b,a
    a+=1;v=n*2;a+=size;b+=size+1
    while a<b:
        if a&1:v=min(v,seg[a]);a+=1
        if b&1:b-=1;v=min(v,seg[b])
    a//=2;b//=2
    return v
levels=max(1,(n-1).bit_length());prefs=[];zeros=[];cur=rank
for bit in range(levels-1,-1,-1):
    pref=[0];z=[];o=[]
    for x in cur:
        iszero=not (x>>bit&1);pref.append(pref[-1]+iszero);(z if iszero else o).append(x)
    prefs.append(pref);zeros.append(len(z));cur=z+o
def kth(l,r,k):
    value=0
    for lev,bit in enumerate(range(levels-1,-1,-1)):
        p=prefs[lev];zl=p[l];zr=p[r];cnt=zr-zl
        if k<=cnt:l=zl;r=zr
        else:k-=cnt;value|=1<<bit;l=zeros[lev]+l-zl;r=zeros[lev]+r-zr
    return value
q=int(d[1]);out=[];at=2
for _ in range(q):
    l=int(d[at])-1;r=int(d[at+1]);k=int(d[at+2]);at+=3;a=kth(l,r,k);v=0 if k==r-l else lcp(a,kth(l,r,l-k))
    print('\n'.join(out))
if __name__=='__main__':main()

```

25. Dynamic Pattern Ledger

Build Aho–Corasick and its failure-link tree. Pattern i matches while the automaton is in state v exactly when its terminal node is an ancestor of v in that tree. Euler-tour every failure subtree. Activating a terminal performs a range add on its subtree; a Fenwick difference array makes the active ancestor count at state v a point query. Scan each query text through the automaton and sum those point values.

The failure-chain property proves that the point value counts exactly the active patterns ending at the current

character. Summing positions counts all occurrences, including overlaps and duplicate indexed patterns. Total time is linear in input text plus $O(\log \text{ states})$ per activation and scanned character; memory is linear in automaton size. Range-add both endpoints, and treat equal patterns as separate activations of the same terminal.

C++

```
#include <bits/stdc++.h>
using namespace std;struct N{array<int,26>to{};int f=0;vector<int>ch;N(){to.fill(0);}};int main(){i
```

Python

```
import sys,collections
def main():
    d=sys.stdin.buffer.read().split();n=int(d[0]);q=int(d[1]);to=[{}];fail=[0];ends=[];at=2
    for s in d[at:at+n]:
        v=0
        for c in s:
            if c not in to[v]:to[v][c]=len(to);to.append({});fail.append(0)
            v=to[v][c]
        ends.append(v)
    at+=n;qq=collections.deque(to[0].values());order=[0]
    while qq:
        v=qq.popleft();order.append(v)
        for c,u in to[v].items():
            f=fail[v]
            while f and c not in to[f]:f=fail[f]
            fail[u]=to[f].get(c,0);qq.append(u)
    child=[[] for _ in to]
    for v in range(1,len(to)):child[fail[v]].append(v)
    tin=[0]*len(to);tout=[0]*len(to);tm=0;st=[(0,0)]
    while st:
        v,i=st[-1]
        if i==0:tm+=1;tin[v]=tm
        if i<len(child[v]):st[-1]=(v,i+1);st.append((child[v][i],0))
        else:tout[v]=tm;st.pop()
    bit=[0]*(len(to)+2)
    def add(x,z):
        while x<len(bit):bit[x]+=z;x+=x&-x
    def get(x):
        z=0
        while x:z+=bit[x];x-=x&-x
        return z
    out=[]
    for _ in range(q):
        op=d[at];x=d[at+1];at+=2
        if op==b'?:':
            v=0;ans=0
```

```

for c in x:
    while v and c not in to[v]:v=fail[v]
    v=to[v].get(c,0);ans+=get(tin[v])
    out.append(str(ans))
else:v=ends[int(x)-1];z=1 if op=='+' else -1;add(tin[v],z);add(tout[v]+1,-z)
print('\n'.join(out))
if __name__=='__main__':main()

```

26. Cyclic Match Convolution

Reverse a and concatenate $b+b$. For a concrete character c , convolution of their c -indicator arrays at coefficient $n-1+d$ counts equal concrete pairs under shift d . Sum this for a, b, c . One more convolution of “is concrete” indicators counts positions where both symbols are concrete. Their difference is exactly the mismatch count; wildcard positions enter neither mismatch term.

The convolution identity pairs reversed index $n-1-i$ with doubled-text index $i+d$, hence considers precisely $a[i]$ and $b[(i+d) \bmod n]$. Indicator products count each eligible position once, proving the subtraction formula. Four NTTs give $O(n \log n)$ time and $O(n)$ memory. Read coefficient $n-1+d$, duplicate b rather than a , and never count wildcard–wildcard as a concrete match.

C++

```

#include <bits/stdc++.h>
using namespace std;const int M=998244353,G=3;int pw(int a,int b){long long z=1;for(;b;b>>=1,a=(long

```

Python

```

import sys

MOD = 998244353
ROOT = 3

def ntt(values, inverse):
    n = len(values)
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            values[i], values[j] = values[j], values[i]
    length = 2
    while length <= n:
        step = pow(ROOT, (MOD - 1) // length, MOD)

```



```

    if inverse:
        step = pow(step, MOD - 2, MOD)
    half = length // 2
    for start in range(0, n, length):
        root = 1
        stop = start + half
        for left in range(start, stop):
            right = left + half
            u = values[left]
            v = values[right] * root % MOD
            values[left] = (u + v) % MOD
            values[right] = (u - v) % MOD
            root = root * step % MOD
        length <= 1
    if inverse:
        scale = pow(n, MOD - 2, MOD)
        for i in range(n):
            values[i] = values[i] * scale % MOD

def transformed_indicators(first, second, predicate, size):
    left = [int(predicate(c)) for c in reversed(first)]
    right = [int(predicate(c)) for c in second + second]
    left.extend([0] * (size - len(left)))
    right.extend([0] * (size - len(right)))
    ntt(left, False)
    ntt(right, False)
    return left, right

def main():
    data = sys.stdin.buffer.read().split()
    n, limit = map(int, data[:2])
    first, second = data[2], data[3]
    size = 1
    while size < 3 * n - 1:
        size <= 1

    left, right = transformed_indicators(
        first, second, lambda c: c != 63, size)
    mismatch = [left[i] * right[i] % MOD for i in range(size)]
    del left, right

    for character in (97, 98, 99):
        left, right = transformed_indicators(
            first, second, lambda c, wanted=character: c == wanted, size)
        for i in range(size):
            mismatch[i] = (mismatch[i] - left[i] * right[i]) % MOD
        del left, right

    ntt(mismatch, True)

```

```

    answer = [shift for shift in range(n)
               if mismatch[n - 1 + shift] <= limit]
    print(len(answer))
    print(*answer)

if __name__ == "__main__":
    main()

```

27. Distinct Substring Rank

Rank blocks in suffix-array order

Sort the suffixes. Suffix `sa[i]` shares its first `lcp[i]` letters with the previous suffix, so it contributes exactly the new distinct substrings of lengths `lcp[i]+1, ..., n-sa[i]`, and they come after every substring from earlier suffixes in lexicographic order. Prefix sums of these block sizes locate the `k`-th substring by binary search: its block `i` and its length `len = lcp[i] + (k - before)`.

Its occurrences are the suffixes `sa[i..hi]`, where `hi` is the last index whose `lcp` values after `i` stay $\geq \text{len}$. (The block of `i` starts the interval because `lcp[i] < len`.) Sparse tables on `lcp` and on `sa` give `hi` by binary search and the leftmost start as a range minimum.

Correctness. Distinct substrings are exactly the prefixes of suffixes, and each is attributed to the first suffix in sorted order that has it as a prefix. That suffix is `sa[i]` iff the length exceeds `lcp[i]`, so the blocks partition the substrings in lexicographic order. The occurrence interval follows from the fact that the LCP of two suffixes is the minimum `lcp` value between their positions.

Complexity and traps. The suffix array costs $O(n \log^2 n)$ with comparison doubling, and each query costs $O(\log n)$. Printing the substring itself would be up to $|s|$ characters per query, so the statement asks for **start length** of the leftmost occurrence instead. Ranks reach $n(n+1)/2 \sim 2 \cdot 10^{10}$, and `k` up to 10^{18} must print `-1`.

C++

```

#include <bits/stdc++.h>
using namespace std;

// In suffix-array order, suffix sa[i] contributes the distinct substrings of
// lengths lcp[i]+1 .. n-sa[i], already in lexicographic order. Prefix sums of
// these counts locate the k-th substring by binary search. Its occurrences
// are the suffixes sa[i..hi] where lcp stays >= its length; a sparse table on
// lcp finds hi and another on sa gives the leftmost start.

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    int q;
    cin >> s >> q;
    int n = s.size();

```

```

vector<int> sa(n), rk(n), tmp(n);
iota(sa.begin(), sa.end(), 0);
for (int i = 0; i < n; i++) rk[i] = s[i];
for (int k = 1;; k <= 1) {
    auto key = [&](int i) { return pair(rk[i], i + k < n ? rk[i + k] : -1); };
    sort(sa.begin(), sa.end(), [&](int a, int b) { return key(a) < key(b); });
    tmp[sa[0]] = 0;
    for (int i = 1; i < n; i++) tmp[sa[i]] = tmp[sa[i - 1]] + (key(sa[i - 1]) < key(sa[i]));
    rk = tmp;
    if (rk[sa[n - 1]] == n - 1) break;
}
vector<int> lcp(n, 0);
for (int i = 0, h = 0; i < n; i++) {
    if (rk[i] == 0) {
        h = 0;
        continue;
    }
    int j = sa[rk[i] - 1];
    while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
    lcp[rk[i]] = h;
    if (h) h--;
}
vector<long long> prefix(n + 1, 0);
for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + (n - sa[i] - lcp[i]);
int LOG = 1;
while ((1 << LOG) <= n) LOG++;
vector<vector<int>> mnL(LOG, lcp), mnS(LOG, sa);
for (int k = 1; k < LOG; k++)
    for (int i = 0; i + (1 << k) <= n; i++) {
        mnL[k][i] = min(mnL[k - 1][i], mnL[k - 1][i + (1 << (k - 1))]);
        mnS[k][i] = min(mnS[k - 1][i], mnS[k - 1][i + (1 << (k - 1))]);
    }
auto rangeMin = [&](vector<vector<int>>& t, int l, int r) { // inclusive
    int k = 31 - __builtin_clz(r - l + 1);
    return min(t[k][l], t[k][r - (1 << k) + 1]);
};
string out;
while (q--) {
    long long k;
    cin >> k;
    if (k > prefix[n]) {
        out += "-1\n";
        continue;
    }
    int i = lower_bound(prefix.begin() + 1, prefix.end(), k) - prefix.begin() - 1;
    int len = lcp[i] + (int)(k - prefix[i]);
    // Largest hi >= i with min(lcp[i+1..hi]) >= len.
    int lo = i, hi = n - 1;
    while (lo < hi) {
        int mid = (lo + hi + 1) / 2;
        if (rangeMin(mnL, i + 1, mid) >= len) lo = mid;
    }
}

```

```

        else hi = mid - 1;
    }
    int start = rangeMin(mnS, i, lo) + 1;
    out += to_string(start) + ' ' + to_string(len) + '\n';
}
cout << out;
}

```

Python

```

import sys
from array import array
from bisect import bisect_left

def main():
    data = sys.stdin.buffer.read().split()
    s = data[0]
    n = len(s)
    q = int(data[1])

    # Suffix array by prefix doubling.
    rank = list(s)
    sa = list(range(n))
    k = 1
    width = max(n, 256) + 1 # ranks (and letter codes) stay below width
    while True:
        key = [r * width + (rank[i + k] + 1 if i + k < n else 0) for i, r in enumerate(rank)]
        sa.sort(key=key.__getitem__)
        new = [0] * n
        for idx in range(1, n):
            new[sa[idx]] = new[sa[idx - 1]] + (key[sa[idx - 1]] != key[sa[idx]])
        rank = new
        if rank[sa[-1]] == n - 1:
            break
        k *= 2
    lcp = [0] * n
    h = 0
    for i in range(n):
        r = rank[i]
        if r == 0:
            h = 0
            continue
        j = sa[r - 1]
        while i + h < n and j + h < n and s[i + h] == s[j + h]:
            h += 1
        lcp[r] = h
        if h:
            h -= 1

```

```

# In suffix-array order, suffix sa[i] adds the distinct substrings of
# lengths lcp[i]+1 .. n-sa[i], already lexicographically sorted.
prefix = [0] * (n + 1)
for i in range(n):
    prefix[i + 1] = prefix[i] + n - sa[i] - lcp[i]
mn_lcp = [array("i", lcp)]
mn_sa = [array("i", sa)]
step = 1
while 2 * step <= n:
    a = mn_lcp[-1]
    b = mn_sa[-1]
    mn_lcp.append(array("i", [min(a[i], a[i + step]) for i in range(n - 2 * step + 1)]))
    mn_sa.append(array("i", [min(b[i], b[i + step]) for i in range(n - 2 * step + 1)]))
    step *= 2

def range_min(table, l, r):
    k = (r - l + 1).bit_length() - 1
    row = table[k]
    x = row[l]
    y = row[r - (1 << k) + 1]
    return x if x < y else y

total = prefix[n]
out = []
for token in data[2:2 + q]:
    k = int(token)
    if k > total:
        out.append("-1")
        continue
    i = bisect_left(prefix, k, 1) - 1
    length = lcp[i] + k - prefix[i]
    # Occurrences are suffixes i..hi, where lcp stays >= length.
    lo, hi = i, n - 1
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if range_min(mn_lcp, i + 1, mid) >= length:
            lo = mid
        else:
            hi = mid - 1
    out.append(f"{range_min(mn_sa, i, lo) + 1} {length}")
sys.stdout.write("\n".join(out) + "\n")

if __name__ == "__main__":
    main()

```

28. Suffix Pair Affinity

Adjacent suffix-array ranks form edges weighted by their LCP. For any two ranks, their suffix LCP is the minimum edge weight on the interval between them. Process these edges from largest weight downward with

DSU. When an edge of weight w joins components of sizes a, b , exactly $a \cdot b$ previously separate suffix pairs acquire bottleneck—and hence LCP— w ; add $w \cdot a \cdot b$.

This is the descending-edge construction of the LCP Cartesian/reconstruction tree. The interval-minimum theorem proves every suffix pair joins at exactly its LCP value, once, so all contributions sum to the requested aggregate. After suffix-array construction the scan is $O(n \log n)$ for sorting (or linear with bucketed LCPs) and near-linear DSU time. Zero-weight merges contribute nothing but remain safe.

C++

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false); cin.tie(nullptr); string s; if(!(cin>>s)) return 0;
// ...
```

Python

```
import sys
def main():
    s=sys.stdin.buffer.readline().strip(); n=len(s); sa=list(range(n)); r=list(s); k=1
    while True:
        sa.sort(key=lambda i:(r[i],r[i+k] if i+k<n else -1)); z=[0]*n
        for i in range(1,n): z[sa[i]]=z[sa[i-1]]+(r[sa[i-1]],r[sa[i-1]+k] if sa[i-1]+k<n else -1)<(r[sa[i]],r[sa[i]+k] if sa[i]+k<n else -1))
        r=z
        if r[sa[-1]]==n-1: break
        k*=2
    e=[]; k=0
    for i in range(n):
        x=r[i]
        if x:
            j=sa[x-1]
            while i+k<n and j+k<n and s[i+k]==s[j+k]: k+=1
            e.append((k,x-1,x)); k=max(0,k-1)
    p=list(range(n)); sz=[1]*n
    def f(x):
        while p[x]!=x: p[x]=p[p[x]]; x=p[x]
        return x
    ans=0
    for w,x,y in sorted(e,reverse=True): x=f(x); y=f(y); ans+=w*sz[x]*sz[y]; p[y]=x; sz[x]+=sz[y]
    print(ans)
if __name__=='__main__': main()
```

29. Editable Palindrome Rope

Store the string in an implicit randomized treap. Each node maintains subtree size, forward and backward polynomial hashes under two moduli, plus a lazy reversal flag. Splitting by size isolates any interval. **REV** swaps children and the two hash directions lazily; **SET** changes the isolated one-node value; **PAL** compares its forward and backward hash pairs. Merge the three pieces after every operation.

Induction on the pull formula shows each aggregate is exactly the polynomial hash of its in-order string. Lazy reversal preserves that invariant by swapping directions and child order. Splits preserve in-order sequence, so each command acts on exactly $[l, r]$. Expected time is $O(\log n)$ per operation and memory is $O(n)$. Push reversal before descending, precompute powers through n , and pull after both pointer and character changes.

C++

```
#include <bits/stdc++.h>
using namespace std; const long long M1=1000000007, M2=1000000009, B=911382323; struct N{int l=0, r=0, z=0;
```

Python

```
import sys, random
sys.setrecursionlimit(1000000); M1=1000000007; M2=1000000009; B=911382323
def main():
    d=sys.stdin.buffer.read().split(); s=d[0]; n=len(s); q=int(d[1]); pw1=[1]*(n+2); pw2=[1]*(n+2)
    for i in range(1, n+2): pw1[i]=pw1[i-1]*B%M1; pw2[i]=pw2[i-1]*B%M2
    L=[0]; R=[0]; P=[0]; V=[0]; Z=[0]; F1=[0]; F2=[0]; G1=[0]; G2=[0]; REV=[0]; rng=random.Random(712367)
    def node(v): L.append(0); R.append(0); P.append(rng.randrange(1<<30)); V.append(v); Z.append(1); F1.append(0)
    def rv(x):
        if x: L[x], R[x]=R[x], L[x]; F1[x], G1[x]=G1[x], F1[x]; F2[x], G2[x]=G2[x], F2[x]; REV[x]^=1
    def push(x):
        if REV[x]: rv(L[x]); rv(R[x]); REV[x]=0
    def pull(x):
        l=L[x]; r=R[x]; Z[x]=Z[l]+Z[r]+1; F1[x]=(F1[l]*pw1[Z[r]+1]+V[x]*pw1[Z[r]]+F1[r])%M1; F2[x]=(F2[l]*pw2[Z[r]+1]+V[x]*pw2[Z[r]]+F2[r])%M2
    def split(x, k):
        if not x: return 0, 0
        push(x)
        if Z[L[x]]>=k: a, L[x]=split(L[x], k); pull(x); return a, x
        R[x], b=split(R[x], k-Z[L[x]]-1); pull(x); return x, b
    def merge(a, b):
        if not a or not b: return a or b
        if P[a]<P[b]: push(a); R[a]=merge(R[a], b); pull(a); return a
        push(b); L[b]=merge(a, L[b]); pull(b); return b
    root=0
    for c in s: root=merge(root, node(c-96))
    out=[]; at=2
    for _ in range(q):
        o=d[at]; l=int(d[at+1]); at+=2; a, b=split(root, l-1)
        if o==b'SET': x, b=split(b, 1); V[x]=d[at][0]-96; at+=1; pull(x)
        else:
            r=int(d[at]); at+=1; x, b=split(b, r-1+1)
            if o==b'REV': rv(x)
            else: out.append('YES' if F1[x]==G1[x] and F2[x]==G2[x] else 'NO')
            root=merge(a, merge(x, b))
    print('\n'.join(out))
if __name__=='__main__': main()
```

30. Forbidden Superstring Count

Build one Aho–Corasick automaton for every pattern. Propagate a forbidden flag and a bitmask of required patterns through failure links. For subset T of required patterns, delete states matching a base-forbidden pattern or any pattern in T . Matrix exponentiation of the remaining transition graph counts length- L strings avoiding all those patterns.

Inclusion–exclusion gives

```
sum over T subset required of  $(-1)^{|T|}$  * avoid(forbidden union T).
```

A walk from the root corresponds bijectively to an alphabet string, and it survives exactly when no deleted state is visited. Thus each matrix count is the required avoidance cardinality, while inclusion–exclusion retains exactly strings containing every required pattern. Complexity is $O(2^R * \text{states}^3 * \log L)$ time and $O(\text{states}^2)$ working memory. Failure outputs must be propagated before matrices are built; $L=0$ is handled by the identity matrix.

C++

```
#include <bits/stdc++.h>
using namespace std; const long long M=998244353; using A=vector<vector<long long>>; A mul(const A&a, const A&b, const long long M)
```

Python

```
import sys, collections
M=998244353
def mul(a,b):
    n=len(a); c=[[0]*n for _ in range(n)]
    for i in range(n):
        for k,x in enumerate(a[i]):
            if x:
                for j,y in enumerate(b[k]):
                    if y: c[i][j]=(c[i][j]+x*y)%M
    return c
def main():
    d=sys.stdin.buffer.read().split(); R=int(d[0]); F=int(d[1]); L=int(d[2]); p=d[3:3+R+F]; to=[{}]; fail=[0]*len(p)
    for i,s in enumerate(p):
        v=0
        for c in s:
            if c not in to[v]: to[v][c]=len(to); to.append({}); fail.append(0); mask.append(0); bad.append(0)
            v=to[v][c]
        if i<R: mask[v]|=1<<i
        else: bad[v]=1
    q=collections.deque()
    for c in b'abc':
        if c in to[0]: q.append(to[0][c])
        else: to[0][c]=0
    while q:
```



```

v=q.popleft();mask[v]|=mask[fail[v]];bad[v]|=bad[fail[v]]
for c in b'abc':
    if c in to[v]:fail[to[v][c]]=to[fail[v]][c];q.append(to[v][c])
    else:to[v][c]=to[fail[v]][c]
ans=0;n=len(to)
for ban in range(1<<R):
    ok=[not bad[i] and not(mask[i]&ban) for i in range(n)];a=[[0]*n for _ in range(n)]
    for i in range(n):
        if ok[i]:
            for c in b'abc':
                j=to[i][c]
                if ok[j]:a[i][j]+=1
    v=[[0]*n for _ in range(n)]
    for i in range(n):v[i][i]=1
    e=L
    while e:
        if e&1:v=mul(v,a)
        a=mul(a,a);e//=2
    z=sum(v[0])%M;ans+=( -z if ban.bit_count()&1 else z)
    print(ans%M)
if __name__=='__main__':main()

```

31. Two-Faction Networks

Count coloured graphs, take the logarithm, halve

A **2-coloured graph** assigns each vertex one of two colours and uses only edges between colours: $b_n = \sum_k C(n,k) 2^{\{k(n-k)\}}$. Every 2-coloured graph splits uniquely into connected 2-coloured graphs, so with exponential generating functions $B = \exp(C)$, where C counts the connected ones. A connected bipartite graph has exactly two proper 2-colourings, so the answer is $n! [x^n] \log B / 2$.

All b_n at once: $k(n-k) = (n^2 - k^2 - (n-k)^2)/2$. Since $998244353 = 1 \pmod{8}$, $s = \sqrt[4]{2}$ exists ($s = z + z^{-1}$ for a primitive 8th root of unity z), and

$$b_n / n! = s^{\{n^2\}} * \sum_k (s^{\{-k^2\}} / k!) (s^{\{-(n-k)^2\}} / (n-k)!),$$

one self-convolution.

Correctness. The exponential formula applies because labelled 2-coloured graphs are sets of connected 2-coloured components. A connected bipartite graph's colouring is fixed once one vertex's colour is chosen.

Complexity and traps. One convolution plus one logarithm, $O(N \log N)$. Reduce exponents n^2 modulo $p - 1$. The $n = 1$ term is 1 (two colourings of a single vertex, divided by two).

C++

```

#include <bits/stdc++.h>
using namespace std;

// A 2-coloured graph puts every vertex in one of two factions and only joins
// vertices of different factions:  $b_n = \sum_k C(n,k) 2^{k(n-k)}$ . Its
// exponential generating function B satisfies  $\log B = \text{EGF of connected}$ 
// 2-coloured graphs, and a connected bipartite graph has exactly two
// colourings. With  $s = \sqrt{2}$  (it exists modulo 998244353), the identity
//  $k(n-k) = (n^2 - k^2 - (n-k)^2)/2$  turns all  $b_n$  into one convolution.

const long long MOD = 998244353, G = 3;

long long power(long long b, long long e) {
    long long r = 1;
    b %= MOD;
    if (b < 0) b += MOD;
    for (; e; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

void ntt(vector<long long>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        long long w = power(G, (MOD - 1) / len);
        if (invert) w = power(w, MOD - 2);
        vector<long long> ws(len / 2, 1);
        for (int k = 1; k < len / 2; k++) ws[k] = ws[k - 1] * w % MOD;
        for (int i = 0; i < n; i += len)
            for (int k = 0; k < len / 2; k++) {
                long long u = a[i + k], v = a[i + k + len / 2] * ws[k] % MOD;
                a[i + k] = u + v < MOD ? u + v : u + v - MOD;
                a[i + k + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
            }
    }
    if (invert) {
        long long inv = power(n, MOD - 2);
        for (auto& x : a) x = x * inv % MOD;
    }
}

// Product truncated to `limit` terms (limit < 0: full product).
vector<long long> multiply(vector<long long> a, vector<long long> b, int limit = -1) {
    if (a.empty() || b.empty()) return {};
    if (limit >= 0) {

```

```

        if ((int)a.size() > limit) a.resize(limit);
        if ((int)b.size() > limit) b.resize(limit);
    }
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <= 1;
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(limit >= 0 ? min(need, limit) : need);
    return a;
}

vector<long long> inverse(const vector<long long>& a, int n) {
    vector<long long> g = {power(a[0], MOD - 2)};
    for (int m = 1; m < n; m++) {
        m *= 2;
        vector<long long> t = multiply(vector<long long>(a.begin(), a.begin() + min<int>(m, a.size())), g, m);
        for (auto& x : t) x = (MOD - x) % MOD;
        t[0] = (t[0] + 2) % MOD;
        g = multiply(g, t, m);
    }
    g.resize(n);
    return g;
}

vector<long long> logarithm(const vector<long long>& a, int n) {
    vector<long long> da;
    for (int i = 1; i < min<int>(a.size(), n); i++) da.push_back(a[i] * i % MOD);
    vector<long long> q = multiply(da, inverse(a, n), n - 1), out(n, 0);
    q.resize(max(n - 1, 0));
    vector<long long> inv(n + 1, 1);
    for (int i = 2; i <= n; i++) inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
    for (int i = 1; i < n; i++) out[i] = q[i - 1] * inv[i] % MOD;
    return out;
}

vector<long long> exponential(const vector<long long>& a, int n) {
    vector<long long> g = {1};
    for (int m = 1; m < n; m++) {
        m *= 2;
        vector<long long> gm = g;
        gm.resize(m, 0);
        vector<long long> lg = logarithm(gm, m), t(m);
        for (int i = 0; i < m; i++) t[i] = ((i < (int)a.size() ? a[i] : 0) - lg[i] + MOD) % MOD;
        t[0] = (t[0] + 1) % MOD;
        g = multiply(g, t, m);
    }
    g.resize(n);
}

```

```

    return g;
}

int main() {
    int N;
    cin >> N;
    long long zeta = power(G, (MOD - 1) / 8); // primitive 8th root of unity
    long long s = (zeta + power(zeta, MOD - 2)) % MOD; // s^2 = 2
    long long sInv = power(s, MOD - 2);
    vector<long long> fact(N + 1, 1), invFact(N + 1, 1);
    for (int i = 1; i <= N; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[N] = power(fact[N], MOD - 2);
    for (int i = N; i > 0; i--) invFact[i - 1] = invFact[i] * i % MOD;
    vector<long long> a(N + 1);
    for (long long k = 0; k <= N; k++) a[k] = power(sInv, k * k % (MOD - 1)) * invFact[k] % MOD;
    vector<long long> B = multiply(a, a, N + 1);
    for (long long n = 0; n <= N; n++) B[n] = B[n] * power(s, n * n % (MOD - 1)) % MOD;
    vector<long long> C = logarithm(B, N + 1);
    long long inv2 = (MOD + 1) / 2;
    string out;
    for (int n = 1; n <= N; n++) {
        out += to_string(C[n] * fact[n] % MOD * inv2 % MOD);
        out += n < N ? ' ' : '\n';
    }
    cout << out;
}

```

Python

```

"""Connected bipartite labelled graphs: log of the 2-coloured-graph EGF."""
import sys

MOD = 998244353
_CTX = None

def _decimal_context():
    global _CTX
    if _CTX is None:
        import decimal
        _CTX = decimal.Context(prec=decimal.MAX_PREC, Emax=decimal.MAX_EMAX, Emin=decimal.MIN_EMIN)
    return _CTX

def multiply(a, b, limit=None):
    """Product of coefficient lists modulo MOD, truncated to `limit` terms.

    Large products pack coefficients into decimal digits and let libmpdec's
    number-theoretic-transform multiplication do the work."""
    if not a or not b:

```

```

        return []
    if limit is not None:
        a = a[:limit]
        b = b[:limit]
    length = len(a) + len(b) - 1
    if min(len(a), len(b)) <= 40:
        out = [0] * length
        for i, x in enumerate(a):
            if x:
                for j, y in enumerate(b):
                    out[i + j] += x * y
        out = [v % MOD for v in out]
    else:
        import decimal
        ctx = _decimal_context()
        A = decimal.Decimal("".join(format(x, "024d") for x in reversed(a)))
        B = decimal.Decimal("".join(format(x, "024d") for x in reversed(b)))
        text = str(ctx.multiply(A, B)).rjust(length * 24, "0")
        top = len(text)
        out = [int(text[top - (i + 1) * 24: top - i * 24]) % MOD for i in range(length)]
    return out[:limit] if limit is not None else out


def inverse(a, n):
    """First n coefficients of 1/a (a[0] != 0)."""
    g = [pow(a[0], MOD - 2, MOD)]
    m = 1
    while m < n:
        m *= 2
        t = multiply(a[:m], g, m)
        t = [(-x) % MOD for x in t]
        t[0] = (t[0] + 2) % MOD
        g = multiply(g, t, m)
    return g[:n]


def log(a, n):
    """First n coefficients of log(a) with a[0] = 1."""
    if n == 1:
        return [0]
    da = [a[i] * i % MOD for i in range(1, min(len(a), n))]
    q = multiply(da, inverse(a, n), n - 1)
    q += [0] * (n - 1 - len(q))
    inv = [0, 1] + [0] * (n - 1)
    for i in range(2, n + 1):
        inv[i] = (MOD - (MOD // i) * inv[MOD % i] % MOD) % MOD
    return [0] + [q[i - 1] * inv[i] % MOD for i in range(1, n)]


def exp(a, n):
    """First n coefficients of exp(a) with a[0] = 0."""

```

```

g = [1]
m = 1
while m < n:
    m *= 2
    lg = log(g + [0] * (m - len(g)), m)
    t = [(a[i] if i < len(a) else 0) - lg[i]) % MOD for i in range(m)]
    t[0] = (t[0] + 1) % MOD
    g = multiply(g, t, m)
return g[:n]

def main():
    N = int(sys.stdin.read().split()[0])
    zeta = pow(3, (MOD - 1) // 8, MOD) # primitive 8th root of unity
    s = (zeta + pow(zeta, MOD - 2, MOD)) % MOD # s^2 = 2
    s_inv = pow(s, MOD - 2, MOD)
    fact = [1] * (N + 1)
    for i in range(1, N + 1):
        fact[i] = fact[i - 1] * i % MOD
    inv_fact = [1] * (N + 1)
    inv_fact[N] = pow(fact[N], MOD - 2, MOD)
    for i in range(N, 0, -1):
        inv_fact[i - 1] = inv_fact[i] * i % MOD
    # 2^{k(n-k)} = s^{n^2} s^{-k^2} s^{-(n-k)^2}
    a = [pow(s_inv, k * k % (MOD - 1), MOD) * inv_fact[k] % MOD for k in range(N + 1)]
    B = multiply(a, a, N + 1)
    B = [B[n] * pow(s, n * n % (MOD - 1), MOD) % MOD for n in range(N + 1)]
    C = log(B, N + 1)
    half = (MOD + 1) // 2
    print(" ".join(str(C[n] * fact[n] % MOD * half % MOD) for n in range(1, N + 1)))

if __name__ == "__main__":
    main()

```

32. Exponential Polynomial Ledger

A polynomial hides behind the geometric factor

Let $S(k) = \sum_{i < k} r^i i^d$. For $r = 1$, S is a polynomial of degree $d+1$ in k . For $r \neq 1$, summation by parts shows $S(k) = r^k G(k) + c$ with $\deg G \leq d$ and a constant c . Compute $S(0), \dots, S(d+1)$ directly; the powers i^d come from a linear sieve with `pow` only at primes. Then $G(i) = (S(i) - c) r^{-i}$, and since the $(d+1)$ -th finite difference of G vanishes,

$$\sum_i (-1)^{d+1-i} C(d+1, i) S(i) r^{-i} = c (r^{-1} - 1)^{d+1},$$

which determines c because $r \neq 1$. Finally evaluate G at $n \bmod p$ by Lagrange interpolation over the points $0..d$, and multiply by $r^{n \bmod (p-1)}$.

Correctness. Polynomial identities hold over the field, so evaluating at $n \bmod p$ is valid. r^n depends only on $n \bmod (p-1)$ for $r \neq 0$. $r = 0$ leaves only the term $i = 0$, with $0^0 = 1$.

Complexity and traps. $O(d)$ field operations plus $O(d / \log d)$ modular powers. The Lagrange formula must allow $n \bmod p \leq d$. Use prefix and suffix products rather than divisions by $x - i$.

C++

```
#include <bits/stdc++.h>
using namespace std;

// S(k) = sum_{i<k} r^i i^d. For r = 1, S is a polynomial of degree d+1 in k.
// For r != 1, S(k) = r^k G(k) + c with deg G <= d (sum by parts). Compute
// S(0..d+1) directly; the (d+1)-th finite difference of G vanishes, which
// determines c, and Lagrange interpolation evaluates G at n mod p.

typedef long long ll;
const ll MOD = 998244353;

ll power(ll b, ll e) {
    ll r = 1;
    b %= MOD;
    if (b < 0) b += MOD;
    for (; e >= 1; b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

vector<ll> fact, invFact;

ll interpolate(const vector<ll>& values, ll x) {
    int k = values.size();
    x %= MOD;
    vector<ll> pre(k + 1, 1), suf(k + 1, 1);
    for (int i = 0; i < k; i++) pre[i + 1] = pre[i] * ((x - i) % MOD + MOD) % MOD;
    for (int i = k - 1; i >= 0; i--) suf[i] = suf[i + 1] * ((x - i) % MOD + MOD) % MOD;
    ll total = 0;
    for (int i = 0; i < k; i++) {
        ll term = values[i] * pre[i] % MOD * suf[i + 1] % MOD * invFact[i] % MOD * invFact[k - 1 - i] % MOD;
        total = ((k - 1 - i) & 1) ? (total - term + MOD) % MOD : (total + term) % MOD;
    }
    return total;
}

int main() {
    ll n, r;
    int d;
    cin >> n >> r >> d;
    r %= MOD;
    if (n == 0) {
        cout << 0 << '\n';
    }
}
```

```

    return 0;
}
if (r == 0) {
    cout << (d == 0 ? 1 : 0) << '\n'; // only i = 0 survives, 0^0 = 1
    return 0;
}
int m = d + 2;
vector<ll> pw(m + 1, 0);
pw[0] = d == 0 ? 1 : 0;
if (m >= 1) pw[1] = 1;
vector<int> primes;
vector<char> composite(m + 1, 0);
for (int i = 2; i <= m; i++) {
    if (!composite[i]) {
        primes.push_back(i);
        pw[i] = power(i, d);
    }
    for (int p : primes) {
        if ((ll)i * p > m) break;
        composite[i * p] = 1;
        pw[i * p] = pw[i] * pw[p] % MOD;
        if (i % p == 0) break;
    }
}
vector<ll> S(m + 1, 0);
ll rp = 1;
for (int i = 0; i < m; i++) {
    S[i + 1] = (S[i] + rp * pw[i]) % MOD;
    rp = rp * r % MOD;
}
fact.assign(m + 2, 1);
invFact.assign(m + 2, 1);
for (int i = 1; i < m + 2; i++) fact[i] = fact[i - 1] * i % MOD;
invFact[m + 1] = power(fact[m + 1], MOD - 2);
for (int i = m + 1; i > 0; i--) invFact[i - 1] = invFact[i] * i % MOD;
if (r == 1) {
    cout << interpolate(vector<ll>(S.begin(), S.begin() + d + 2), n) << '\n';
    return 0;
}
ll rInv = power(r, MOD - 2);
vector<ll> t(d + 2);
ll rk = 1;
for (int i = 0; i < d + 2; i++) {
    t[i] = S[i] * rk % MOD;
    rk = rk * rInv % MOD;
}
ll num = 0;
for (int i = 0; i < d + 2; i++) {
    ll w = fact[d + 1] * invFact[i] % MOD * invFact[d + 1 - i] % MOD * t[i] % MOD;
    num = ((d + 1 - i) & 1) ? (num - w + MOD) % MOD : (num + w) % MOD;
}

```



```

ll c = num * power(power((rInv - 1 + MOD) % MOD, d + 1), MOD - 2) % MOD;
vector<ll> G(d + 1);
rk = 1;
for (int i = 0; i <= d; i++) {
    G[i] = ((t[i] - c * rk) % MOD + MOD) % MOD;
    rk = rk * rInv % MOD;
}
cout << (power(r, n % (MOD - 1)) * interpolate(G, n) + c) % MOD << '\n';
}

```

Python

```

import sys

MOD = 998244353

def main():
    n, r, d = map(int, sys.stdin.read().split())
    r %= MOD
    if n == 0:
        print(0)
        return
    if r == 0:
        print(1 if d == 0 else 0) # only the term i = 0 survives, and 0^0 = 1
        return
    m = d + 2 # sample points 0..d+1
    # i^d for i = 0..m via a linear sieve (powers only at primes).
    pw = [0] * (m + 1)
    pw[0] = 1 if d == 0 else 0
    if m >= 1:
        pw[1] = 1
    primes = []
    composite = bytearray(m + 1)
    for i in range(2, m + 1):
        if not composite[i]:
            primes.append(i)
            pw[i] = pow(i, d, MOD)
        for p in primes:
            if i * p > m:
                break
            composite[i * p] = 1
            pw[i * p] = pw[i] * pw[p] % MOD
            if i % p == 0:
                break
    # Prefix sums S(k) = sum_{i<k} r^i i^d for k = 0..m.
    S = [0] * (m + 1)
    rp = 1
    for i in range(m):
        S[i + 1] = (S[i] + rp * pw[i]) % MOD

```

```

    rp = rp * r % MOD
fact = [1] * (m + 2)
for i in range(1, m + 2):
    fact[i] = fact[i - 1] * i % MOD
inv_fact = [1] * (m + 2)
inv_fact[m + 1] = pow(fact[m + 1], MOD - 2, MOD)
for i in range(m + 1, 0, -1):
    inv_fact[i - 1] = inv_fact[i] * i % MOD

def interpolate(values, x):
    """Value at x of the polynomial through (i, values[i]), i = 0..k-1."""
    k = len(values)
    x %= MOD
    pre = [1] * (k + 1)
    for i in range(k):
        pre[i + 1] = pre[i] * (x - i) % MOD
    suf = [1] * (k + 1)
    for i in range(k - 1, -1, -1):
        suf[i] = suf[i + 1] * (x - i) % MOD
    total = 0
    for i in range(k):
        term = values[i] * pre[i] % MOD * suf[i + 1] % MOD * inv_fact[i] % MOD * inv_fact[k - i] % MOD
        total += -term if (k - 1 - i) & 1 else term
    return total % MOD

if r == 1:
    # sum_{i<n} i^d is a polynomial of degree d + 1 in n.
    print(interpolate(S[:d + 2], n))
    return
# S(k) = r^k G(k) + c with deg G <= d. The (d+1)-th finite difference of
# G vanishes, which pins down c; then interpolate G at n.
r_inv = pow(r, MOD - 2, MOD)
t = [0] * (d + 2)
rk = 1
for i in range(d + 2):
    t[i] = S[i] * rk % MOD
    rk = rk * r_inv % MOD
num = 0
for i in range(d + 2):
    w = fact[d + 1] * inv_fact[i] % MOD * inv_fact[d + 1 - i] % MOD
    num += -w * t[i] if (d + 1 - i) & 1 else w * t[i]
c = num % MOD * pow(pow((r_inv - 1) % MOD, d + 1, MOD), MOD - 2, MOD) % MOD
G = []
rk = 1
for i in range(d + 1):
    G.append((t[i] - c * rk) % MOD)
    rk = rk * r_inv % MOD
print((pow(r, n % (MOD - 1), MOD) * interpolate(G, n) + c) % MOD)

```

```

if __name__ == "__main__":

```

```
main()
```

33. Branching Family Census

Newton's method on a functional equation

A record is a root with s ordered sub-records and c_s styles, so its generating function satisfies $T = x \phi(T)$ with $\phi(z) = \sum c_s z^s$. The equation $F(T) = T - x \phi(T) = 0$ has $F'(T) = 1 - x \phi'(T)$, which is invertible as a power series. Newton's iteration $T \leftarrow T - F(T)/F'(T)$ doubles the number of correct coefficients. Each round evaluates $\phi(T)$ and $\phi'(T)$ by Horner's rule at the current precision.

Correctness. If T is correct modulo x^m , then $F(T + e) = F(T) + F'(T) e + O(e^2)$ with $e = O(x^m)$, so the update is correct modulo x^{2m} . The start $T = c_0 x$ is correct modulo x^2 .

Complexity and traps. $O(D M(N))$ over all rounds, where M is the cost of a multiplication. With $c_0 = 0$ every count is zero. The sample $(1, 2, 1)$ gives the Catalan numbers, and $(1, 1, 1)$ gives Motzkin numbers shifted by one.

C++

```
#include <bits/stdc++.h>
using namespace std;

// A plane tree is a root with s ordered subtrees, weighted by c_s, so its
// generating function satisfies T = x * phi(T) with phi(z) = sum c_s z^s.
// Newton's method doubles the number of correct coefficients each round:
// T <- T - (T - x phi(T)) / (1 - x phi'(T)), with phi(T), phi'(T) by Horner.

const long long MOD = 998244353, G = 3;

long long power(long long b, long long e) {
    long long r = 1;
    b %= MOD;
    if (b < 0) b += MOD;
    for (; e; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

void ntt(vector<long long>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long w = power(G, (MOD - 1) / len);
```

```

        if (invert) w = power(w, MOD - 2);
        vector<long long> ws(len / 2, 1);
        for (int k = 1; k < len / 2; k++) ws[k] = ws[k - 1] * w % MOD;
        for (int i = 0; i < n; i += len)
            for (int k = 0; k < len / 2; k++) {
                long long u = a[i + k], v = a[i + k + len / 2] * ws[k] % MOD;
                a[i + k] = u + v < MOD ? u + v : u + v - MOD;
                a[i + k + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
            }
    }
    if (invert) {
        long long inv = power(n, MOD - 2);
        for (auto& x : a) x = x * inv % MOD;
    }
}

// Product truncated to `limit` terms (limit < 0: full product).
vector<long long> multiply(vector<long long> a, vector<long long> b, int limit = -1) {
    if (a.empty() || b.empty()) return {};
    if (limit >= 0) {
        if ((int)a.size() > limit) a.resize(limit);
        if ((int)b.size() > limit) b.resize(limit);
    }
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <= 1;
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(limit >= 0 ? min(need, limit) : need);
    return a;
}

vector<long long> inverse(const vector<long long>& a, int n) {
    vector<long long> g = {power(a[0], MOD - 2)};
    for (int m = 1; m < n; m++) {
        m *= 2;
        vector<long long> t = multiply(vector<long long>(a.begin(), a.begin() + min<int>(m, a.size())), g, m);
        for (auto& x : t) x = (MOD - x) % MOD;
        t[0] = (t[0] + 2) % MOD;
        g = multiply(g, t, m);
    }
    g.resize(n);
    return g;
}

vector<long long> logarithm(const vector<long long>& a, int n) {
    vector<long long> da;
    for (int i = 1; i < min<int>(a.size(), n); i++) da.push_back(a[i] * i % MOD);
}

```

```

    vector<long long> q = multiply(da, inverse(a, n), n - 1), out(n, 0);
    q.resize(max(n - 1, 0));
    vector<long long> inv(n + 1, 1);
    for (int i = 2; i <= n; i++) inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
    for (int i = 1; i < n; i++) out[i] = q[i - 1] * inv[i] % MOD;
    return out;
}

vector<long long> exponential(const vector<long long>& a, int n) {
    vector<long long> g = {1};
    for (int m = 1; m < n;) {
        m *= 2;
        vector<long long> gm = g;
        gm.resize(m, 0);
        vector<long long> lg = logarithm(gm, m), t(m);
        for (int i = 0; i < m; i++) t[i] = ((i < (int)a.size() ? a[i] : 0) - lg[i] + MOD) % MOD;
        t[0] = (t[0] + 1) % MOD;
        g = multiply(g, t, m);
    }
    g.resize(n);
    return g;
}

vector<long long> horner(const vector<long long>& coef, const vector<long long>& T, int m) {
    vector<long long> acc = {coef.back()};
    for (int s = (int)coef.size() - 2; s >= 0; s--) {
        acc = multiply(acc, T, m);
        if (acc.empty()) acc = {0};
        acc[0] = (acc[0] + coef[s]) % MOD;
    }
    acc.resize(m, 0);
    return acc;
}

int main() {
    int N, D;
    cin >> N >> D;
    vector<long long> c(D + 1), dc;
    for (auto& x : c) cin >> x, x %= MOD;
    for (int s = 1; s <= D; s++) dc.push_back(s * c[s] % MOD);
    int total = N + 1;
    vector<long long> T = {0, c[0]};
    for (int m = 2; m < total;) {
        m = min(2 * m, total);
        T.resize(m, 0);
        vector<long long> P = horner(c, T, m), Q = dc.empty() ? vector<long long>(m, 0) : horner(dc,
        vector<long long> F(m), Fp(m);
        for (int i = 0; i < m; i++) {
            F[i] = (T[i] - (i ? P[i - 1] : 0) + MOD) % MOD;
            Fp[i] = ((i == 0 ? 1 : 0) - (i ? Q[i - 1] : 0) + MOD) % MOD;
        }
    }
}

```

```

        vector<long long> step = multiply(F, inverse(Fp, m), m);
        step.resize(m, 0);
        for (int i = 0; i < m; i++) T[i] = (T[i] - step[i] + MOD) % MOD;
    }
    T.resize(total, 0);
    string out;
    for (int n = 1; n <= N; n++) {
        out += to_string(T[n]);
        out += n < N ? ' ' : '\n';
    }
    cout << out;
}

```

Python

```

"""Weighted plane trees: Newton iteration on  $T = x * \phi(T)$ ."""
import sys

MOD = 998244353
_CTX = None

def _decimal_context():
    global _CTX
    if _CTX is None:
        import decimal
        _CTX = decimal.Context(prec=decimal.MAX_PREC, Emax=decimal.MAX_EMAX, Emin=decimal.MIN_EMIN)
    return _CTX

def multiply(a, b, limit=None):
    """Product of coefficient lists modulo MOD, truncated to `limit` terms.

    Large products pack coefficients into decimal digits and let libmpdec's
    number-theoretic-transform multiplication do the work."""
    if not a or not b:
        return []
    if limit is not None:
        a = a[:limit]
        b = b[:limit]
    length = len(a) + len(b) - 1
    if min(len(a), len(b)) <= 40:
        out = [0] * length
        for i, x in enumerate(a):
            if x:
                for j, y in enumerate(b):
                    out[i + j] += x * y
        out = [v % MOD for v in out]
    else:
        import decimal

```

```

    ctx = _decimal_context()
    A = decimal.Decimal("".join(format(x, "024d") for x in reversed(a)))
    B = decimal.Decimal("".join(format(x, "024d") for x in reversed(b)))
    text = str(ctx.multiply(A, B)).rjust(length * 24, "0")
    top = len(text)
    out = [int(text[top - (i + 1) * 24: top - i * 24]) % MOD for i in range(length)]
    return out[:limit] if limit is not None else out

def inverse(a, n):
    """First n coefficients of 1/a (a[0] != 0)."""
    g = [pow(a[0], MOD - 2, MOD)]
    m = 1
    while m < n:
        m *= 2
        t = multiply(a[:m], g, m)
        t = [(-x) % MOD for x in t]
        t[0] = (t[0] + 2) % MOD
        g = multiply(g, t, m)
    return g[:n]

def log(a, n):
    """First n coefficients of log(a) with a[0] = 1."""
    if n == 1:
        return [0]
    da = [a[i] * i % MOD for i in range(1, min(len(a), n))]
    q = multiply(da, inverse(a, n), n - 1)
    q += [0] * (n - 1 - len(q))
    inv = [0, 1] + [0] * (n - 1)
    for i in range(2, n + 1):
        inv[i] = (MOD - (MOD // i) * inv[MOD % i] % MOD) % MOD
    return [0] + [q[i - 1] * inv[i] % MOD for i in range(1, n)]

def exp(a, n):
    """First n coefficients of exp(a) with a[0] = 0."""
    g = [1]
    m = 1
    while m < n:
        m *= 2
        lg = log(g + [0] * (m - len(g)), m)
        t = [(a[i] if i < len(a) else 0) - lg[i]] % MOD for i in range(m)
        t[0] = (t[0] + 1) % MOD
        g = multiply(g, t, m)
    return g[:n]

def main():
    data = list(map(int, sys.stdin.read().split()))
    N, D = data[0], data[1]

```

```

c = [x % MOD for x in data[2:3 + D]]
dc = [s * c[s] % MOD for s in range(1, D + 1)] # coefficients of phi'

def horner(coef, T, m):
    acc = [coef[-1]]
    for s in range(len(coef) - 2, -1, -1):
        acc = multiply(acc, T, m) if len(acc) > 1 or acc[0] else [0]
        acc += [0] * (1 - len(acc))
        acc[0] = (acc[0] + coef[s]) % MOD
    return acc + [0] * (m - len(acc))

total = N + 1
T = [0, c[0]]
m = 2
while m < total:
    m = min(2 * m, total)
    T = T + [0] * (m - len(T))
    P = horner(c, T, m)
    Q = horner(dc, T, m) if dc else [0] * m
    F = [(T[i] - (P[i - 1] if i else 0)) % MOD for i in range(m)]
    Fp = [((1 if i == 0 else 0) - (Q[i - 1] if i else 0)) % MOD for i in range(m)]
    step = multiply(F, inverse(Fp, m), m)
    step += [0] * (m - len(step))
    T = [(T[i] - step[i]) % MOD for i in range(m)]
T += [0] * (total - len(T))
print(" ".join(str(T[n]) for n in range(1, N + 1)))

if __name__ == "__main__":
    main()

```

34. Coin Bag Census

The logarithm is a harmonic sum

The bag generating function is $\prod_i 1/(1 - x^{w_i})$. Its logarithm is

$$\sum_{\text{over values } w} \text{cnt}[w] * \sum_{k \geq 1} x^{w k} / k,$$

which costs $\sum_w S/w = O(S \log S)$ to write down. One power-series exponential then returns every count up to S .

Correctness. $\log(1/(1 - y)) = \sum y^k / k$, and the logarithm turns the product into a sum. Values above S contribute nothing below x^{S+1} .

Complexity and traps. $O(S \log S)$ for the harmonic sum plus $O(S \log S)$ for the exponential. Coin types are distinct even with equal values, so count multiplicities and do not deduplicate. With one coin of each value the counts are the partition numbers.

C++

```
#include <bits/stdc++.h>
using namespace std;

// prod_i 1/(1 - x^{w_i}) = exp( sum_w cnt[w] * sum_{k>=1} x^{wk} / k ).
// The logarithm is a harmonic sum costing O(S log S); one power-series
// exponential then gives every count up to S.

const long long MOD = 998244353, G = 3;

long long power(long long b, long long e) {
    long long r = 1;
    b %= MOD;
    if (b < 0) b += MOD;
    for (; e; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

void ntt(vector<long long>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        long long w = power(G, (MOD - 1) / len);
        if (invert) w = power(w, MOD - 2);
        vector<long long> ws(len / 2, 1);
        for (int k = 1; k < len / 2; k++) ws[k] = ws[k - 1] * w % MOD;
        for (int i = 0; i < n; i += len)
            for (int k = 0; k < len / 2; k++) {
                long long u = a[i + k], v = a[i + k + len / 2] * ws[k] % MOD;
                a[i + k] = u + v < MOD ? u + v : u + v - MOD;
                a[i + k + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
            }
    }
    if (invert) {
        long long inv = power(n, MOD - 2);
        for (auto& x : a) x = x * inv % MOD;
    }
}

// Product truncated to `limit` terms (limit < 0: full product).
vector<long long> multiply(vector<long long> a, vector<long long> b, int limit = -1) {
    if (a.empty() || b.empty()) return {};
    if (limit >= 0) {
        if ((int)a.size() > limit) a.resize(limit);
    }
}
```

```

        if ((int)b.size() > limit) b.resize(limit);
    }
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <= 1;
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(limit >= 0 ? min(need, limit) : need);
    return a;
}

vector<long long> inverse(const vector<long long>& a, int n) {
    vector<long long> g = {power(a[0], MOD - 2)};
    for (int m = 1; m < n; m *= 2) {
        vector<long long> t = multiply(vector<long long>(a.begin(), a.begin() + min<int>(m, a.size())), g, m);
        for (auto& x : t) x = (MOD - x) % MOD;
        t[0] = (t[0] + 2) % MOD;
        g = multiply(g, t, m);
    }
    g.resize(n);
    return g;
}

vector<long long> logarithm(const vector<long long>& a, int n) {
    vector<long long> da;
    for (int i = 1; i < min<int>(a.size(), n); i++) da.push_back(a[i] * i % MOD);
    vector<long long> q = multiply(da, inverse(a, n), n - 1), out(n, 0);
    q.resize(max(n - 1, 0));
    vector<long long> inv(n + 1, 1);
    for (int i = 2; i <= n; i++) inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
    for (int i = 1; i < n; i++) out[i] = q[i - 1] * inv[i] % MOD;
    return out;
}

vector<long long> exponential(const vector<long long>& a, int n) {
    vector<long long> g = {1};
    for (int m = 1; m < n; m *= 2) {
        vector<long long> gm = g;
        gm.resize(m, 0);
        vector<long long> lg = logarithm(gm, m), t(m);
        for (int i = 0; i < m; i++) t[i] = ((i < (int)a.size() ? a[i] : 0) - lg[i] + MOD) % MOD;
        t[0] = (t[0] + 1) % MOD;
        g = multiply(g, t, m);
    }
    g.resize(n);
    return g;
}

```

```

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, S;
    cin >> n >> S;
    vector<long long> cnt(S + 1, 0);
    for (int i = 0; i < n; i++) {
        int w;
        cin >> w;
        if (w <= S) cnt[w]++;
    }
    vector<long long> inv(S + 2, 1);
    for (int i = 2; i <= S + 1; i++) inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
    vector<long long> L(S + 1, 0);
    for (int w = 1; w <= S; w++)
        if (cnt[w])
            for (int e = w, k = 1; e <= S; e += w, k++) L[e] = (L[e] + cnt[w] * inv[k]) % MOD;
    vector<long long> P = exponential(L, S + 1);
    string out;
    for (int s = 1; s <= S; s++) {
        out += to_string(P[s]);
        out += s < S ? ' ' : '\n';
    }
    cout << out;
}

```

Python

```

"""Coin multisets: log of prod 1/(1-x^w) is a harmonic sum; then exp."""
import sys

MOD = 998244353
_CTX = None

def _decimal_context():
    global _CTX
    if _CTX is None:
        import decimal
        _CTX = decimal.Context(prec=decimal.MAX_PREC, Emax=decimal.MAX_EMAX, Emin=decimal.MIN_EMIN)
    return _CTX

def multiply(a, b, limit=None):
    """Product of coefficient lists modulo MOD, truncated to `limit` terms.

    Large products pack coefficients into decimal digits and let libmpdec's
    number-theoretic-transform multiplication do the work."""

```

```

if not a or not b:
    return []
if limit is not None:
    a = a[:limit]
    b = b[:limit]
length = len(a) + len(b) - 1
if min(len(a), len(b)) <= 40:
    out = [0] * length
    for i, x in enumerate(a):
        if x:
            for j, y in enumerate(b):
                out[i + j] += x * y
    out = [v % MOD for v in out]
else:
    import decimal
    ctx = _decimal_context()
    A = decimal.Decimal("".join(format(x, "024d") for x in reversed(a)))
    B = decimal.Decimal("".join(format(x, "024d") for x in reversed(b)))
    text = str(ctx.multiply(A, B)).rjust(length * 24, "0")
    top = len(text)
    out = [int(text[top - (i + 1) * 24: top - i * 24]) % MOD for i in range(length)]
return out[:limit] if limit is not None else out

```

```

def inverse(a, n):
    """First n coefficients of 1/a (a[0] != 0)."""
    g = [pow(a[0], MOD - 2, MOD)]
    m = 1
    while m < n:
        m *= 2
        t = multiply(a[:m], g, m)
        t = [(-x) % MOD for x in t]
        t[0] = (t[0] + 2) % MOD
        g = multiply(g, t, m)
    return g[:n]

```

```

def log(a, n):
    """First n coefficients of log(a) with a[0] = 1."""
    if n == 1:
        return [0]
    da = [a[i] * i % MOD for i in range(1, min(len(a), n))]
    q = multiply(da, inverse(a, n), n - 1)
    q += [0] * (n - 1 - len(q))
    inv = [0, 1] + [0] * (n - 1)
    for i in range(2, n + 1):
        inv[i] = (MOD - (MOD // i) * inv[MOD % i] % MOD) % MOD
    return [0] + [q[i - 1] * inv[i] % MOD for i in range(1, n)]

```

```

def exp(a, n):

```

```

"""First n coefficients of exp(a) with a[0] = 0."""
g = [1]
m = 1
while m < n:
    m *= 2
    lg = log(g + [0] * (m - len(g)), m)
    t = [(a[i] if i < len(a) else 0) - lg[i]) % MOD for i in range(m)]
    t[0] = (t[0] + 1) % MOD
    g = multiply(g, t, m)
return g[:n]

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, S = data[0], data[1]
    count = [0] * (S + 1)
    for w in data[2:2 + n]:
        if w <= S:
            count[w] += 1
    inv = [0, 1] + [0] * S
    for i in range(2, S + 1):
        inv[i] = (MOD - (MOD // i) * inv[MOD % i] % MOD) % MOD
    # log prod 1/(1 - x^w) = sum_w count[w] sum_k x^(wk) / k
    L = [0] * (S + 1)
    for w in range(1, S + 1):
        cw = count[w]
        if cw:
            k = 1
            for e in range(w, S + 1, w):
                L[e] += cw * inv[k]
                k += 1
    L = [x % MOD for x in L]
    P = exp(L, S + 1)
    print(" ".join(map(str, P[1:S + 1])))

if __name__ == "__main__":
    main()

```

35. Staircase Moments

Euclid on lattice points

Let F , G , H be the sums of q , $x \cdot q$, and q^2 with $q = \text{floor}((a \cdot x + b) / c)$. If $a \geq c$ or $b \geq c$, write $a = A \cdot c + a'$ and $b = B \cdot c + b'$. Then $q = A \cdot x + B + q'$, and the moments of q' together with closed forms for $\sum x^j$ give the new values. Otherwise, count the lattice points under the line by rows instead of columns: with $m = \text{floor}((a \cdot n + b) / c)$,

$$F = n \cdot m - F', \quad G = (m \cdot n \cdot (n+1) - H' - F') / 2, \quad H = n \cdot m \cdot (m+1) - 2G' - 2F' - F,$$

where (F', G', H') are the moments for $(c, c - b - 1, a, m - 1)$. The pair (a, c) becomes $(c \bmod a, a)$ exactly as in Euclid's algorithm.

Correctness. $\text{sum}_x q(x)$ counts the pairs (x, y) with $1 \leq y \leq q(x)$, and $y \leq q(x)$ iff $x > \text{floor}((c y - b - 1) / a)$. Summing the same set over y gives the swapped recursion, and weighting by x or by $2y - 1$ gives G and H .

Complexity and traps. $O(\log \max(a, c))$ per query. Reduce n modulo p only inside polynomial closed forms. The product $a n + b$ needs 128 bits in C++.

C++

```
#include <bits/stdc++.h>
using namespace std;

// F, G, H = sums over x = 0..n of q, x q, q^2 with q = floor((a x + b) / c).
// Reduce a, b below c (pulling out the linear parts), then exchange the roles
// of x and q: the lattice points under the line are counted by rows instead
// of columns, which turns (a, c) into (c, a) exactly like Euclid's algorithm.

typedef long long ll;
const ll MOD = 998244353, INV2 = (MOD + 1) / 2;
ll INV6;

struct Triple {
    ll f, g, h;
};

ll power(ll b, ll e) {
    ll r = 1;
    for (b %= MOD; e; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

Triple solve(ll a, ll b, ll c, ll n) {
    ll nm = n % MOD;
    ll s1 = nm * ((nm + 1) % MOD) % MOD * INV2 % MOD;
    ll s2 = nm * ((nm + 1) % MOD) % MOD * ((2 * nm + 1) % MOD) % MOD * INV6 % MOD;
    if (a == 0) {
        ll q = b / c % MOD, n1 = (nm + 1) % MOD;
        return {n1 * q % MOD, q * s1 % MOD, n1 * q % MOD * q % MOD};
    }
    if (a >= c || b >= c) {
        ll A = a / c % MOD, B = b / c % MOD;
        Triple t = solve(a % c, b % c, c, n);
        ll n1 = (nm + 1) % MOD;
        Triple r;
        r.f = (t.f + A * s1 + B * n1) % MOD;
        r.g = (t.g + A * s2 + B * s1) % MOD;
        r.h = (t.h + A * A % MOD * s2 + B * B % MOD * n1 + 2 * A % MOD * B % MOD * s1 + 2 * B * t.f
```

```

        return r;
    }
    ll m = (ll)((__int128)a * n + b) / c;
    if (m == 0) return {0, 0, 0};
    Triple t = solve(c, c - b - 1, a, m - 1);
    ll mm = m % MOD;
    Triple r;
    r.f = ((nm * mm - t.f) % MOD + MOD) % MOD;
    r.g = ((mm * nm % MOD * ((nm + 1) % MOD) - t.h - t.f) % MOD + 2 * MOD) % MOD * INV2 % MOD;
    r.h = ((nm * mm % MOD * ((mm + 1) % MOD) - 2 * t.g - 2 * t.f - r.f) % MOD + 5 * MOD) % MOD;
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    INV6 = power(6, MOD - 2);
    int t;
    cin >> t;
    string out;
    while (t--) {
        ll n, a, b, c;
        cin >> n >> a >> b >> c;
        Triple r = solve(a, b, c, n);
        out += to_string(r.f) + ' ' + to_string(r.g) + ' ' + to_string(r.h) + '\n';
    }
    cout << out;
}

```

Python

```

import sys

MOD = 998244353
INV2 = (MOD + 1) // 2
INV6 = pow(6, MOD - 2, MOD)

def solve(a, b, c, n):
    """(F, G, H) = sums over x = 0..n of q, x*q, q*q with q = (a x + b) // c.

    Reduce a, b below c, then swap the roles of x and q (the lattice points
    under the line counted by columns instead of rows), like Euclid."""
    # Iterative version of the recursion: record frames, then unwind.
    frames = []
    while True:
        if a == 0:
            q = b // c % MOD
            n1 = (n + 1) % MOD
            base = (n1 * q % MOD, q * (n % MOD) % MOD * n1 % MOD * INV2 % MOD, n1 * q % MOD * q % MOD)
            frames.append((q, n1, base))
            b, c = c - b, b
            a, b = b, a
            n = n - 1
            if n < 0:
                break

```

```

        break
    if a >= c or b >= c:
        frames.append((0, a // c, b // c, n))
        a %= c
        b %= c
        continue
    m = (a * n + b) // c
    if m == 0:
        base = (0, 0, 0)
        break
    frames.append((1, m, None, n))
    a, b, c, n = c, c - b - 1, a, m - 1
F, G, H = base
for kind, x, y, n in reversed(frames):
    nm = n % MOD
    s1 = nm * (nm + 1) % MOD * INV2 % MOD
    s2 = nm * (nm + 1) % MOD * (2 * nm + 1) % MOD * INV6 % MOD
    if kind == 0:
        A = x % MOD
        B = y % MOD
        f = (F + A * s1 + B * (nm + 1)) % MOD
        g = (G + A * s2 + B * s1) % MOD
        h = (H + A * A % MOD * s2 + B * B % MOD * (nm + 1) + 2 * A * B % MOD * s1
            + 2 * B * F + 2 * A * G) % MOD
    else:
        m = x % MOD
        f = (nm * m - F) % MOD
        g = (m * nm % MOD * (nm + 1) - H - F) % MOD * INV2 % MOD
        h = (nm * m % MOD * (m + 1) - 2 * G - 2 * F - f) % MOD
    F, G, H = f, g, h
return F, G, H

```

```

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    t = data[0]
    out = []
    for i in range(t):
        n, a, b, c = data[1 + 4 * i:5 + 4 * i]
        out.append("%d %d %d" % solve(a, b, c, n))
    sys.stdout.write("\n".join(out) + "\n")

```

```

if __name__ == "__main__":
    main()

```


36. Binomial Ledger

Prime powers, then CRT

Factor $m = \prod p^e$. Modulo p^e , write $n! = p^{\{v(n)\}} u(n)$, where v is Legendre's valuation and $u(n)$ is coprime to p . Grouping the factors of $n!$ by blocks of length p^e gives

$$u(n) = \text{pre}[p^e]^{\lfloor n/p^e \rfloor} * \text{pre}[n \bmod p^e] * u(\lfloor n/p \rfloor),$$

with $\text{pre}[j]$ the product of the integers $\leq j$ coprime to p (modulo p^e). Then $C(n, k) = p^{\{v(n) - v(k) - v(n-k)\}} u(n) / (u(k) u(n-k))$, which is 0 once the valuation reaches e . Combine the prime powers by CRT.

Correctness. The integers in each full block of length p^e , with the multiples of p removed, are a reduced residue system. The removed multiples of p contribute $p^{\lfloor n/p \rfloor} \lfloor n/p \rfloor!$, which is where the recursion comes from.

Complexity and traps. Tables of total size at most m , and $O(\sum_p \log_p n)$ per query. $\text{pre}[p^e]$ is ± 1 (Wilson's theorem), so its power costs nothing. When $k > n$ the answer is 0. When $m = 1$ every answer is 0.

C++

```
#include <bits/stdc++.h>
using namespace std;

// C(n, k) mod p^e: write n! = p^{\{v_p(n!)\}} * u(n), where u(n) (the factorial
// with every factor p removed) satisfies the generalized Lucas recursion
// u(n) = full^{\lfloor n/p^e \rfloor} * pre[n mod p^e] * u(\lfloor n/p \rfloor), with pre[j] the
// product of integers <= j coprime to p. Combine the prime powers by CRT.

typedef long long ll;

ll egcdInverse(ll a, ll m) { // inverse of a modulo m, gcd(a, m) = 1
    ll old_r = a % m, r = m, old_s = 1, s = 0;
    while (r) {
        ll q = old_r / r;
        tie(old_r, r) = make_pair(r, old_r - q * r);
        tie(old_s, s) = make_pair(s, old_s - q * s);
    }
    return (old_s % m + m) % m;
}

struct PrimePower {
    ll p, e, pe, full;
    vector<ll> pre;
    PrimePower(ll p, ll e) : p(p), e(e) {
        pe = 1;
        for (int i = 0; i < e; i++) pe *= p;
        pre.assign(pe + 1, 1);
        for (ll i = 1; i <= pe; i++) pre[i] = i % p ? pre[i - 1] * i % pe : pre[i - 1];
    }
};
```

```

        full = pre[pe];
    }
    ll unitFactorial(ll x) const {
        ll result = 1;
        while (x) {
            ll qq = x / pe, r = x % pe;
            if (full != 1 && (qq & 1)) result = result * full % pe;
            result = result * pre[r] % pe;
            x /= p;
        }
        return result;
    }
    ll legendre(ll x) const {
        ll v = 0;
        while (x) x /= p, v += x;
        return v;
    }
    ll binom(ll n, ll k) const {
        ll v = legendre(n) - legendre(k) - legendre(n - k);
        if (v >= e) return 0;
        ll top = unitFactorial(n), bottom = unitFactorial(k) * unitFactorial(n - k) % pe;
        ll r = top * egcdInverse(bottom, pe) % pe;
        for (int i = 0; i < v; i++) r = r * p % pe;
        return r;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll m;
    int q;
    cin >> m >> q;
    vector<PrimePower> parts;
    {
        ll x = m;
        for (ll p = 2; p * p <= x; p++)
            if (x % p == 0) {
                int e = 0;
                while (x % p == 0) x /= p, e++;
                parts.emplace_back(p, e);
            }
        if (x > 1) parts.emplace_back(x, 1);
    }
    vector<ll> coef;
    for (auto& part : parts) {
        ll Mi = m / part.pe;
        coef.push_back(part.pe > 1 ? Mi * egcdInverse(Mi % part.pe, part.pe) % m : 0);
    }
    string out;
    while (q--) {

```

```

    ll n, k;
    cin >> n >> k;
    ll total = 0;
    if (k <= n && m > 1)
        for (size_t i = 0; i < parts.size(); i++)
            total = (total + (__int128)parts[i].binom(n, k) * coef[i]) % m;
    out += to_string(total) + '\n';
}
cout << out;
}

```

Python

```

import sys

def factorize(m):
    out = []
    p = 2
    while p * p <= m:
        if m % p == 0:
            e = 0
            while m % p == 0:
                m //= p
                e += 1
            out.append((p, e))
        p += 1
    if m > 1:
        out.append((m, 1))
    return out

class PrimePower:
    """C(n, k) mod p^e by removing every factor p from the factorials."""

    def __init__(self, p, e):
        self.p = p
        self.e = e
        self.pe = p ** e
        pe = self.pe
        pre = [1] * (pe + 1)
        for i in range(1, pe + 1):
            pre[i] = pre[i - 1] * i % pe if i % p else pre[i - 1]
        self.pre = pre
        self.full = pre[pe] # +-1 by Wilson's theorem for prime powers

    def unit_factorial(self, x):
        """x! with all factors p removed, modulo p^e."""
        pe, p, pre, full = self.pe, self.p, self.pre, self.full
        result = 1

```

```

    while x:
        q, r = divmod(x, pe)
        if full != 1 and q & 1:
            result = result * full % pe
        result = result * pre[r] % pe
        x //= p
    return result

def legendre(self, x):
    p = self.p
    v = 0
    while x:
        x //= p
        v += x
    return v

def binom(self, n, k):
    v = self.legendre(n) - self.legendre(k) - self.legendre(n - k)
    if v >= self.e:
        return 0
    pe = self.pe
    top = self.unit_factorial(n)
    bottom = self.unit_factorial(k) * self.unit_factorial(n - k) % pe
    return top * pow(bottom, -1, pe) % pe * self.p ** v % pe

def main():
    data = sys.stdin.buffer.read().split()
    m, q = int(data[0]), int(data[1])
    parts = [PrimePower(p, e) for p, e in factorize(m)]
    # CRT coefficients: x = sum r_i * M_i * (M_i^{-1} mod m_i)
    coef = []
    for part in parts:
        Mi = m // part.pe
        coef.append(Mi * pow(Mi % part.pe, -1, part.pe) % m if part.pe > 1 else 0)
    out = []
    for i in range(q):
        n = int(data[2 + 2 * i])
        k = int(data[3 + 2 * i])
        if k > n or m == 1:
            out.append(0)
            continue
        total = 0
        for part, cf in zip(parts, coef):
            total += part.binom(n, k) * cf
        out.append(total % m)
    sys.stdout.write("\n".join(map(str, out)) + "\n")

if __name__ == "__main__":
    main()

```

37. Rabbit Cycle

A matrix order with a provable multiple

$F(i + L) = F(i)$ for all i iff $M^L = I$ for $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$ modulo m . By CRT the period is the lcm over the prime powers of m . Modulo a prime p , the roots of $x^2 - x - 1$ lie in F_p when $p \equiv \pm 1 \pmod{5}$ (so $M^{p-1} = I$), are swapped by Frobenius when $p \equiv \pm 2 \pmod{5}$ (so $M^{2(p+1)} = I$), and coincide for $p = 5$ (period 20). Lifting to p^e multiplies the order by at most p^{e-1} . Factor this multiple L with Pollard's rho, then divide out each prime while $M^{L/r}$ is still the identity.

Correctness. The candidate is a multiple of the true order. Removing primes while the power stays the identity yields the exact order, since the order divides every exponent E with $M^E = I$. The method needs no unproved conjecture (such as Wall's).

Complexity and traps. Factoring numbers up to 10^{18} with Pollard's rho, plus $O(\log^2)$ matrix powers per prime. The answer can exceed m (up to $6m$). Matrix products need 128-bit multiplications in C++.

C++

```
#include <bits/stdc++.h>
using namespace std;

// The Pisano period of m is the order of F = [[1,1],[1,0]] modulo m, and by
// CRT it is the lcm of the orders modulo each prime power p^e. Over F_p the
// roots of x^2 - x - 1 lie in F_p (p = +-1 mod 5), in F_{p^2} with the
// Frobenius swapping them (p = +-2 mod 5), or coincide (p = 5); this makes
// p^{e-1} * (p-1), p^{e-1} * 2(p+1), or 5^{e-1} * 20 a multiple of the order.
// Factor that multiple with Pollard's rho and strip prime factors while F to
// the reduced power is still the identity.

typedef unsigned long long u64;
typedef __uint128_t u128;

u64 mulmod(u64 a, u64 b, u64 m) { return (u128)a * b % m; }
u64 powmod(u64 b, u64 e, u64 m) {
    u64 r = 1 % m;
    for (b %= m; e >= 1; b = mulmod(b, b, m))
        if (e & 1) r = mulmod(r, b, m);
    return r;
}

bool isPrime(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
        if (n % p == 0) return n == p;
    u64 d = n - 1;
    int s = 0;
    while (d % 2 == 0) d /= 2, s++;
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = powmod(a, d, n);
```

```

    if (x == 1 || x == n - 1) continue;
    bool composite = true;
    for (int i = 1; i < s && composite; i++) {
        x = mulmod(x, x, n);
        if (x == n - 1) composite = false;
    }
    if (composite) return false;
}
return true;
}

mt19937_64 rng(20261001);

u64 rho(u64 n) {
    if (n % 2 == 0) return 2;
    while (true) {
        u64 c = rng() % (n - 1) + 1, y = rng() % n, g = 1, q = 1, x = y, ys = y;
        auto f = [&](u64 v) { return (mulmod(v, v, n) + c) % n; };
        for (u64 r = 1; g == 1; r <= 1) {
            x = y;
            for (u64 i = 0; i < r; i++) y = f(y);
            for (u64 k = 0; k < r && g == 1; k += 128) {
                ys = y;
                for (u64 i = 0; i < min<u64>(128, r - k); i++) {
                    y = f(y);
                    q = mulmod(q, x > y ? x - y : y - x, n);
                }
                g = __gcd(q, n);
            }
        }
        if (g == n) {
            g = 1;
            while (g == 1) {
                ys = f(ys);
                g = __gcd(x > ys ? x - ys : ys - x, n);
            }
        }
        if (g != n) return g;
    }
}

void factor(u64 n, map<u64, int>& out) {
    if (n == 1) return;
    if (isPrime(n)) {
        out[n]++;
        return;
    }
    for (u64 p : {2, 3, 5, 7, 11, 13})
        if (n % p == 0) {
            out[p]++;
            factor(n / p, out);
        }
}

```

```

        return;
    }
    u64 d = rho(n);
    factor(d, out);
    factor(n / d, out);
}

typedef array<u64, 4> Mat;
Mat matmul(const Mat& x, const Mat& y, u64 m) {
    return {(mulmod(x[0], y[0], m) + mulmod(x[1], y[2], m)) % m, (mulmod(x[0], y[1], m) + mulmod(x[1], y[2], m)) % m, (mulmod(x[2], y[0], m) + mulmod(x[3], y[2], m)) % m, (mulmod(x[2], y[1], m) + mulmod(x[3], y[2], m)) % m};
}
bool isIdentityPower(u64 e, u64 m) {
    Mat r = {1 % m, 0, 0, 1 % m}, b = {1 % m, 1 % m, 1 % m, 0};
    for (; e >= 1, b = matmul(b, b, m))
        if (e & 1) r = matmul(r, b, m);
    return r == Mat{1 % m, 0, 0, 1 % m};
}

u64 periodPrimePower(u64 p, int e) {
    u64 mod = 1, pk = 1;
    for (int i = 0; i < e; i++) mod *= p;
    for (int i = 1; i < e; i++) pk *= p;
    u64 base = p == 5 ? 20 : (p % 5 == 1 || p % 5 == 4) ? p - 1 : 2 * (p + 1);
    map<u64, int> fac;
    factor(base, fac);
    if (e > 1) fac[p] += e - 1;
    u64 L = base * pk;
    for (auto& [r, cnt] : fac)
        while (L % r == 0 && isIdentityPower(L / r, mod)) L /= r;
    return L;
}

int main() {
    int q;
    cin >> q;
    while (q--) {
        u64 m;
        cin >> m;
        map<u64, int> fac;
        factor(m, fac);
        u64 answer = 1;
        for (auto& [p, e] : fac) {
            u64 L = periodPrimePower(p, e);
            answer = answer / __gcd(answer, L) * L;
        }
        cout << answer << '\n';
    }
}

```

Python

```
import math
import random
import sys

_rng = random.Random(20261001)

def is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for a in (2, 325, 9375, 28178, 450775, 9780504, 1795265022):
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True

def rho(n):
    if n % 2 == 0:
        return 2
    while True:
        c = _rng.randrange(1, n)
        y = _rng.randrange(n)
        g = 1
        q = 1
        r = 1
        x = y
        ys = y
        while g == 1:
            x = y
            for _ in range(r):
                y = (y * y + c) % n
            k = 0
```



```

        while k < r and g == 1:
            ys = y
            for _ in range(min(128, r - k)):
                y = (y * y + c) % n
                q = q * abs(x - y) % n
                g = math.gcd(q, n)
                k += 128
            r *= 2
        if g == n:
            g = 1
            while g == 1:
                ys = (ys * ys + c) % n
                g = math.gcd(abs(x - ys), n)
        if g != n:
            return g

def factor(n, out):
    if n == 1:
        return
    if is_prime(n):
        out[n] = out.get(n, 0) + 1
        return
    for p in (2, 3, 5, 7, 11, 13):
        if n % p == 0:
            out[p] = out.get(p, 0) + 1
            factor(n // p, out)
            return
    d = rho(n)
    factor(d, out)
    factor(n // d, out)

def fib_matrix_power(e, mod):
    """[[1,1],[1,0]]^e modulo mod, as (a, b, c, d)."""
    ra, rb, rc, rd = 1, 0, 0, 1
    a, b, c, d = 1, 1, 1, 0
    while e:
        if e & 1:
            ra, rb, rc, rd = (ra * a + rb * c) % mod, (ra * b + rb * d) % mod, (rc * a + rd * c) % mod, (rc * b + rd * d) % mod
            a, b, c, d = (a * a + b * c) % mod, (a * b + b * d) % mod, (c * a + d * c) % mod, (c * b + d * d) % mod
        e >>= 1
    return ra, rb, rc, rd

def period_prime_power(p, e):
    """Order of the Fibonacci matrix modulo p^e.

    p^(e-1) * L is a multiple of it, with L = p - 1, 2(p + 1), or 20 for p = 5
    by the Frobenius action on the roots of x^2 - x - 1; strip prime factors
    while the matrix power is still the identity."""

```

```

mod = p ** e
if p == 5:
    base = 20
elif p % 5 in (1, 4):
    base = p - 1
else:
    base = 2 * (p + 1)
fac = {}
factor(base, fac)
if e > 1:
    fac[p] = fac.get(p, 0) + e - 1
L = base * p ** (e - 1)
for r in fac:
    while L % r == 0 and fib_matrix_power(L // r, mod) == (1, 0, 0, 1):
        L //= r
return L

def main():
    data = sys.stdin.read().split()
    q = int(data[0])
    out = []
    for i in range(q):
        m = int(data[1 + i])
        fac = {}
        factor(m, fac)
        answer = 1
        for p, e in fac.items():
            L = period_prime_power(p, e)
            answer = answer // math.gcd(answer, L) * L
        out.append(answer)
    print("\n".join(map(str, out)))

if __name__ == "__main__":
    main()

```

38. Exclusive Range Ledger

Subset XORs form a vector space

The XORs of subsets of $a[1..r]$ form a $\text{GF}(2)$ -space V , and each element of V is produced by exactly $2^{\{\text{len} - \dim V\}}$ subsets (a coset of the kernel). So the answer is $2^{\{\text{len} - \text{rank}\}}$ when x is in V , and 0 otherwise.

Process queries offline by r , keeping a basis indexed by leading bit in which each slot stores the **newest** possible vector. An incoming element swaps itself into a slot whenever it is newer than the stored vector, and the older vector continues down. Then the stored vectors with position ≥ 1 form an echelon basis of the span of $a[1..r]$.

Correctness. By induction on r , the vectors with position ≥ 1 span exactly the elements of $a[1..r]$ and are independent. The swap keeps every slot as recent as possible, which is the standard exchange argument.

Complexity and traps. $O((n + q) \cdot B)$ with $B = 30$. The empty subset counts, so $x = 0$ always has an answer of at least 1.

C++

```
#include <bits/stdc++.h>
using namespace std;

// The XORs of subsets of a[l..r] form a vector space V over GF(2); every
// element of V is hit by exactly  $2^{\{\text{len} - \text{dim } V\}}$  subsets. Sweep r and keep a
// prefix basis in which each leading bit holds the most recent vector
// possible: an incoming element is swapped in whenever it is newer than the
// stored one. Then the stored vectors with position  $\geq l$  form an echelon
// basis of V, giving both dim V and membership of x.

const long long MOD = 1000000007;
const int BITS = 30;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (auto& v : a) cin >> v;
    vector<array<int, 4>> queries(q);
    for (int i = 0; i < q; i++) {
        int l, r, x;
        cin >> l >> r >> x;
        queries[i] = {r, l, x, i};
    }
    sort(queries.begin(), queries.end());
    vector<long long> pow2(n + 1, 1);
    for (int i = 0; i < n; i++) pow2[i + 1] = pow2[i] * 2 % MOD;
    array<int, BITS> basis{}, where{};
    vector<long long> answer(q);
    int done = 0;
    for (auto [r, l, x, idx] : queries) {
        while (done < r) {
            int v = a[done], p = ++done;
            for (int b = BITS - 1; b >= 0; b--) {
                if (!(v >> b & 1)) continue;
                if (!basis[b]) {
                    basis[b] = v;
                    where[b] = p;
                    break;
                }
            }
            if (where[b] < p) swap(basis[b], v), swap(where[b], p);
            v ^= basis[b];
        }
        answer[idx] = pow2[r - l + 1];
    }
}
```

```

    }
    int rank = 0;
    for (int b = BITS - 1; b >= 0; b--)
        if (basis[b] && where[b] >= 1) {
            rank++;
            if (x >> b & 1) x ^= basis[b];
        }
    answer[idx] = x ? 0 : pow2[r - 1 + 1 - rank];
}
string out;
for (long long v : answer) out += to_string(v) + '\n';
cout << out;
}

```

Python

```

import sys

MOD = 1_000_000_007
BITS = 30

def main():
    data = sys.stdin.buffer.read().split()
    n, q = int(data[0]), int(data[1])
    a = data[2:2 + n]
    queries = []
    pos = 2 + n
    for i in range(q):
        l = int(data[pos])
        r = int(data[pos + 1])
        x = int(data[pos + 2])
        pos += 3
        queries.append((r, l, x, i))
    queries.sort()
    pow2 = [1] * (n + 1)
    for i in range(n):
        pow2[i + 1] = pow2[i] * 2 % MOD
    # basis[b]: vector with leading bit b; where[b]: its (latest) position.
    # Inserting a newer element swaps it into the basis, so the vectors with
    # position >= l always span exactly a[l..r].
    basis = [0] * BITS
    where = [0] * BITS
    answer = [0] * q
    done = 0
    for r, l, x, idx in queries:
        while done < r:
            done += 1
            v = int(a[done - 1])
            p = done

```

```

        for b in range(BITS - 1, -1, -1):
            if not v >> b & 1:
                continue
            if not basis[b]:
                basis[b] = v
                where[b] = p
                break
            if where[b] < p:
                basis[b], v = v, basis[b]
                where[b], p = p, where[b]
            v ^= basis[b]
rank = 0
for b in range(BITS - 1, -1, -1):
    if basis[b] and where[b] >= 1:
        rank += 1
        if x >> b & 1:
            x ^= basis[b]
    answer[idx] = 0 if x else pow2[r - 1 + 1 - rank]
sys.stdout.write("\n".join(map(str, answer)) + "\n")

if __name__ == "__main__":
    main()

```

39. Divisor Echo Power

A derivation for Dirichlet convolution

Let $\Omega(n)$ count prime factors with multiplicity, and set $D(h)(n) = \Omega(n) \cdot h(n)$. Because $\Omega(xy) = \Omega(x) + \Omega(y)$, $D(a \# b) = D(a) \# b + a \# D(b)$. For $g = f^k$ this gives $f \# D(g) = k \cdot g \# D(f)$. Comparing the values at n , and using $f(1) = 1$ and $\Omega(1) = 0$,

$$\Omega(n) \cdot g(n) = \sum_{d \mid n, d < n} g(d) \cdot f(n/d) \cdot (k \cdot \Omega(n/d) - \Omega(d)).$$

Push each finished $g(d)$ to its multiples.

Correctness. The recurrence determines $g(n)$ from smaller divisors because $\Omega(n) \geq 1$ is invertible modulo the prime. The coefficients of f^k are polynomials in k , so k may be reduced modulo the prime. For $k = -1$ the result is the Dirichlet inverse.

Complexity and traps. Harmonic $O(N \log N)$. With $f = 1$, $k = 2$ gives the divisor count and $k = -1$ gives the Möbius function.

C++

```

#include <bits/stdc++.h>
using namespace std;

// D(h)(n) = Omega(n) h(n) (prime factors counted with multiplicity) is a

```

```
// derivation of Dirichlet convolution:  $D(a*b) = D(a)*b + a*D(b)$ . For
//  $g = f^k$  this gives  $f * D(g) = k g * D(f)$ , and comparing values at  $n$ :
//  $\Omega(n) g(n) = \sum_{d | n, d < n} g(d) f(n/d) (k \Omega(n/d) - \Omega(d))$ .
// Push each finished  $g(d)$  to its multiples:  $O(N \log N)$ ,  $k$  only mod  $p$ .
```

```
typedef long long ll;
const ll MOD = 998244353;

ll power(ll b, ll e) {
    ll r = 1;
    for (b %= MOD; e; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int N;
    ll k;
    cin >> N >> k;
    k %= MOD;
    vector<ll> f(N + 1, 0);
    for (int i = 1; i <= N; i++) cin >> f[i], f[i] %= MOD;
    vector<int> omega(N + 1, 0);
    for (int p = 2; p <= N; p++)
        if (!omega[p])
            for (ll pk = p; pk <= N; pk *= p)
                for (ll m = pk; m <= N; m += pk) omega[m]++;
    vector<ll> inv(64, 1);
    for (int i = 1; i < 64; i++) inv[i] = power(i, MOD - 2);
    vector<ll> acc(N + 1, 0), g(N + 1, 0);
    g[1] = 1;
    for (int d = 1; d <= N; d++) {
        if (d > 1) g[d] = acc[d] % MOD * inv[omega[d]] % MOD;
        if (!g[d]) continue;
        for (ll n = 2LL * d, m = 2; n <= N; n += d, m++) {
            ll coef = (k * omega[m] - omega[d] + MOD) % MOD;
            acc[n] = (acc[n] + g[d] * f[m] % MOD * coef) % MOD;
        }
    }
    string out;
    for (int n = 1; n <= N; n++) {
        out += to_string(g[n]);
        out += n < N ? ' ' : '\n';
    }
    cout << out;
}
```

Python

```
import sys

MOD = 998244353

def main():
    data = sys.stdin.buffer.read().split()
    N, k = int(data[0]), int(data[1]) % MOD
    f = [0] + [int(x) % MOD for x in data[2:2 + N]]
    # Omega(n): number of prime factors counted with multiplicity.
    omega = [0] * (N + 1)
    for p in range(2, N + 1):
        if omega[p] == 0:
            pk = p
            while pk <= N:
                for m in range(pk, N + 1, pk):
                    omega[m] += 1
                pk *= p
    inv = [0, 1] + [0] * 64
    for i in range(2, 66):
        inv[i] = pow(i, MOD - 2, MOD)
    # D(h)(n) = Omega(n) h(n) is a derivation of Dirichlet convolution, so
    # g = f^k satisfies f * D(g) = k g * D(f). Reading this at n isolates
    # Omega(n) g(n) in terms of g(d), d | n, d < n.
    weight = [k * omega[m] % MOD * f[m] % MOD for m in range(N + 1)] # k Omega(m) f(m)
    acc = [0] * (N + 1)
    g = [0] * (N + 1)
    g[1] = 1
    for d in range(1, N + 1):
        if d > 1:
            g[d] = acc[d] % MOD * inv[omega[d]] % MOD
        gd = g[d]
        if not gd or 2 * d > N:
            continue
        od = omega[d]
        top = N // d
        # contribution to n = d*m for m = 2..top: g(d) f(m) (k Omega(m) - Omega(d));
        # sums stay exact (Python integers) and are reduced when g(n) is read.
        acc[2 * d::d] = [x + gd * (w - od * fm) for x, w, fm in zip(acc[2 * d::d], weight[2:top + 1])]
    sys.stdout.write(" ".join(map(str, g[1:]))) + "\n"

if __name__ == "__main__":
    main()
```

40. Root Census Modulo p

The distinct roots are a gcd

Over F_p , $x^p - x = \prod \text{ over } a \text{ of } (x - a)$. So $\gcd(f, x^p - x)$ is the product of $(x - a)$ over the distinct roots of f , and its degree is the answer. Compute $x^p \bmod f$ by square-and-multiply. Each reduction modulo f uses the precomputed series inverse of f reversed (Barrett for polynomials), so a step costs a few polynomial products. Finish with Euclid's algorithm.

Correctness. The roots of $x^p - x$ are exactly all elements of F_p , each simple, so the gcd keeps each root of f once. The zero polynomial is the special case in which every residue is a root.

Complexity and traps. $O(d^2 \log p)$ with schoolbook products (C++ accumulates 128-bit sums lazily), or Kronecker substitution in Python. Normalize the coefficients modulo p first; the leading coefficient may vanish.

C++

```
#include <bits/stdc++.h>
using namespace std;

//  $x^p - x = \prod \text{ over } a \text{ in } F_p \text{ of } (x - a)$ , so  $\gcd(f, x^p - x)$  is the product
// of  $(x - a)$  over the distinct roots  $a$  of  $f$ , and its degree is the answer.
// Compute  $h = x^p \bmod f$  by square-and-multiply; polynomial remainders use a
// precomputed inverse of reversed  $f$  (Barrett), so every step is three
// polynomial products. Products accumulate 128-bit sums lazily.

typedef unsigned long long u64;
typedef __uint128_t u128;
u64 P;

u64 mulmod(u64 a, u64 b) { return (u128)a * b % P; }
u64 powmod(u64 b, u64 e) {
    u64 r = 1;
    for (b %= P; e; e >>= 1, b = mulmod(b, b))
        if (e & 1) r = mulmod(r, b);
    return r;
}
u64 sub(u64 a, u64 b) { return a >= b ? a - b : a + P - b; }

typedef vector<u64> Poly;

// Each product is  $< 2^{120}$ , so 256 of them fit in an unsigned 128-bit sum.
Poly mul(const Poly& a, const Poly& b, size_t limit = SIZE_MAX) {
    if (a.empty() || b.empty()) return {};
    size_t n = min(limit, a.size() + b.size() - 1);
    Poly out(n);
    for (size_t k = 0; k < n; k++) {
        size_t lo = k >= b.size() ? k - b.size() + 1 : 0, hi = min(k, a.size() - 1);
        u128 acc = 0;
        u64 res = 0;
```



```

    int cnt = 0;
    for (size_t i = lo; i <= hi; i++) {
        acc += (u128)a[i] * b[k - i];
        if (++cnt == 256) {
            res = (res + (u64)(acc % P)) % P;
            acc = 0;
            cnt = 0;
        }
    }
    out[k] = (res + (u64)(acc % P)) % P;
}
return out;
}

Poly seriesInverse(const Poly& a, size_t n) {
    Poly g = {powmod(a[0], P - 2)};
    for (size_t k = 1; k < n; k++) {
        k *= 2;
        Poly t = mul(Poly(a.begin(), a.begin() + min(k, a.size())), g, k);
        for (auto& x : t) x = x ? P - x : 0;
        t[0] = (t[0] + 2) % P;
        g = mul(g, t, k);
    }
    g.resize(n);
    return g;
}

int main() {
    int d;
    cin >> P >> d;
    Poly f(d + 1);
    for (auto& c : f) cin >> c, c %= P;
    while (!f.empty() && f.back() == 0) f.pop_back();
    if (f.empty()) {
        cout << P << '\n';
        return 0;
    }
    if (f.size() == 1) {
        cout << 0 << '\n';
        return 0;
    }
    u64 leadInv = powmod(f.back(), P - 2);
    for (auto& c : f) c = mulmod(c, leadInv);
    size_t m = f.size() - 1;
    Poly revF(f.rbegin(), f.rend());
    Poly invRev = seriesInverse(revF, max<size_t>(1, m));

    auto reduce = [&](Poly A) {
        while (!A.empty() && A.back() == 0) A.pop_back();
        if (A.size() <= m) {
            A.resize(m, 0);

```

```

        return A;
    }
    size_t n = A.size() - 1, qlen = n - m + 1;
    Poly revA(A.rbegin(), A.rbegin() + qlen);
    Poly qRev = mul(revA, Poly(invRev.begin(), invRev.begin() + qlen), qlen);
    qRev.resize(qlen, 0);
    Poly Q(qRev.rbegin(), qRev.rend());
    Poly prod = mul(Q, f, m);
    Poly R(m);
    for (size_t i = 0; i < m; i++) R[i] = sub(A[i], i < prod.size() ? prod[i] : 0);
    return R;
};

Poly h(m, 0);
h[0] = 1;
for (int bit = 63 - __builtin_clzll(P); bit >= 0; bit--) {
    h = reduce(mul(h, h));
    if (P >> bit & 1) {
        Poly shifted(h.size() + 1, 0);
        copy(h.begin(), h.end(), shifted.begin() + 1);
        h = reduce(shifted);
    }
}
Poly g = h;
if (g.size() < 2) g.resize(2, 0);
g[1] = sub(g[1], 1);
Poly a = f, b = g;
while (!b.empty() && b.back() == 0) b.pop_back();
while (!b.empty()) {
    u64 invLead = powmod(b.back(), P - 2);
    size_t db = b.size() - 1;
    for (size_t i = a.size(); i-- > db;) {
        u64 c = mulmod(a[i], invLead);
        if (!c) continue;
        size_t shift = i - db;
        for (size_t j = 0; j <= db; j++) a[shift + j] = sub(a[shift + j], mulmod(c, b[j]));
    }
    a.resize(min(a.size(), db));
    while (!a.empty() && a.back() == 0) a.pop_back();
    swap(a, b);
}
cout << a.size() - 1 << '\n';
}

```

Python

```
import sys
```

```
def main():
```

```

data = list(map(int, sys.stdin.buffer.read().split()))
p, d = data[0], data[1]
f = [c % p for c in data[2:3 + d]]
while f and f[-1] == 0:
    f.pop()
if not f:
    print(p) # the zero polynomial vanishes everywhere
    return
if len(f) == 1:
    print(0)
    return
# Make f monic.
lead_inv = pow(f[-1], p - 2, p)
f = [c * lead_inv % p for c in f]
m = len(f) - 1
slot = (2 * p.bit_length() + m.bit_length() + 8 + 3) // 4 # hex digits per coefficient

def mul(a, b):
    """Polynomial product modulo p by Kronecker substitution."""
    if not a or not b:
        return []
    length = len(a) + len(b) - 1
    fmt = "0%dx" % slot
    A = int("".join(format(x, fmt) for x in reversed(a)), 16)
    B = int("".join(format(x, fmt) for x in reversed(b)), 16)
    text = format(A * B, "x").rjust(length * slot, "0")
    top = len(text)
    return [int(text[top - (i + 1) * slot: top - i * slot], 16) % p for i in range(length)]

def series_inverse(a, n):
    g = [pow(a[0], p - 2, p)]
    k = 1
    while k < n:
        k *= 2
        t = mul(a[:k], g)[:k]
        t = [(-x) % p for x in t]
        t[0] = (t[0] + 2) % p
        g = mul(g, t)[:k]
    return g[:n]

rev_f = f[::-1]
inv_rev = series_inverse(rev_f, max(1, m))

def reduce(A):
    """A mod f via a reversed-series quotient (Barrett for polynomials)."""
    while A and A[-1] == 0:
        A.pop()
    n = len(A) - 1
    if n < m:
        return A + [0] * (m - len(A))
    qlen = n - m + 1

```

```

    q_rev = mul(A[::-1][:qlen], inv_rev[:qlen])[:qlen]
    q_rev += [0] * (qlen - len(q_rev))
    Q = q_rev[::-1]
    prod = mul(Q, f)
    return [(A[i] - prod[i]) % p for i in range(m)]

# h = x^p mod f by square-and-multiply.
h = [1] + [0] * (m - 1) if m > 0 else []
for bit in bin(p)[2:]:
    h = reduce(mul, h, h)
    if bit == "1":
        h = reduce([0] + h, # multiply by x
# gcd(f, x^p - x) has exactly the distinct roots of f as its roots.
g = h[:] + [0] * max(0, 2 - len(h))
g[1] = (g[1] - 1) % p
a, b = f[:], g
while b and b[-1] == 0:
    b.pop()
while b:
    # a <- a mod b
    inv_lead = pow(b[-1], p - 2, p)
    a = a[:]
    db = len(b) - 1
    for i in range(len(a) - 1, db - 1, -1):
        c = a[i] * inv_lead % p
        if c:
            shift = i - db
            for j in range(db + 1):
                a[shift + j] = (a[shift + j] - c * b[j]) % p
    a = a[:db]
    while a and a[-1] == 0:
        a.pop()
    a, b = b, a
print(len(a) - 1)

if __name__ == "__main__":
    main()

```

41. Sprinkler Coverage Area

Green's theorem on the uncovered arcs

The area of the union equals half the boundary integral of $x \, dy - y \, dx$. The boundary consists of the arcs of each circle not covered by the other discs. Remove duplicates and discs contained in others. For circle i , every overlapping circle j covers one angular interval centred at the direction of j , with half-width $\arccos((r_i^2 + d^2 - r_j^2) / (2 r_i d))$. Sweep the sorted intervals and integrate each uncovered arc in closed form.

Correctness. After the removals, every boundary point of the union lies on exactly one circle's uncovered arcs (tangencies have measure zero), and the arcs are oriented counterclockwise.

Complexity and traps. $O(n^2 \log n)$. Identical circles must be merged, or both would be considered covered. Externally tangent circles must not produce intervals.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Green's theorem: the area of the union is (1/2) * the integral of
// (x dy - y dx) along its boundary, and the boundary consists of the arcs of
// each circle not covered by another circle. After removing duplicates and
// circles contained in others, each other circle covers one angular interval
// of circle i (law of cosines); sweep the sorted intervals and integrate the
// uncovered arcs in closed form.  $O(n^2 \log n)$ .

int main() {
    int n;
    cin >> n;
    vector<array<long long, 3>> raw(n);
    for (auto& c : raw) cin >> c[0] >> c[1] >> c[2];
    sort(raw.begin(), raw.end());
    raw.erase(unique(raw.begin(), raw.end()), raw.end());
    sort(raw.begin(), raw.end(), [](const auto& a, const auto& b) { return a[2] > b[2]; });
    vector<array<long long, 3>> kept;
    for (auto& c : raw) {
        bool inside = false;
        for (auto& k : kept) {
            long long dx = c[0] - k[0], dy = c[1] - k[1], dr = k[2] - c[2];
            if (dr >= 0 && dr * dr >= dx * dx + dy * dy) {
                inside = true;
                break;
            }
        }
        if (!inside) kept.push_back(c);
    }
    const double TWO_PI = 2 * acos(-1.0);
    double total = 0;
    int m = kept.size();
    vector<pair<double, double>> events;
    for (int i = 0; i < m; i++) {
        double x = kept[i][0], y = kept[i][1], r = kept[i][2];
        events.clear();
        for (int j = 0; j < m; j++) {
            if (i == j) continue;
            long long dx = kept[j][0] - kept[i][0], dy = kept[j][1] - kept[i][1], sr = kept[i][2] +
            long long d2 = dx * dx + dy * dy;
            if (d2 >= sr * sr) continue; // disjoint or externally tangent
            double d = sqrt((double)d2), R = kept[j][2];
            double cosA = (r * r + d2 - R * R) / (2 * r * d);
            double alpha = acos(max(-1.0, min(1.0, cosA)));
        }
    }
}
```

```

        double lo = fmod(atan2((double)dy, (double)dx) - alpha + 2 * TWO_PI, TWO_PI), hi = lo +
        if (hi > TWO_PI) {
            events.push_back({lo, TWO_PI});
            events.push_back({0.0, hi - TWO_PI});
        } else {
            events.push_back({lo, hi});
        }
    }
    sort(events.begin(), events.end());
    auto arc = [&](double a, double b) {
        total += 0.5 * (r * r * (b - a) + x * r * (sin(b) - sin(a)) - y * r * (cos(b) - cos(a)));
    };
    double cur = 0;
    for (auto [lo, hi] : events) {
        if (lo > cur) arc(cur, lo);
        cur = max(cur, hi);
    }
    if (cur < TWO_PI) arc(cur, TWO_PI);
}
printf("%.10f\n", total);
}

```

Python

```

import math
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    circles = sorted(set((data[1 + 3 * i], data[2 + 3 * i], data[3 + 3 * i]) for i in range(n)), key=
    # Drop circles inside another one (the list is sorted by decreasing radius,
    # and duplicates are already gone).
    kept = []
    for x, y, r in circles:
        inside = False
        for X, Y, R in kept:
            dx = x - X
            dy = y - Y
            if R >= r and (R - r) * (R - r) >= dx * dx + dy * dy:
                inside = True
                break
        if not inside:
            kept.append((x, y, r))
    total = 0.0
    two_pi = 2 * math.pi
    for i, (x, y, r) in enumerate(kept):
        events = []
        for j, (X, Y, R) in enumerate(kept):

```

```

        if i == j:
            continue
        dx = X - x
        dy = Y - y
        d2 = dx * dx + dy * dy
        if d2 >= (r + R) * (r + R):
            continue # disjoint or externally tangent
        d = math.sqrt(d2)
        # The part of circle i inside circle j is the arc around the
        # direction to j with half-angle alpha (law of cosines).
        cos_a = (r * r + d2 - R * R) / (2 * r * d)
        alpha = math.acos(max(-1.0, min(1.0, cos_a)))
        mid = math.atan2(dy, dx)
        lo = mid - alpha
        hi = mid + alpha
        lo %= two_pi
        hi = lo + 2 * alpha
        if hi > two_pi:
            events.append((lo, two_pi))
            events.append((0.0, hi - two_pi))
        else:
            events.append((lo, hi))
    events.sort()
    # Uncovered arcs contribute (1/2) * integral of (x dy - y dx).
    cur = 0.0
    free = []
    for lo, hi in events:
        if lo > cur:
            free.append((cur, lo))
        if hi > cur:
            cur = hi
    if cur < two_pi:
        free.append((cur, two_pi))
    for a, b in free:
        total += 0.5 * (r * r * (b - a) + x * r * (math.sin(b) - math.sin(a)) - y * r * (math.cos(b) - math.cos(a)))
    print("%.10f" % total)

if __name__ == "__main__":
    main()

```

42. Crystal Hull Surface

Incremental hull with a horizon

Start from a non-degenerate tetrahedron, with faces stored counterclockwise from outside. For each further point, find the faces it sees (strictly positive orientation). Delete them, and connect every horizon edge (a visible face adjacent to an invisible one) to the point, keeping the edge's orientation. Points that see no face are inside and are skipped. The answer is the sum of the triangle areas.

Correctness. The visible faces form a connected cap. Replacing it by the cone from the new point over its boundary yields the hull of the old hull plus the point. The statement's guarantee (no supporting plane contains four points) makes every face a triangle and every orientation test decisive.

Complexity and traps. $O(n F) = O(n^2)$ face tests. Orientations of coordinates up to 10^9 need 128 bits. The initial tetrahedron needs four affinely independent points.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Incremental 3D convex hull. Faces are stored counterclockwise as seen from
// outside, so a point sees a face exactly when it lies strictly on the
// outward side (exact 128-bit orientation tests). Adding a point removes its
// visible faces and connects the horizon -- edges between a visible and an
// invisible face -- to the point. The surface area is the sum of triangle
// areas; the statement guarantees that hull faces are triangles.

typedef long long ll;
typedef __int128 i128;

int n;
vector<array<ll, 3>> pts;

i128 orient(int a, int b, int c, int d) {
    i128 ux = pts[b][0] - pts[a][0], uy = pts[b][1] - pts[a][1], uz = pts[b][2] - pts[a][2];
    i128 vx = pts[c][0] - pts[a][0], vy = pts[c][1] - pts[a][1], vz = pts[c][2] - pts[a][2];
    i128 wx = pts[d][0] - pts[a][0], wy = pts[d][1] - pts[a][1], wz = pts[d][2] - pts[a][2];
    return ux * (vy * wz - vz * wy) - uy * (vx * wz - vz * wx) + uz * (vx * wy - vy * wx);
}

int main() {
    scanf("%d", &n);
    pts.resize(n);
    for (auto& p : pts) scanf("%lld %lld %lld", &p[0], &p[1], &p[2]);
    int i0 = 0, i1 = -1, i2 = -1, i3 = -1;
    for (int i = 1; i < n && i1 < 0; i++)
        if (pts[i] != pts[i0]) i1 = i;
    for (int i = 0; i < n && i2 < 0; i++) {
        i128 ux = pts[i1][0] - pts[i0][0], uy = pts[i1][1] - pts[i0][1], uz = pts[i1][2] - pts[i0][2];
        i128 vx = pts[i][0] - pts[i0][0], vy = pts[i][1] - pts[i0][1], vz = pts[i][2] - pts[i0][2];
        if (uy * vz - uz * vy != 0 || uz * vx - ux * vz != 0 || ux * vy - uy * vx != 0) i2 = i;
    }
    for (int i = 0; i < n && i3 < 0; i++)
        if (orient(i0, i1, i2, i) != 0) i3 = i;
    if (orient(i0, i1, i2, i3) > 0) swap(i1, i2); // outward normal away from i3

    vector<array<int, 3>> faces;
    vector<char> alive;
    map<pair<int, int>, int> edgeFace;
```



```

auto addFace = [&](int a, int b, int c) {
    int id = faces.size();
    faces.push_back({a, b, c});
    alive.push_back(1);
    edgeFace[{a, b}] = id;
    edgeFace[{b, c}] = id;
    edgeFace[{c, a}] = id;
};

addFace(i0, i1, i2);
addFace(i0, i3, i1);
addFace(i1, i3, i2);
addFace(i2, i3, i0);
vector<int> live = {0, 1, 2, 3};
vector<char> isVisible;
for (int p = 0; p < n; p++) {
    if (p == i0 || p == i1 || p == i2 || p == i3) continue;
    vector<int> visible;
    isVisible.assign(faces.size(), 0);
    for (int id : live)
        if (orient(faces[id][0], faces[id][1], faces[id][2], p) > 0) {
            visible.push_back(id);
            isVisible[id] = 1;
        }
    if (visible.empty()) continue;
    vector<pair<int, int>> horizon;
    for (int id : visible) {
        auto [a, b, c] = faces[id];
        for (auto [u, v] : {pair(a, b), pair(b, c), pair(c, a)}) {
            int other = edgeFace[{v, u}];
            if (!isVisible[other]) horizon.push_back({u, v});
        }
    }
    for (int id : visible) {
        alive[id] = 0;
        auto [a, b, c] = faces[id];
        for (auto e : {pair(a, b), pair(b, c), pair(c, a)}) {
            auto it = edgeFace.find(e);
            if (it != edgeFace.end() && it->second == id) edgeFace.erase(it);
        }
    }
    for (auto [u, v] : horizon) addFace(u, v, p);
    vector<int> next;
    for (int id : live)
        if (alive[id]) next.push_back(id);
    for (int id = (int)faces.size() - (int)horizon.size(); id < (int)faces.size(); id++) next.push_back(id);
    live.swap(next);
}

double area = 0;
for (int id : live) {
    auto [a, b, c] = faces[id];
    double ux = pts[b][0] - pts[a][0], uy = pts[b][1] - pts[a][1], uz = pts[b][2] - pts[a][2];

```

```

    double vx = pts[c][0] - pts[a][0], vy = pts[c][1] - pts[a][1], vz = pts[c][2] - pts[a][2];
    double cx = uy * vz - uz * vy, cy = uz * vx - ux * vz, cz = ux * vy - uy * vx;
    area += sqrt(cx * cx + cy * cy + cz * cz) / 2;
}
printf("%.10f\n", area);
}

```

Python

```

import math
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    pts = [(data[1 + 3 * i], data[2 + 3 * i], data[3 + 3 * i]) for i in range(n)]

    def orient(a, b, c, d):
        """Positive when d is on the side of plane abc that the normal
        (b - a) x (c - a) points to."""
        ax, ay, az = pts[a]
        ux, uy, uz = pts[b][0] - ax, pts[b][1] - ay, pts[b][2] - az
        vx, vy, vz = pts[c][0] - ax, pts[c][1] - ay, pts[c][2] - az
        wx, wy, wz = pts[d][0] - ax, pts[d][1] - ay, pts[d][2] - az
        return (ux * (vy * wz - vz * wy) - uy * (vx * wz - vz * wx) + uz * (vx * wy - vy * wx))

    # Initial tetrahedron: four affinely independent points.
    i0 = 0
    i1 = next(i for i in range(1, n) if pts[i] != pts[i0])
    i2 = None
    for i in range(n):
        ux = [pts[i1][k] - pts[i0][k] for k in range(3)]
        vx = [pts[i][k] - pts[i0][k] for k in range(3)]
        cr = (ux[1] * vx[2] - ux[2] * vx[1], ux[2] * vx[0] - ux[0] * vx[2], ux[0] * vx[1] - ux[1] * vx[0])
        if cr != (0, 0, 0):
            i2 = i
            break
    i3 = next(i for i in range(n) if orient(i0, i1, i2, i) != 0)
    if orient(i0, i1, i2, i3) > 0:
        i1, i2 = i2, i1 # make face (i0, i1, i2) point away from i3
    faces = {}
    edge_face = {}
    next_id = [0]

    def add_face(a, b, c):
        fid = next_id[0]
        next_id[0] += 1
        faces[fid] = (a, b, c)
        edge_face[(a, b)] = fid

```

```

    edge_face[(b, c)] = fid
    edge_face[(c, a)] = fid

add_face(i0, i1, i2)
add_face(i0, i3, i1)
add_face(i1, i3, i2)
add_face(i2, i3, i0)
used = {i0, i1, i2, i3}
for p in range(n):
    if p in used:
        continue
    visible = [fid for fid, (a, b, c) in faces.items() if orient(a, b, c, p) > 0]
    if not visible:
        continue # inside (or on) the current hull
    vis = set(visible)
    horizon = []
    for fid in visible:
        a, b, c = faces[fid]
        for u, v in ((a, b), (b, c), (c, a)):
            if edge_face.get((v, u)) not in vis:
                horizon.append((u, v))
    for fid in visible:
        a, b, c = faces.pop(fid)
        for u, v in ((a, b), (b, c), (c, a)):
            if edge_face.get((u, v)) == fid:
                del edge_face[(u, v)]
    for u, v in horizon:
        add_face(u, v, p)
area = 0.0
for a, b, c in faces.values():
    ux, uy, uz = (pts[b][k] - pts[a][k] for k in range(3))
    vx, vy, vz = (pts[c][k] - pts[a][k] for k in range(3))
    cx, cy, cz = uy * vz - uz * vy, uz * vx - ux * vz, ux * vy - uy * vx
    area += math.sqrt(cx * cx + cy * cy + cz * cz) / 2
print("%.10f" % area)

if __name__ == "__main__":
    main()

```

43. Triangle Census Queries

Points below segments

For $x_i < x_j$, let $\text{below}(i, j)$ be the number of points strictly between them in x and strictly below segment ij . Around each i , sort the points to its right by slope. Point k is below ij iff its slope from i is smaller and $x_k < x_j$, a dominance count kept with a Fenwick tree or sorted insertion. For a triangle with vertices sorted by x as a, b, c , the answer is $\text{below}(a,b) + \text{below}(b,c) - \text{below}(a,c)$ when b is above ac , and $\text{below}(a,c) - \text{below}(a,b) - \text{below}(b,c) - 1$ otherwise.

Correctness. The region under the path a, b, c minus the region under ac is the triangle when b is above. In the other case the roles swap and b itself is counted once under ac . Distinct x coordinates and no three collinear points leave no boundary cases.

Complexity and traps. $O(n^2 \log n)$ preprocessing, $O(1)$ per query, and $O(n^2)$ memory. Compare slopes with exact cross products.

C++

```
#include <bits/stdc++.h>
using namespace std;

// below[i][j] ( $x_i < x_j$ ) = points strictly between  $i$  and  $j$  in  $x$  and strictly
// below segment  $ij$ . Around each  $i$ , sort the points to its right by slope:
//  $k$  lies below segment  $ij$  iff  $\text{slope}(i,k) < \text{slope}(i,j)$  and  $x_k < x_j$ , a
// dominance count kept with a Fenwick tree over  $x$ -ranks. For a triangle with
// vertices  $a, b, c$  sorted by  $x$ , the inside count is
//   below( $a,b$ ) + below( $b,c$ ) - below( $a,c$ )           if  $b$  is above segment  $ac$ ,
//   below( $a,c$ ) - below( $a,b$ ) - below( $b,c$ ) - 1       otherwise ( $b$  itself is below).

typedef long long ll;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];
    vector<int> order(n), rank(n);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int a, int b) { return x[a] < x[b]; });
    for (int r = 0; r < n; r++) rank[order[r]] = r;
    vector<vector<int>> below(n, vector<int>(n, 0));
    vector<int> bit(n + 1);
    for (int i = 0; i < n; i++) {
        vector<int> right;
        for (int j = 0; j < n; j++)
            if (x[j] > x[i]) right.push_back(j);
        sort(right.begin(), right.end(), [&](int a, int b) {
            return (x[a] - x[i]) * (__int128)(y[b] - y[i]) - (__int128)(y[a] - y[i]) * (x[b] - x[i]);
        });
        fill(bit.begin(), bit.end(), 0);
        for (int j : right) {
            int s = 0;
            for (int k = rank[j]; k > 0; k -= k & -k) s += bit[k];
            below[i][j] = s;
            for (int k = rank[j] + 1; k <= n; k += k & -k) bit[k]++;
        }
    }
    auto under = [&](int a, int b) { return x[a] < x[b] ? below[a][b] : below[b][a]; };
}
```

```

string out;
while (q--) {
    int t[3];
    cin >> t[0] >> t[1] >> t[2];
    for (int& v : t) v--;
    sort(t, t + 3, [&](int a, int b) { return x[a] < x[b]; });
    int a = t[0], b = t[1], c = t[2];
    __int128 cr = (__int128)(x[c] - x[a]) * (y[b] - y[a]) - (__int128)(y[c] - y[a]) * (x[b] - x[a]);
    int ans = cr > 0 ? under(a, b) + under(b, c) - under(a, c) : under(a, c) - under(a, b) - under(b, c);
    out += to_string(ans) + '\n';
}
cout << out;
}

```

Python

```

import sys
from array import array
from bisect import bisect_left, insort
from functools import cmp_to_key

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, q = data[0], data[1]
    xs = data[2:2 + 2 * n:2]
    ys = data[3:3 + 2 * n:2]
    # below[i][j] (x_i < x_j): points strictly between them in x and strictly
    # below segment ij. Around i, sort the points to its right by slope; k is
    # below segment ij iff slope(i,k) < slope(i,j) and x_k < x_j.
    order_x = sorted(range(n), key=xs.__getitem__)
    rank = [0] * n
    for r, i in enumerate(order_x):
        rank[i] = r
    below = [None] * n
    for i in range(n):
        xi, yi = xs[i], ys[i]
        right = [j for j in range(n) if xs[j] > xi]

        def cmp(a, b):
            # slope(i,a) < slope(i,b)  <=>  cross(a - i, b - i) > 0 (dx > 0)
            c = (xs[a] - xi) * (ys[b] - yi) - (ys[a] - yi) * (xs[b] - xi)
            return -1 if c > 0 else 1 if c < 0 else 0

        right.sort(key=cmp_to_key(cmp))
        row = array("i", bytes(4 * n))
        seen = [] # x-ranks of points with smaller slope, kept sorted
        for j in right:
            row[j] = bisect_left(seen, rank[j])
            insort(seen, rank[j])

```

```

        below[i] = row

def under(a, b):
    return below[a][b] if xs[a] < xs[b] else below[b][a]

out = []
pos = 2 + 2 * n
for _ in range(q):
    a, b, c = sorted((data[pos] - 1, data[pos + 1] - 1, data[pos + 2] - 1), key=xs.__getitem__)
    pos += 3
    # Is b above segment ac?
    cross = (xs[c] - xs[a]) * (ys[b] - ys[a]) - (ys[c] - ys[a]) * (xs[b] - xs[a])
    if cross > 0:
        out.append(under(a, b) + under(b, c) - under(a, c))
    else:
        out.append(under(a, c) - under(a, b) - under(b, c) - 1)
sys.stdout.write("\n".join(map(str, out)) + "\n")

if __name__ == "__main__":
    main()

```

44. Polygon Triangulation Count

Interval DP over internal diagonals

Exactly one triangle of every triangulation sits on the edge $(0, n-1)$. Its apex k splits the polygon into the chains $0..k$ and $k..n-1$, each closed by an internal diagonal. So $T[i][j] = \sum_k T[i][k] T[k][j]$ over valid chords (i,k) and (k,j) . A chord is an internal diagonal iff it lies in the interior angle at both endpoints (cone test) and properly crosses no edge.

Correctness. In a triangulation the triangles adjacent to a chord split the chord's region into two independent sub-polygons, and the decomposition by the apex is unique.

Complexity and traps. $O(n^3)$ for both validity and the DP. Normalize to counterclockwise order. Reflex vertices need the complemented cone test.

C++

```

#include <bits/stdc++.h>
using namespace std;

// Every triangulation contains exactly one triangle on the edge (0, n-1); its
// apex k splits the polygon into the chains 0..k and k..n-1, each closed by
// an internal diagonal. So T[i][j] (chain i..j closed by a valid chord) is
// the sum over k of T[i][k] T[k][j] with (i,k), (k,j) valid. A chord is an
// internal diagonal iff it lies in the interior angle at both ends and
// properly crosses no polygon edge (no three vertices are collinear).

```

```

typedef long long ll;
const ll MOD = 1000000007;

int n;
vector<ll> X, Y;

ll cross(int a, int b, int c) { return (X[b] - X[a]) * (Y[c] - Y[a]) - (Y[b] - Y[a]) * (X[c] - X[a]); }
int sgn(__int128 v) { return v > 0 ? 1 : v < 0 ? -1 : 0; }
int crossSign(int a, int b, int c) {
    return sgn((__int128)(X[b] - X[a]) * (Y[c] - Y[a]) - (__int128)(Y[b] - Y[a]) * (X[c] - X[a]));
}

bool inCone(int a, int b) {
    int a0 = (a + n - 1) % n, a1 = (a + 1) % n;
    if (crossSign(a0, a, a1) >= 0) return crossSign(a, b, a0) > 0 && crossSign(b, a, a1) > 0;
    return !(crossSign(a, b, a1) >= 0 && crossSign(b, a, a0) >= 0);
}

bool crossesBoundary(int a, int b) {
    for (int c = 0; c < n; c++) {
        int d = (c + 1) % n;
        if (c == a || c == b || d == a || d == b) continue;
        if ((crossSign(a, b, c) > 0) == (crossSign(a, b, d) > 0)) continue;
        if ((crossSign(c, d, a) > 0) != (crossSign(c, d, b) > 0)) return true;
    }
    return false;
}

int main() {
    cin >> n;
    X.resize(n);
    Y.resize(n);
    for (int i = 0; i < n; i++) cin >> X[i] >> Y[i];
    __int128 area2 = 0;
    for (int i = 0; i < n; i++) area2 += (__int128)X[i] * Y[(i + 1) % n] - (__int128)X[(i + 1) % n] * Y[i];
    if (area2 < 0) {
        reverse(X.begin(), X.end());
        reverse(Y.begin(), Y.end());
    }
    vector<vector<char>> ok(n, vector<char>(n, 0));
    for (int i = 0; i < n; i++) {
        ok[i][(i + 1) % n] = ok[(i + 1) % n][i] = 1;
        for (int j = i + 2; j < n; j++) {
            if (i == 0 && j == n - 1) continue;
            if (inCone(i, j) && inCone(j, i) && !crossesBoundary(i, j)) ok[i][j] = ok[j][i] = 1;
        }
    }
    vector<vector<ll>> T(n, vector<ll>(n, 0));
    for (int i = 0; i + 1 < n; i++) T[i][i + 1] = 1;
    for (int len = 2; len < n; len++)
        for (int i = 0; i + len < n; i++) {

```

```

        int j = i + len;
        if (!ok[i][j]) continue;
        ll total = 0;
        for (int k = i + 1; k < j; k++)
            if (ok[i][k] && ok[k][j]) total = (total + T[i][k] * T[k][j]) % MOD;
        T[i][j] = total;
    }
    cout << T[0][n - 1] << '\n';
}

```

Python

```

import sys

MOD = 1_000_000_007

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    X = data[1:1 + 2 * n:2]
    Y = data[2:2 + 2 * n:2]
    # Orient counterclockwise.
    area2 = sum(X[i] * Y[(i + 1) % n] - X[(i + 1) % n] * Y[i] for i in range(n))
    if area2 < 0:
        X.reverse()
        Y.reverse()

    def cross(a, b, c):
        return (X[b] - X[a]) * (Y[c] - Y[a]) - (Y[b] - Y[a]) * (X[c] - X[a])

    def in_cone(a, b):
        """Is the segment from vertex a towards b inside the interior angle at a?"""
        a0 = (a - 1) % n
        a1 = (a + 1) % n
        if cross(a0, a, a1) >= 0: # convex vertex
            return cross(a, b, a0) > 0 and cross(b, a, a1) > 0
        return not (cross(a, b, a1) >= 0 and cross(b, a, a0) >= 0)

    edges = [(k, (k + 1) % n) for k in range(n)]

    def crosses_boundary(a, b):
        for c, d in edges:
            if c in (a, b) or d in (a, b):
                continue
            d1 = cross(a, b, c)
            d2 = cross(a, b, d)
            if (d1 > 0) == (d2 > 0):
                continue
            d3 = cross(c, d, a)

```



```

        d4 = cross(c, d, b)
        if (d3 > 0) != (d4 > 0):
            return True
    return False

ok = [[False] * n for _ in range(n)]
for i in range(n):
    ok[i][(i + 1) % n] = ok[(i + 1) % n][i] = True
    for j in range(i + 2, n):
        if i == 0 and j == n - 1:
            continue
        if in_cone(i, j) and in_cone(j, i) and not crosses_boundary(i, j):
            ok[i][j] = ok[j][i] = True
# T[i][j]: triangulations of the sub-polygon i, i+1, ..., j closed by chord (i, j).
T = [[0] * n for _ in range(n)]
for i in range(n - 1):
    T[i][i + 1] = 1
for length in range(2, n):
    for i in range(0, n - length):
        j = i + length
        if not ok[i][j]:
            continue
        Ti = T[i]
        oki = ok[i]
        total = 0
        for k in range(i + 1, j):
            if oki[k] and ok[k][j]:
                total += Ti[k] * T[k][j]
        T[i][j] = total % MOD
print(T[0][n - 1])

if __name__ == "__main__":
    main()

```

45. Obstacle Shortcut

Shortest paths bend only at corners

A shortest path avoiding open convex obstacles is a polyline whose interior vertices are obstacle corners: anywhere else it could be shortened locally. Build the visibility graph on S , T , and all corners, then run Dijkstra. Segment PQ is blocked iff some point of it is strictly inside an obstacle. Intersect t in $[0, 1]$ with the open half-planes of the obstacle's edges, and compare the resulting bounds as exact fractions.

Correctness. Touching walls and corners is allowed, so only strict interior points block. The clipping interval is exactly the set of parameters whose points are strictly inside.

Complexity and traps. $O(V^2 E)$ for visibility and $O(V^2 \log V)$ for Dijkstra. Segments between two corners of the same building are edges (allowed) or diagonals (blocked). Use 128-bit integers for the fractions.

C++

```
#include <bits/stdc++.h>
using namespace std;

// A shortest obstacle-avoiding path is a polyline whose bends are obstacle
// corners (shorten any other bend), so it lives in the visibility graph on
// {S, T, corners}. Segment PQ is blocked iff some point of it is strictly
// inside an obstacle: intersect t in [0,1] with the open half-planes of the
// obstacle's counterclockwise edges, comparing the bounds as exact fractions.
// Dijkstra on the visibility graph gives the answer.

typedef long long ll;
typedef __int128 i128;
typedef pair<ll, ll> Pt;

int main() {
    ll sx, sy, tx, ty;
    int k;
    cin >> sx >> sy >> tx >> ty >> k;
    vector<vector<Pt>> polys(k);
    for (auto& poly : polys) {
        int m;
        cin >> m;
        poly.resize(m);
        for (auto& p : poly) cin >> p.first >> p.second;
        i128 area2 = 0;
        for (int i = 0; i < m; i++)
            area2 += (i128)poly[i].first * poly[(i + 1) % m].second - (i128)poly[(i + 1) % m].first * poly[i].second;
        if (area2 < 0) reverse(poly.begin(), poly.end());
    }
    vector<Pt> nodes = {{sx, sy}, {tx, ty}};
    for (auto& poly : polys) nodes.insert(nodes.end(), poly.begin(), poly.end());

    auto blocked = [&](Pt P, Pt Q) {
        ll dx = Q.first - P.first, dy = Q.second - P.second;
        for (auto& poly : polys) {
            int m = poly.size();
            i128 loN = -1, loD = 1, hiN = 2, hiD = 1;
            bool feasible = true;
            for (int i = 0; i < m && feasible; i++) {
                ll ax = poly[i].first, ay = poly[i].second;
                ll ex = poly[(i + 1) % m].first - ax, ey = poly[(i + 1) % m].second - ay;
                i128 c0 = (i128)ex * (P.second - ay) - (i128)ey * (P.first - ax);
                i128 c1 = (i128)ex * dy - (i128)ey * dx;
                if (c1 == 0) {
                    if (c0 <= 0) feasible = false;
                } else if (c1 > 0) {
                    if (-c0 * loD > loN * c1) loN = -c0, loD = c1;
                } else {
                    if (c0 * hiD < hiN * (-c1)) hiN = c0, hiD = -c1;
                }
            }
        }
        return !feasible;
    };
}
```

```

    }
    }
    if (feasible && loN * hiD < hiN * loD && loN < loD && hiN > 0) return true;
}
return false;
};

int V = nodes.size();
vector<vector<pair<int, double>>> adj(V);
for (int i = 0; i < V; i++)
    for (int j = i + 1; j < V; j++)
        if (!blocked(nodes[i], nodes[j])) {
            double w = hypot((double)(nodes[i].first - nodes[j].first), (double)(nodes[i].second - nodes[j].second));
            adj[i].push_back({j, w});
            adj[j].push_back({i, w});
        }
vector<double> dist(V, 1e300);
priority_queue<pair<double, int>, vector<pair<double, int>>, greater<>> pq;
dist[0] = 0;
pq.push({0, 0});
while (!pq.empty()) {
    auto [d, v] = pq.top();
    pq.pop();
    if (d > dist[v]) continue;
    for (auto [w, c] : adj[v])
        if (d + c < dist[w]) {
            dist[w] = d + c;
            pq.push({dist[w], w});
        }
}
printf("%.10f\n", dist[1]);
}

```

Python

```

import heapq
import math
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    sx, sy, tx, ty, k = data[:5]
    pos = 5
    polys = []
    for _ in range(k):
        m = data[pos]
        pos += 1
        poly = [(data[pos + 2 * i], data[pos + 2 * i + 1]) for i in range(m)]
        pos += 2 * m

```

```

    area2 = sum(poly[i][0] * poly[(i + 1) % m][1] - poly[(i + 1) % m][0] * poly[i][1] for i in range(m))
    if area2 < 0:
        poly.reverse()
    polys.append(poly)
nodes = [(sx, sy), (tx, ty)]
for poly in polys:
    nodes.extend(poly)

def blocked(P, Q):
    """Does segment PQ meet the interior of some obstacle?

    Clip t in [0,1] with the open half-planes of each (CCW) edge; the open
    constraints t > lo, t < hi are kept as exact fractions."""
    px, py = P
    dx, dy = Q[0] - px, Q[1] - py
    for poly in polys:
        m = len(poly)
        lo_n, lo_d = -1, 1 # lower bound, as a fraction (t > lo)
        hi_n, hi_d = 2, 1 # upper bound (t < hi)
        feasible = True
        for i in range(m):
            ax, ay = poly[i]
            bx, by = poly[(i + 1) % m]
            ex, ey = bx - ax, by - ay
            c0 = ex * (py - ay) - ey * (px - ax) # f(0)
            c1 = ex * dy - ey * dx # slope
            # need c0 + t * c1 > 0
            if c1 == 0:
                if c0 <= 0:
                    feasible = False
                    break
            elif c1 > 0: # t > -c0 / c1
                num, den = -c0, c1
                if num * lo_d > lo_n * den:
                    lo_n, lo_d = num, den
            else: # t < -c0 / c1 = c0 / -c1
                num, den = c0, -c1
                if num * hi_d < hi_n * den:
                    hi_n, hi_d = num, den
        if not feasible:
            continue
    # nonempty: lo < hi, lo < 1, hi > 0
    if lo_n * hi_d < hi_n * lo_d and lo_n < lo_d and hi_n > 0:
        return True
    return False

V = len(nodes)
adj = [[] for _ in range(V)]
for i in range(V):
    for j in range(i + 1, V):
        if not blocked(nodes[i], nodes[j]):

```

```

        w = math.hypot(nodes[i][0] - nodes[j][0], nodes[i][1] - nodes[j][1])
        adj[i].append((j, w))
        adj[j].append((i, w))
dist = [math.inf] * V
dist[0] = 0.0
heap = [(0.0, 0)]
while heap:
    d, v = heapq.heappop(heap)
    if d > dist[v]:
        continue
    if v == 1:
        break
    for w, c in adj[v]:
        nd = d + c
        if nd < dist[w]:
            dist[w] = nd
            heapq.heappush(heap, (nd, w))
print("%.10f" % dist[1])

if __name__ == "__main__":
    main()

```

46. Lantern Grid

Light chasing

Presses commute and are linear over $\text{GF}(2)$. Treat the first row's presses as unknowns. Lamp (r, c) can then only be turned off by the press below it, which determines row $r+1$ as an affine function of the unknowns (a bitset of m coefficients plus a constant). The last row of lamps gives m equations. If they are consistent, the answer is $2^{m - \text{rank}}$, and otherwise 0. Transpose so that $m \leq n$.

Correctness. Every press set satisfying the first $n-1$ lamp rows is determined by its first row, and the remaining equations are exactly the last-row lamps.

Complexity and traps. $O(n \cdot m^2 / w)$ with word size w . Known values: the empty 4×4 board has 16 solutions, and the 19×19 board has nullity 16.

C++

```

#include <bits/stdc++.h>
using namespace std;

// Pressing is linear over GF(2) and presses commute. Fix the presses of the
// first row as unknowns; then each lamp of row r can only be switched off by
// the press below it, which determines row r+1 as an affine function of the
// unknowns. The last row of lamps yields m equations in m unknowns: the
// answer is  $2^{(m - \text{rank})}$  if they are consistent and 0 otherwise. Rows are
// bitsets of the m unknowns plus a constant bit.

```

```

const long long MOD = 1000000007;
const int MAXM = 1001;
typedef bitset<MAXM + 1> Row;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto& s : g) cin >> s;
    if (m > n) {
        vector<string> t(m, string(n, '0'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) t[j][i] = g[i][j];
        g = t;
        swap(n, m);
    }
    vector<Row> prev(m), cur(m), nxt(m);
    for (int c = 0; c < m; c++) cur[c][c] = 1;
    auto lampRow = [&](int r, vector<Row>& out) {
        for (int c = 0; c < m; c++) {
            Row v = prev[c] ^ cur[c];
            if (c) v ^= cur[c - 1];
            if (c + 1 < m) v ^= cur[c + 1];
            if (g[r][c] == '1') v.flip(m);
            out[c] = v;
        }
    };
    for (int r = 0; r + 1 < n; r++) {
        lampRow(r, nxt);
        swap(prev, cur);
        swap(cur, nxt);
    }
    vector<Row> eqs(m);
    lampRow(n - 1, eqs);
    int rank = 0;
    for (int bit = 0; bit < m; bit++) {
        int pivot = -1;
        for (int i = rank; i < m; i++)
            if (eqs[i][bit]) {
                pivot = i;
                break;
            }
        if (pivot < 0) continue;
        swap(eqs[rank], eqs[pivot]);
        for (int i = 0; i < m; i++)
            if (i != rank && eqs[i][bit]) eqs[i] ^= eqs[rank];
        rank++;
    }
    for (int i = rank; i < m; i++)
        if (eqs[i][m]) {
            cout << 0 << '\n';
        }
}

```

```

        return 0;
    }
    long long ans = 1;
    for (int i = 0; i < m - rank; i++) ans = ans * 2 % MOD;
    cout << ans << '\n';
}

```

Python

```

import sys

MOD = 1_000_000_007

def main():
    data = sys.stdin.buffer.read().split()
    n, m = int(data[0]), int(data[1])
    rows = [data[2 + i].decode() for i in range(n)]
    grid = [[ch == "1" for ch in row] for row in rows]
    if m > n: # keep the shorter side as the free variables
        grid = [list(col) for col in zip(*grid)]
        n, m = m, n
    # Every press is an affine function of the first-row presses: an integer
    # whose bits 0..m-1 are coefficients and bit m is the constant.
    const = 1 << m
    prev = [0] * m
    cur = [1 << c for c in range(m)]
    for r in range(n - 1):
        lamp = grid[r]
        nxt = [0] * m
        for c in range(m):
            # lamp (r, c) must end off; only press (r+1, c) can still change it
            v = prev[c] ^ cur[c]
            if c:
                v ^= cur[c - 1]
            if c + 1 < m:
                v ^= cur[c + 1]
            if lamp[c]:
                v ^= const
            nxt[c] = v
        prev, cur = cur, nxt
    # The last row of lamps gives m equations in the m first-row variables.
    eqs = []
    lamp = grid[n - 1]
    for c in range(m):
        v = prev[c] ^ cur[c]
        if c:
            v ^= cur[c - 1]
        if c + 1 < m:
            v ^= cur[c + 1]

```

```

    if lamp[c]:
        v ^= const
    eqs.append(v)
# Gaussian elimination over GF(2).
rank = 0
for bit in range(m):
    mask = 1 << bit
    pivot = next((i for i in range(rank, m) if eqs[i] & mask), -1)
    if pivot < 0:
        continue
    eqs[rank], eqs[pivot] = eqs[pivot], eqs[rank]
    pv = eqs[rank]
    for i in range(m):
        if i != rank and eqs[i] & mask:
            eqs[i] ^= pv
    rank += 1
if any(e == const for e in eqs[rank:]):
    print(0)
else:
    print(pow(2, m - rank, MOD))

if __name__ == "__main__":
    main()

```

47. Quorum Circle

Binary search, pin a point to the boundary

Feasibility is monotone in R . An optimal disc can be translated until one of its points, p , lies on the boundary. The centre then lies on the circle of radius R around p , and every other point q with $|pq| \leq 2R$ restricts the centre to an arc of half-width $\arccos(|pq| / 2R)$ around the direction of q . The deepest overlap of these arcs, plus coincident points, decides feasibility.

Correctness. Translating a disc while it keeps its points never loses any, until a point reaches the boundary. Each arc is exactly the set of centres at distance at most R from q .

Complexity and traps. $O(n^2 \log n)$ per probe, and about 60 probes. Coincident points belong to every candidate disc. $k = 1$ gives 0.

C++

```

#include <bits/stdc++.h>
using namespace std;

// Binary search the radius R. If some closed disc of radius R holds k points,
// it can be translated until one of them, p, lies on its boundary; the centre
// is then on the circle of radius R around p, and every other point q within
// 2R allows an arc of centre angles of half-width  $\arccos(|pq| / 2R)$ . An

```



```

// angular sweep finds the deepest overlap.   $O(n^2 \log n)$  per probe.

int main() {
    int n, k;
    scanf("%d %d", &n, &k);
    vector<double> x(n), y(n);
    for (int i = 0; i < n; i++) scanf("%lf %lf", &x[i], &y[i]);
    if (k <= 1) {
        printf("%.10f\n", 0.0);
        return 0;
    }
    vector<int> same(n, 0);
    vector<vector<pair<double, double>>> near(n);
    double far = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            double d = hypot(x[j] - x[i], y[j] - y[i]);
            if (d == 0) same[i]++;
            else near[i].push_back({d, atan2(y[j] - y[i], x[j] - x[i])});
            far = max(far, d);
        }
        sort(near[i].begin(), near[i].end());
    }
    const double PI = acos(-1.0);
    vector<pair<double, int>> events;
    auto feasible = [&](double R) {
        double limit = 2 * R * (1 + 1e-12);
        for (int i = 0; i < n; i++) {
            if (same[i] + 1 >= k) return true;
            events.clear();
            for (auto [d, ang] : near[i]) {
                if (d > limit) break;
                double w = acos(min(1.0, d / (2 * R)));
                double s = ang - w, e = ang + w;
                if (s < -PI) s += 2 * PI, e += 2 * PI;
                if (e > PI) {
                    events.push_back({s, 0});
                    events.push_back({PI, 1});
                    events.push_back({-PI, 0});
                    events.push_back({e - 2 * PI, 1});
                } else {
                    events.push_back({s, 0});
                    events.push_back({e, 1});
                }
            }
        }
        sort(events.begin(), events.end());
        int cur = same[i] + 1, best = cur;
        for (auto [a, kind] : events) {
            if (kind == 0) best = max(best, ++cur);
            else cur--;
        }
    };
}

```

```

    }
    if (best >= k) return true;
}
return false;
};
double lo = 0, hi = far + 1;
for (int it = 0; it < 60; it++) {
    double mid = (lo + hi) / 2;
    if (feasible(mid)) hi = mid;
    else lo = mid;
}
printf("%.10f\n", hi);
}

```

Python

```

import math
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, k = data[0], data[1]
    pts = [(data[2 + 2 * i], data[3 + 2 * i]) for i in range(n)]
    if k <= 1:
        print("%.10f" % 0.0)
        return
    # Pairwise distances and directions, computed once.
    near = []
    for i, (x, y) in enumerate(pts):
        row = []
        same = 0
        for j, (u, v) in enumerate(pts):
            if i == j:
                continue
            d = math.hypot(u - x, v - y)
            if d == 0:
                same += 1
            else:
                row.append((d, math.atan2(v - y, u - x)))
        row.sort()
        near.append((same, row))
    pi = math.pi

    def feasible(R):
        """Some closed disc of radius R holds k points? An optimal disc can be
        moved until a point p lies on its boundary; then the centre is on the
        circle of radius R around p, and each q constrains it to an arc."""
        limit = 2 * R * (1 + 1e-12)
        for same, row in near:

```

```

    if same + 1 >= k:
        return True
    events = []
    for d, ang in row:
        if d > limit:
            break
        w = math.acos(min(1.0, d / (2 * R)))
        s = ang - w
        e = ang + w
        if s < -pi:
            s += 2 * pi
            e += 2 * pi
        if e > pi:
            events.append((s, 0))
            events.append((pi, 1))
            events.append((-pi, 0))
            events.append((e - 2 * pi, 1))
        else:
            events.append((s, 0))
            events.append((e, 1))
    events.sort()
    cur = best = same + 1
    for _, kind in events:
        if kind == 0:
            cur += 1
            if cur > best:
                best = cur
        else:
            cur -= 1
    if best >= k:
        return True
    return False

```

```

lo = 0.0
hi = max(r[-1][0] if r else 0.0 for _, r in near) + 1 # beyond the diameter
for _ in range(60):
    mid = (lo + hi) / 2
    if feasible(mid):
        hi = mid
    else:
        lo = mid
print("%.10f" % hi)

```

```

if __name__ == "__main__":
    main()

```

48. Quiet Committee Census

Meet in the middle

Split the members into halves A and B. For every subset M of B, compute `best[M]` and `ways[M]` (the optimum and the number of optimal committees inside M) by branching on the lowest member `v`: either `v` is out (`M - v`), or `v` is in (`M - N[v]`, plus `w[v]`). Then enumerate the committees `S` inside A. Each leaves exactly `B - N(S)` available, so combine with `best` and `ways` of that set.

Correctness. Every committee splits uniquely into its A part and its B part. The B part ranges over exactly the committees inside `B - N(S)`. The branching on `v` is a partition, so counts add.

Complexity and traps. $O(2^{\{n/2\}})$ time and memory. With zero weights every committee ties, so the count is the number of independent sets (for example 2^{40} on the empty graph). Repeated rivalries are harmless.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Meet in the middle. For the second half B, best[mask] / ways[mask] give the
// optimum and the number of optimal independent sets inside the vertex subset
// mask (branch on the lowest vertex: unused, or used with its neighbours
// removed). Then enumerate the independent subsets S of the first half; each
// one leaves exactly B minus N(S) available.  $O(2^{(n/2)})$  time and memory.

typedef long long ll;
const ll MOD = 1000000007;

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    vector<ll> w(n);
    for (auto& x : w) scanf("%lld", &x);
    vector<unsigned long long> adj(n, 0);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        --u;
        --v;
        adj[u] |= 1ULL << v;
        adj[v] |= 1ULL << u;
    }
    int h = n / 2, b = n - h;
    unsigned fullB = (1u << b) - 1;
    vector<unsigned> nbB(b);
    for (int i = 0; i < b; i++) nbB[i] = (unsigned)(adj[h + i] >> h) | (1u << i);
    vector<ll> best(1u << b, 0);
    vector<int> ways(1u << b, 1);
    for (unsigned mask = 1; mask < (1u << b); mask++) {
        int v = __builtin_ctz(mask);
```

```

    unsigned a = mask ^ (1u << v), c = mask & ~nbB[v];
    ll x = best[a], y = best[c] + w[h + v];
    if (x > y) best[mask] = x, ways[mask] = ways[a];
    else if (y > x) best[mask] = y, ways[mask] = ways[c];
    else best[mask] = x, ways[mask] = (ways[a] + ways[c]) % MOD;
}
unsigned size = 1u << h;
vector<char> ok(size, 0);
vector<ll> sum(size, 0);
vector<unsigned> forb(size, 0);
ok[0] = 1;
ll top = -1, count = 0;
for (unsigned mask = 0; mask < size; mask++) {
    if (mask) {
        int v = __builtin_ctz(mask);
        unsigned rest = mask ^ (1u << v);
        if (!ok[rest] || (adj[v] & rest)) continue;
        ok[mask] = 1;
        sum[mask] = sum[rest] + w[v];
        forb[mask] = forb[rest] | (unsigned)(adj[v] >> h);
    }
    unsigned freeB = fullB & ~forb[mask];
    ll value = sum[mask] + best[freeB];
    if (value > top) top = value, count = ways[freeB];
    else if (value == top) count = (count + ways[freeB]) % MOD;
}
printf("%lld %lld\n", top, count % MOD);
}

```

Python

```

import sys

MOD = 1_000_000_007

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[0], data[1]
    w = data[2:2 + n]
    adj = [0] * n
    pos = 2 + n
    for _ in range(m):
        u = data[pos] - 1
        v = data[pos + 1] - 1
        pos += 2
        adj[u] |= 1 << v
        adj[v] |= 1 << u
    h = n // 2          # first half A = vertices 0..h-1, second half B = h..n-1
    b = n - h

```

```

full_b = (1 << b) - 1
# best[mask] / ways[mask]: optimum and number of optimal independent sets
# inside the B-subset `mask`. Split on the lowest vertex v of the mask:
# either v is unused, or v is used and its neighbours are removed.
nb_b = [(adj[h + i] >> h) | (1 << i) for i in range(b)]
best = [0] * (1 << b)
ways = [1] * (1 << b)
wb = w[h:]
for mask in range(1, 1 << b):
    v = (mask & -mask).bit_length() - 1
    a = mask ^ (1 << v)
    c = mask & ~nb_b[v]
    x = best[a]
    y = best[c] + wb[v]
    if x > y:
        best[mask] = x
        ways[mask] = ways[a]
    elif y > x:
        best[mask] = y
        ways[mask] = ways[c]
    else:
        best[mask] = x
        ways[mask] = (ways[a] + ways[c]) % MOD
# Enumerate independent subsets S of A; each leaves B minus N(S) free.
adj_a = [adj[i] & ((1 << h) - 1) for i in range(h)]
to_b = [adj[i] >> h for i in range(h)]
size = 1 << h
ok = bytearray(size)
ok[0] = 1
total_w = [0] * size
forbidden = [0] * size
top = -1
count = 0
for mask in range(size):
    if mask:
        v = (mask & -mask).bit_length() - 1
        rest = mask ^ (1 << v)
        if not ok[rest] or adj_a[v] & rest:
            continue
        ok[mask] = 1
        total_w[mask] = total_w[rest] + w[v]
        forbidden[mask] = forbidden[rest] | to_b[v]
    free = full_b & ~forbidden[mask]
    value = total_w[mask] + best[free]
    if value > top:
        top = value
        count = ways[free]
    elif value == top:
        count += ways[free]
print(top, count % MOD)

```

```
if __name__ == "__main__":
    main()
```

49. Glacier Cut

Binary searches on a convex polygon

Let $f(p) = \text{cross}(B - A, p - A)$ (positive means left of the line). Edge directions of a counterclockwise convex polygon increase in angle, so the vertex maximizing f is the start of the first edge whose angle is at least the direction of the line. The minimizing vertex is found symmetrically. Between them f is monotone along both chains, and binary searches find the two edges where it changes sign. The left piece is a range of vertices plus the two crossing points, so its area is a prefix shoelace sum plus three correction terms. Computing the crossing points as exact rationals keeps even tiny slivers accurate.

Correctness. Unimodality of a linear function on a convex polygon makes all searches valid. The piece left of the line is exactly the convex polygon formed by the crossing points and the vertices with $f > 0$.

Complexity and traps. $O(\log n)$ per query. Lines that miss the polygon or only touch it give 0 or the full area. The numerators reach about 10^{33} , so C++ uses 128-bit integers.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Let f(p) = cross(b - a, p - a) (positive = left of the line). On a convex
// polygon f rises from its minimum vertex to its maximum vertex and falls
// back; both extremes are found by a binary search over edge directions,
// which are sorted by angle. Two more binary searches find the edges where
// f changes sign. The left piece is the vertex range between them plus the
// two crossing points, whose area follows from prefix shoelace sums. The
// crossing points are exact rationals, so 128-bit integers give the area of
// even tiny pieces without cancellation error.

typedef long long ll;
typedef __int128 i128;

int n;
vector<ll> xs, ys, ex, ey;
vector<int> eh;

int half(ll x, ll y) { return (y > 0 || (y == 0 && x > 0)) ? 0 : 1; }

int firstEdgeNotBefore(ll tx, ll ty) {
    int th = half(tx, ty), lo = 0, hi = n;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        bool before = eh[mid] < th || (eh[mid] == th && ex[mid] * ty - ey[mid] * tx > 0);
        if (before) lo = mid + 1;
    }
}
```

```

        else hi = mid;
    }
    return lo % n;
}

int main() {
    int q;
    scanf("%d %d", &n, &q);
    xs.resize(n);
    ys.resize(n);
    for (int i = 0; i < n; i++) scanf("%lld %lld", &xs[i], &ys[i]);
    i128 area = 0;
    for (int i = 0; i < n; i++) area += (i128)xs[i] * ys[(i + 1) % n] - (i128)xs[(i + 1) % n] * ys[i];
    if (area < 0) reverse(xs.begin(), xs.end()), reverse(ys.begin(), ys.end());
    ex.resize(n);
    ey.resize(n);
    for (int i = 0; i < n; i++) ex[i] = xs[(i + 1) % n] - xs[i], ey[i] = ys[(i + 1) % n] - ys[i];
    int best = 0;
    for (int i = 1; i < n; i++) {
        int hi_ = half(ex[i], ey[i]), hb = half(ex[best], ey[best]);
        if (hi_ < hb || (hi_ == hb && ex[best] * ey[i] - ey[best] * ex[i] < 0)) best = i;
    }
    rotate(xs.begin(), xs.begin() + best, xs.end());
    rotate(ys.begin(), ys.begin() + best, ys.end());
    rotate(ex.begin(), ex.begin() + best, ex.end());
    rotate(ey.begin(), ey.begin() + best, ey.end());
    eh.resize(n);
    for (int i = 0; i < n; i++) eh[i] = half(ex[i], ey[i]);
    vector<ll> prefix(2 * n + 1, 0);
    for (int k = 0; k < 2 * n; k++) {
        int i = k % n, j = (k + 1) % n;
        prefix[k + 1] = prefix[k] + xs[i] * ys[j] - xs[j] * ys[i];
    }
    ll total2 = prefix[n];
    string out;
    char buf[64];
    while (q--) {
        ll ax, ay, bx, by;
        scanf("%lld %lld %lld %lld", &ax, &ay, &bx, &by);
        ll dx = bx - ax, dy = by - ay;
        auto f = [&](int i) { return dx * (ys[i] - ay) - dy * (xs[i] - ax); };
        int imax = firstEdgeNotBefore(-dx, -dy), imin = firstEdgeNotBefore(dx, dy);
        if (f(imax) <= 0) {
            out += "0\n";
            continue;
        }
        if (f(imin) >= 0) {
            snprintf(buf, sizeof buf, "%.10Lf\n", (long double)total2 / 2);
            out += buf;
            continue;
        }
    }
}

```



```

int lo = 0, hi = ((imax - imin) % n + n) % n;
while (lo < hi) {
    int mid = (lo + hi + 1) / 2;
    if (f((imin + mid) % n) <= 0) lo = mid;
    else hi = mid - 1;
}
int i = (imin + lo) % n;
lo = 0, hi = ((imin - imax) % n + n) % n;
while (lo < hi) {
    int mid = (lo + hi + 1) / 2;
    if (f((imax + mid) % n) > 0) lo = mid;
    else hi = mid - 1;
}
int j = (imax + lo) % n, i1 = (i + 1) % n, j1 = (j + 1) % n;
i128 fi = f(i), fi1 = f(i1), fj = f(j), fj1 = f(j1);
i128 d1 = fi1 - fi, d2 = fj - fj1;
i128 X0 = xs[i] * d1 + (xs[i1] - xs[i]) * (-fi), X1 = ys[i] * d1 + (ys[i1] - ys[i]) * (-fi);
i128 Y0 = xs[j] * d2 + (xs[j1] - xs[j]) * fj, Y1 = ys[j] * d2 + (ys[j1] - ys[j]) * fj;
int k0 = i1, k1 = j >= i1 ? j : j + n;
i128 inner = prefix[k1] - prefix[k0];
i128 num = (X0 * ys[i1] - X1 * xs[i1]) * d2 + inner * d1 * d2 + (xs[j] * Y1 - ys[j] * Y0) *
long double value = (long double)num / ((long double)d1 * (long double)d2 * 2);
snprintf(buf, sizeof buf, "%.10Lf\n", value);
out += buf;
}
fputs(out.c_str(), stdout);
}

```

Python

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, q = data[0], data[1]
    xs = data[2:2 + 2 * n:2]
    ys = data[3:3 + 2 * n:2]
    if sum(xs[i] * ys[(i + 1) % n] - xs[(i + 1) % n] * ys[i] for i in range(n)) < 0:
        xs.reverse()
        ys.reverse()

    def half(x, y):
        return 0 if y > 0 or (y == 0 and x > 0) else 1

    # Rotate so that edge directions increase in angle over [0, 2*pi).
    ex = [xs[(i + 1) % n] - xs[i] for i in range(n)]
    ey = [ys[(i + 1) % n] - ys[i] for i in range(n)]
    best = 0
    for i in range(n):

```

```

        if half(ex[i], ey[i]) < half(ex[best], ey[best]) or (
            half(ex[i], ey[i]) == half(ex[best], ey[best]) and ex[best] * ey[i] - ey[best] * ex
        ):
            best = i
    xs = xs[best:] + xs[:best]
    ys = ys[best:] + ys[:best]
    ex = ex[best:] + ex[:best]
    ey = ey[best:] + ey[:best]
    eh = [half(ex[i], ey[i]) for i in range(n)]
    # prefix[k] = sum of cross(p_i, p_{i+1}) for i < k over the doubled cycle
    prefix = [0] * (2 * n + 1)
    for k in range(2 * n):
        i = k % n
        j = (k + 1) % n
        prefix[k + 1] = prefix[k] + xs[i] * ys[j] - xs[j] * ys[i]
    total2 = prefix[n]

def first_edge_not_before(tx, ty):
    """Index of the first edge whose angle is >= angle(t) (cyclic)."""
    th = half(tx, ty)
    lo, hi = 0, n
    while lo < hi:
        mid = (lo + hi) // 2
        h = eh[mid]
        before = h < th or (h == th and ex[mid] * ty - ey[mid] * tx > 0)
        if before:
            lo = mid + 1
        else:
            hi = mid
    return lo % n

out = []
pos = 2 + 2 * n
for _ in range(q):
    ax, ay, bx, by = data[pos:pos + 4]
    pos += 4
    dx, dy = bx - ax, by - ay

    def f(i):
        return dx * (ys[i] - ay) - dy * (xs[i] - ax)

    # f is the dot product with the left normal (-dy, dx); its maximum is
    # at the start of the first edge pointing past (-dy, dx) rotated by +90.
    imax = first_edge_not_before(-dx, -dy)
    imin = first_edge_not_before(dx, dy)
    fmax = f(imax)
    fmin = f(imin)
    if fmax <= 0:
        out.append("0")
        continue
    if fmin >= 0:
        out.append(str(total2 / 2))

```

```

        continue
# chain imin -> imax: f increases; last vertex with f <= 0
lo, hi = 0, (imax - imin) % n
while lo < hi:
    mid = (lo + hi + 1) // 2
    if f((imin + mid) % n) <= 0:
        lo = mid
    else:
        hi = mid - 1
i = (imin + lo) % n
# chain imax -> imin: f decreases; last vertex with f > 0
lo, hi = 0, (imin - imax) % n
while lo < hi:
    mid = (lo + hi + 1) // 2
    if f((imax + mid) % n) > 0:
        lo = mid
    else:
        hi = mid - 1
j = (imax + lo) % n
i1 = (i + 1) % n
j1 = (j + 1) % n
# X on edge (i, i1), Y on edge (j, j1), as exact fractions
fi, fi1, fj, fj1 = f(i), f(i1), f(j), f(j1)
d1 = fi1 - fi
d2 = fj - fj1
X0 = xs[i] * d1 + (xs[i1] - xs[i]) * (-fi)
X1 = ys[i] * d1 + (ys[i1] - ys[i]) * (-fi)
Y0 = xs[j] * d2 + (xs[j1] - xs[j]) * fj
Y1 = ys[j] * d2 + (ys[j1] - ys[j]) * fj
k0 = i1
k1 = j if j >= i1 else j + n
inner = prefix[k1] - prefix[k0]
num = ((X0 * ys[i1] - X1 * xs[i1]) * d2
        + inner * d1 * d2
        + (xs[j] * Y1 - ys[j] * Y0) * d1
        + (Y0 * X1 - Y1 * X0))
out.append(repr(num / (2 * d1 * d2)))
sys.stdout.write("\n".join(out) + "\n")

if __name__ == "__main__":
    main()

```

50. Chronicle Path Dictionary

Turn path occurrences into root-path suffix tests

Let $w = \text{LCA}(u, v)$. Split $S(u, v)$ into the upward part A (from u up to w , inclusive) and the downward part B (below w , down to v). Every occurrence lies inside A, inside B, or crosses the junction between them.

Read upward from its lowest vertex x , an occurrence inside A spells p . Equivalently, the root-to- x string **ends with reverse**(p). An occurrence inside B that ends at vertex y means the root-to- y string ends with p . So both kinds of occurrence become “ends with” tests on root-to-vertex strings.

Build one Aho–Corasick automaton for all patterns and their reversals, and run it down the tree: the state of x is the automaton state reached on the root-to- x string. A string ends with a dictionary word t exactly when its state lies in the fail-tree subtree of t ’s node, which is an interval of the fail-tree Euler order.

Occurrences inside A are the vertices x on the vertical segment from u up to depth $\text{depth}(w) + |p| - 1$ whose state lies in that interval. That count is (count on the root path of u) minus (count on the root path of the segment top’s parent). One DFS maintains a Fenwick tree over the Euler positions of the states on the current root path and evaluates every such event. B is symmetric, with bottom vertex v and top depth $\text{depth}(w) + |p|$.

A crossing occurrence uses at most $|p| - 1$ vertices on each side of the junction. Extract those at most $2|p| - 2$ letters (one ancestor jump plus a walk) and run KMP, counting only matches that start before the junction and end after it.

Correctness proof

Lemma 1. For a vertex x and a word t , the root-to- x string ends with t iff the automaton state of x is in the fail-subtree of $\text{node}(t)$.

Proof. The automaton state is the longest suffix of the read text that is a trie prefix. Its fail chain lists every suffix of the text that is a trie prefix, longest first. t is such a suffix iff $\text{node}(t)$ is on that chain, i.e. iff $\text{node}(t)$ is an ancestor of the state in the fail tree.

Lemma 2. The three counted families partition all occurrences, and each family is counted exactly.

Proof. An occurrence occupies consecutive positions of $A+B$. It lies before the junction, after it, or covers both sides of it. An occurrence inside A has a unique lowest vertex, on the stated segment, and it satisfies Lemma 1 with **reverse**(p). Conversely, such a vertex yields an occurrence that stays inside A by the depth bound. The case of B is symmetric. Crossing occurrences are exactly the matches of the extracted window that start before the junction and end after it.

Theorem. The printed value is the number of occurrences of p in $S(u, v)$.

Proof. The Fenwick tree holds exactly the states of the current root path, so each event returns the count of Lemma 1 on that root path. Differences of root paths give vertical segments. Lemma 2 completes the sum.

Complexity and traps

The automaton has at most $2S+1$ states for total pattern length S . Building it, the Euler tour, and the DFS take $O((n + S) \log S)$ with the Fenwick tree, and each query adds $O(\log n + |p|)$. Letters on the upward side must be read toward the root. $S(v, u)$ is the reverse of $S(u, v)$. The crossing window must include w itself on the A side. Recursion depth reaches n , so every DFS is iterative.

C++

```

#include <bits/stdc++.h>
using namespace std;

// Split the path u -> v at w = lca(u, v) into the upward part u..w and the
// downward part below w. An occurrence inside the upward part, read upward
// from its lowest vertex x, means that the root-to-x string ends with the
// reversed pattern; one inside the downward part, ending at vertex y, means
// that the root-to-y string ends with the pattern. Run one Aho--Corasick
// automaton (all patterns and their reversals) down the tree: "ends with" is
// membership of the vertex's state in a fail-tree subtree, so each part is a
// difference of two root-path counts, answered offline by a DFS with a Fenwick
// tree over fail-tree Euler positions. Occurrences crossing the junction lie
// within |p| - 1 vertices of w on each side and are found by KMP.

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    string label;
    cin >> label;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> qu(q), qv(q);
    vector<string> pats(q);
    for (int i = 0; i < q; i++) cin >> qu[i] >> qv[i] >> pats[i];

    vector<int> parent(n + 1, 0), depth(n + 1, 0), order = {1};
    {
        vector<char> seen(n + 1, 0);
        seen[1] = 1;
        for (size_t i = 0; i < order.size(); i++) {
            int v = order[i];
            for (int w : adj[v])
                if (!seen[w]) {
                    seen[w] = 1;
                    parent[w] = v;
                    depth[w] = depth[v] + 1;
                    order.push_back(w);
                }
        }
    }
    int LOG = 1;
    while ((1 << LOG) <= n) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n + 1));
    up[0] = parent;

```

```

for (int k = 1; k < LOG; k++)
    for (int v = 0; v <= n; v++) up[k][v] = up[k - 1][up[k - 1][v]];
auto ancestorAtDepth = [&](int v, int d) {
    int diff = depth[v] - d;
    for (int k = 0; diff; k++, diff >>= 1)
        if (diff & 1) v = up[k][v];
    return v;
};

auto lca = [&](int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = ancestorAtDepth(a, depth[b]);
    if (a == b) return a;
    for (int k = LOG - 1; k >= 0; k--)
        if (up[k][a] != up[k][b]) a = up[k][a], b = up[k][b];
    return parent[a];
};

// Aho--Corasick with full transition table.
vector<array<int, 26>> go(1);
go[0].fill(-1);
auto insert = [&](const string& w) {
    int s = 0;
    for (char ch : w) {
        int c = ch - 'a';
        if (go[s][c] < 0) {
            go[s][c] = go.size();
            go.emplace_back();
            go.back().fill(-1);
        }
        s = go[s][c];
    }
    return s;
};

vector<int> nodeFwd(q), nodeRev(q);
for (int i = 0; i < q; i++) {
    nodeFwd[i] = insert(pats[i]);
    nodeRev[i] = insert(string(pats[i].rbegin(), pats[i].rend()));
}

int T = go.size();
vector<int> fail(T, 0), bfs = {0};
for (size_t h = 0; h < bfs.size(); h++) {
    int s = bfs[h];
    for (int c = 0; c < 26; c++) {
        int t = go[s][c];
        if (t >= 0) {
            fail[t] = s ? go[fail[s]][c] : 0;
            bfs.push_back(t);
        } else {
            go[s][c] = s ? go[fail[s]][c] : 0;
        }
    }
}

```

```

}
vector<vector<int>> fchildren(T);
for (int t = 1; t < T; t++) fchildren[fail[t]].push_back(t);
vector<int> tin(T), tout(T);
{
    int timer = 0;
    vector<pair<int, int>> stack = {{0, 0}};
    while (!stack.empty()) {
        auto& [s, i] = stack.back();
        if (i == 0) tin[s] = ++timer;
        if (i < (int)fchildren[s].size()) {
            int t = fchildren[s][i++];
            stack.push_back({t, 0});
        } else {
            tout[s] = timer;
            stack.pop_back();
        }
    }
}

struct Event {
    int node, sign, id;
};
vector<vector<Event>> events(n + 1);
vector<long long> answer(q, 0);
for (int i = 0; i < q; i++) {
    int u = qu[i], v = qv[i];
    const string& p = pats[i];
    int L = p.size(), l = lca(u, v), dl = depth[l];
    if (depth[u] >= dl + L - 1) {
        int w = ancestorAtDepth(u, dl + L - 1);
        events[u].push_back({nodeRev[i], 1, i});
        if (parent[w]) events[parent[w]].push_back({nodeRev[i], -1, i});
    }
    if (v != 1 && depth[v] >= dl + L) {
        int w = ancestorAtDepth(v, dl + L);
        events[v].push_back({nodeFwd[i], 1, i});
        events[parent[w]].push_back({nodeFwd[i], -1, i});
    }
    if (L >= 2 && v != 1) {
        int a = min(L - 1, depth[u] - dl + 1), b = min(L - 1, depth[v] - dl);
        if (a + b >= L) {
            string text;
            for (int x = ancestorAtDepth(u, dl + a - 1); x = parent[x]) {
                text += label[x - 1];
                if (x == 1) break;
            }
            string right;
            for (int y = ancestorAtDepth(v, dl + b); y != 1; y = parent[y]) right += label[y - 1];
            text.append(right.rbegin(), right.rend());
            vector<int> pf(L, 0);

```

```

        for (int idx = 1, k = 0; idx < L; idx++) {
            while (k && p[idx] != p[k]) k = pf[k - 1];
            if (p[idx] == p[k]) k++;
            pf[idx] = k;
        }
        int k = 0;
        for (int idx = 0; idx < (int)text.size(); idx++) {
            while (k && text[idx] != p[k]) k = pf[k - 1];
            if (text[idx] == p[k]) k++;
            if (k == L) {
                int start = idx - L + 1;
                if (start < a && a <= idx) answer[i]++;
                k = pf[k - 1];
            }
        }
    }
}

vector<int> bit(T + 1, 0);
auto bitAdd = [&](int i, int x) {
    for (; i <= T; i += i & -i) bit[i] += x;
};
auto bitSum = [&](int i) {
    int s = 0;
    for (; i > 0; i -= i & -i) s += bit[i];
    return s;
};
vector<int> state(n + 1, 0);
vector<pair<int, int>> stack = {{1, 0}};
while (!stack.empty()) {
    auto& [v, i] = stack.back();
    if (i == 0) {
        int from = v == 1 ? 0 : state[parent[v]];
        state[v] = go[from][label[v - 1] - 'a'];
        bitAdd(tin[state[v]], 1);
        for (const Event& e : events[v])
            answer[e.id] += (long long)e.sign * (bitSum(tout[e.node]) - bitSum(tin[e.node] - 1));
    }
    if (i < (int)adj[v].size()) {
        int w = adj[v][i++];
        if (w != parent[v]) stack.push_back({w, 0});
    } else {
        bitAdd(tin[state[v]], -1);
        stack.pop_back();
    }
}
string out;
for (long long x : answer) out += to_string(x) + '\n';
cout << out;
}

```


Python

```
import sys
from array import array

def main():
    data = sys.stdin.buffer.read().split()
    n, q = int(data[0]), int(data[1])
    label = data[2]
    adj = [[] for _ in range(n + 1)]
    ptr = 3
    for _ in range(n - 1):
        u = int(data[ptr])
        v = int(data[ptr + 1])
        ptr += 2
        adj[u].append(v)
        adj[v].append(u)
    qu = [0] * q
    qv = [0] * q
    pats = [b""] * q
    for i in range(q):
        qu[i] = int(data[ptr])
        qv[i] = int(data[ptr + 1])
        pats[i] = data[ptr + 2]
        ptr += 3
    del data

    # Rooted tree, depths, binary lifting.
    parent = [0] * (n + 1)
    depth = [0] * (n + 1)
    order = [1]
    parent[1] = 0
    seen = [False] * (n + 1)
    seen[1] = True
    for v in order:
        for w in adj[v]:
            if not seen[w]:
                seen[w] = True
                parent[w] = v
                depth[w] = depth[v] + 1
                order.append(w)
    LOG = max(1, (n).bit_length())
    up = [array("i", parent)]
    for k in range(1, LOG):
        prev = up[-1]
        up.append(array("i", [prev[prev[v]] for v in range(n + 1)]))

    def ancestor_at_depth(v, d):
        diff = depth[v] - d
        k = 0
```

```

while diff:
    if diff & 1:
        v = up[k][v]
        diff >>= 1
        k += 1
return v

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    a = ancestor_at_depth(a, depth[b])
    if a == b:
        return a
    for k in range(LOG - 1, -1, -1):
        if up[k][a] != up[k][b]:
            a = up[k][a]
            b = up[k][b]
    return parent[a]

# Aho--Corasick automaton of every pattern and its reversal.
child = [{}]
```

```

def insert(word):
    s = 0
    for c in word:
        nxt = child[s].get(c)
        if nxt is None:
            nxt = len(child)
            child.append({})
            child[s][c] = nxt
        s = nxt
    return s

node_fwd = [insert(p) for p in pats]
node_rev = [insert(p[::-1]) for p in pats]
T = len(child)
fail = [0] * T
bfs = [0]
for s in bfs:
    for c, t in child[s].items():
        if s:
            f = fail[s]
            while f and c not in child[f]:
                f = fail[f]
            g = child[f].get(c, 0)
            fail[t] = g if g != t else 0
        bfs.append(t)

# Euler tour of the fail tree.
fchildren = [[] for _ in range(T)]
for t in range(1, T):
    fchildren[fail[t]].append(t)
tin = [0] * T

```

```

tout = [0] * T
timer = 0
stack = [(0, False)]
while stack:
    s, done = stack.pop()
    if done:
        tout[s] = timer
        continue
    timer += 1
    tin[s] = timer
    stack.append((s, True))
    for t in fchildren[s]:
        stack.append((t, False))
del fchildren

memo = [None] * T

def goto(s, c):
    path = []
    t = s
    while True:
        nxt = child[t].get(c)
        if nxt is not None:
            break
        m = memo[t]
        if m is not None and c in m:
            nxt = m[c]
            break
        if t == 0:
            nxt = 0
            break
        path.append(t)
        t = fail[t]
    for x in path:
        if memo[x] is None:
            memo[x] = {}
        memo[x][c] = nxt
    return nxt

# Occurrences are counted by their bottom vertex x on a vertical segment:
# the root-to-x string must end with the pattern (downward half) or with
# its reversal (upward half). Events: (vertex, automaton node, sign, id).
events = [[] for _ in range(n + 1)]
answer = [0] * q
for i in range(q):
    u, v, p = qu[i], qv[i], pats[i]
    L = len(p)
    l = lca(u, v)
    dl = depth[l]
    if depth[u] >= dl + L - 1:
        w = ancestor_at_depth(u, dl + L - 1)

```

```

        events[u].append((node_rev[i], 1, i))
    if parent[w]:
        events[parent[w]].append((node_rev[i], -1, i))
if v != 1 and depth[v] >= dl + L:
    w = ancestor_at_depth(v, dl + L)
    events[v].append((node_fwd[i], 1, i))
    events[parent[w]].append((node_fwd[i], -1, i))
if L >= 2 and v != 1:
    a = min(L - 1, depth[u] - dl + 1)
    b = min(L - 1, depth[v] - dl)
    if a + b >= L:
        x = ancestor_at_depth(u, dl + a - 1)
        left = []
        while True:
            left.append(label[x - 1])
            if x == 1:
                break
            x = parent[x]
        y = ancestor_at_depth(v, dl + b)
        right = []
        while y != 1:
            right.append(label[y - 1])
            y = parent[y]
        right.reverse()
        text = left + right
        # KMP: occurrences starting before the junction and ending after it.
        fail_p = [0] * L
        k = 0
        for idx in range(1, L):
            while k and p[idx] != p[k]:
                k = fail_p[k - 1]
            if p[idx] == p[k]:
                k += 1
            fail_p[idx] = k
        k = 0
        count = 0
        for idx, c in enumerate(text):
            while k and c != p[k]:
                k = fail_p[k - 1]
            if c == p[k]:
                k += 1
            if k == L:
                start = idx - L + 1
                if start < a <= idx:
                    count += 1
                k = fail_p[k - 1]
        answer[i] += count

# DFS down the tree with a Fenwick tree over fail-tree Euler positions.
bit = [0] * (T + 1)

```

```

def bit_add(i, x):
    while i <= T:
        bit[i] += x
        i += i & -i

def bit_sum(i):
    s = 0
    while i > 0:
        s += bit[i]
        i &= i - 1
    return s

state = [0] * (n + 1)
stack = [(1, False)]
while stack:
    v, done = stack.pop()
    if done:
        bit_add(tin[state[v]], -1)
        continue
    st = goto(state[parent[v]] if v != 1 else 0, label[v - 1])
    state[v] = st
    bit_add(tin[st], 1)
    for node, sign, i in events[v]:
        answer[i] += sign * (bit_sum(tout[node]) - bit_sum(tin[node] - 1))
    stack.append((v, True))
    for w in adj[v]:
        if w != parent[v]:
            stack.append((w, False))
sys.stdout.write("\n".join(map(str, answer)) + "\n")

if __name__ == "__main__":
    main()

```

51. Echo Census

From squares to runs

A **run** is a maximal substring $s[a..b)$ with smallest period p and length at least $2p$. Write an echo as ww with $w = u^k$, where u is primitive. Then ww has smallest period $|u|$ and length at least $2|u|$, so it lies inside a unique run of period $|u|$, with half-length $k|u|$.

Inside one run of period p , the echoes of half-length $L = kp$ start at $a, a+1, \dots, b-2L$, and the echoes starting at x and $x+p$ are equal. Hence the first $\min(p, b-2L-a+1)$ starting points already give every such echo of that run. Enumerate those for every run and multiple L , and deduplicate by `(hash, length)`.

The runs themselves come from the runs theorem. Fix one of the two letter orders; the end of the string is smaller than every letter in both. Let $lyn[i]$ be the length of the longest Lyndon word starting at i . Every run of period p contains, for one of the two orders, a position i with $lyn[i] = p$. Extending that root left and right with longest-common-extension queries recovers the whole run. The Lyndon array follows from the next-smaller-suffix recurrence $j += lyn[j]$, whose suffix comparisons are LCE queries.

Correctness proof

Lemma 1. Every echo lies in some run whose period is the length of its primitive root.

Proof. ww with $w = u^k$ has period $|u|$ and length $2k|u| \geq 2|u|$, and $|u|$ is its smallest period by Fine and Wilf. Extending it maximally while keeping that period gives a run containing it.

Lemma 2. In a run of period p , the echo of half-length L starting at x equals the one starting at $x+p$, whenever both lie in the run.

Proof. Both substrings have length $2L$ and lie in a string of period p . Shifting by the period does not change the letters.

Lemma 3. The Lyndon-root procedure outputs exactly the runs.

Proof. This is the runs theorem: each run has an L-root for one of the two orders, and the longest Lyndon word starting there is exactly that root. Conversely, every reported interval has period p and is maximal because both extensions stop at a mismatch or at an end of the string.

Theorem. The program prints the number of distinct echoes.

Proof. By Lemmas 1 and 3 every echo is some ww enumerated from its run. By Lemma 2 the restricted starting points lose none. Deduplication counts each string once.

Complexity and traps

The number of runs is below n , and the enumerated (start, L) pairs are bounded by the sum over runs of $\min(p, \text{len} - 2L + 1)$ over multiples L , which stays $O(n \log n)$. Each LCE costs $O(\log n)$ by galloping plus binary search on prefix hashes, giving $O(n \log n)$ overall. Trap: in the Lyndon recurrence, a suffix that is a prefix of the other suffix is the **smaller** one in both orders, so the comparison must be handled before comparing letters.

C++

```
#include <bits/stdc++.h>
using namespace std;

// A square ww whose root w = u^k (u primitive) lies inside a run of minimal
// period |u|, and inside one run the squares of half-length k|u| starting at
// x and x + |u| are equal. So enumerate all runs, and for each run and each
// multiple L of its period, only the first min(p, #starts) starting positions;
// deduplicate those squares by hash. Runs come from the runs theorem: under
// one of the two letter orders, every run contains a position i whose longest
// Lyndon word has length exactly the period. Lyndon arrays and all
// longest-common-extension queries use polynomial hashing modulo 2^61 - 1.

typedef unsigned long long ull;
const ull MOD = (1ULL << 61) - 1, BASE = 1000003;

ull mulmod(ull a, ull b) {
    __uint128_t c = (__uint128_t)a * b;
    ull lo = (ull)(c & MOD), hi = (ull)(c >> 61);
    ull r = lo + hi;
```

```

    return r >= MOD ? r - MOD : r;
}

int n;
string s, rs;
vector<ull> pw, fh, rh;

ull getHash(const vector<ull>& h, int i, int len) {
    ull v = h[i + len] + MOD - mulmod(h[i], pw[len]);
    return v >= MOD ? v - MOD : v;
}

// Longest common prefix of suffixes starting at i and j of the string with
// prefix hashes h.
int extend(const string& t, const vector<ull>& h, int i, int j) {
    if (i == j) return n - i;
    int limit = n - max(i, j);
    if (limit == 0 || t[i] != t[j]) return 0;
    int good = 1, step = 1, hi;
    while (true) {
        long long nxt = (long long)good + step;
        if (nxt > limit || getHash(h, i, nxt) != getHash(h, j, nxt)) {
            hi = (int)min<long long>(nxt, limit + 1);
            break;
        }
        good = nxt;
        step *= 2;
    }
    int lo = good;
    while (hi - lo > 1) {
        int mid = (lo + hi) / 2;
        if (getHash(h, i, mid) == getHash(h, j, mid)) lo = mid;
        else hi = mid;
    }
    return lo;
}

int lce(int i, int j) { return extend(s, fh, i, j); }
int lcs(int i, int j) { // common suffix of prefixes ending at i and j
    if (i < 0 || j < 0) return 0;
    return extend(rs, rh, n - 1 - i, n - 1 - j);
}

int main() {
    cin >> s;
    n = s.size();
    rs = string(s.rbegin(), s.rend());
    pw.assign(n + 1, 1);
    fh.assign(n + 1, 0);
    rh.assign(n + 1, 0);
    for (int i = 0; i < n; i++) {

```

```

        pw[i + 1] = mulmod(pw[i], BASE);
        fh[i + 1] = (mulmod(fh[i], BASE) + (ull)s[i]) % MOD;
        rh[i + 1] = (mulmod(rh[i], BASE) + (ull)rs[i]) % MOD;
    }
    vector<array<int, 3>> runs;
    vector<int> lyn(n);
    for (int flip = 0; flip < 2; flip++) {
        for (int i = n - 1; i >= 0; i--) {
            int j = i + 1;
            while (j < n) {
                int l = lce(i, j);
                if (j + l == n) break; // suffix j is a prefix of suffix i
                bool smaller = s[i + l] < s[j + l];
                if (smaller != (bool)flip) j += lyn[j];
                else break;
            }
            lyn[i] = j - i;
        }
        for (int i = 0; i < n; i++) {
            int p = lyn[i], j = i + p;
            if (j > n) continue;
            int r = j < n ? lce(i, j) : 0;
            int l = lcs(i - 1, j - 1);
            if (l + r >= p) runs.push_back({i - l, j + r, p});
        }
    }
    sort(runs.begin(), runs.end());
    runs.erase(unique(runs.begin(), runs.end()), runs.end());
    vector<pair<ull, int>> squares;
    for (auto [a, b, p] : runs) {
        int length = b - a;
        for (int L = p; 2 * L <= length; L += p) {
            int lastStart = b - 2 * L;
            int upto = min(lastStart, a + p - 1);
            for (int x = a; x <= upto; x++) squares.push_back({getHash(fh, x, 2 * L), L});
        }
    }
    sort(squares.begin(), squares.end());
    squares.erase(unique(squares.begin(), squares.end()), squares.end());
    cout << squares.size() << '\n';
}

```

Python

```

import sys

MOD = (1 << 61) - 1
BASE = 1000003

```



```

def main():
    s = sys.stdin.buffer.read().split()[0]
    n = len(s)
    # Forward prefix hashes, and prefix hashes of the reversed string for
    # longest-common-suffix queries.
    pw = [1] * (n + 1)
    for i in range(n):
        pw[i + 1] = pw[i] * BASE % MOD
    h = [0] * (n + 1)
    for i in range(n):
        h[i + 1] = (h[i] * BASE + s[i]) % MOD
    rs = s[::-1]
    rh = [0] * (n + 1)
    for i in range(n):
        rh[i + 1] = (rh[i] * BASE + rs[i]) % MOD

    def fwd(i, length):
        return (h[i + length] - h[i] * pw[length]) % MOD

    def lce(i, j):
        """Longest common prefix of suffixes i and j (galloping + binary search)."""
        if i == j:
            return n - i
        limit = n - max(i, j)
        if limit == 0 or s[i] != s[j]:
            return 0
        step = 1
        good = 1
        while True:
            nxt = good + step
            if nxt > limit or fwd(i, nxt) != fwd(j, nxt):
                hi = min(nxt, limit + 1)
                break
            good = nxt
            step *= 2
        lo = good
        while hi - lo > 1:
            mid = (lo + hi) >> 1
            if fwd(i, mid) == fwd(j, mid):
                lo = mid
            else:
                hi = mid
        return lo

    def lcs(i, j):
        """Longest common suffix of prefixes ending at i and j (inclusive)."""
        if i < 0 or j < 0:
            return 0
        if s[i] != s[j]:
            return 0
        a = n - 1 - i

```

```

b = n - 1 - j
limit = n - max(a, b)
step = 1
good = 1
while True:
    nxt = good + step
    if nxt > limit or (rh[a + nxt] - rh[a] * pw[nxt]) % MOD != (rh[b + nxt] - rh[b] * pw[nxt]) % MOD:
        hi = min(nxt, limit + 1)
        break
    good = nxt
    step *= 2
lo = good
while hi - lo > 1:
    mid = (lo + hi) >> 1
    if (rh[a + mid] - rh[a] * pw[mid]) % MOD == (rh[b + mid] - rh[b] * pw[mid]) % MOD:
        lo = mid
    else:
        hi = mid
return lo

runs = set()
for flip in (False, True):
    # Lyndon array under the letter order (reversed when flip), with the
    # end of the string smaller than every letter in both orders.
    lyn = [1] * n
    for i in range(n - 1, -1, -1):
        j = i + 1
        while j < n:
            l = lce(i, j)
            if j + 1 == n:
                break # suffix j is a prefix of suffix i: suffix j is smaller
            smaller = s[i + 1] < s[j + 1]
            if smaller != flip:
                j += lyn[j] # suffix i < suffix j: extend the Lyndon word
            else:
                break
        lyn[i] = j - i
    for i in range(n):
        p = lyn[i]
        j = i + p
        if j > n:
            continue
        r = lce(i, j) if j < n else 0
        l = lcs(i - 1, j - 1)
        if l + r >= p:
            runs.add((i - l, j + r, p))

squares = set()
for a, b, p in runs:
    length = b - a
    L = p

```

```

    while 2 * L <= length:
        last_start = b - 2 * L
        for x in range(a, min(last_start, a + p - 1) + 1):
            squares.add(fwd(x, 2 * L) * (n + 1) + L)
        L += p
    print(len(squares))

if __name__ == "__main__":
    main()

```

52. Tariff Revision Network

Divide and conquer over the revision timeline

Recursively solve a time interval $[l, r]$ of revisions. Links revised inside the interval are *dynamic*; every other link has a fixed tariff there.

1. **Contraction.** Union all dynamic links first, then run Kruskal on the static links. A static link still accepted is in *every* minimum spanning tree of every moment in $[l, r]$. Add its tariff to a running total and contract its endpoints.
2. **Reduction.** On the contracted graph, run Kruskal on the static links alone. A static link rejected now closes a cycle of cheaper static links and is never needed in $[l, r]$, so drop it.

After both steps an interval with k dynamic links keeps $O(k)$ vertices and static edges. Split the interval, and at a single revision apply it and run Kruskal on the tiny remaining graph.

Correctness proof

Lemma 1. A static link chosen during contraction belongs to some MST at every moment of $[l, r]$.

Proof. Whatever tariffs the dynamic links take, it is at most as good as treating them as infinitely cheap. Kruskal with the dynamic links first accepts the static link e only if its endpoints are not yet connected by dynamic links plus cheaper static links. By the cut property, e is then a minimum edge across some cut for every assignment of dynamic tariffs.

Lemma 2. A static link rejected during reduction belongs to no needed MST at any moment of $[l, r]$.

Proof. It closes a cycle of static links that are no more expensive and whose tariffs are fixed in the interval. By the cycle property it can be replaced in every spanning tree.

Theorem. Each printed value is the MST weight after that revision.

Proof. By Lemmas 1 and 2 each recursion level preserves the MST weight of every moment of its interval (plus the accumulated contracted weight). A leaf computes it directly.

Complexity and traps

Every level processes $O(q)$ edges in total, so the time is $O((m + q) \log q \log m)$ with sorting and the memory is $O(m + q \log q)$. Revisions must be applied in time order: the left half runs before the right half reads the updated tariffs. Self-loops and parallel links are handled by the DSU. Stamp dynamic links with a fresh counter per call, not with 1.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Offline dynamic MST by divide and conquer over the update timeline.
// At a node covering updates [l, r], the edges modified inside the range are
// "dynamic"; every other edge has a fixed weight throughout the range.
// * Contraction: static edges chosen by Kruskal even when all dynamic edges
//   are forced first belong to every MST of the range -> contract them.
// * Reduction: static edges rejected by Kruskal on static edges alone can
//   never enter an MST of the range -> delete them.
// After both steps a node with k dynamic edges keeps  $O(k)$  vertices and edges.

struct Edge {
    int u, v, id;
};

int n, m, q;
vector<long long> weight;
vector<int> qe;
vector<long long> qw;
vector<int> dynamicStamp;
vector<long long> answers;

struct DSU {
    vector<int> parent;
    void reset(int size) {
        parent.resize(size);
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) x = parent[x] = parent[parent[x]];
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        parent[a] = b;
        return true;
    }
};

void solve(int l, int r, int vertices, vector<Edge> edges, long long base) {
    auto byWeight = [](const Edge& a, const Edge& b) {
        return weight[a.id] < weight[b.id];
    };
    if (l == r) {
        weight[qe[l]] = qw[l];
        sort(edges.begin(), edges.end(), byWeight);
    }
```

```

    DSU dsu;
    dsu.reset(vertices);
    long long total = base;
    for (const Edge& e : edges)
        if (dsu.unite(e.u, e.v)) total += weight[e.id];
    answers[l] = total;
    return;
}

static int stampCounter = 0;
int stamp = ++stampCounter;
for (int i = l; i <= r; i++) dynamicStamp[qe[i]] = stamp;

vector<Edge> dyn, stat;
for (const Edge& e : edges)
    (dynamicStamp[e.id] == stamp ? dyn : stat).push_back(e);
sort(stat.begin(), stat.end(), byWeight);

// Contraction.
DSU dsu;
dsu.reset(vertices);
for (const Edge& e : dyn) dsu.unite(e.u, e.v);
DSU forced;
forced.reset(vertices);
for (const Edge& e : stat)
    if (dsu.unite(e.u, e.v)) {
        forced.unite(e.u, e.v);
        base += weight[e.id];
    }
vector<int> label(vertices, -1);
int newVertices = 0;
for (int v = 0; v < vertices; v++) {
    int root = forced.find(v);
    if (label[root] < 0) label[root] = newVertices++;
    label[v] = label[root];
}

// Reduction on the contracted graph.
dsu.reset(newVertices);
vector<Edge> next;
next.reserve(dyn.size() * 2 + 1);
for (const Edge& e : stat) {
    int a = label[e.u], b = label[e.v];
    if (a != b && dsu.unite(a, b)) next.push_back({a, b, e.id});
}
for (const Edge& e : dyn) next.push_back({label[e.u], label[e.v], e.id});

int mid = (l + r) / 2;
solve(l, mid, newVertices, next, base);
solve(mid + 1, r, newVertices, next, base);
}

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m >> q;
    weight.resize(m);
    vector<Edge> edges(m);
    for (int i = 0; i < m; i++) {
        cin >> edges[i].u >> edges[i].v >> weight[i];
        edges[i].u--;
        edges[i].v--;
        edges[i].id = i;
    }
    qe.resize(q);
    qw.resize(q);
    for (int i = 0; i < q; i++) {
        cin >> qe[i] >> qw[i];
        qe[i]--;
    }
    dynamicStamp.assign(m, 0);
    answers.resize(q);
    solve(0, q - 1, n, edges, 0);
    string out;
    for (long long x : answers) out += to_string(x) + '\n';
    cout << out;
}

```

Python

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    n, m, q = int(data[0]), int(data[1]), int(data[2])
    pos = 3
    weight = [0] * m
    eu = [0] * m
    ev = [0] * m
    for i in range(m):
        eu[i] = int(data[pos]) - 1
        ev[i] = int(data[pos + 1]) - 1
        weight[i] = int(data[pos + 2])
        pos += 3
    qe = [0] * q
    qw = [0] * q
    for i in range(q):
        qe[i] = int(data[pos]) - 1
        qw[i] = int(data[pos + 1])
        pos += 2
    answers = [0] * q

```

```

stamp_of = [0] * m
counter = [0]
key = weight.__getitem__

def find(parent, x):
    root = x
    while parent[root] != root:
        root = parent[root]
    while parent[x] != root:
        parent[x], x = root, parent[x]
    return root

# Each edge list is three parallel lists: local endpoints and edge ids.
def solve(l, r, vertices, us, vs, ids, base):
    if l == r:
        weight[qe[l]] = qw[l]
        order = sorted(range(len(ids)), key=lambda i: weight[ids[i]])
        parent = list(range(vertices))
        total = base
        for i in order:
            a = find(parent, us[i])
            b = find(parent, vs[i])
            if a != b:
                parent[a] = b
                total += weight[ids[i]]
        answers[l] = total
        return
    counter[0] += 1
    stamp = counter[0]
    for i in range(l, r + 1):
        stamp_of[qe[i]] = stamp
    dyn = []
    stat = []
    for i in range(len(ids)):
        (dyn if stamp_of[ids[i]] == stamp else stat).append(i)
    stat.sort(key=lambda i: weight[ids[i]])

    # Contraction: static edges still needed after all dynamic edges.
    parent = list(range(vertices))
    for i in dyn:
        a = find(parent, us[i])
        b = find(parent, vs[i])
        if a != b:
            parent[a] = b
    forced = list(range(vertices))
    for i in stat:
        a = find(parent, us[i])
        b = find(parent, vs[i])
        if a != b:
            parent[a] = b
            a = find(forced, us[i])

```

```

        b = find(forced, vs[i])
        forced[a] = b
        base += weight[ids[i]]
label = [-1] * vertices
count = 0
for v in range(vertices):
    root = find(forced, v)
    if label[root] < 0:
        label[root] = count
        count += 1
    label[v] = label[root]

# Reduction: static edges outside the static-only forest are useless.
parent = list(range(count))
nu = []
nv = []
nid = []
for i in stat:
    a = label[us[i]]
    b = label[vs[i]]
    if a == b:
        continue
    ra = find(parent, a)
    rb = find(parent, b)
    if ra != rb:
        parent[ra] = rb
        nu.append(a)
        nv.append(b)
        nid.append(ids[i])
for i in dyn:
    nu.append(label[us[i]])
    nv.append(label[vs[i]])
    nid.append(ids[i])
mid = (l + r) // 2
solve(l, mid, count, nu, nv, nid, base)
solve(mid + 1, r, count, nu, nv, nid, base)

sys.setrecursionlimit(10000)
solve(0, q - 1, n, eu, ev, list(range(m)), 0)
sys.stdout.write("\n".join(map(str, answers)) + "\n")

if __name__ == "__main__":
    main()

```


53. Spanning Weight Spectrum

Weights as exponents

Give a cable of weight w the formal weight x^w in the Laplacian. By the matrix-tree theorem any cofactor equals

$$F(x) = \sum \text{over spanning trees } T \text{ of } x^{\text{weight}(T)},$$

so the coefficient of x^W is the requested count. F has degree at most $D = (n-1)K$. Evaluate $F(0), \dots, F(D)$ by modular Gaussian elimination and interpolate the coefficients in $O(D^2)$ with the master polynomial $\text{prod}(x - i)$ and synthetic division.

Correctness proof

Lemma 1. For a numeric x , the determinant of the reduced Laplacian with entries built from x^w equals $F(x)$.

Proof. The weighted matrix-tree theorem states that the cofactor equals the sum over spanning trees of the product of edge weights, here $x^{\text{sum of } w}$. Self-loops contribute nothing to a Laplacian.

Lemma 2. $D+1$ distinct evaluation points determine a polynomial of degree at most D over the field $\mathbb{Z}/998244353$.

Proof. The Vandermonde matrix of distinct points is invertible; Lagrange's formula writes the unique interpolant.

Theorem. The output lists the number of spanning trees of each weight modulo the prime.

Proof. Lemma 1 gives correct values of F modulo the prime, and Lemma 2 recovers its coefficients.

Complexity and traps

$O(Dn^3 + D^2)$ time, which is at most about $9.5 \cdot 10^7$ field operations at the limits, and $O(n^2 + D)$ memory. $0^0 = 1$ makes weight-0 cables count at $x = 0$. A disconnected graph gives the zero polynomial, and $n = 1$ prints the single empty tree.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Give an edge of weight w the formal weight x^w. By the matrix-tree theorem
// any cofactor of the Laplacian is F(x) = sum_T x^{weight(T)}, a polynomial of
// degree at most D = (n-1)K. Evaluate F at x = 0..D by Gaussian elimination
// modulo the prime, then recover its coefficients by Lagrange interpolation.

const long long MOD = 998244353;

long long power(long long b, long long e) {
```

```

    long long r = 1;
    b %= MOD;
    if (b < 0) b += MOD;
    while (e) {
        if (e & 1) r = r * b % MOD;
        b = b * b % MOD;
        e >>= 1;
    }
    return r;
}

long long determinant(vector<vector<long long>> a) {
    int n = a.size();
    long long det = 1;
    for (int c = 0; c < n; c++) {
        int pivot = -1;
        for (int r = c; r < n; r++)
            if (a[r][c]) {
                pivot = r;
                break;
            }
        if (pivot < 0) return 0;
        if (pivot != c) {
            swap(a[pivot], a[c]);
            det = (MOD - det) % MOD;
        }
        det = det * a[c][c] % MOD;
        long long inv = power(a[c][c], MOD - 2);
        for (int r = c + 1; r < n; r++) {
            if (!a[r][c]) continue;
            long long f = a[r][c] * inv % MOD;
            for (int k = c; k < n; k++) a[r][k] = (a[r][k] - f * a[c][k]) % MOD;
            for (int k = c; k < n; k++)
                if (a[r][k] < 0) a[r][k] += MOD;
        }
    }
    return det;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, K;
    cin >> n >> m >> K;
    vector<array<int, 3>> edges(m);
    for (auto& [u, v, w] : edges) {
        cin >> u >> v >> w;
        --u;
        --v;
    }
    int D = (n - 1) * K;

```

```

vector<long long> values(D + 1);
for (int x = 0; x <= D; x++) {
    vector<long long> pw(K + 1, 1);
    for (int i = 1; i <= K; i++) pw[i] = pw[i - 1] * x % MOD;
    vector<vector<long long>> lap(n - 1, vector<long long>(n - 1, 0));
    for (auto [u, v, w] : edges) {
        if (u == v) continue;
        long long t = pw[w];
        if (u < n - 1) lap[u][u] = (lap[u][u] + t) % MOD;
        if (v < n - 1) lap[v][v] = (lap[v][v] + t) % MOD;
        if (u < n - 1 && v < n - 1) {
            lap[u][v] = (lap[u][v] - t + MOD) % MOD;
            lap[v][u] = (lap[v][u] - t + MOD) % MOD;
        }
    }
    values[x] = determinant(lap);
}

// Interpolate through (i, values[i]), i = 0..D.
vector<long long> master(D + 2, 0); // prod (x - i)
master[0] = 1;
for (int i = 0; i <= D; i++) {
    for (int k = i + 1; k >= 1; k--) master[k] = (master[k - 1] - master[k] * i) % MOD;
    master[0] = master[0] * (MOD - i) % MOD;
}
for (auto& c : master) c = (c % MOD + MOD) % MOD;
vector<long long> fact(D + 1, 1);
for (int i = 1; i <= D; i++) fact[i] = fact[i - 1] * i % MOD;
vector<long long> coef(D + 1, 0), quotient(D + 1);
for (int i = 0; i <= D; i++) {
    if (!values[i]) continue;
    long long denom = fact[i] * fact[D - i] % MOD;
    if ((D - i) % 2) denom = (MOD - denom) % MOD;
    long long scale = values[i] * power(denom, MOD - 2) % MOD;
    // quotient = master / (x - i), synthetic division from the top.
    long long carry = 0;
    for (int k = D + 1; k >= 1; k--) {
        carry = (master[k] + carry * i) % MOD;
        quotient[k - 1] = carry;
    }
    for (int k = 0; k <= D; k++) coef[k] = (coef[k] + scale * quotient[k]) % MOD;
}
string out;
for (int k = 0; k <= D; k++) {
    out += to_string(coef[k]);
    out += k < D ? ' ' : '\n';
}
cout << out;
}

```

Python

```
import sys

MOD = 998244353

def determinant(a):
    n = len(a)
    det = 1
    for c in range(n):
        pivot = -1
        for r in range(c, n):
            if a[r][c]:
                pivot = r
                break
        if pivot < 0:
            return 0
        if pivot != c:
            a[pivot], a[c] = a[c], a[pivot]
            det = -det
        row = a[c]
        det = det * row[c] % MOD
        inv = pow(row[c], MOD - 2, MOD)
        tail = row[c:]
        for r in range(c + 1, n):
            other = a[r]
            if other[c]:
                f = other[c] * inv % MOD
                other[c:] = [(x - f * y) % MOD for x, y in zip(other[c:], tail)]
    return det % MOD

def main():
    data = sys.stdin.buffer.read().split()
    n, m, K = int(data[0]), int(data[1]), int(data[2])
    edges = []
    for i in range(m):
        u = int(data[3 + 3 * i]) - 1
        v = int(data[4 + 3 * i]) - 1
        w = int(data[5 + 3 * i])
        if u != v:
            edges.append((u, v, w))
    D = (n - 1) * K
    values = []
    size = n - 1
    for x in range(D + 1):
        pw = [1] * (K + 1)
        for i in range(1, K + 1):
            pw[i] = pw[i - 1] * x % MOD
        lap = [[0] * size for _ in range(size)]
```

```

    for u, v, w in edges:
        t = pw[w]
        if u < size:
            lap[u][u] += t
        if v < size:
            lap[v][v] += t
        if u < size and v < size:
            lap[u][v] -= t
            lap[v][u] -= t
    lap = [[value % MOD for value in row] for row in lap]
    values.append(determinant(lap))

# Lagrange interpolation through (i, values[i]), i = 0..D.
master = [1]
for i in range(D + 1):
    nxt = [0] * (len(master) + 1)
    for k, c in enumerate(master):
        nxt[k + 1] += c
        nxt[k] -= c * i
    master = [c % MOD for c in nxt]
fact = [1] * (D + 1)
for i in range(1, D + 1):
    fact[i] = fact[i - 1] * i % MOD
coef = [0] * (D + 1)
for i in range(D + 1):
    if not values[i]:
        continue
    denom = fact[i] * fact[D - i] % MOD
    if (D - i) % 2:
        denom = MOD - denom
    scale = values[i] * pow(denom, MOD - 2, MOD) % MOD
    quotient = [0] * (D + 1)
    carry = 0
    for k in range(D + 1, 0, -1):
        carry = (master[k] + carry * i) % MOD
        quotient[k - 1] = carry
    coef = [(c + scale * t) % MOD for c, t in zip(coef, quotient)]
sys.stdout.write(" ".join(map(str, coef)) + "\n")

if __name__ == "__main__":
    main()

```

54. Fleet Pairing

Primal–dual blossom algorithm

Maximum-weight matching in a general graph is solved by Edmonds' blossom algorithm with dual variables. Every vertex carries y_v (initially the maximum weight) and every odd set B carries $z_B \geq 0$. We keep all edges feasible, $y_u + y_v + \sum z_B \geq 2w(uv)$, and grow alternating trees from all free vertices using only

tight edges. An edge between two outer vertices of different trees augments; one inside a tree closes an odd cycle, which is shrunk to a blossom. When no tight edge helps, change the duals by the smallest of four bounds:

1. an outer vertex dual reaches zero;
2. an edge from an outer vertex to a free vertex becomes tight;
3. an edge between two outer blossoms becomes tight;
4. an inner blossom's z reaches zero, which expands it.

Parallel offers keep only the best one. Weights are doubled so that every dual change stays an integer.

Correctness proof

Lemma 1. Throughout the algorithm the duals are feasible and every matched edge and every blossom's internal matched structure stay tight.

Proof. The delta choice is the minimum of exactly the quantities that would first violate feasibility or nonnegativity. Tree vertices move in opposite directions along every tight tree edge.

Lemma 2. At termination the complementary-slackness conditions hold: matched edges are tight, positive- z blossoms are full, and every free vertex has $y_v = 0$.

Proof. A stage ends either by augmentation or by delta type 1, which drives the duals of all outer vertices, in particular all free vertices, to zero. Blossoms with positive z are only created as tight odd cycles, and augmentation preserves their near-perfect matchings.

Theorem. The final matching has maximum weight.

Proof. By Edmonds' LP description of the matching polytope, feasible duals bound every matching's weight from above. Lemma 2 gives equality for the constructed matching.

Complexity and traps

There are $O(n)$ stages with $O(n)$ dual changes each, and each change costs $O(n)$ thanks to the per-blossom best-edge lists. That is $O(n^3)$ time and $O(n + m)$ memory. Handling blossoms inside blossoms is where implementations go wrong: relabel with the stored endpoint lists, rotate children on augmentation, and expand zero-dual blossoms at the end of a stage.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Maximum-weight matching in a general graph: Edmonds' primal--dual blossom
// algorithm in Galil's  $O(n^3)$  formulation (same structure as the Python
// reference). Vertex duals start at the maximum weight; each stage grows
// alternating trees on tight edges, shrinks odd cycles into blossoms, and
// otherwise moves the duals by the smallest of four slack bounds.

typedef long long ll;

struct Matcher {
    int nv, ne;
```

```

vector<array<ll, 3>> edges; // u, v, doubled weight
vector<int> endpoint, mate, label, labelend, inblossom, blossomparent, blossombase, bestedge,
    unusedblossoms, queue;
vector<vector<int>> neighbend, blossomchilds, blossomendps, blossombestedges;
vector<char> hasBestList, allowedge;
vector<ll> dualvar;

ll slack(int k) { return dualvar[edges[k][0]] + dualvar[edges[k][1]] - 2 * edges[k][2]; }

void leaves(int b, vector<int>& out) {
    if (b < nv) {
        out.push_back(b);
        return;
    }
    for (int t : blossomchilds[b]) leaves(t, out);
}

void assignLabel(int w, int t, int p) {
    while (true) {
        int b = inblossom[w];
        label[w] = label[b] = t;
        labelend[w] = labelend[b] = p;
        bestedge[w] = bestedge[b] = -1;
        if (t == 1) {
            leaves(b, queue);
            return;
        }
        int base = blossombase[b];
        w = endpoint[mate[base]];
        t = 1;
        p = mate[base] ^ 1;
    }
}

int scanBlossom(int v, int w) {
    vector<int> path;
    int base = -1;
    while (v != -1 || w != -1) {
        int b = inblossom[v];
        if (label[b] & 4) {
            base = blossombase[b];
            break;
        }
        path.push_back(b);
        label[b] = 5;
        if (labelend[b] == -1) {
            v = -1;
        } else {
            v = endpoint[labelend[b]];
            b = inblossom[v];
            v = endpoint[labelend[b]];
        }
    }
}

```

```

    }
    if (w != -1) swap(v, w);
}
for (int b : path) label[b] = 1;
return base;
}

void addBlossom(int base, int k) {
    int v = edges[k][0], w = edges[k][1];
    int bb = inblossom[base], bv = inblossom[v], bw = inblossom[w];
    int b = unusedblossoms.back();
    unusedblossoms.pop_back();
    blossombase[b] = base;
    blossomparent[b] = -1;
    blossomparent[bb] = b;
    vector<int> path, endps;
    while (bv != bb) {
        blossomparent[bv] = b;
        path.push_back(bv);
        endps.push_back(labelend[bv]);
        v = endpoint[labelend[bv]];
        bv = inblossom[v];
    }
    path.push_back(bb);
    reverse(path.begin(), path.end());
    reverse(endps.begin(), endps.end());
    endps.push_back(2 * k);
    while (bw != bb) {
        blossomparent[bw] = b;
        path.push_back(bw);
        endps.push_back(labelend[bw] ^ 1);
        w = endpoint[labelend[bw]];
        bw = inblossom[w];
    }
    blossomchilds[b] = path;
    blossomendps[b] = endps;
    label[b] = 1;
    labelend[b] = labelend[bb];
    dualvar[b] = 0;
    vector<int> lv;
    leaves(b, lv);
    for (int x : lv) {
        if (label[inblossom[x]] == 2) queue.push_back(x);
        inblossom[x] = b;
    }
    vector<int> bestedgeto(2 * nv, -1);
    for (int sub : path) {
        vector<int> candidates;
        if (!hasBestList[sub]) {
            vector<int> sl;
            leaves(sub, sl);

```



```

        for (int x : sl)
            for (int p : neighbend[x]) candidates.push_back(p >> 1);
    } else {
        candidates = blossombestedges[sub];
    }
    for (int kk : candidates) {
        int i = edges[kk][0], j = edges[kk][1];
        if (inblossom[j] == b) swap(i, j);
        int bj = inblossom[j];
        if (bj != b && label[bj] == 1 && (bestedgeto[bj] == -1 || slack(kk) < slack(bestedgeto[bj]))
            bestedgeto[bj] = kk;
    }
    hasBestList[sub] = 0;
    blossombestedges[sub].clear();
    bestedge[sub] = -1;
}
blossombestedges[b].clear();
for (int kk : bestedgeto)
    if (kk != -1) blossombestedges[b].push_back(kk);
hasBestList[b] = 1;
bestedge[b] = -1;
for (int kk : blossombestedges[b])
    if (bestedge[b] == -1 || slack(kk) < slack(bestedge[b])) bestedge[b] = kk;
}

void expandBlossom(int b, bool endstage) {
    for (int s : blossomchilds[b]) {
        blossomparent[s] = -1;
        if (s < nv) {
            inblossom[s] = s;
        } else if (endstage && dualvar[s] == 0) {
            expandBlossom(s, endstage);
        } else {
            vector<int> lv;
            leaves(s, lv);
            for (int x : lv) inblossom[x] = s;
        }
    }
    if (!endstage && label[b] == 2) {
        vector<int>& childs = blossomchilds[b];
        vector<int>& endps = blossomendps[b];
        int L = childs.size();
        auto at = [&](vector<int>& vec, int idx) -> int& { return vec[((idx % L) + L) % L]; };
        int entrychild = inblossom[endpoint[labelend[b] ^ 1]];
        int j = find(childs.begin(), childs.end(), entrychild) - childs.begin();
        int jstep, endptrick;
        if (j & 1) {
            j -= L;
            jstep = 1;
            endptrick = 0;
        } else {

```

```

        jstep = -1;
        endptrick = 1;
    }
    int p = labelend[b];
    while (j != 0) {
        label[endpoint[p ^ 1]] = 0;
        label[endpoint[at(endps, j - endptrick) ^ endptrick ^ 1]] = 0;
        assignLabel(endpoint[p ^ 1], 2, p);
        allowedge[at(endps, j - endptrick) >> 1] = 1;
        j += jstep;
        p = at(endps, j - endptrick) ^ endptrick;
        allowedge[p >> 1] = 1;
        j += jstep;
    }
    int bv = at(childs, j);
    label[endpoint[p ^ 1]] = label[bv] = 2;
    labelend[endpoint[p ^ 1]] = labelend[bv] = p;
    bestedge[bv] = -1;
    j += jstep;
    while (at(childs, j) != entrychild) {
        bv = at(childs, j);
        if (label[bv] == 1) {
            j += jstep;
            continue;
        }
        vector<int> lv;
        leaves(bv, lv);
        int found = -1;
        for (int x : lv)
            if (label[x] != 0) {
                found = x;
                break;
            }
        if (found >= 0) {
            label[found] = 0;
            label[endpoint[mate[blossombase[bv]]]] = 0;
            assignLabel(found, 2, labelend[found]);
        }
        j += jstep;
    }
}
label[b] = labelend[b] = -1;
blossomchilds[b].clear();
blossomendps[b].clear();
blossombase[b] = -1;
blossombestedges[b].clear();
hasBestList[b] = 0;
bestedge[b] = -1;
unusedblossoms.push_back(b);
}

```

```

void augmentBlossom(int b, int v) {
    int t = v;
    while (blossomparent[t] != b) t = blossomparent[t];
    if (t >= nv) augmentBlossom(t, v);
    vector<int>& childs = blossomchilds[b];
    vector<int>& endps = blossomendps[b];
    int L = childs.size();
    auto at = [&](vector<int>& vec, int idx) -> int& { return vec[((idx % L) + L) % L]; };
    int i = find(childs.begin(), childs.end(), t) - childs.begin();
    int j = i, jstep, endptrick;
    if (i & 1) {
        j -= L;
        jstep = 1;
        endptrick = 0;
    } else {
        jstep = -1;
        endptrick = 1;
    }
    while (j != 0) {
        j += jstep;
        t = at(childs, j);
        int p = at(endps, j - endptrick) ^ endptrick;
        if (t >= nv) augmentBlossom(t, endpoint[p]);
        j += jstep;
        t = at(childs, j);
        if (t >= nv) augmentBlossom(t, endpoint[p ^ 1]);
        mate[endpoint[p]] = p ^ 1;
        mate[endpoint[p ^ 1]] = p;
    }
    rotate(childs.begin(), childs.begin() + i, childs.end());
    rotate(endps.begin(), endps.begin() + i, endps.end());
    blossombase[b] = blossombase[childs[0]];
}

void augmentMatching(int k) {
    int ends[2][2] = {{(int)edges[k][0], 2 * k + 1}, {(int)edges[k][1], 2 * k}};
    for (auto& sp : ends) {
        int s = sp[0], p = sp[1];
        while (true) {
            int bs = inblossom[s];
            if (bs >= nv) augmentBlossom(bs, s);
            mate[s] = p;
            if (labelend[bs] == -1) break;
            int t = endpoint[labelend[bs]];
            int bt = inblossom[t];
            s = endpoint[labelend[bt]];
            int j = endpoint[labelend[bt] ^ 1];
            if (bt >= nv) augmentBlossom(bt, j);
            mate[j] = labelend[bt];
            p = labelend[bt] ^ 1;
        }
    }
}

```

```

    }
}

vector<int> solve(int n, const vector<array<ll, 3>>& input) {
    nv = n;
    edges = input;
    ne = edges.size();
    mate.assign(nv, -1);
    if (ne == 0) return mate;
    ll maxweight = 0;
    for (auto& e : edges) maxweight = max(maxweight, e[2]);
    endpoint.resize(2 * ne);
    for (int p = 0; p < 2 * ne; p++) endpoint[p] = edges[p >> 1][p & 1];
    neighbend.assign(nv, {});
    for (int k = 0; k < ne; k++) {
        neighbend[edges[k][0]].push_back(2 * k + 1);
        neighbend[edges[k][1]].push_back(2 * k);
    }
    label.assign(2 * nv, 0);
    labelend.assign(2 * nv, -1);
    inblossom.resize(nv);
    iota(inblossom.begin(), inblossom.end(), 0);
    blossomparent.assign(2 * nv, -1);
    blossomchilds.assign(2 * nv, {});
    blossombase.assign(2 * nv, -1);
    for (int v = 0; v < nv; v++) blossombase[v] = v;
    blossomendps.assign(2 * nv, {});
    bestedge.assign(2 * nv, -1);
    blossombestedges.assign(2 * nv, {});
    hasBestList.assign(2 * nv, 0);
    unusedblossoms.clear();
    for (int b = nv; b < 2 * nv; b++) unusedblossoms.push_back(b);
    dualvar.assign(2 * nv, 0);
    for (int v = 0; v < nv; v++) dualvar[v] = maxweight;
    allowededge.assign(ne, 0);

    for (int stage = 0; stage < nv; stage++) {
        fill(label.begin(), label.end(), 0);
        fill(bestedge.begin(), bestedge.end(), -1);
        for (int b = nv; b < 2 * nv; b++) {
            blossombestedges[b].clear();
            hasBestList[b] = 0;
        }
        fill(allowededge.begin(), allowededge.end(), 0);
        queue.clear();
        for (int v = 0; v < nv; v++)
            if (mate[v] == -1 && label[inblossom[v]] == 0) assignLabel(v, 1, -1);
        bool augmented = false;
        while (true) {
            while (!queue.empty() && !augmented) {
                int v = queue.back();

```

```

queue.pop_back();
for (int p : neighbend[v]) {
    int k = p >> 1, w = endpoint[p];
    if (inblossom[v] == inblossom[w]) continue;
    ll kslack = 0;
    if (!allowededge[k]) {
        kslack = slack(k);
        if (kslack <= 0) allowededge[k] = 1;
    }
    if (allowededge[k]) {
        if (label[inblossom[w]] == 0) {
            assignLabel(w, 2, p ^ 1);
        } else if (label[inblossom[w]] == 1) {
            int base = scanBlossom(v, w);
            if (base >= 0) {
                addBlossom(base, k);
            } else {
                augmentMatching(k);
                augmented = true;
                break;
            }
        } else if (label[w] == 0) {
            label[w] = 2;
            labelend[w] = p ^ 1;
        }
    } else if (label[inblossom[w]] == 1) {
        int b = inblossom[v];
        if (bestedge[b] == -1 || kslack < slack(bestedge[b])) bestedge[b] = k;
    } else if (label[w] == 0) {
        if (bestedge[w] == -1 || kslack < slack(bestedge[w])) bestedge[w] = k;
    }
}
}
if (augmented) break;
int deltatype = 1, deltaedge = -1, deltablossom = -1;
ll delta = *min_element(dualvar.begin(), dualvar.begin() + nv);
for (int v = 0; v < nv; v++)
    if (label[inblossom[v]] == 0 && bestedge[v] != -1) {
        ll d = slack(bestedge[v]);
        if (d < delta) delta = d, deltatype = 2, deltaedge = bestedge[v];
    }
for (int b = 0; b < 2 * nv; b++)
    if (blossomparent[b] == -1 && label[b] == 1 && bestedge[b] != -1) {
        ll d = slack(bestedge[b]) / 2;
        if (d < delta) delta = d, deltatype = 3, deltaedge = bestedge[b];
    }
for (int b = nv; b < 2 * nv; b++)
    if (blossombase[b] >= 0 && blossomparent[b] == -1 && label[b] == 2 && dualvar[b] < delta)
        delta = dualvar[b], deltatype = 4, deltablossom = b;
for (int v = 0; v < nv; v++) {
    int lb = label[inblossom[v]];

```

```

        if (lb == 1) dualvar[v] -= delta;
        else if (lb == 2) dualvar[v] += delta;
    }
    for (int b = nv; b < 2 * nv; b++)
        if (blossombase[b] >= 0 && blossomparent[b] == -1) {
            if (label[b] == 1) dualvar[b] += delta;
            else if (label[b] == 2) dualvar[b] -= delta;
        }
    if (deltatype == 1) break;
    if (deltatype == 2) {
        allowedge[deltaedge] = 1;
        int i = edges[deltaedge][0], j = edges[deltaedge][1];
        if (label[inblossom[i]] == 0) swap(i, j);
        queue.push_back(i);
    } else if (deltatype == 3) {
        allowedge[deltaedge] = 1;
        queue.push_back(edges[deltaedge][0]);
    } else {
        expandBlossom(deltablossom, false);
    }
}
if (!augmented) break;
for (int b = nv; b < 2 * nv; b++)
    if (blossomparent[b] == -1 && blossombase[b] >= 0 && label[b] == 1 && dualvar[b] ==
        expandBlossom(b, true);
}
vector<int> result(nv, -1);
for (int v = 0; v < nv; v++)
    if (mate[v] >= 0) result[v] = endpoint[mate[v]];
return result;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    map<pair<int, int>, ll> best;
    for (int i = 0; i < m; i++) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        --u;
        --v;
        if (u == v) continue;
        if (u > v) swap(u, v);
        auto it = best.find({u, v});
        if (it == best.end()) best[{u, v}] = w;
        else it->second = max(it->second, w);
    }
}

```

```

// Parallel offers collapse to the best one; doubled weights keep every
// dual update integral.
vector<array<ll, 3>> edges;
for (auto& [key, w] : best) edges.push_back({key.first, key.second, 2 * w});
Matcher matcher;
vector<int> mate = matcher.solve(n, edges);
ll total = 0;
for (int v = 0; v < n; v++)
    if (mate[v] > v) total += best[{v, mate[v]}];
cout << total << '\n';
}

```

Python

```

import sys

# Maximum-weight matching in a general graph: Edmonds' primal--dual blossom
# algorithm in the  $O(n^3)$  formulation of Galil, following the structure of
# van Rantwijk's reference implementation. Vertex duals start at max weight;
# every stage grows alternating trees with tight edges, shrinks odd cycles into
# blossoms, and otherwise changes duals by the smallest of four slack bounds.

def max_weight_matching(nvertex, edges):
    nedge = len(edges)
    if nedge == 0:
        return [-1] * nvertex
    maxweight = max(0, max(w for _, _, w in edges))
    endpoint = [edges[p >> 1][p & 1] for p in range(2 * nedge)]
    neighbend = [[] for _ in range(nvertex)]
    for k, (i, j, _) in enumerate(edges):
        neighbend[i].append(2 * k + 1)
        neighbend[j].append(2 * k)
    mate = [-1] * nvertex
    label = [0] * (2 * nvertex)
    labelend = [-1] * (2 * nvertex)
    inblossom = list(range(nvertex))
    blossomparent = [-1] * (2 * nvertex)
    blossomchilds = [None] * (2 * nvertex)
    blossombase = list(range(nvertex)) + [-1] * nvertex
    blossomendps = [None] * (2 * nvertex)
    bestedge = [-1] * (2 * nvertex)
    blossombestedges = [None] * (2 * nvertex)
    unusedblossoms = list(range(nvertex, 2 * nvertex))
    dualvar = [maxweight] * nvertex + [0] * nvertex
    allowedge = [False] * nedge
    queue = []

    def slack(k):
        i, j, wt = edges[k]

```

```

    return dualvar[i] + dualvar[j] - 2 * wt

def leaves(b):
    if b < nvertex:
        return [b]
    out = []
    stack = [b]
    while stack:
        t = stack.pop()
        if t < nvertex:
            out.append(t)
        else:
            stack.extend(blossomchilds[t])
    return out

def assign_label(w, t, p):
    while True:
        b = inblossom[w]
        label[w] = label[b] = t
        labelend[w] = labelend[b] = p
        bestedge[w] = bestedge[b] = -1
        if t == 1:
            queue.extend(leaves(b))
            return
        base = blossombase[b]
        w = endpoint[mate[base]]
        t = 1
        p = mate[base] ^ 1

def scan_blossom(v, w):
    path = []
    base = -1
    while v != -1 or w != -1:
        b = inblossom[v]
        if label[b] & 4:
            base = blossombase[b]
            break
        path.append(b)
        label[b] = 5
        if labelend[b] == -1:
            v = -1
        else:
            v = endpoint[labelend[b]]
            b = inblossom[v]
            v = endpoint[labelend[b]]
        if w != -1:
            v, w = w, v
    for b in path:
        label[b] = 1
    return base

```



```

def add_blossom(base, k):
    v, w, _ = edges[k]
    bb = inblossom[base]
    bv = inblossom[v]
    bw = inblossom[w]
    b = unusedblossoms.pop()
    blossombase[b] = base
    blossomparent[b] = -1
    blossomparent[bb] = b
    path = []
    endps = []
    while bv != bb:
        blossomparent[bv] = b
        path.append(bv)
        endps.append(labelend[bv])
        v = endpoint[labelend[bv]]
        bv = inblossom[v]
    path.append(bb)
    path.reverse()
    endps.reverse()
    endps.append(2 * k)
    while bw != bb:
        blossomparent[bw] = b
        path.append(bw)
        endps.append(labelend[bw] ^ 1)
        w = endpoint[labelend[bw]]
        bw = inblossom[w]
    blossomchilds[b] = path
    blossomendps[b] = endps
    label[b] = 1
    labelend[b] = labelend[bb]
    dualvar[b] = 0
    for v in leaves(b):
        if label[inblossom[v]] == 2:
            queue.append(v)
            inblossom[v] = b
    bestedgeto = [-1] * (2 * nvertex)
    for bv in path:
        if blossombestedges[bv] is None:
            nblists = [[p >> 1 for p in neighbend[v]] for v in leaves(bv)]
        else:
            nblists = [blossombestedges[bv]]
        for nblast in nblists:
            for k in nblast:
                i, j, _ = edges[k]
                if inblossom[j] == b:
                    i, j = j, i
                bj = inblossom[j]
                if bj != b and label[bj] == 1 and (bestedgeto[bj] == -1 or slack(k) < slack(bestedgeto[bj])):
                    bestedgeto[bj] = k
    blossombestedges[bv] = None

```

```

    bestedge[bv] = -1
    blossombestedges[b] = [k for k in bestedgeto if k != -1]
    bestedge[b] = -1
    for k in blossombestedges[b]:
        if bestedge[b] == -1 or slack(k) < slack(bestedge[b]):
            bestedge[b] = k

def expand_blossom(b, endstage):
    for s in blossomchilds[b]:
        blossomparent[s] = -1
        if s < nvertex:
            inblossom[s] = s
        elif endstage and dualvar[s] == 0:
            expand_blossom(s, endstage)
        else:
            for v in leaves(s):
                inblossom[v] = s
    if not endstage and label[b] == 2:
        childs = blossomchilds[b]
        endps = blossomendps[b]
        entrychild = inblossom[endpoint[labelend[b] ^ 1]]
        j = childs.index(entrychild)
        if j & 1:
            j -= len(childs)
            jstep = 1
            endptrick = 0
        else:
            jstep = -1
            endptrick = 1
        p = labelend[b]
        while j != 0:
            label[endpoint[p ^ 1]] = 0
            label[endpoint[endps[j - endptrick] ^ endptrick ^ 1]] = 0
            assign_label(endpoint[p ^ 1], 2, p)
            allowedge[endps[j - endptrick] >> 1] = True
            j += jstep
            p = endps[j - endptrick] ^ endptrick
            allowedge[p >> 1] = True
            j += jstep
        bv = childs[j]
        label[endpoint[p ^ 1]] = label[bv] = 2
        labelend[endpoint[p ^ 1]] = labelend[bv] = p
        bestedge[bv] = -1
        j += jstep
        while childs[j] != entrychild:
            bv = childs[j]
            if label[bv] == 1:
                j += jstep
                continue
            found = -1
            for v in leaves(bv):

```

```

        if label[v] != 0:
            found = v
            break
    if found >= 0:
        label[found] = 0
        label[endpoint[mate[blossombase[bv]]]] = 0
        assign_label(found, 2, labelend[found])
    j += jstep
label[b] = labelend[b] = -1
blossomchilds[b] = blossomendps[b] = None
blossombase[b] = -1
blossombestedges[b] = None
bestedge[b] = -1
unusedblossoms.append(b)

def augment_blossom(b, v):
    t = v
    while blossomparent[t] != b:
        t = blossomparent[t]
    if t >= nvertex:
        augment_blossom(t, v)
    i = j = blossomchilds[b].index(t)
    if i & 1:
        j -= len(blossomchilds[b])
        jstep = 1
        endptrick = 0
    else:
        jstep = -1
        endptrick = 1
    while j != 0:
        j += jstep
        t = blossomchilds[b][j]
        p = blossomendps[b][j - endptrick] ^ endptrick
        if t >= nvertex:
            augment_blossom(t, endpoint[p])
        j += jstep
        t = blossomchilds[b][j]
        if t >= nvertex:
            augment_blossom(t, endpoint[p ^ 1])
        mate[endpoint[p]] = p ^ 1
        mate[endpoint[p ^ 1]] = p
    blossomchilds[b] = blossomchilds[b][i:] + blossomchilds[b][:i]
    blossomendps[b] = blossomendps[b][i:] + blossomendps[b][:i]
    blossombase[b] = blossombase[blossomchilds[b][0]]

def augment_matching(k):
    v, w, _ = edges[k]
    for s, p in ((v, 2 * k + 1), (w, 2 * k)):
        while True:
            bs = inblossom[s]
            if bs >= nvertex:

```

```

        augment_blossom(bs, s)
    mate[s] = p
    if labelend[bs] == -1:
        break
    t = endpoint[labelend[bs]]
    bt = inblossom[t]
    s = endpoint[labelend[bt]]
    j = endpoint[labelend[bt] ^ 1]
    if bt >= nvertex:
        augment_blossom(bt, j)
    mate[j] = labelend[bt]
    p = labelend[bt] ^ 1

for _ in range(nvertex):
    label[:] = [0] * (2 * nvertex)
    bestedge[:] = [-1] * (2 * nvertex)
    blossombestedges[nvertex:] = [None] * nvertex
    allowededge[:] = [False] * nedge
    queue[:] = []
    for v in range(nvertex):
        if mate[v] == -1 and label[inblossom[v]] == 0:
            assign_label(v, 1, -1)
    augmented = False
    while True:
        while queue and not augmented:
            v = queue.pop()
            for p in neighbend[v]:
                k = p >> 1
                w = endpoint[p]
                if inblossom[v] == inblossom[w]:
                    continue
                if not allowededge[k]:
                    kslack = slack(k)
                    if kslack <= 0:
                        allowededge[k] = True
                if allowededge[k]:
                    if label[inblossom[w]] == 0:
                        assign_label(w, 2, p ^ 1)
                    elif label[inblossom[w]] == 1:
                        base = scan_blossom(v, w)
                        if base >= 0:
                            add_blossom(base, k)
                        else:
                            augment_matching(k)
                            augmented = True
                            break
                    elif label[w] == 0:
                        label[w] = 2
                        labelend[w] = p ^ 1
                    elif label[inblossom[w]] == 1:
                        b = inblossom[w]

```

```

        if bestedge[b] == -1 or kslack < slack(bestedge[b]):
            bestedge[b] = k
        elif label[w] == 0:
            if bestedge[w] == -1 or kslack < slack(bestedge[w]):
                bestedge[w] = k
    if augmented:
        break
    deltatype = 1
    delta = min(dualvar[:nvertex])
    deltaedge = deltablossom = -1
    for v in range(nvertex):
        if label[inblossom[v]] == 0 and bestedge[v] != -1:
            d = slack(bestedge[v])
            if d < delta:
                delta = d
                deltatype = 2
                deltaedge = bestedge[v]
    for b in range(2 * nvertex):
        if blossomparent[b] == -1 and label[b] == 1 and bestedge[b] != -1:
            d = slack(bestedge[b]) >> 1
            if d < delta:
                delta = d
                deltatype = 3
                deltaedge = bestedge[b]
    for b in range(nvertex, 2 * nvertex):
        if blossombase[b] >= 0 and blossomparent[b] == -1 and label[b] == 2 and dualvar[b] <
            delta = dualvar[b]
            deltatype = 4
            deltablossom = b
    for v in range(nvertex):
        lb = label[inblossom[v]]
        if lb == 1:
            dualvar[v] -= delta
        elif lb == 2:
            dualvar[v] += delta
    for b in range(nvertex, 2 * nvertex):
        if blossombase[b] >= 0 and blossomparent[b] == -1:
            if label[b] == 1:
                dualvar[b] += delta
            elif label[b] == 2:
                dualvar[b] -= delta
    if deltatype == 1:
        break
    if deltatype == 2:
        allowedge[deltaedge] = True
        i, j, _ = edges[deltaedge]
        if label[inblossom[i]] == 0:
            i, j = j, i
        queue.append(i)
    elif deltatype == 3:
        allowedge[deltaedge] = True

```

```

        i, j, _ = edges[deltaedge]
        queue.append(i)
    else:
        expand_blossom(deltablossom, False)
if not augmented:
    break
for b in range(nvertex, 2 * nvertex):
    if blossomparent[b] == -1 and blossombase[b] >= 0 and label[b] == 1 and dualvar[b] == 0:
        expand_blossom(b, True)
return [endpoint[m] if m >= 0 else -1 for m in mate]

def main():
    data = sys.stdin.buffer.read().split()
    n, m = int(data[0]), int(data[1])
    best = {}
    for i in range(m):
        u = int(data[2 + 3 * i]) - 1
        v = int(data[3 + 3 * i]) - 1
        w = int(data[4 + 3 * i])
        if u != v:
            key = (u, v) if u < v else (v, u)
            if best.get(key, 0) < w:
                best[key] = w
    # Parallel offers collapse to the best one; doubled weights keep every
    # dual update integral.
    edges = [(u, v, 2 * w) for (u, v), w in best.items()]
    mate = max_weight_matching(n, edges)
    total = 0
    for v in range(n):
        if mate[v] > v:
            total += best[(v, mate[v])]
    print(total)

if __name__ == "__main__":
    main()

```

55. Monsoon Diameter

Paths as points

A path with totals $(A, B) = (\sum a, \sum b)$ takes $A \cdot t + B$ on day t . The answer on day t is the maximum of these linear functions: the upper envelope of the point set restricted to $t \geq 0$. There are $O(n^2)$ paths, but only $O(n \log n)$ candidate points are needed.

Split every vertex of degree above three with zero-cost dummy vertices; path totals do not change. Now choose an edge that splits the component into two parts of sizes at least about a third each. The paths through that edge are sums of one root-to-vertex vector from each side plus the edge itself. The useful ones lie on the Minkowski

sum of the two sides' upper hulls, which has linear size and is computed by merging slopes. Recurse on both sides. Finally take the hull of all candidates and sweep t upward with a pointer.

Correctness proof

Lemma 1. For vectors P and Q and every $t \geq 0$, $\max_{p+q \in P+Q} f_t(p+q) = \max_P f_t + \max_Q f_t$, where $f_t(A, B) = At + B$. So only hull points of each side matter, and $\text{hull}(P+Q)$ is the Minkowski sum of the hulls.

Proof. The function f_t is linear, and a linear maximum over a sum set separates into the two maxima. The vertices of the Minkowski sum of two convex polygons are sums of vertices, merged by edge slope.

Lemma 2. Every path is represented at the unique decomposition edge where its two endpoints are first separated, or it has zero length.

Proof. Recursion removes edges. The first removed edge on the path splits it into two root-to-vertex pieces inside that component.

Theorem. The printed sequence is the maximum path length on each day.

Proof. By Lemma 2 every path's value is dominated by some candidate, and by Lemma 1 no maximum is lost in the hulls. The final sweep evaluates the envelope correctly because its optimal point moves monotonically to larger A as t grows.

Complexity and traps

Degree at most three makes the edge-centroid split balanced, so the depth is $O(\log n)$ and the total work is $O(n \log^2 n + m)$. Coordinates reach 10^{16} , so cross products need 128 bits in C++. Keep the point $(0,0)$ for a single-village path, and drop the hull part before the maximum- B point, because it never wins for $t \geq 0$.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Every path is a point (A, B) = (sum a, sum b) and has length A t + B on day
// t, so the answers are the upper envelope of all path points. Only O(n log n)
// candidate points are needed. After splitting high-degree vertices with
// zero-cost dummy vertices (degree <= 3), pick an edge that splits the current
// component evenly. Paths through that edge are Minkowski sums of the two
// sides' upper hulls of root-to-vertex points, shifted by the edge itself;
// paths not through it are handled recursively.

typedef long long ll;
typedef pair<ll, ll> P;

__int128 cross(P o, P a, P b) {
    return (__int128)(a.first - o.first) * (b.second - o.second) -
           (__int128)(a.second - o.second) * (b.first - o.first);
}
```

```

// Upper hull restricted to the part useful for t >= 0: A increasing, B
// decreasing, slopes strictly decreasing.
vector<P> usefulHull(vector<P> pts) {
    sort(pts.begin(), pts.end());
    vector<P> h;
    for (const P& p : pts) {
        while (!h.empty() && h.back().first == p.first) h.pop_back(); // same A, larger B wins
        while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), p) >= 0) h.pop_back();
        h.push_back(p);
    }
    size_t best = 0;
    for (size_t i = 1; i < h.size(); i++)
        if (h[i].second >= h[best].second) best = i;
    return vector<P>(h.begin() + best, h.end());
}

vector<P> minkowski(const vector<P>& a, const vector<P>& b) {
    vector<P> out = {{a[0].first + b[0].first, a[0].second + b[0].second}};
    size_t i = 0, j = 0;
    while (i + 1 < a.size() || j + 1 < b.size()) {
        bool takeA;
        if (i + 1 == a.size()) takeA = false;
        else if (j + 1 == b.size()) takeA = true;
        else {
            P da = {a[i + 1].first - a[i].first, a[i + 1].second - a[i].second};
            P db = {b[j + 1].first - b[j].first, b[j + 1].second - b[j].second};
            // larger slope dB/dA first
            takeA = (__int128)da.second * db.first >= (__int128)db.second * da.first;
        }
        if (takeA) {
            out.push_back({out.back().first + a[i + 1].first - a[i].first, out.back().second + a[i + 1].second - a[i].second});
            i++;
        } else {
            out.push_back({out.back().first + b[j + 1].first - b[j].first, out.back().second + b[j + 1].second - b[j].second});
            j++;
        }
    }
    return out;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<array<ll, 3>>> raw(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        ll a, b;
        cin >> u >> v >> a >> b;
        --u;
    }
}

```



```

    --v;
    raw[u].push_back({v, a, b});
    raw[v].push_back({u, a, b});
}
// Binarize: rooted at 0, give each vertex at most two children.
vector<int> eu, ev;
vector<ll> ea, eb;
int V = n;
auto addEdge = [&](int u, int v, ll a, ll b) {
    eu.push_back(u);
    ev.push_back(v);
    ea.push_back(a);
    eb.push_back(b);
};
{
    vector<int> parent(n, -1), stack = {0};
    parent[0] = 0;
    while (!stack.empty()) {
        int v = stack.back();
        stack.pop_back();
        vector<array<ll, 3>> kids;
        for (auto& e : raw[v])
            if (parent[e[0]] < 0) {
                parent[e[0]] = v;
                kids.push_back(e);
                stack.push_back(e[0]);
            }
        int cur = v;
        int k = kids.size();
        for (int i = 0; i < k; i++) {
            addEdge(cur, kids[i][0], kids[i][1], kids[i][2]);
            if (k - i > 2) { // keep at most two children per vertex
                int d = V++;
                addEdge(cur, d, 0, 0);
                cur = d;
            }
        }
    }
}
int E = eu.size();
vector<vector<int>> adj(V);
for (int e = 0; e < E; e++) {
    adj[eu[e]].push_back(e);
    adj[ev[e]].push_back(e);
}
vector<char> removed(E, 0);
vector<P> candidates = {{0, 0}};
vector<int> sz(V), parentEdge(V), order;
vector<ll> accA(V), accB(V);

auto collect = [&](int root, vector<int>& out) { // vertices of component
    out.clear();

```

```

    out.push_back(root);
    parentEdge[root] = -1;
    for (size_t i = 0; i < out.size(); i++) {
        int v = out[i];
        for (int e : adj[v]) {
            if (removed[e] || e == parentEdge[v]) continue;
            int w = eu[e] ^ ev[e] ^ v;
            parentEdge[w] = e;
            out.push_back(w);
        }
    }
};

auto sidePoints = [&](int root) {
    vector<int> vs;
    collect(root, vs);
    vector<P> pts;
    pts.reserve(vs.size());
    accA[root] = accB[root] = 0;
    for (int v : vs) {
        if (v != root) {
            int e = parentEdge[v], p = eu[e] ^ ev[e] ^ v;
            accA[v] = accA[p] + ea[e];
            accB[v] = accB[p] + eb[e];
        }
        pts.push_back({accA[v], accB[v]});
    }
    return usefulHull(pts);
};

vector<int> work = {0}, vs;
while (!work.empty()) {
    int root = work.back();
    work.pop_back();
    collect(root, vs);
    int total = vs.size();
    if (total == 1) continue;
    for (int i = total - 1; i >= 0; i--) {
        int v = vs[i];
        sz[v] = 1;
        for (int e : adj[v])
            if (!removed[e] && e != parentEdge[v]) sz[v] += sz[eu[e] ^ ev[e] ^ v];
    }
    int bestEdge = -1, bestVal = INT_MAX, child = -1;
    for (int i = 1; i < total; i++) {
        int v = vs[i];
        int value = max(sz[v], total - sz[v]);
        if (value < bestVal) bestVal = value, bestEdge = parentEdge[v], child = v;
    }
    int other = eu[bestEdge] ^ ev[bestEdge] ^ child;
    removed[bestEdge] = 1;
    vector<P> h1 = sidePoints(child), h2 = sidePoints(other);

```

```

        vector<P> sum = minkowski(h1, h2);
        for (auto& p : sum) candidates.push_back({p.first + ea[bestEdge], p.second + eb[bestEdge]});
        work.push_back(child);
        work.push_back(other);
    }
    vector<P> hull = usefulHull(candidates);
    string out;
    size_t k = 0;
    for (ll t = 0; t < m; t++) {
        while (k + 1 < hull.size() && hull[k + 1].first * t + hull[k + 1].second >= hull[k].first *
            out += to_string(hull[k].first * t + hull[k].second);
            out += t + 1 < m ? ' ' : '\n';
        }
    }
    cout << out;
}

```

Python

```

import sys

def useful_hull(points):
    """Upper hull part that matters for t >= 0 (A up, B down, concave)."""
    points.sort()
    h = []
    for p in points:
        pa, pb = p
        while h and h[-1][0] == pa:
            h.pop()
        while len(h) >= 2:
            oa, ob = h[-2]
            aa, ab = h[-1]
            if (aa - oa) * (pb - ob) - (ab - ob) * (pa - oa) >= 0:
                h.pop()
            else:
                break
        h.append(p)
    best = 0
    for i in range(1, len(h)):
        if h[i][1] >= h[best][1]:
            best = i
    return h[best:]

def minkowski(a, b, shift_a, shift_b):
    x = a[0][0] + b[0][0] + shift_a
    y = a[0][1] + b[0][1] + shift_b
    out = [(x, y)]
    i = j = 0
    la = len(a) - 1

```

```

lb = len(b) - 1
while i < la or j < lb:
    if i == la:
        take_a = False
    elif j == lb:
        take_a = True
    else:
        dax = a[i + 1][0] - a[i][0]
        day = a[i + 1][1] - a[i][1]
        dbx = b[j + 1][0] - b[j][0]
        dby = b[j + 1][1] - b[j][1]
        take_a = day * dbx >= dby * dax
    if take_a:
        x += a[i + 1][0] - a[i][0]
        y += a[i + 1][1] - a[i][1]
        i += 1
    else:
        x += b[j + 1][0] - b[j][0]
        y += b[j + 1][1] - b[j][1]
        j += 1
    out.append((x, y))
return out

```

```

def main():
    data = sys.stdin.buffer.read().split()
    n, m = int(data[0]), int(data[1])
    raw = [[] for _ in range(n)]
    ptr = 2
    for _ in range(n - 1):
        u = int(data[ptr]) - 1
        v = int(data[ptr + 1]) - 1
        a = int(data[ptr + 2])
        b = int(data[ptr + 3])
        ptr += 4
        raw[u].append((v, a, b))
        raw[v].append((u, a, b))
    del data

    # Binarize with zero-cost dummy vertices.
    eu = []
    ev = []
    ea = []
    eb = []
    V = n
    parent = [-1] * n
    parent[0] = 0
    stack = [0]
    while stack:
        v = stack.pop()
        kids = []

```

```

    for e in row[v]:
        if parent[e[0]] < 0:
            parent[e[0]] = v
            kids.append(e)
            stack.append(e[0])
    cur = v
    k = len(kids)
    for i, (w, a, b) in enumerate(kids):
        eu.append(cur)
        ev.append(w)
        ea.append(a)
        eb.append(b)
        if k - i > 2:
            eu.append(cur)
            ev.append(V)
            ea.append(0)
            eb.append(0)
            cur = V
            V += 1

del row, parent
E = len(eu)
adj = [[] for _ in range(V)]
for e in range(E):
    adj[eu[e]].append(e)
    adj[ev[e]].append(e)
removed = [False] * E
parent_edge = [-1] * V
size = [0] * V
acc_a = [0] * V
acc_b = [0] * V

def collect(root):
    out = [root]
    parent_edge[root] = -1
    for v in out:
        pe = parent_edge[v]
        for e in adj[v]:
            if e != pe and not removed[e]:
                w = eu[e] ^ ev[e] ^ v
                parent_edge[w] = e
                out.append(w)
    return out

def side_hull(root):
    vs = collect(root)
    acc_a[root] = 0
    acc_b[root] = 0
    pts = [(0, 0)]
    for idx in range(1, len(vs)):
        v = vs[idx]
        e = parent_edge[v]

```

```

        p = eu[e] ^ ev[e] ^ v
        x = acc_a[p] + ea[e]
        y = acc_b[p] + eb[e]
        acc_a[v] = x
        acc_b[v] = y
        pts.append((x, y))
    return useful_hull(pts)

candidates = [(0, 0)]
work = [0]
while work:
    root = work.pop()
    vs = collect(root)
    total = len(vs)
    if total == 1:
        continue
    for idx in range(total - 1, -1, -1):
        v = vs[idx]
        s = 1
        pe = parent_edge[v]
        for e in adj[v]:
            if e != pe and not removed[e]:
                s += size[eu[e] ^ ev[e] ^ v]
        size[v] = s
    best_value = total + 1
    child = -1
    for idx in range(1, total):
        v = vs[idx]
        s = size[v]
        value = s if s > total - s else total - s
        if value < best_value:
            best_value = value
            child = v
    edge = parent_edge[child]
    other = eu[edge] ^ ev[edge] ^ child
    removed[edge] = True
    h1 = side_hull(child)
    h2 = side_hull(other)
    candidates.extend(minkowski(h1, h2, ea[edge], eb[edge]))
    work.append(child)
    work.append(other)

hull = useful_hull(candidates)
del candidates, adj, eu, ev, ea, eb
write = sys.stdout.write
out = []
k = 0
last = len(hull) - 1
ha, hb = hull[0]
for t in range(m):
    while k < last:

```

```

na, nb = hull[k + 1]
if na * t + nb >= ha * t + hb:
    k += 1
    ha, hb = na, nb
else:
    break
out.append(ha * t + hb)
if len(out) == 65536:
    write(" ".join(map(str, out)) + " ")
    out = []
write(" ".join(map(str, out)) + "\n")

if __name__ == "__main__":
    main()

```

56. Window Substring Census

Count each substring at its last occurrence

Sweep r from left to right. For each distinct substring t of $s[1..r]$, look at its last occurrence ending at or before r . That occurrence starts at ≥ 1 exactly when t occurs in $s[1..r]$. So give $+1$ to the start position of every such last occurrence; the answer to $(1, r)$ is then the sum over starts in $[1, r]$.

In the suffix automaton of s , the suffixes of $s[1..r]$ are the states on the suffix-link path from the prefix state P_r to the root. The substrings of one state share their end positions, so they also share their last end $e \leq r$. A state with lengths $(\text{len}(\text{link}), \text{len}]$ contributes the starts $[e - \text{len} + 1, e - \text{len}(\text{link})]$.

Keep the suffix-link tree in a link-cut tree in which every preferred path has one colour, its common e . Moving from $r-1$ to r , $\text{access}(P_r)$ recolours the root path of P_r to r . Each preferred-path chain $\text{top}..x$ that it absorbs contributes a contiguous length block, so its old starts are removed with one range update. The whole path then contributes the starts $[1, r]$.

Correctness proof

Lemma 1. After processing r , the colour of a state is the largest end position $\leq r$ of its substrings, or 0 if none has occurred yet.

Proof. A state's substrings end at r iff the state is on the root path of P_r . Access recolours exactly that path with r .

Lemma 2. The range-add structure holds, for every start position, the number of distinct substrings whose last occurrence (ending at or before r) starts there.

Proof. By Lemma 1 the contribution of every state is determined by its colour. Access removes each changed chain's old block and adds the new one. Chains in one splay tree cover contiguous length ranges because lengths increase strictly along the root path.

Theorem. For every window the output is the number of distinct substrings of $s[1..r]$.

Proof. A substring occurs in $s[1..r]$ iff its last occurrence ending at or before r starts at or after 1, and by Lemma 2 the range sum over $[1, r]$ counts exactly those.

Complexity and traps

Access performs $O(\log n)$ amortized preferred-child changes, each with a Fenwick update, so the time is $O(n \log^2 n + q \log n)$ and the memory is $O(n \sigma)$. The root state has length 0 and contributes nothing. Use the leftmost node of the splayed subtree as the chain top, and push the colour tags before reading them.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Count each distinct substring of s[l..r] at its LAST occurrence ending at or
// before r; it is inside the window iff that occurrence starts at >= l.
// Sweep r. In the suffix automaton, the suffixes of s[l..r] are the states on
// the suffix-link path from the prefix state to the root, and a state's
// substrings share their last end position. A link-cut tree over the
// suffix-link tree keeps every preferred path uniformly coloured by that last
// end e: a coloured chain top..x covers lengths (len(link(top)), len(x)], i.e.
// start positions [e - len(x) + 1, e - len(link(top))]. Accessing the prefix
// state recolours its root path with r, removing the old chains' starts from a
// range-add/range-sum Fenwick tree and adding starts [1, r].

typedef long long ll;

struct Fenwick { // range add, range sum over positions 1..n
    int n;
    vector<ll> a, b;
    Fenwick(int n) : n(n), a(n + 2, 0), b(n + 2, 0) {}
    void addPoint(vector<ll>& t, int i, ll v) {
        for (; i <= n + 1; i += i & -i) t[i] += v;
    }
    void rangeAdd(int l, int r, ll v) { // add v to [l, r]
        if (l > r) return;
        addPoint(a, l, v);
        addPoint(a, r + 1, -v);
        addPoint(b, l, v * (l - 1));
        addPoint(b, r + 1, -v * r);
    }
    ll prefix(int i) {
        ll sa = 0, sb = 0;
        for (int k = i; k > 0; k -= k & -k) sa += a[k], sb += b[k];
        return sa * i - sb;
    }
    ll rangeSum(int l, int r) { return prefix(r) - prefix(l - 1); }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
```



```

int q;
cin >> s >> q;
int n = s.size();
// Suffix automaton.
vector<int> len(2 * n + 1, 0), link(2 * n + 1, -1), prefixState(n + 1);
vector<array<int, 26>> nxt(2 * n + 1);
for (auto& row : nxt) row.fill(-1);
int states = 1, last = 0;
for (int i = 0; i < n; i++) {
    int c = s[i] - 'a', cur = states++;
    len[cur] = len[last] + 1;
    int p = last;
    while (p != -1 && nxt[p][c] == -1) nxt[p][c] = cur, p = link[p];
    if (p == -1) {
        link[cur] = 0;
    } else {
        int qq = nxt[p][c];
        if (len[p] + 1 == len[qq]) {
            link[cur] = qq;
        } else {
            int clone = states++;
            len[clone] = len[p] + 1;
            nxt[clone] = nxt[qq];
            link[clone] = link[qq];
            while (p != -1 && nxt[p][c] == qq) nxt[p][c] = clone, p = link[p];
            link[qq] = link[cur] = clone;
        }
    }
    last = cur;
    prefixState[i + 1] = cur;
}
nxt.clear();
nxt.shrink_to_fit();

// Link-cut tree over the suffix-link tree (rooted, no reversal needed).
// Node ids are state + 1 so that 0 is the null node.
int N = states + 1;
vector<int> ch0(N, 0), ch1(N, 0), fa(N, 0), colour(N, 0), tag(N, 0), leftmost(N);
for (int v = 1; v < N; v++) {
    leftmost[v] = v;
    fa[v] = link[v - 1] >= 0 ? link[v - 1] + 1 : 0;
}
auto isRoot = [&](int x) { return !fa[x] || (ch0[fa[x]] != x && ch1[fa[x]] != x); };
auto pull = [&](int x) { leftmost[x] = ch0[x] ? leftmost[ch0[x]] : x; };
auto apply = [&](int x, int c) {
    if (x) colour[x] = tag[x] = c;
};
auto push = [&](int x) {
    if (tag[x]) {
        apply(ch0[x], tag[x]);
        apply(ch1[x], tag[x]);
    }
}

```

```

        tag[x] = 0;
    }
};

auto rotate = [&](int x) {
    int p = fa[x], g = fa[p];
    bool right = ch1[p] == x;
    if (!isRoot(p)) (ch0[g] == p ? ch0[g] : ch1[g]) = x;
    fa[x] = g;
    if (right) {
        ch1[p] = ch0[x];
        if (ch0[x]) fa[ch0[x]] = p;
        ch0[x] = p;
    } else {
        ch0[p] = ch1[x];
        if (ch1[x]) fa[ch1[x]] = p;
        ch1[x] = p;
    }
    fa[p] = x;
    pull(p);
    pull(x);
};

vector<int> path;
auto splay = [&](int x) {
    path.clear();
    for (int y = x;; y = fa[y]) {
        path.push_back(y);
        if (isRoot(y)) break;
    }
    for (int i = (int)path.size() - 1; i >= 0; i--) push(path[i]);
    while (!isRoot(x)) {
        int p = fa[x], g = fa[p];
        if (!isRoot(p)) rotate((ch0[g] == p) == (ch0[p] == x) ? p : x);
        rotate(x);
    }
};

Fenwick fen(n);
auto access = [&](int x, int r) {
    int start = x;
    for (int y = 0; x; y = x, x = fa[x]) {
        splay(x);
        int c = colour[x];
        if (c) {
            int top = ch0[x] ? leftmost[ch0[x]] : x;
            int lowLen = link[top - 1] >= 0 ? len[link[top - 1]] : 0;
            fen.rangeAdd(c - len[x - 1] + 1, c - lowLen, -1);
        }
        ch1[x] = y;
        pull(x);
    }
    splay(start);
    apply(start, r);
};

```

```

    fen.rangeAdd(1, r, 1); // the whole root path: lengths 1..r ending at r
};

vector<int> ql(q), qr(q), order(q);
for (int i = 0; i < q; i++) cin >> ql[i] >> qr[i];
iota(order.begin(), order.end(), 0);
sort(order.begin(), order.end(), [&](int a, int b) { return qr[a] < qr[b]; });
vector<ll> answer(q);
int done = 0;
for (int id : order) {
    while (done < qr[id]) {
        done++;
        access(prefixState[done] + 1, done);
    }
    answer[id] = fen.rangeSum(ql[id], qr[id]);
}
string out;
for (ll x : answer) out += to_string(x) + '\n';
cout << out;
}

```

Python

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    s = data[0]
    n = len(s)
    q = int(data[1])

    # Suffix automaton with dict transitions.
    size = 2 * n + 1
    length = [0] * size
    link = [-1] * size
    trans = [None] * size
    trans[0] = {}
    prefix_state = [0] * (n + 1)
    states = 1
    last = 0
    for i in range(n):
        c = s[i]
        cur = states
        states += 1
        length[cur] = length[last] + 1
        trans[cur] = {}
        p = last
        while p != -1 and c not in trans[p]:
            trans[p][c] = cur

```

```

        p = link[p]
    if p == -1:
        link[cur] = 0
    else:
        qq = trans[p][c]
        if length[p] + 1 == length[qq]:
            link[cur] = qq
        else:
            clone = states
            states += 1
            length[clone] = length[p] + 1
            trans[clone] = dict(trans[qq])
            link[clone] = link[qq]
            while p != -1 and trans[p].get(c) == qq:
                trans[p][c] = clone
                p = link[p]
            link[qq] = clone
            link[cur] = clone
    last = cur
    prefix_state[i + 1] = cur
del trans

# Link-cut tree over suffix links; node id = state + 1, 0 is null.
N = states + 1
L = [0] * N
R = [0] * N
fa = [0] * N
colour = [0] * N
tag = [0] * N
leftmost = list(range(N))
low_len = [0] * N # len(link(state)) or 0 for the root
node_len = [0] * N
for v in range(1, N):
    st = v - 1
    node_len[v] = length[st]
    if link[st] >= 0:
        fa[v] = link[st] + 1
        low_len[v] = length[link[st]]

def is_root(x):
    p = fa[x]
    return not p or (L[p] != x and R[p] != x)

def push(x):
    t = tag[x]
    if t:
        a = L[x]
        if a:
            colour[a] = tag[a] = t
        b = R[x]
        if b:

```

```

        colour[b] = tag[b] = t
        tag[x] = 0

def rotate(x):
    p = fa[x]
    g = fa[p]
    if g and (L[g] == p or R[g] == p):
        if L[g] == p:
            L[g] = x
        else:
            R[g] = x
    fa[x] = g
    if R[p] == x:
        b = L[x]
        R[p] = b
        if b:
            fa[b] = p
        L[x] = p
    else:
        b = R[x]
        L[p] = b
        if b:
            fa[b] = p
        R[x] = p
    fa[p] = x
    leftmost[p] = leftmost[L[p]] if L[p] else p
    leftmost[x] = leftmost[L[x]] if L[x] else x

def splay(x):
    path = [x]
    y = x
    while not is_root(y):
        y = fa[y]
        path.append(y)
    for y in reversed(path):
        push(y)
    while not is_root(x):
        p = fa[x]
        if not is_root(p):
            g = fa[p]
            if (L[g] == p) == (L[p] == x):
                rotate(p)
            else:
                rotate(x)
        rotate(x)

# Fenwick trees for range add / range sum over start positions.
A = [0] * (n + 2)
B = [0] * (n + 2)
top_index = n + 1

```

```

def range_add(l, r, v):
    if l > r:
        return
    i = l
    vb = v * (l - 1)
    while i <= top_index:
        A[i] += v
        B[i] += vb
        i += i & -i
    i = r + 1
    vb = v * r
    while i <= top_index:
        A[i] -= v
        B[i] -= vb
        i += i & -i

def prefix(i):
    sa = 0
    sb = 0
    k = i
    while k > 0:
        sa += A[k]
        sb += B[k]
        k &= k - 1
    return sa * i - sb

def access(start, r):
    x = start
    y = 0
    while x:
        splay(x)
        c = colour[x]
        if c:
            top = leftmost[L[x]] if L[x] else x
            range_add(c - node_len[x] + 1, c - low_len[top], -1)
        R[x] = y
        leftmost[x] = leftmost[L[x]] if L[x] else x
        y = x
        x = fa[x]
    splay(start)
    colour[start] = tag[start] = r
    range_add(1, r, 1)

ql = [0] * q
qr = [0] * q
for i in range(q):
    ql[i] = int(data[2 + 2 * i])
    qr[i] = int(data[3 + 2 * i])
answer = [0] * q
done = 0
for i in sorted(range(q), key=qr.__getitem__):

```

```

    r = qr[i]
    while done < r:
        done += 1
        access(prefix_state[done] + 1, done)
    answer[i] = prefix(r) - prefix(ql[i] - 1)
    sys.stdout.write("\n".join(map(str, answer)) + "\n")

if __name__ == "__main__":
    main()

```

57. Prime Power Xor Sum

Min_25 sieve

f is multiplicative with $f(p) = p \text{ xor } 1$, which is $p - 1$ for odd p and 3 for $p = 2$. The only values that matter are the $O(\sqrt{N})$ distinct quotients $v = \text{floor}(N / i)$.

1. **Lucy DP.** For each such v , compute the number and the sum of primes up to v by sieving out, for each prime $p \leq \sqrt{N}$, the numbers whose least prime factor is p : $g(v) -= p^j (g(v/p) - g(p-1))$.
2. **Composite part.** Let $S(v, j)$ be the sum of $f(i)$ over $2 \leq i \leq v$ whose least prime factor exceeds p_j . Then

$$\begin{aligned}
 S(v, j) = & F_{\text{prime}}(v) - F_{\text{prime}}(p_j) \\
 & + \sum_{\{k > j, p_k^2 \leq v\}} \sum_{\{e \geq 1, p_k^{e+1} \leq v\}} \\
 & \quad (f(p_k^e) S(v/p_k^e, k) + f(p_k^{e+1})) .
 \end{aligned}$$

The answer is $S(N, 0) + 1$.

Correctness proof

Lemma 1. After processing all primes $\leq \sqrt{v}$, $g(v)$ counts (or sums) exactly the primes up to v .

Proof. Processing p removes the numbers in $[2, v]$ whose least prime factor is p . They are p times the numbers m with least prime factor at least p , i.e. the terms of $g(v/p) - g(p-1)$. Composite numbers up to v have least prime factor $\leq \sqrt{v}$.

Lemma 2. The recurrence for S counts every i with least prime factor $> p_j$ exactly once.

Proof. A prime i is counted in the prime term. A composite i factors uniquely as $p_k^e * m$ with p_k its least prime factor and m having least prime factor $> p_k$. Either $m > 1$, counted in $S(v/p_k^e, k)$, or $m = 1$ and $e \geq 2$, counted by the $f(p_k^{e+1})$ terms. Multiplicativity gives $f(i) = f(p_k^e) f(m)$.

Theorem. The output equals $\sum_{i \leq N} f(i)$ modulo the prime.

Proof. By Lemmas 1 and 2, plus $f(1) = 1$.

Complexity and traps

$O(N^{3/4} / \log N)$ time and $O(\sqrt{N})$ memory. $f(2) = 3$ differs from $p - 1$, so correct it in the prime sums. Values of f can be 0 (for example $f(4) = 2 \text{ xor } 2$), so never divide by f . Compute the square root exactly.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Min_25 sieve. Phase one (Lucy DP) finds, for every value  $v = N / i$ , the
// number and the sum of primes up to  $v$ ; since  $f(p) = p \bmod 1$  equals  $p - 1$  for
// odd  $p$  and  $3$  for  $p = 2$ , these give the prime part of the answer. Phase two
// adds composites by their least prime factor, recursing only while  $p^2 \leq v$ .

typedef long long ll;
const ll MOD = 1000000007;

ll N, sq;
vector<ll> primes, vals, gCount, gSum;
vector<int> idxSmall, idxLarge;

int index(ll v) { return v <= sq ? idxSmall[v] : idxLarge[N / v]; }

ll primeF(ll v) { // sum of  $f(p)$  over primes  $p \leq v$ , modulo MOD
    if (v < 2) return 0;
    int i = index(v);
    return ((gSum[i] - gCount[i] + 2) % MOD + MOD) % MOD;
}

ll S(ll v, int j) { // sum of  $f(i)$ ,  $2 \leq i \leq v$ , least prime factor  $> \text{primes}[j-1]$ 
    ll pj = j ? primes[j - 1] : 1;
    if (v < 2 || pj >= v) return 0;
    ll result = (primeF(v) - primeF(pj) + MOD) % MOD;
    for (int k = j; k < (int)primes.size() && primes[k] * primes[k] <= v; k++) {
        ll p = primes[k];
        ll pe = p;
        for (int e = 1; pe * p <= v; e++, pe *= p) {
            ll fe = (p ^ e) % MOD, fe1 = (p ^ (e + 1)) % MOD;
            result = (result + fe * S(v / pe, k + 1) + fe1) % MOD;
        }
    }
    return result;
}

int main() {
    cin >> N;
    sq = sqrtl((long double)N);
    while (sq * sq > N) sq--;
    while ((sq + 1) * (sq + 1) <= N) sq++;
    vector<char> composite(sq + 1, 0);
    for (ll i = 2; i <= sq; i++) {
        if (composite[i]) continue;
        primes.push_back(i);
        for (ll j = i * i; j <= sq; j += i) composite[j] = 1;
    }
}
```



```

idxSmall.assign(sq + 2, 0);
idxLarge.assign(sq + 2, 0);
for (ll i = 1; i <= N; i = N / (N / i) + 1) {
    ll v = N / i;
    int id = vals.size();
    vals.push_back(v);
    if (v <= sq) idxSmall[v] = id;
    else idxLarge[N / v] = id;
    gCount.push_back((v - 1) % MOD);
    // 2 + 3 + ... + v = v(v+1)/2 - 1
    __int128 t = (__int128)v * (v + 1) / 2 - 1;
    gSum.push_back((ll)(t % MOD));
}
for (ll p : primes) {
    ll pc = gCount[index(p - 1)], ps = gSum[index(p - 1)];
    for (size_t i = 0; i < vals.size() && vals[i] >= p * p; i++) {
        int k = index(vals[i] / p);
        gCount[i] = ((gCount[i] - (gCount[k] - pc)) % MOD + MOD) % MOD;
        gSum[i] = ((gSum[i] - p % MOD * ((gSum[k] - ps + MOD) % MOD)) % MOD + MOD) % MOD;
    }
}
cout << (S(N, 0) + 1) % MOD << '\n';
}

```

Python

```

import sys
from math import isqrt

MOD = 1000000007

def main():
    N = int(sys.stdin.read().split()[0])
    sq = isqrt(N)
    sieve = bytearray([1]) * (sq + 1)
    primes = []
    for i in range(2, sq + 1):
        if sieve[i]:
            primes.append(i)
            sieve[i * i::i] = bytearray(len(range(i * i, sq + 1, i)))

    # Lucy DP over the distinct values N // i (kept exact; Python ints are
    # unbounded, and reduction happens at the end).
    vals = []
    i = 1
    while i <= N:
        v = N // i
        vals.append(v)
        i = N // v + 1

```

```

small = [0] * (sq + 2)
large = [0] * (sq + 2)
for idx, v in enumerate(vals):
    if v <= sq:
        small[v] = idx
    else:
        large[N // v] = idx
count = [v - 1 for v in vals]
total = [v * (v + 1) // 2 - 1 for v in vals]
for p in primes:
    p2 = p * p
    if p2 > N:
        break
    base = small[p - 1]
    pc = count[base]
    ps = total[base]
    for idx, v in enumerate(vals):
        if v < p2:
            break
        w = v // p
        k = small[w] if w <= sq else large[N // w]
        count[idx] -= count[k] - pc
        total[idx] -= p * (total[k] - ps)
# Prime part: f(p) = p - 1 for odd p, f(2) = 3.
prime_f = [(s - c + 2) % MOD if v >= 2 else 0 for v, s, c in zip(vals, total, count)]

def pf(v):
    if v < 2:
        return 0
    return prime_f[small[v] if v <= sq else large[N // v]]

sys.setrecursionlimit(10000)
prime_count = len(primes)

def S(v, j):
    pj = primes[j - 1] if j else 1
    if v < 2 or pj >= v:
        return 0
    result = pf(v) - pf(pj)
    k = j
    while k < prime_count:
        p = primes[k]
        if p * p > v:
            break
        pe = p
        e = 1
        while pe * p <= v:
            result += (p ^ e) * S(v // pe, k + 1) + (p ^ (e + 1))
            pe *= p
            e += 1
        k += 1

```

```

    return result % MOD

    print((S(N, 0) + 1) % MOD)

if __name__ == "__main__":
    main()

```

58. Critical Link Audit

Dominators after subdividing links

Replace every link $u \rightarrow v$ by $u \rightarrow x_e \rightarrow v$ with a new vertex x_e . Closing link e equals deleting x_e . A station becomes unreachable exactly when every path to it from 1 passes through x_e , i.e. when x_e dominates it. So the answer for e is the number of original stations in the subtree of x_e in the dominator tree. Compute the tree with Lengauer–Tarjan (semi-dominators, path-compressed link-eval) on $n + m$ vertices.

Correctness proof

Lemma 1. A reachable station t is disconnected by closing e iff x_e dominates t in the subdivided graph.

Proof. Paths in the original graph avoiding e correspond to paths in the subdivided graph avoiding x_e , and conversely.

Lemma 2. The dominators of t are exactly its ancestors in the dominator tree.

Proof. This is the defining property of the tree built from immediate dominators, and Lengauer–Tarjan computes those immediate dominators correctly.

Theorem. Each output is the number of stations lost when that link closes.

Proof. By Lemmas 1 and 2 the lost stations are the original vertices in the subtree of x_e . An unreachable x_e loses nothing.

Complexity and traps

$O((n + m) \log(n + m))$ with simple path compression, and linear memory. The DFS and `eval` must be iterative. Parallel links correctly lose nothing because their subdivision vertices do not dominate each other. Unreachable links print 0.

C++

```

#include <bits/stdc++.h>
using namespace std;

// Subdivide every link u->v into u->x->v. Removing the link cuts exactly the
// vertices dominated by x, so the answer is the number of original stations in
// x's subtree of the dominator tree (Lengauer--Tarjan, path compression).

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int total = n + m;
    vector<int> head(total, -1), nxt(2 * m), to(2 * m);
    int edgeCount = 0;
    auto addEdge = [&](int a, int b) {
        to[edgeCount] = b;
        nxt[edgeCount] = head[a];
        head[a] = edgeCount++;
    };
    vector<pair<int, int>> links(m);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        links[i] = {u, v};
        addEdge(u, n + i);
        addEdge(n + i, v);
    }

    // Iterative DFS from station 1; vertices are renamed to preorder indices.
    vector<int> dfn(total, -1), vertexAt, parent;
    vertexAt.reserve(total);
    parent.reserve(total);
    {
        vector<pair<int, int>> stack;
        dfn[0] = 0;
        vertexAt.push_back(0);
        parent.push_back(-1);
        stack.push_back({0, head[0]});
        while (!stack.empty()) {
            auto& [v, e] = stack.back();
            if (e < 0) {
                stack.pop_back();
                continue;
            }
            int w = to[e];
            e = nxt[e];
            if (dfn[w] < 0) {
                dfn[w] = vertexAt.size();
                vertexAt.push_back(w);
                parent.push_back(dfn[v]);
                stack.push_back({w, head[w]});
            }
        }
    }
    int T = vertexAt.size();
}

```

```

// Predecessor lists in preorder numbering.
vector<int> predStart(T + 1, 0), predList;
for (int a = 0; a < total; a++)
    if (dfn[a] >= 0)
        for (int e = head[a]; e >= 0; e = nxt[e]) predStart[dfn[to[e]] + 1]++;
for (int i = 0; i < T; i++) predStart[i + 1] += predStart[i];
predList.resize(predStart[T]);
{
    vector<int> fill(predStart.begin(), predStart.end() - 1);
    for (int a = 0; a < total; a++)
        if (dfn[a] >= 0)
            for (int e = head[a]; e >= 0; e = nxt[e])
                predList[fill[dfn[to[e]]]++] = dfn[a];
}

vector<int> sdom(T), idom(T, 0), anc(T, -1), best(T), bucketHead(T, -1), bucketNext(T, -1);
iota(sdom.begin(), sdom.end(), 0);
iota(best.begin(), best.end(), 0);
vector<int> path;
auto eval = [&](int v) {
    if (anc[v] < 0) return v;
    path.clear();
    int x = v;
    while (anc[anc[x]] >= 0) {
        path.push_back(x);
        x = anc[x];
    }
    for (int i = (int)path.size() - 1; i >= 0; i--) {
        int y = path[i];
        if (sdom[best[anc[y]]] < sdom[best[y]]) best[y] = best[anc[y]];
        anc[y] = anc[anc[y]];
    }
    return best[v];
};
for (int w = T - 1; w >= 1; w--) {
    for (int i = predStart[w]; i < predStart[w + 1]; i++) {
        int u = eval(predList[i]);
        if (sdom[u] < sdom[w]) sdom[w] = sdom[u];
    }
    bucketNext[w] = bucketHead[sdom[w]];
    bucketHead[sdom[w]] = w;
    int p = parent[w];
    anc[w] = p;
    for (int v = bucketHead[p]; v >= 0; v = bucketNext[v]) {
        int u = eval(v);
        idom[v] = sdom[u] < sdom[v] ? u : p;
    }
    bucketHead[p] = -1;
}
for (int w = 1; w < T; w++)
    if (idom[w] != sdom[w]) idom[w] = idom[idom[w]];

```

```

vector<long long> below(T, 0);
for (int w = T - 1; w >= 0; w--) {
    if (vertexAt[w] < n) below[w]++;
    if (w) below[idom[w]] += below[w];
}
string out;
for (int i = 0; i < m; i++) {
    int x = dfn[n + i];
    out += to_string(x < 0 ? 0 : below[x]);
    out += '\n';
}
cout << out;
}

```

Python

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    n, m = int(data[0]), int(data[1])
    total = n + m
    # Subdivide link i (u -> v) as u -> n+i -> v.
    adj_start = [0] * (total + 1)
    src = [0] * (2 * m)
    dst = [0] * (2 * m)
    for i in range(m):
        u = int(data[2 + 2 * i]) - 1
        v = int(data[3 + 2 * i]) - 1
        x = n + i
        src[2 * i] = u
        dst[2 * i] = x
        src[2 * i + 1] = x
        dst[2 * i + 1] = v
    for a in src:
        adj_start[a + 1] += 1
    for i in range(total):
        adj_start[i + 1] += adj_start[i]
    adj = [0] * (2 * m)
    fill = adj_start[:]
    for a, b in zip(src, dst):
        adj[fill[a]] = b
        fill[a] += 1

    # Iterative preorder DFS from station 1.
    dfn = [-1] * total
    vertex_at = [0]
    parent = [-1]

```

```

dfn[0] = 0
stack_v = [0]
stack_e = [adj_start[0]]
while stack_v:
    v = stack_v[-1]
    e = stack_e[-1]
    if e == adj_start[v + 1]:
        stack_v.pop()
        stack_e.pop()
        continue
    stack_e[-1] = e + 1
    w = adj[e]
    if dfn[w] < 0:
        dfn[w] = len(vertex_at)
        vertex_at.append(w)
        parent.append(dfn[v])
        stack_v.append(w)
        stack_e.append(adj_start[w])
T = len(vertex_at)
# Predecessor lists in preorder numbering, stored contiguously.
pred_start = [0] * (T + 1)
for a, b in zip(src, dst):
    if dfn[a] >= 0:
        pred_start[dfn[b] + 1] += 1
for i in range(T):
    pred_start[i + 1] += pred_start[i]
pred = [0] * pred_start[T]
fill = pred_start[:]
for a, b in zip(src, dst):
    da = dfn[a]
    if da >= 0:
        db = dfn[b]
        pred[fill[db]] = da
        fill[db] += 1
del fill, src, dst, adj, adj_start

sdom = list(range(T))
idom = [0] * T
anc = [-1] * T
best = list(range(T))
bucket_head = [-1] * T
bucket_next = [-1] * T

def evaluate(v):
    if anc[v] < 0:
        return v
    path = []
    x = v
    while anc[anc[x]] >= 0:
        path.append(x)
        x = anc[x]

```

```

    for y in reversed(path):
        a = anc[y]
        if sdom[best[a]] < sdom[best[y]]:
            best[y] = best[a]
        anc[y] = anc[a]
    return best[v]

for w in range(T - 1, 0, -1):
    s = sdom[w]
    for i in range(pred_start[w], pred_start[w + 1]):
        v = pred[i]
        u = evaluate(v) if anc[v] >= 0 else v
        if sdom[u] < s:
            s = sdom[u]
    sdom[w] = s
    bucket_next[w] = bucket_head[s]
    bucket_head[s] = w
    p = parent[w]
    anc[w] = p
    v = bucket_head[p]
    while v >= 0:
        u = evaluate(v)
        idom[v] = u if sdom[u] < sdom[v] else p
        v = bucket_next[v]
    bucket_head[p] = -1
for w in range(1, T):
    if idom[w] != sdom[w]:
        idom[w] = idom[idom[w]]

below = [1 if vertex_at[w] < n else 0 for w in range(T)]
for w in range(T - 1, 0, -1):
    below[idom[w]] += below[w]
out = []
for i in range(m):
    x = dfn[n + i]
    out.append(below[x] if x >= 0 else 0)
sys.stdout.write("\n".join(map(str, out)) + "\n")

if __name__ == "__main__":
    main()

```

59. Walk Count Oracle

A recurrence hidden in the matrix

Let $a_i = e_s^T A^i e_t$ be the number of journeys of length i . By the Cayley–Hamilton theorem, the characteristic polynomial of A annihilates A , so (a_i) satisfies a linear recurrence of order at most n . Compute $a_0, \dots, a_{2n+1}$ by $2n + 2$ sparse vector steps. Berlekamp–Massey then returns the shortest recurrence Q consistent with them, which is the true minimal recurrence. Finally $[x^N] P(x)/Q(x)$ with $P = (a \cdot Q) \bmod$

x^L comes from Bostan–Mori: replace P/Q by $P(x)Q(-x) / Q(x)Q(-x)$ and keep the even or odd part, halving N each round.

Correctness proof

Lemma 1. The sequence has a linear recurrence of order at most n , valid from index 0, possibly with a nilpotent transient.

Proof. Take the minimal polynomial μ of A , of degree $\leq n$. Then $e_s^T A^i \mu(A) e_t = 0$ for all i , which is exactly such a recurrence.

Lemma 2. Given $2L$ terms of a sequence with a recurrence of order at most L , Berlekamp–Massey outputs a recurrence valid for the whole sequence.

Proof. Two recurrences of orders L_1, L_2 that agree on $L_1 + L_2$ terms generate the same sequence. The algorithm returns the shortest recurrence for the prefix, whose order is at most L .

Theorem. The output is a_N modulo the prime.

Proof. By Lemmas 1 and 2, $A(x) = P(x)/Q(x)$ as formal series. Each Bostan–Mori step preserves the coefficient $[x^N]$ of P/Q .

Complexity and traps

$O(nm + n^2 \log N)$ with schoolbook products ($O(n \log n \log N)$ with FFT-based products). Powering the $n \times n$ matrix directly, at $n^3 \log N$, is far too slow. Keep the transient: build P from the actual first L terms instead of assuming a pure recurrence. $N = 0$ counts the empty journey when $s = t$.

C++

```
#include <bits/stdc++.h>
using namespace std;

// a[i] = number of walks of length i from s to t = e_s^T A^i e_t.
// By Cayley--Hamilton the sequence satisfies a linear recurrence of order at
// most n, so 2n terms (each one sparse vector step) determine it through
// Berlekamp--Massey. The N-th term is then [x^N] P(x)/Q(x), extracted by
// Bostan--Mori halving in O(n^2 log N).

typedef long long ll;
const ll MOD = 998244353;

ll power(ll b, ll e) {
    ll r = 1;
    b %= MOD;
    while (e) {
        if (e & 1) r = r * b % MOD;
        b = b * b % MOD;
        e >>= 1;
    }
    return r;
}
```

```

}

vector<ll> berlekampMassey(const vector<ll>& s) {
    vector<ll> C = {1}, B = {1};
    int L = 0, m = 1;
    ll b = 1;
    for (size_t i = 0; i < s.size(); i++) {
        ll d = 0;
        for (int j = 0; j <= L; j++) d = (d + C[j] * s[i - j]) % MOD;
        if (d == 0) {
            m++;
            continue;
        }
        vector<ll> T = C;
        ll coef = d * power(b, MOD - 2) % MOD;
        if (C.size() < B.size() + m) C.resize(B.size() + m, 0);
        for (size_t j = 0; j < B.size(); j++) C[j + m] = (C[j + m] - coef * B[j] % MOD + MOD) % MOD;
        if (2 * L <= (int)i) {
            L = i + 1 - L;
            B = T;
            b = d;
            m = 1;
        } else {
            m++;
        }
    }
    C.resize(L + 1);
    return C; // Q(x) = C(x): sum_j C[j] a[i-j] = 0 for i >= L
}

vector<ll> multiply(const vector<ll>& a, const vector<ll>& b) {
    vector<unsigned long long> r(a.size() + b.size() - 1, 0);
    for (size_t i = 0; i < a.size(); i++) {
        if (!a[i]) continue;
        for (size_t j = 0; j < b.size(); j++) {
            r[i + j] += (unsigned long long)a[i] * b[j];
            if (r[i + j] >= (1ULL << 62)) r[i + j] %= MOD;
        }
    }
    vector<ll> out(r.size());
    for (size_t i = 0; i < r.size(); i++) out[i] = r[i] % MOD;
    return out;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    ll N;
    cin >> n >> m >> N >> s >> t;
    --s;

```

```

--t;
vector<int> eu(m), ev(m);
for (int i = 0; i < m; i++) {
    cin >> eu[i] >> ev[i];
    --eu[i];
    --ev[i];
}
int terms = 2 * n + 2;
vector<ll> seq(terms), cur(n, 0), nxt(n);
cur[s] = 1;
for (int i = 0; i < terms; i++) {
    seq[i] = cur[t];
    fill(nxt.begin(), nxt.end(), 0);
    for (int e = 0; e < m; e++) {
        nxt[ev[e]] += cur[eu[e]];
        if (nxt[ev[e]] >= MOD) nxt[ev[e]] -= MOD;
    }
    swap(cur, nxt);
}
if (N < terms) {
    cout << seq[N] << '\n';
    return 0;
}
vector<ll> Q = berlekampMassey(seq);
int L = Q.size() - 1;
if (L == 0) {
    cout << 0 << '\n';
    return 0;
}
vector<ll> P = multiply(vector<ll>(seq.begin(), seq.begin() + L), Q);
P.resize(L);
while (N > 0) {
    vector<ll> Qm = Q;
    for (size_t i = 1; i < Qm.size(); i += 2) Qm[i] = (MOD - Qm[i]) % MOD;
    vector<ll> U = multiply(P, Qm), V = multiply(Q, Qm);
    vector<ll> nP, nQ;
    for (size_t i = N & 1; i < U.size(); i += 2) nP.push_back(U[i]);
    for (size_t i = 0; i < V.size(); i += 2) nQ.push_back(V[i]);
    P = nP;
    Q = nQ;
    N >>= 1;
    if (P.empty()) {
        cout << 0 << '\n';
        return 0;
    }
}
cout << P[0] * power(Q[0], MOD - 2) % MOD << '\n';
}

```

Python

```
import sys

MOD = 998244353
SLOT = 20 # hex digits per packed coefficient (80 bits > log2(n * MOD^2))

def pack(a):
    return int("".join(format(x, "020x") for x in reversed(a)), 16) if a else 0

def multiply(a, b):
    """Polynomial product modulo MOD by Kronecker substitution."""
    if not a or not b:
        return []
    length = len(a) + len(b) - 1
    text = format(pack(a) * pack(b), "x").rjust(length * SLOT, "0")
    top = len(text)
    return [int(text[top - (i + 1) * SLOT: top - i * SLOT], 16) % MOD for i in range(length)]

def berlekamp_massey(s):
    C = [1]
    B = [1]
    L = 0
    m = 1
    b = 1
    for i in range(len(s)):
        d = 0
        for j in range(L + 1):
            d += C[j] * s[i - j]
        d %= MOD
        if d == 0:
            m += 1
            continue
        T = C[:]
        coef = d * pow(b, MOD - 2, MOD) % MOD
        if len(C) < len(B) + m:
            C.extend([0] * (len(B) + m - len(C)))
        for j, x in enumerate(B):
            C[j + m] = (C[j + m] - coef * x) % MOD
        if 2 * L <= i:
            L = i + 1 - L
            B = T
            b = d
            m = 1
        else:
            m += 1
    return C[:L + 1]
```

```

def main():
    data = sys.stdin.buffer.read().split()
    n, m, N, s, t = int(data[0]), int(data[1]), int(data[2]), int(data[3]) - 1, int(data[4]) - 1
    edges = [(int(data[5 + 2 * i]) - 1, int(data[6 + 2 * i]) - 1) for i in range(m)]
    edges.sort()
    terms = 2 * n + 2
    seq = []
    cur = [0] * n
    cur[s] = 1
    for _ in range(terms):
        seq.append(cur[t])
        nxt = [0] * n
        for u, v in edges:
            nxt[v] += cur[u]
        cur = [x % MOD for x in nxt]
    if N < terms:
        print(seq[N])
        return
    Q = berlekamp_massey(seq)
    L = len(Q) - 1
    if L == 0:
        print(0)
        return
    P = multiply(seq[:L], Q)[:L]
    while N:
        Qm = [x if i % 2 == 0 else (MOD - x) % MOD for i, x in enumerate(Q)]
        U = multiply(P, Qm)
        V = multiply(Q, Qm)
        P = U[N & 1::2]
        Q = V[0::2]
        N >>= 1
        if not P:
            print(0)
            return
    print(P[0] * pow(Q[0], MOD - 2, MOD) % MOD)

if __name__ == "__main__":
    main()

```

60. Taxicab Backbone

Bottlenecks live on a Manhattan MST

The minimum possible largest hop between two beacons equals the largest edge on their path in any minimum spanning tree of the complete Manhattan graph (the minimax path property of MSTs). Some MST uses only candidate edges from each point to its nearest neighbour in each of the eight 45-degree octants. A sweep in decreasing x with a Fenwick tree over $y - x$, minimizing $x + y$, finds one octant's nearest neighbours, and four coordinate transforms cover all eight octants. Answer queries during Kruskal: each component keeps its

pending queries, merging moves the smaller list, and a query whose endpoints join is answered by the current edge weight.

Correctness proof

Lemma 1. For each point and each octant, the nearest point in that octant is enough: some MST uses only these edges.

Proof. If q and r are both in point p 's octant and q is the nearest, then $|q - r| \leq |p - r|$ in Manhattan distance. So edge pr is a maximum on the cycle p, q, r and can be discarded by the cycle property.

Lemma 2. Kruskal answers a query (s, t) with the weight of the edge that first joins their components, which is the minimax bottleneck.

Proof. Components after all edges of weight $< w$ are exactly the classes of routes with maximum hop $< w$.

Theorem. Every printed value is the minimum required strength.

Proof. Combine Lemmas 1 and 2. Small-to-large moving keeps every pending query in the list of one of its endpoints' components.

Complexity and traps

$O(n \log n + q \log q)$ time. The sweep's ties decide which of two equal points sees the other, so sort by (x, y) and query before inserting. Distances reach $4 * 10^9$, beyond 32 bits. $s = t$ answers 0.

C++

```
#include <bits/stdc++.h>
using namespace std;

// The bottleneck between two beacons equals the largest edge on their path in
// a minimum spanning tree of the complete Manhattan graph. Only O(n)
// candidate edges are needed: for each of the eight octants around a point,
// its nearest neighbour in that octant. Four coordinate transforms plus a
// Fenwick tree of prefix minima find them. Queries are answered while
// Kruskal merges components, moving the smaller query list each time.

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];

    vector<tuple<long long, int, int>> edges;
    edges.reserve(4 * (size_t)n);
    vector<long long> px = x, py = y;
    vector<int> order(n);
    for (int dir = 0; dir < 4; dir++) {
```

```

if (dir == 1 || dir == 3)
    for (int i = 0; i < n; i++) swap(px[i], py[i]);
else if (dir == 2)
    for (int i = 0; i < n; i++) px[i] = -px[i];
iota(order.begin(), order.end(), 0);
sort(order.begin(), order.end(), [&](int a, int b) {
    return px[a] != px[b] ? px[a] < px[b] : py[a] < py[b];
});
vector<long long> keys(n);
for (int i = 0; i < n; i++) keys[i] = py[i] - px[i];
vector<long long> sortedKeys = keys;
sort(sortedKeys.begin(), sortedKeys.end());
sortedKeys.erase(unique(sortedKeys.begin(), sortedKeys.end()), sortedKeys.end());
int K = sortedKeys.size();
// Fenwick over reversed key order: prefix = all keys >= given key.
vector<long long> bestValue(K + 1, LLONG_MAX);
vector<int> bestId(K + 1, -1);
for (int t = n - 1; t >= 0; t--) {
    int i = order[t];
    int pos = lower_bound(sortedKeys.begin(), sortedKeys.end(), keys[i]) - sortedKeys.begin();
    int r = K - pos; // 1-based index in reversed order
    long long value = LLONG_MAX;
    int id = -1;
    for (int k = r; k > 0; k -= k & -k)
        if (bestValue[k] < value) value = bestValue[k], id = bestId[k];
    if (id >= 0) edges.emplace_back(1llabs(x[i] - x[id]) + 1llabs(y[i] - y[id]), i, id);
    long long mine = px[i] + py[i];
    for (int k = r; k <= K; k += k & -k)
        if (mine < bestValue[k]) bestValue[k] = mine, bestId[k] = i;
}
}
sort(edges.begin(), edges.end());

vector<long long> answer(q, -1);
vector<vector<int>> pending(n);
vector<int> qa(q), qb(q);
for (int i = 0; i < q; i++) {
    cin >> qa[i] >> qb[i];
    --qa[i];
    --qb[i];
    if (qa[i] == qb[i]) {
        answer[i] = 0;
    } else {
        pending[qa[i]].push_back(i);
        pending[qb[i]].push_back(i);
    }
}

vector<int> parent(n);
iota(parent.begin(), parent.end(), 0);
auto find = [&](int v) {
    while (parent[v] != v) v = parent[v] = parent[parent[v]];

```

```

        return v;
};
for (auto& [w, a, b] : edges) {
    a = find(a);
    b = find(b);
    if (a == b) continue;
    if (pending[a].size() > pending[b].size()) swap(a, b);
    // Queries of the smaller side either finish now or move to b.
    for (int id : pending[a]) {
        if (answer[id] >= 0) continue;
        int other = find(qa[id]) == a ? find(qb[id]) : find(qa[id]);
        if (other == b) answer[id] = w;
        else pending[b].push_back(id);
    }
    vector<int>().swap(pending[a]);
    parent[a] = b;
}
string out;
for (long long v : answer) out += to_string(v) + '\n';
cout << out;
}

```

Python

```

import sys
from bisect import bisect_left

def main():
    data = sys.stdin.buffer.read().split()
    n, q = int(data[0]), int(data[1])
    x = [int(v) for v in data[2:2 + 2 * n:2]]
    y = [int(v) for v in data[3:3 + 2 * n:2]]
    pos = 2 + 2 * n

    # Nearest neighbour in each octant: four transforms, one Fenwick sweep each.
    # Candidate edges are packed as weight << 36 | i << 18 | j to save memory.
    edges = []
    px = x[:]
    py = y[:]
    for direction in range(4):
        if direction == 1 or direction == 3:
            px, py = py, px
        elif direction == 2:
            px = [-v for v in px]
        order = sorted(range(n), key=lambda i: (px[i], py[i]))
        keys = [b - a for a, b in zip(px, py)]
        sorted_keys = sorted(set(keys))
        K = len(sorted_keys)
        rank = {k: K - i for i, k in enumerate(sorted_keys)}

```



```

del sorted_keys
big = 1 << 62
best_value = [big] * (K + 1)
best_id = [-1] * (K + 1)
for t in range(n - 1, -1, -1):
    i = order[t]
    r = rank[keys[i]]
    value = big
    j = -1
    k = r
    while k > 0:
        if best_value[k] < value:
            value = best_value[k]
            j = best_id[k]
        k &= k - 1
    if j >= 0:
        edges.append((abs(x[i] - x[j]) + abs(y[i] - y[j]))) << 36 | i << 18 | j)
    mine = px[i] + py[i]
    k = r
    while k <= K:
        if mine < best_value[k]:
            best_value[k] = mine
            best_id[k] = i
        k += k & -k
edges.sort()

answer = [-1] * q
qa = [0] * q
qb = [0] * q
pending = [[] for _ in range(n)]
for i in range(q):
    a = int(data[pos]) - 1
    b = int(data[pos + 1]) - 1
    pos += 2
    qa[i] = a
    qb[i] = b
    if a == b:
        answer[i] = 0
    else:
        pending[a].append(i)
        pending[b].append(i)

parent = list(range(n))

def find(v):
    root = v
    while parent[root] != root:
        root = parent[root]
    while parent[v] != root:
        parent[v], v = root, parent[v]
    return root

```

```

mask = (1 << 18) - 1
for packed in edges:
    w = packed >> 36
    a = find(packed >> 18 & mask)
    b = find(packed & mask)
    if a == b:
        continue
    if len(pending[a]) > len(pending[b]):
        a, b = b, a
    target = pending[b]
    for i in pending[a]:
        if answer[i] >= 0:
            continue
        u = find(qa[i])
        other = find(qb[i]) if u == a else u
        if other == b:
            answer[i] = w
        else:
            target.append(i)
    pending[a] = []
    parent[a] = b
sys.stdout.write("\n".join(map(str, answer)) + "\n")

if __name__ == "__main__":
    main()

```

61. Cavern Descent

Lines from descendants

$dp[v] = \min$ over proper descendants u of $a[v] * b[u] + dp[u]$, and exits have $dp = 0$. The descendant u is the line $y = b[u] X + dp[u]$, queried at $X = a[v]$. Each vertex needs a structure holding its subtree's lines that supports minimum queries at a point and merging with its children's structures. Li Chao trees over X in $[-10^5, 10^5]$ merge by re-inserting the line of each node of the smaller tree into the other while walking both trees in parallel.

Correctness proof

Lemma 1. A Li Chao tree query returns the minimum over all inserted lines at X .

Proof. Each node keeps the line that is better at its midpoint. The loser can only win on one side, where it is inserted. The query visits every node whose interval contains X .

Lemma 2. Merging tree B into tree A yields a Li Chao tree over the union of their lines.

Proof. Each line of B is inserted into A at the position of its node or above it. Insertion is valid from any node whose interval contains the line's useful region, and the recursion processes every node of B .

Theorem. dp is computed correctly for every chamber.

Proof. Process vertices in post-order. Before v is queried, its structure contains exactly the lines of its proper descendants (Lemma 2). Lemma 1 gives the minimum.

Complexity and traps

A line only moves downward during merges, so the total merge work is $O(n \log C)$ with $C = 2 * 10^5 + 1$, plus $O(n \log C)$ for queries. Recycle the emptied nodes of merged trees to keep the memory linear. dp values reach $2 * 10^{15}$, so use 64 bits. The root is not an exit unless $n = 1$.

C++

```
#include <bits/stdc++.h>
using namespace std;

// dp[v] = min over proper descendants u of a[v] * b[u] + dp[u].
// Each descendant is the line y = b[u] * X + dp[u], queried at X = a[v].
// Every vertex owns a Li Chao tree over X in [-LIM, LIM]; children's trees are
// merged by re-inserting the lines of one tree's nodes into the other.
// Merging is O(n log C) amortized: a line only ever moves downward.

const int LIM = 100000;

struct Node {
    long long k, m;
    int left, right;
};

vector<Node> pool;
vector<int> freeNodes; // nodes emptied by merges are recycled

long long eval(const Node& nd, long long x) { return nd.k * x + nd.m; }

int newNode(long long k, long long m) {
    if (!freeNodes.empty()) {
        int t = freeNodes.back();
        freeNodes.pop_back();
        pool[t] = {k, m, 0, 0};
        return t;
    }
    pool.push_back({k, m, 0, 0});
    return pool.size() - 1;
}

// Insert line (k, m) into the subtree rooted at t covering [lo, hi].
int insertLine(int t, int lo, int hi, long long k, long long m) {
    if (!t) return newNode(k, m);
    int root = t;
    while (true) {
        int mid = (lo + hi) >> 1; // arithmetic shift: floor division
        bool leftBetter = k * lo + m < eval(pool[t], lo);
```

```

    bool midBetter = k * mid + m < eval(pool[t], mid);
    if (midBetter) {
        swap(pool[t].k, k);
        swap(pool[t].m, m);
    }
    if (lo == hi) break;
    if (leftBetter != midBetter) {
        if (!pool[t].left) {
            int c = newNode(k, m);
            pool[t].left = c;
            break;
        }
        t = pool[t].left;
        hi = mid;
    } else {
        if (!pool[t].right) {
            int c = newNode(k, m);
            pool[t].right = c;
            break;
        }
        t = pool[t].right;
        lo = mid + 1;
    }
}
return root;
}

int mergeTrees(int a, int b, int lo, int hi) {
    if (!a || !b) return a ^ b;
    a = insertLine(a, lo, hi, pool[b].k, pool[b].m);
    int bl = pool[b].left, br = pool[b].right;
    freeNodes.push_back(b);
    if (lo == hi) return a;
    int mid = (lo + hi) >> 1; // arithmetic shift: floor division
    int nl = mergeTrees(pool[a].left, bl, lo, mid);
    pool[a].left = nl;
    int nr = mergeTrees(pool[a].right, br, mid + 1, hi);
    pool[a].right = nr;
    return a;
}

long long query(int t, long long x) {
    long long best = LLONG_MAX;
    int lo = -LIM, hi = LIM;
    while (t) {
        best = min(best, eval(pool[t], x));
        int mid = (lo + hi) >> 1;
        if (x <= mid) t = pool[t].left, hi = mid;
        else t = pool[t].right, lo = mid + 1;
    }
    return best;
}

```

```

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n), b(n);
    for (auto& v : a) cin >> v;
    for (auto& v : b) cin >> v;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    pool.reserve(n + 1);
    pool.push_back({0, 0, 0, 0}); // node 0 is the null sentinel
    vector<int> order, parent(n, -1);
    order.reserve(n);
    vector<int> stack = {0};
    parent[0] = 0;
    while (!stack.empty()) {
        int v = stack.back();
        stack.pop_back();
        order.push_back(v);
        for (int w : adj[v])
            if (parent[w] < 0) parent[w] = v, stack.push_back(w);
    }
    vector<int> tree(n, 0);
    vector<long long> dp(n, 0);
    for (int i = n - 1; i >= 0; i--) {
        int v = order[i];
        dp[v] = tree[v] ? query(tree[v], a[v]) : 0;
        tree[v] = insertLine(tree[v], -LIM, LIM, b[v], dp[v]);
        if (v) tree[parent[v]] = mergeTrees(tree[parent[v]], tree[v], -LIM, LIM);
    }
    string out;
    for (int i = 0; i < n; i++) {
        out += to_string(dp[i]);
        out += i + 1 < n ? ' ' : '\n';
    }
    cout << out;
}

```

Python

```
import sys

LIM = 100000

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = [int(v) for v in data[1:1 + n]]
    b = [int(v) for v in data[1 + n:1 + 2 * n]]
    adj = [[] for _ in range(n)]
    pos = 1 + 2 * n
    for _ in range(n - 1):
        u = int(data[pos]) - 1
        v = int(data[pos + 1]) - 1
        pos += 2
        adj[u].append(v)
        adj[v].append(u)

    # Li Chao node pool; index 0 is the null node. Nodes emptied by a merge
    # are recycled, so the pool never holds more than n + 1 nodes.
    K = [0]
    M = [0]
    L = [0]
    R = [0]
    free = []

    def new_node(k, m):
        if free:
            t = free.pop()
            K[t] = k
            M[t] = m
            L[t] = 0
            R[t] = 0
            return t
        K.append(k)
        M.append(m)
        L.append(0)
        R.append(0)
        return len(K) - 1

    def insert(t, lo, hi, k, m):
        if not t:
            return new_node(k, m)
        root = t
        while True:
            mid = (lo + hi) >> 1
            tk = K[t]
            tm = M[t]
```

```

    left_better = k * lo + m < tk * lo + tm
    mid_better = k * mid + m < tk * mid + tm
    if mid_better:
        K[t] = k
        M[t] = m
        k = tk
        m = tm
    if lo == hi:
        return root
    if left_better != mid_better:
        child = L[t]
        if not child:
            L[t] = new_node(k, m)
            return root
        t = child
        hi = mid
    else:
        child = R[t]
        if not child:
            R[t] = new_node(k, m)
            return root
        t = child
        lo = mid + 1

def merge(x, y, lo, hi):
    if not x or not y:
        return x or y
    x = insert(x, lo, hi, K[y], M[y])
    yl = L[y]
    yr = R[y]
    free.append(y)
    if lo == hi:
        return x
    mid = (lo + hi) >> 1
    L[x] = merge(L[x], yl, lo, mid)
    R[x] = merge(R[x], yr, mid + 1, hi)
    return x

def query(t, x):
    best = None
    lo = -LIM
    hi = LIM
    while t:
        value = K[t] * x + M[t]
        if best is None or value < best:
            best = value
        mid = (lo + hi) >> 1
        if x <= mid:
            t = L[t]
            hi = mid
        else:

```

```

        t = R[t]
        lo = mid + 1
    return best

parent = [-1] * n
parent[0] = 0
order = []
stack = [0]
while stack:
    v = stack.pop()
    order.append(v)
    for w in adj[v]:
        if parent[w] < 0:
            parent[w] = v
            stack.append(w)
tree = [0] * n
dp = [0] * n
for v in reversed(order):
    t = tree[v]
    value = query(t, a[v]) if t else 0
    dp[v] = value
    t = insert(t, -LIM, LIM, b[v], value)
    if v:
        p = parent[v]
        tree[p] = merge(tree[p], t, -LIM, LIM)
    tree[v] = 0
sys.stdout.write(" ".join(map(str, dp)) + "\n")

if __name__ == "__main__":
    main()

```

62. Window Component Census

Keep the newest spanning forest

Scan cables $1, 2, \dots, m$ while maintaining a spanning forest that prefers newer cables. When cable i joins two vertices that are already connected, remove the oldest cable on the path between them and insert i . Record the removed index as $\text{pre}[i]$: 0 when nothing is removed, and i for a self-loop. The forest lives in a link-cut tree whose cable nodes carry their index as weight, with path-minimum aggregates.

A window $[l, r]$ has $n - \#\{i \text{ in } [l, r] : \text{pre}[i] < l\}$ components, counted offline by sorting queries by l and cables by pre .

Correctness proof

Lemma 1. After processing cable r , for every $l \leq r$ the forest cables with index $\geq l$ form a spanning forest of the graph on cables $1..r$.

Proof. The forest is a maximum spanning forest with respect to cable index, and the cut and cycle properties hold for that order. Restricting a maximum spanning forest to weights ≥ 1 gives a spanning forest of the subgraph with those weights.

Lemma 2. Cable $i \leq r$ is in the forest after step r with index ≥ 1 , and contributes to that spanning forest, iff $\text{pre}[i] < 1$.

Proof. Cable i joins two components of the graph on cables $\text{pre}[i]+1..i$ but not of the graph on cables $\text{pre}[i]..i$. Hence it merges components within $[1, i]$ iff $1 > \text{pre}[i]$, and cables merging components are exactly the forest edges of Lemma 1.

Theorem. Each answer equals the number of components of the window's graph.

Proof. Components equal n minus the forest size. By Lemma 2 the forest size is the counted quantity.

Complexity and traps

$O((m + q) \log n)$ amortized with the link-cut tree and a Fenwick tree, and $O(n + m + q)$ memory. Cable nodes separate parallel cables so that the oldest one can be cut. Self-loops never merge components. Cut both halves of the replaced cable node.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Scan cables in order while keeping the spanning forest that prefers newer
// cables (a link-cut tree over vertex nodes and cable nodes, path minimum of
// cable index). When cable  $i$  closes a cycle, the oldest cable on that cycle
// leaves the forest; call its index  $\text{pre}[i]$  ( $\text{pre}[i] = 0$  when nothing leaves,
//  $\text{pre}[i] = i$  for a self-loop). For a window  $[l, r]$ , cable  $i$  in the window
// merges two components of the window graph iff  $\text{pre}[i] < l$ , so the answer is
//  $n - \#\{i \text{ in } [l, r] : \text{pre}[i] < l\}$ , counted offline with a Fenwick tree.

const int INF = INT_MAX;
vector<int> ch[2], par, val, mn;
vector<char> rev;

bool isRoot(int x) { return !par[x] || (ch[0][par[x]] != x && ch[1][par[x]] != x); }
void pull(int x) {
    mn[x] = x;
    for (int d = 0; d < 2; d++)
        if (ch[d][x] && val[mn[ch[d][x]]] < val[mn[x]]) mn[x] = mn[ch[d][x]];
}
void flip(int x) {
    if (x) {
        swap(ch[0][x], ch[1][x]);
        rev[x] ^= 1;
    }
}
void push(int x) {
```

```

    if (rev[x]) {
        flip(ch[0][x]);
        flip(ch[1][x]);
        rev[x] = 0;
    }
}

void rotate(int x) {
    int p = par[x], g = par[p];
    int d = ch[1][p] == x;
    if (!isRoot(p)) ch[ch[1][g] == p][g] = x;
    par[x] = g;
    ch[d][p] = ch[!d][x];
    if (ch[!d][x]) par[ch[!d][x]] = p;
    ch[!d][x] = p;
    par[p] = x;
    pull(p);
    pull(x);
}

void splay(int x) {
    static vector<int> path;
    path.clear();
    for (int y = x;; y = par[y]) {
        path.push_back(y);
        if (isRoot(y)) break;
    }
    for (int i = (int)path.size() - 1; i >= 0; i--) push(path[i]);
    while (!isRoot(x)) {
        int p = par[x], g = par[p];
        if (!isRoot(p)) rotate((ch[0][g] == p) == (ch[0][p] == x) ? p : x);
        rotate(x);
    }
}

void access(int x) {
    for (int last = 0; x; last = x, x = par[x]) {
        splay(x);
        ch[1][x] = last;
        pull(x);
    }
}

void makeRoot(int x) {
    access(x);
    splay(x);
    flip(x);
}

int findRoot(int x) {
    access(x);
    splay(x);
    while (true) {
        push(x);
        if (!ch[0][x]) break;
        x = ch[0][x];
    }
}

```

```

    }
    splay(x);
    return x;
}

void link(int x, int y) {
    makeRoot(x);
    par[x] = y;
}

void cut(int x, int y) {
    makeRoot(x);
    access(y);
    splay(y);
    ch[0][y] = 0;
    par[x] = 0;
    pull(y);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    int total = n + m + 1;
    for (auto& c : ch) c.assign(total, 0);
    par.assign(total, 0);
    val.assign(total, INF);
    mn.resize(total);
    rev.assign(total, 0);
    for (int i = 0; i < total; i++) mn[i] = i;
    vector<int> eu(m + 1), ev(m + 1), pre(m + 1, 0);
    for (int i = 1; i <= m; i++) {
        cin >> eu[i] >> ev[i];
        int u = eu[i], v = ev[i], node = n + i;
        val[node] = i;
        if (u == v) {
            pre[i] = i;
            continue;
        }
        if (findRoot(u) == findRoot(v)) {
            makeRoot(u);
            access(v);
            splay(v);
            int oldest = mn[v];
            int j = oldest - n;
            pre[i] = j;
            cut(eu[j], oldest);
            cut(oldest, ev[j]);
        }
        link(u, node);
        link(node, v);
    }
}

```

```

// Offline count of i in [l, r] with pre[i] < l: sweep l upward.
vector<int> ql(q), qr(q), order(q);
for (int i = 0; i < q; i++) cin >> ql[i] >> qr[i];
iota(order.begin(), order.end(), 0);
sort(order.begin(), order.end(), [&](int a, int b) { return ql[a] < ql[b]; });
vector<int> byPre(m);
iota(byPre.begin(), byPre.end(), 1);
sort(byPre.begin(), byPre.end(), [&](int a, int b) { return pre[a] < pre[b]; });
vector<int> bit(m + 1, 0);
vector<int> answer(q);
size_t ptr = 0;
for (int id : order) {
    while (ptr < byPre.size() && pre[byPre[ptr]] < ql[id]) {
        for (int k = byPre[ptr]; k <= m; k += k & -k) bit[k]++;
        ptr++;
    }
    auto prefix = [&](int k) {
        int s = 0;
        for (; k > 0; k -= k & -k) s += bit[k];
        return s;
    };
    answer[id] = n - (prefix(qr[id]) - prefix(ql[id] - 1));
}
string out;
for (int x : answer) out += to_string(x) + '\n';
cout << out;
}

```

Python

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    n, m, q = int(data[0]), int(data[1]), int(data[2])
    total = n + m + 1
    INF = 1 << 60
    L = [0] * total
    R = [0] * total
    P = [0] * total
    val = [INF] * total
    mn = list(range(total))
    rev = [False] * total

    def pull(x):
        best = x
        bv = val[x]
        c = L[x]

```

```

    if c and val[mn[c]] < bv:
        best = mn[c]
        bv = val[best]
    c = R[x]
    if c and val[mn[c]] < bv:
        best = mn[c]
    mn[x] = best

def push(x):
    if rev[x]:
        a = L[x]
        b = R[x]
        if a:
            L[a], R[a] = R[a], L[a]
            rev[a] = not rev[a]
        if b:
            L[b], R[b] = R[b], L[b]
            rev[b] = not rev[b]
        rev[x] = False

def is_root(x):
    p = P[x]
    return not p or (L[p] != x and R[p] != x)

def rotate(x):
    p = P[x]
    g = P[p]
    if L[p] == x:
        b = R[x]
        L[p] = b
        R[x] = p
    else:
        b = L[x]
        R[p] = b
        L[x] = p
    if b:
        P[b] = p
    if g and (L[g] == p or R[g] == p):
        if L[g] == p:
            L[g] = x
        else:
            R[g] = x
    P[x] = g
    P[p] = x
    pull(p)
    pull(x)

def splay(x):
    path = [x]
    y = x
    while not is_root(y):

```

```

        y = P[y]
        path.append(y)
    for y in reversed(path):
        push(y)
    while not is_root(x):
        p = P[x]
        if not is_root(p):
            g = P[p]
            if (L[g] == p) == (L[p] == x):
                rotate(p)
            else:
                rotate(x)
        rotate(x)

def access(x):
    last = 0
    while x:
        splay(x)
        R[x] = last
        pull(x)
        last = x
        x = P[x]

def make_root(x):
    access(x)
    splay(x)
    L[x], R[x] = R[x], L[x]
    rev[x] = not rev[x]

def find_root(x):
    access(x)
    splay(x)
    while True:
        push(x)
        if not L[x]:
            break
        x = L[x]
    splay(x)
    return x

def link(x, y):
    make_root(x)
    P[x] = y

def cut(x, y):
    make_root(x)
    access(y)
    splay(y)
    L[y] = 0
    P[x] = 0
    pull(y)

```

```

eu = [0] * (m + 1)
ev = [0] * (m + 1)
pre = [0] * (m + 1)
pos = 3
for i in range(1, m + 1):
    u = int(data[pos])
    v = int(data[pos + 1])
    pos += 2
    eu[i] = u
    ev[i] = v
    node = n + i
    val[node] = i
    mn[node] = node
    if u == v:
        pre[i] = i
        continue
    if find_root(u) == find_root(v):
        make_root(u)
        access(v)
        splay(v)
        oldest = mn[v]
        j = oldest - n
        pre[i] = j
        cut(eu[j], oldest)
        cut(oldest, ev[j])
    link(u, node)
    link(node, v)

# Offline: count i in [l, r] with pre[i] < l, sweeping l upward.
ql = [0] * q
qr = [0] * q
for i in range(q):
    ql[i] = int(data[pos])
    qr[i] = int(data[pos + 1])
    pos += 2
order = sorted(range(q), key=ql.__getitem__)
by_pre = sorted(range(1, m + 1), key=pre.__getitem__)
bit = [0] * (m + 1)
answer = [0] * q
ptr = 0
for qi in order:
    l = ql[qi]
    while ptr < m and pre[by_pre[ptr]] < l:
        k = by_pre[ptr]
        while k <= m:
            bit[k] += 1
            k += k & -k
        ptr += 1
    s = 0
    k = qr[qi]
    while k > 0:

```

```

        s += bit[k]
        k &= k - 1
    k = l - 1
    while k > 0:
        s -= bit[k]
        k &= k - 1
    answer[qi] = n - s
    sys.stdout.write("\n".join(map(str, answer)) + "\n")

if __name__ == "__main__":
    main()

```

63. Periodic Loop Frontier

Plug DP produces a recurrence

Process the floor row by row and cell by cell with a broken-profile DP. A state stores $W + 1$ plugs, each empty, an opening bracket, or a closing bracket. Matching brackets are the two ends of one path fragment. Every cell must have degree two. Joining two opening (or two closing) brackets re-matches the partner, and joining a matched pair would close a loop, which is allowed only at the last cell of a row when no other plug is open. That closing transition contributes a_N , the count for N rows.

Row-boundary profiles form a finite set U , and $a_N = c^T T^{(N-1)} v$ for the row transfer matrix T . Thus (a_N) has a linear recurrence of order at most $|U|$ (1117 for $W = 10$). Generate $2|U| + 8$ terms, recover the recurrence with Berlekamp–Massey, and jump to N with Bostan–Mori.

Correctness proof

Lemma 1. The DP counts sets of moves in which every processed cell has degree two and the moves form one cycle.

Proof. Brackets encode the non-crossing pairing of path ends on the frontier. Each transition preserves that encoding. Forbidding a pair to close early prevents extra cycles, and requiring an empty profile at closing forces all cells to be covered by that single cycle.

Lemma 2. Once the cumulative set of reachable row profiles stops growing for one row, it is closed under the row transition.

Proof. The successors of the new row’s profiles are successors of profiles already seen, which were already recorded.

Theorem. The output is the number of patrols modulo the prime.

Proof. By Lemma 1 the generated terms are correct. Lemma 2 bounds the recurrence order by the recorded $|U|$, so $2|U|$ terms determine it (as in Problem 59).

Complexity and traps

With S the number of mid-row states, the plug DP costs $O(|U| W S)$, Berlekamp–Massey costs $O(|U|^2)$, and Bostan–Mori costs $O(|U|^2 \log N)$. Directly powering the 1117×1117 transfer matrix would need about 10^{11} operations. The DP matches the known counts (37 for 4×6 , 4638576 for 8×8) and $a(W, N) = a(N, W)$.

C++

```
#include <bits/stdc++.h>
using namespace std;

// a_N = number of Hamiltonian cycles of the W x N grid. A broken-profile
// (plug) DP with bracket-encoded plugs produces a_1, a_2, ... row by row: the
// single cycle may only close at the last cell of a row with no other plug
// open. Row-boundary profiles form a finite set U, so a_N = c^T T^{(N-1)} v
// satisfies a linear recurrence of order <= |U|. Generate 2|U| + 8 terms,
// recover the recurrence by Berlekamp--Massey, and jump to N with
// Bostan--Mori. (|U| = 1117 for W = 10, so powering T directly is too slow.)

typedef long long ll;
const ll MOD = 998244353;

ll power(ll b, ll e) {
    ll r = 1;
    b %= MOD;
    while (e) {
        if (e & 1) r = r * b % MOD;
        b = b * b % MOD;
        e >>= 1;
    }
    return r;
}

vector<ll> berlekampMassey(const vector<ll>& s) {
    vector<ll> C = {1}, B = {1};
    int L = 0, m = 1;
    ll b = 1;
    for (size_t i = 0; i < s.size(); i++) {
        ll d = 0;
        for (int j = 0; j <= L; j++) d = (d + C[j] * s[i - j]) % MOD;
        if (d == 0) {
            m++;
            continue;
        }
        vector<ll> T = C;
        ll coef = d * power(b, MOD - 2) % MOD;
        if (C.size() < B.size() + m) C.resize(B.size() + m, 0);
        for (size_t j = 0; j < B.size(); j++) C[j + m] = (C[j + m] - coef * B[j] % MOD + MOD) % MOD;
        if (2 * L <= (int)i) {
```

```

        L = i + 1 - L;
        B = T;
        b = d;
        m = 1;
    } else {
        m++;
    }
}
C.resize(L + 1);
return C;
}

vector<ll> multiply(const vector<ll>& a, const vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    vector<unsigned long long> r(a.size() + b.size() - 1, 0);
    for (size_t i = 0; i < a.size(); i++) {
        if (!a[i]) continue;
        for (size_t j = 0; j < b.size(); j++) {
            r[i + j] += (unsigned long long)a[i] * b[j];
            if (r[i + j] >= (1ULL << 62)) r[i + j] %= MOD;
        }
    }
    vector<ll> out(r.size());
    for (size_t i = 0; i < r.size(); i++) out[i] = r[i] % MOD;
    return out;
}

ll nthTerm(const vector<ll>& seq, ll index) {
    if (index < (ll)seq.size()) return seq[index];
    vector<ll> Q = berlekampMassey(seq);
    int L = Q.size() - 1;
    if (L == 0) return 0;
    vector<ll> P = multiply(vector<ll>(seq.begin(), seq.begin() + L), Q);
    P.resize(L);
    while (index) {
        vector<ll> Qm = Q;
        for (size_t i = 1; i < Qm.size(); i += 2) Qm[i] = (MOD - Qm[i]) % MOD;
        vector<ll> U = multiply(P, Qm), V = multiply(Q, Qm), nP, nQ;
        for (size_t i = index & 1; i < U.size(); i += 2) nP.push_back(U[i]);
        for (size_t i = 0; i < V.size(); i += 2) nQ.push_back(V[i]);
        P = nP;
        Q = nQ;
        index >>= 1;
        if (P.empty()) return 0;
    }
    return P[0] * power(Q[0], MOD - 2) % MOD;
}

int W;

int partner(unsigned st, int k) {

```

```

int depth = 0;
if (((st >> (2 * k)) & 3) == 1) {
    for (int t = k;; t++) {
        unsigned v = (st >> (2 * t)) & 3;
        if (v == 1) depth++;
        else if (v == 2 && --depth == 0) return t;
    }
}
for (int t = k;; t--) {
    unsigned v = (st >> (2 * t)) & 3;
    if (v == 2) depth++;
    else if (v == 1 && --depth == 0) return t;
}
}

int main() {
    ll N;
    cin >> W >> N;
    unsigned full = (1u << (2 * (W + 1))) - 1;
    unordered_map<unsigned, ll> cur, nxt;
    cur[0] = 1;
    unordered_set<unsigned> seen = {0};
    vector<ll> seq;
    while (true) {
        nxt.clear();
        for (auto& [st, c] : cur) {
            if (st >> (2 * W)) continue;
            unsigned ns = (st << 2) & full;
            nxt[ns] = (nxt[ns] + c) % MOD;
        }
        swap(cur, nxt);
        ll closed = 0;
        for (int j = 0; j < W; j++) {
            nxt.clear();
            int sj = 2 * j, sk = sj + 2;
            bool last = j + 1 == W;
            auto put = [&](unsigned key, ll c) {
                ll& slot = nxt[key];
                slot += c;
                if (slot >= MOD) slot -= MOD;
            };
            for (auto& [st, c] : cur) {
                unsigned L = (st >> sj) & 3, U = (st >> sk) & 3;
                unsigned base = st & ~(15u << sj);
                if (!L && !U) {
                    if (!last) put(base | (1u << sj) | (2u << sk), c);
                } else if (!L || !U) {
                    unsigned x = L | U;
                    put(base | (x << sj), c);
                    if (!last) put(base | (x << sk), c);
                } else if (L == 1 && U == 1) {

```

```

        int p = partner(st, j + 1);
        put((base & ~(3u << (2 * p))) | (1u << (2 * p)), c);
    } else if (L == 2 && U == 2) {
        int p = partner(st, j);
        put((base & ~(3u << (2 * p))) | (2u << (2 * p)), c);
    } else if (L == 2) {
        put(base, c);
    } else if (last && base == 0) {
        closed = (closed + c) % MOD; // the single cycle closes here
    }
    }
    swap(cur, nxt);
}
for (auto& [st, c] : cur) seen.insert(st);
seq.push_back(closed);
if ((ll)seq.size() > N || seq.size() >= 2 * seen.size() + 8) break;
}
cout << nthTerm(seq, N - 1) << '\n';
}

```

Python

```

import sys

MOD = 998244353
SLOT = 20 # hex digits per packed coefficient in Kronecker multiplication

def pack(a):
    return int("".join(format(x, "020x") for x in reversed(a)), 16) if a else 0

def multiply(a, b):
    if not a or not b:
        return []
    length = len(a) + len(b) - 1
    text = format(pack(a) * pack(b), "x").rjust(length * SLOT, "0")
    top = len(text)
    return [int(text[top - (i + 1) * SLOT: top - i * SLOT], 16) % MOD for i in range(length)]

def berlekamp_massey(s):
    C = [1]
    B = [1]
    L = 0
    m = 1
    b = 1
    for i in range(len(s)):
        d = 0
        for j in range(L + 1):

```

```

        d += C[j] * s[i - j]
    d %= MOD
    if d == 0:
        m += 1
        continue
    T = C[:]
    coef = d * pow(b, MOD - 2, MOD) % MOD
    if len(C) < len(B) + m:
        C.extend([0] * (len(B) + m - len(C)))
    for j, x in enumerate(B):
        C[j + m] = (C[j + m] - coef * x) % MOD
    if 2 * L <= i:
        L = i + 1 - L
        B = T
        b = d
        m = 1
    else:
        m += 1
    return C[:L + 1]

```

```

def nth_term(seq, index):
    """seq satisfies a linear recurrence; return its term at `index`."""
    if index < len(seq):
        return seq[index]
    Q = berlekamp_massey(seq)
    L = len(Q) - 1
    if L == 0:
        return 0
    P = multiply(seq[:L], Q)[:L]
    while index:
        Qm = [x if i % 2 == 0 else (MOD - x) % MOD for i, x in enumerate(Q)]
        U = multiply(P, Qm)
        V = multiply(Q, Qm)
        P = U[index & 1::2]
        Q = V[0::2]
        index >>= 1
        if not P:
            return 0
    return P[0] * pow(Q[0], MOD - 2, MOD) % MOD

```

```

def cycle_counts(W):
    """Yield (a_N, number of distinct row-boundary profiles so far) for N=1,2,...

    A profile stores W+1 plugs in 2-bit fields (0 none, 1 '(', 2 ')').
    Field j is the left plug of cell j and field j+1 its upper plug.
    """
    top_shift = 2 * W
    full = (1 << (2 * (W + 1))) - 1

```

```

def partner(st, k):
    if (st >> (2 * k)) & 3 == 1:
        depth = 0
        t = k
        while True:
            v = (st >> (2 * t)) & 3
            if v == 1:
                depth += 1
            elif v == 2:
                depth -= 1
                if depth == 0:
                    return t
            t += 1
    depth = 0
    t = k
    while True:
        v = (st >> (2 * t)) & 3
        if v == 2:
            depth += 1
        elif v == 1:
            depth -= 1
            if depth == 0:
                return t
        t -= 1

cur = {0: 1}
seen = {0}
while True:
    nxt = {}
    for st, c in cur.items():
        if st >> top_shift:
            continue
        ns = (st << 2) & full
        nxt[ns] = (nxt.get(ns, 0) + c) % MOD
    cur = nxt
    closed = 0
    for j in range(W):
        sj = 2 * j
        sk = sj + 2
        clear = ~(15 << sj)
        last = j + 1 == W
        nxt = {}
        get = nxt.get
        for st, c in cur.items():
            L = (st >> sj) & 3
            U = (st >> sk) & 3
            base = st & clear
            if not L and not U:
                if not last:
                    key = base | (1 << sj) | (2 << sk)
                    nxt[key] = (get(key, 0) + c) % MOD

```

```

        elif not L or not U:
            x = L | U
            key = base | (x << sj)
            nxt[key] = (get(key, 0) + c) % MOD
            if not last:
                key = base | (x << sk)
                nxt[key] = (get(key, 0) + c) % MOD
        elif L == 1 and U == 1:
            p = partner(st, j + 1)
            key = (base & ~(3 << (2 * p))) | (1 << (2 * p))
            nxt[key] = (get(key, 0) + c) % MOD
        elif L == 2 and U == 2:
            p = partner(st, j)
            key = (base & ~(3 << (2 * p))) | (2 << (2 * p))
            nxt[key] = (get(key, 0) + c) % MOD
        elif L == 2:
            nxt[base] = (get(base, 0) + c) % MOD
        elif last and base == 0:
            closed = (closed + c) % MOD # the single cycle closes here
    cur = nxt
    seen.update(cur.keys())
    yield closed, len(seen)

def main():
    W, N = map(int, sys.stdin.read().split())
    seq = []
    for value, profiles in cycle_counts(W):
        seq.append(value)
        if len(seq) > N or len(seq) >= 2 * profiles + 8:
            break
    print(nth_term(seq, N - 1))

if __name__ == "__main__":
    main()

```

64. Beacon Placement

Binary search plus 2-SAT

Binary search the answer d : is there a placement with all pairwise distances $\geq d$? Sort the $2n$ sites. Literal k says “the owner of site k uses site k ”; its negation is the owner’s other site. Choosing site k forbids every other site strictly closer than d , which is a contiguous block of the sorted order. Build a segment tree whose leaf for site j implies “the owner of j uses its other site”. Then k implies $O(\log n)$ tree nodes, and 2-SAT (Tarjan’s SCC) decides the question.

Correctness proof

Lemma 1. A placement with minimum distance $\geq d$ is exactly an assignment satisfying, for every pair of chosen sites $j \neq k$ with distance $< d$, the clause $\text{not}(k) \text{ or } \text{not}(j)$.

Proof. The clause forbids choosing both sites of a close pair. Each beacon picks exactly one of its two literals by the pairing of negations.

Lemma 2. The segment-tree graph has a path from k to $\text{not}(j)$ iff j is within distance $< d$ of k and $j \neq k$.

Proof. Tree edges only go downward, from a covering node to its leaves, and each leaf leads to the negation of its site. The range of k covers exactly the close sites other than k .

Theorem. The binary search returns the maximum achievable minimum distance.

Proof. Feasibility is monotone in d . By Lemmas 1 and 2 the 2-SAT instance is satisfiable iff d is feasible, and 2-SAT is satisfiable iff no literal shares a strongly connected component with its negation.

Complexity and traps

$O(n \log n \log C)$ with $C = 10^9$. Coinciding sites are within distance $0 < d$. The two sites of one beacon may coincide, which must not create a clause against itself. Recursive SCC overflows the stack, so use an iterative Tarjan.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Binary search the answer d. Sort all 2n candidate sites; literal k means
// "the beacon owning site k is placed at site k", and its negation is the
// owner's other site. Choosing site k forbids every other site closer than d,
// a contiguous block of the sorted order. A segment tree whose leaves point to
// "the owner uses its other site" turns each block into  $O(\log n)$  implication
// edges, and Tarjan's SCC algorithm decides the resulting 2-SAT instance.

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    int S = 2 * n;
    vector<pair<long long, int>> sites(S);
    for (int i = 0; i < n; i++) {
        long long a, b;
        cin >> a >> b;
        sites[2 * i] = {a, i};
        sites[2 * i + 1] = {b, i};
    }
    sort(sites.begin(), sites.end());
    vector<long long> pos(S);
    vector<int> partner(S), firstOf(n, -1);
```



```

for (int k = 0; k < S; k++) {
    pos[k] = sites[k].first;
    int owner = sites[k].second;
    if (firstOf[owner] < 0) {
        firstOf[owner] = k;
    } else {
        partner[k] = firstOf[owner];
        partner[firstOf[owner]] = k;
    }
}

int size = 1;
while (size < S) size *= 2;
int V = S + 2 * size; // literals, then segment-tree nodes
auto treeNode = [&](int t) { return S + t; };

vector<int> head(V), nxt, to;
auto feasible = [&](long long d) {
    fill(head.begin(), head.end(), -1);
    nxt.clear();
    to.clear();
    auto add = [&](int a, int b) {
        to.push_back(b);
        nxt.push_back(head[a]);
        head[a] = to.size() - 1;
    };
    for (int t = 1; t < size; t++) {
        add(treeNode(t), treeNode(2 * t));
        add(treeNode(t), treeNode(2 * t + 1));
    }
    for (int k = 0; k < S; k++) add(treeNode(size + k), partner[k]);
    auto cover = [&](int k, int l, int r) { // k -> every leaf in [l, r)
        for (l += size, r += size; l < r; l >>= 1, r >>= 1) {
            if (l & 1) add(k, treeNode(l++));
            if (r & 1) add(k, treeNode(--r));
        }
    };
    for (int k = 0; k < S; k++) {
        int lo = lower_bound(pos.begin(), pos.end(), pos[k] - d + 1) - pos.begin();
        int hi = lower_bound(pos.begin(), pos.end(), pos[k] + d) - pos.begin();
        cover(k, lo, k);
        cover(k, k + 1, hi);
    }
    // Iterative Tarjan.
    vector<int> index(V, -1), low(V), comp(V, -1), stackV, callV, callE;
    vector<char> onStack(V, 0);
    int counter = 0, comps = 0;
    for (int s = 0; s < V; s++) {
        if (index[s] >= 0) continue;
        callV.push_back(s);
        callE.push_back(head[s]);
        index[s] = low[s] = counter++;
    }
}

```

```

    stackV.push_back(s);
    onStack[s] = 1;
    while (!callV.empty()) {
        int v = callV.back();
        int& e = callE.back();
        if (e >= 0) {
            int w = to[e];
            e = nxt[e];
            if (index[w] < 0) {
                index[w] = low[w] = counter++;
                stackV.push_back(w);
                onStack[w] = 1;
                callV.push_back(w);
                callE.push_back(head[w]);
            } else if (onStack[w]) {
                low[v] = min(low[v], index[w]);
            }
            continue;
        }
        callV.pop_back();
        callE.pop_back();
        if (!callV.empty()) low[callV.back()] = min(low[callV.back()], low[v]);
        if (low[v] == index[v]) {
            while (true) {
                int w = stackV.back();
                stackV.pop_back();
                onStack[w] = 0;
                comp[w] = comps;
                if (w == v) break;
            }
            comps++;
        }
    }
}

for (int k = 0; k < S; k++)
    if (comp[k] == comp[partner[k]]) return false;
return true;
};

long long lo = 0, hi = pos.back() - pos.front() + 1; // lo feasible, hi not
while (hi - lo > 1) {
    long long mid = (lo + hi) / 2;
    if (feasible(mid)) lo = mid;
    else hi = mid;
}
cout << lo << '\n';
}

```

Python

```
import sys
from bisect import bisect_left

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    S = 2 * n
    sites = sorted((int(data[1 + k]), k >> 1) for k in range(S))
    pos = [p for p, _ in sites]
    partner = [0] * S
    first = [-1] * n
    for k, (_, owner) in enumerate(sites):
        if first[owner] < 0:
            first[owner] = k
        else:
            partner[k] = first[owner]
            partner[first[owner]] = k
    size = 1
    while size < S:
        size *= 2
    V = S + 2 * size

    # Fixed part of the implication graph: tree edges and leaf -> negation.
    base = [[] for _ in range(V)]
    for t in range(1, size):
        base[S + t] = [S + 2 * t, S + 2 * t + 1]
    for k in range(S):
        base[S + size + k] = [partner[k]]

    def feasible(d):
        adj = base[:]
        for k in range(S):
            out = []
            p = pos[k]
            for l, r in ((bisect_left(pos, p - d + 1), k), (k + 1, bisect_left(pos, p + d))):
                l += size
                r += size
                while l < r:
                    if l & 1:
                        out.append(S + l)
                        l += 1
                    if r & 1:
                        r -= 1
                        out.append(S + r)
                l >>= 1
                r >>= 1
            adj[k] = out
        # Iterative Tarjan.
```

```

index = [-1] * V
low = [0] * V
comp = [-1] * V
on_stack = [False] * V
stack = []
counter = 0
comps = 0
for s in range(V):
    if index[s] >= 0:
        continue
    index[s] = low[s] = counter
    counter += 1
    stack.append(s)
    on_stack[s] = True
    call = [(s, iter(adj[s]))]
    while call:
        v, it = call[-1]
        advanced = False
        for w in it:
            if index[w] < 0:
                index[w] = low[w] = counter
                counter += 1
                stack.append(w)
                on_stack[w] = True
                call.append((w, iter(adj[w])))
                advanced = True
                break
            if on_stack[w] and index[w] < low[v]:
                low[v] = index[w]
        if advanced:
            continue
        call.pop()
    if call:
        u = call[-1][0]
        if low[v] < low[u]:
            low[u] = low[v]
    if low[v] == index[v]:
        while True:
            w = stack.pop()
            on_stack[w] = False
            comp[w] = comps
            if w == v:
                break
        comps += 1
for k in range(S):
    if comp[k] == comp[partner[k]]:
        return False
return True

```

```

lo, hi = 0, pos[-1] - pos[0] + 1
while hi - lo > 1:

```

```

    mid = (lo + hi) // 2
    if feasible(mid):
        lo = mid
    else:
        hi = mid
print(lo)

if __name__ == "__main__":
    main()

```

65. Temporal Steiner Span

Union of root paths, minus the part above the LCA

Root the tree at 1, and let each vertex own the cable to its parent. The minimal connecting subgraph of the sites $1..r$ is the union of their root paths minus the root path of $\text{LCA}(1..r)$. Sweep r upward and paint the root path of r with colour r . Then a cable lies on the root path of some site in $[1, r]$ iff its colour is at least 1. A Fenwick tree over colours stores the total cable length of each colour, so the union has length $\text{sum}(\text{colours} \geq 1)$.

With heavy–light decomposition, every root path is a sequence of chain **prefixes**. Each chain keeps a stack of equally coloured blocks from its head downward. Painting a prefix pops fully covered blocks, trims one partial block, and pushes the new block, each change updating the Fenwick tree. $\text{LCA}(1..r)$ is the LCA of the two sites with the smallest and largest DFS entry time, found with sparse tables.

Correctness proof

Lemma 1. After painting $1..r$, every cable’s colour is the largest site $\leq r$ whose root path contains it.

Proof. Painting in increasing order overwrites older colours.

Lemma 2. The union of the root paths of $1..r$ minus the root path of their LCA is the minimal connecting subgraph.

Proof. Every such root path passes through the LCA. The part above it is used by no connecting path, and every cable below the LCA on a root path separates some site from the LCA.

Lemma 3. The LCA of a vertex set is the LCA of its minimum and maximum DFS entry times.

Proof. Both extremes lie in the LCA’s subtree and in different child subtrees (unless all sites share one), and every site lies between them in DFS order.

Theorem. Each answer is the minimum lit length.

Proof. By Lemma 1 the Fenwick suffix gives the union’s length, by Lemma 3 the depth of the correct LCA is subtracted, and Lemma 2 identifies the result.

Complexity and traps

Each paint touches $O(\log n)$ chains, with amortized $O(1)$ block changes per chain, so the time is $O((n + q) \log^2 n)$ and the memory is $O(n \log n)$ for the sparse tables. HLD LCA must compare the unweighted depths of chain heads, not weighted depths. Sum cable lengths in 64 bits.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Root the tree at 1 and let vertex v own the edge to its parent. Sweep r
// upward and paint the root path of r with colour r. The union of the root
// paths of labels l..r is exactly the set of edges whose current colour is
// >= l; subtracting the root path of LCA(l..r) leaves the Steiner tree.
// With heavy-light decomposition every root path is a sequence of chain
// prefixes, so each chain keeps a stack of equally coloured blocks and a
// Fenwick tree over colours stores the total edge length of each colour.
// LCA(l..r) is the LCA of the labels with smallest and largest DFS entry time.

typedef long long ll;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<vector<pair<int, ll>>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    vector<int> parent(n + 1, 0), order, heavy(n + 1, 0), sz(n + 1, 1);
    vector<ll> up(n + 1, 0), depth(n + 1, 0);
    vector<int> level(n + 1, 0);
    order.reserve(n);
    {
        vector<int> stack = {1};
        parent[1] = 0;
        vector<char> seen(n + 1, 0);
        seen[1] = 1;
        while (!stack.empty()) {
            int v = stack.back();
            stack.pop_back();
            order.push_back(v);
            for (auto [w, len] : adj[v])
                if (!seen[w]) {
                    seen[w] = 1;
                    parent[w] = v;
                    up[w] = len;
                    depth[w] = depth[v] + len;
                    level[w] = level[v] + 1;
                    stack.push_back(w);
                }
        }
    }
}
```

```

    }
}
for (int i = n - 1; i > 0; i--) sz[parent[order[i]]] += sz[order[i]];
for (int i = n - 1; i > 0; i--) {
    int v = order[i], p = parent[v];
    if (!heavy[p] || sz[v] > sz[heavy[p]]) heavy[p] = v;
}
// Positions: each heavy chain occupies a contiguous block, head first.
vector<int> head(n + 1), pos(n + 1), tin(n + 1), tout(n + 1);
{
    int cur = 0;
    vector<int> stack = {1};
    head[1] = 1;
    while (!stack.empty()) {
        int v = stack.back();
        stack.pop_back();
        for (int x = v; x; x = heavy[x]) {
            if (x != v) head[x] = head[v];
            pos[x] = cur++;
            for (auto [w, len] : adj[x])
                if (w != parent[x] && w != heavy[x]) {
                    head[w] = w;
                    stack.push_back(w);
                }
        }
    }
}
// Euler entry times for the range-LCA trick.
{
    int timer = 0;
    vector<pair<int, int>> stack = {{1, 0}};
    while (!stack.empty()) {
        auto& [v, i] = stack.back();
        if (i == 0) tin[v] = timer++;
        if (i < (int)adj[v].size()) {
            int w = adj[v][i++].first;
            if (w != parent[v]) stack.push_back({w, 0});
        } else {
            tout[v] = timer;
            stack.pop_back();
        }
    }
}
auto lca = [&](int a, int b) {
    while (head[a] != head[b]) {
        if (level[head[a]] < level[head[b]]) swap(a, b);
        a = parent[head[a]];
    }
    return pos[a] < pos[b] ? a : b;
};
vector<ll> prefix(n + 1, 0); // prefix[k] = sum of up[] over positions < k

```

```

{
    vector<ll> byPos(n);
    for (int v = 1; v <= n; v++) byPos[pos[v]] = up[v];
    for (int k = 0; k < n; k++) prefix[k + 1] = prefix[k] + byPos[k];
}
// Sparse tables over labels for min / max entry time.
int LOG = 1;
while ((1 << LOG) <= n) LOG++;
vector<vector<int>> mnT(LOG, vector<int>(n + 1)), mxT(LOG, vector<int>(n + 1));
for (int v = 1; v <= n; v++) mnT[0][v] = mxT[0][v] = v;
for (int k = 1; k < LOG; k++)
    for (int v = 1; v + (1 << k) - 1 <= n; v++) {
        int a = mnT[k - 1][v], b = mnT[k - 1][v + (1 << (k - 1))];
        mnT[k][v] = tin[a] < tin[b] ? a : b;
        a = mxT[k - 1][v];
        b = mxT[k - 1][v + (1 << (k - 1))];
        mxT[k][v] = tin[a] > tin[b] ? a : b;
    }
auto rangeLca = [&](int l, int r) {
    int k = 31 - __builtin_clz(r - l + 1);
    int a = mnT[k][l], b = mnT[k][r - (1 << k) + 1];
    int lo = tin[a] < tin[b] ? a : b;
    a = mxT[k][l];
    b = mxT[k][r - (1 << k) + 1];
    int hi = tin[a] > tin[b] ? a : b;
    return lca(lo, hi);
};

vector<ll> bit(n + 1, 0);
auto bitAdd = [&](int i, ll x) {
    for (; i <= n; i += i & -i) bit[i] += x;
};
auto bitSum = [&](int i) {
    ll s = 0;
    for (; i > 0; i -= i & -i) s += bit[i];
    return s;
};
struct Block {
    int from, to, colour; // positions [from, to] on one chain
};
vector<vector<Block>> blocks(n + 1); // per chain head; top block = back()
auto paint = [&](int v, int colour) {
    while (v) {
        int h = head[v], a = pos[h], b = pos[v];
        auto& st = blocks[h];
        while (!st.empty() && st.back().to <= b) {
            Block& blk = st.back();
            bitAdd(blk.colour, -(prefix[blk.to + 1] - prefix[blk.from]));
            st.pop_back();
        }
        if (!st.empty() && st.back().from <= b) {

```



```

        Block& blk = st.back();
        bitAdd(blk.colour, -(prefix[b + 1] - prefix[blk.from]));
        blk.from = b + 1;
    }
    st.push_back({a, b, colour});
    bitAdd(colour, prefix[b + 1] - prefix[a]);
    v = parent[h];
}

};

vector<int> ql(q), qr(q), byR(q);
for (int i = 0; i < q; i++) cin >> ql[i] >> qr[i];
iota(byR.begin(), byR.end(), 0);
sort(byR.begin(), byR.end(), [&](int a, int b) { return qr[a] < qr[b]; });
vector<ll> answer(q);
int painted = 0;
for (int id : byR) {
    while (painted < qr[id]) {
        painted++;
        paint(painted, painted);
    }
    ll covered = bitSum(n) - bitSum(ql[id] - 1);
    answer[id] = covered - depth[rangeLca(ql[id], qr[id])];
}
string out;
for (ll x : answer) out += to_string(x) + '\n';
cout << out;
}

```

Python

```

import sys
from array import array

def main():
    data = sys.stdin.buffer.read().split()
    n, q = int(data[0]), int(data[1])
    adj = [[] for _ in range(n + 1)]
    ptr = 2
    for _ in range(n - 1):
        u = int(data[ptr])
        v = int(data[ptr + 1])
        w = int(data[ptr + 2])
        ptr += 3
        adj[u].append((v, w))
        adj[v].append((u, w))
    ql = array("i", map(int, data[ptr:ptr + 2 * q:2]))
    qr = array("i", map(int, data[ptr + 1:ptr + 2 * q:2]))
    del data

```

```

parent = [0] * (n + 1)
up = [0] * (n + 1)
depth = [0] * (n + 1)
level = [0] * (n + 1)
order = []
seen = [False] * (n + 1)
seen[1] = True
stack = [1]
while stack:
    v = stack.pop()
    order.append(v)
    for w, length in adj[v]:
        if not seen[w]:
            seen[w] = True
            parent[w] = v
            up[w] = length
            depth[w] = depth[v] + length
            level[w] = level[v] + 1
            stack.append(w)
size = [1] * (n + 1)
heavy = [0] * (n + 1)
for v in reversed(order):
    p = parent[v]
    if p:
        size[p] += size[v]
for v in reversed(order):
    p = parent[v]
    if p and (not heavy[p] or size[v] > size[heavy[p]]):
        heavy[p] = v

# Heavy chains get contiguous positions, head first.
head = [0] * (n + 1)
pos = [0] * (n + 1)
head[1] = 1
cur = 0
stack = [1]
while stack:
    v = stack.pop()
    x = v
    while x:
        head[x] = v
        pos[x] = cur
        cur += 1
        for w, _ in adj[x]:
            if w != parent[x] and w != heavy[x]:
                stack.append(w)
        x = heavy[x]

# Preorder entry times (any DFS order works for the range-LCA trick).
tin = [0] * (n + 1)
for t, v in enumerate(order):
    tin[v] = t

```

```

del order, size, adj

def lca(a, b):
    while head[a] != head[b]:
        if level[head[a]] < level[head[b]]:
            a, b = b, a
        a = parent[head[a]]
    return a if pos[a] < pos[b] else b

prefix = [0] * (n + 1)
by_pos = [0] * n
for v in range(1, n + 1):
    by_pos[pos[v]] = up[v]
for k in range(n):
    prefix[k + 1] = prefix[k] + by_pos[k]

# Sparse tables over labels: vertex with min / max entry time. Rows are
# compact int arrays to keep memory low.
mn = [array("i", range(n + 1))]
mx = [array("i", range(n + 1))]
k = 1
while (1 << k) <= n:
    pm = mn[-1]
    px = mx[-1]
    half = 1 << (k - 1)
    cm = list(pm)
    cx = list(px)
    for v in range(1, n - (1 << k) + 2):
        a = pm[v]
        b = pm[v + half]
        cm[v] = a if tin[a] < tin[b] else b
        a = px[v]
        b = px[v + half]
        cx[v] = a if tin[a] > tin[b] else b
    mn.append(array("i", cm))
    mx.append(array("i", cx))
    del cm, cx
    k += 1

def range_lca(l, r):
    k = (r - l + 1).bit_length() - 1
    a = mn[k][l]
    b = mn[k][r - (1 << k) + 1]
    lo = a if tin[a] < tin[b] else b
    a = mx[k][l]
    b = mx[k][r - (1 << k) + 1]
    hi = a if tin[a] > tin[b] else b
    return lca(lo, hi)

bit = [0] * (n + 1)

```

```

def bit_add(i, x):
    while i <= n:
        bit[i] += x
        i += i & -i

def bit_sum(i):
    s = 0
    while i > 0:
        s += bit[i]
        i &= i - 1
    return s

# Per chain: stack of blocks [from, to, colour]; the top block is nearest
# to the chain head.
blocks = [[] for _ in range(n + 1)]

def paint(v, colour):
    while v:
        h = head[v]
        a = pos[h]
        b = pos[v]
        st = blocks[h]
        while st and st[-1][1] <= b:
            blk = st.pop()
            bit_add(blk[2], prefix[blk[0]] - prefix[blk[1] + 1])
        if st and st[-1][0] <= b:
            blk = st[-1]
            bit_add(blk[2], prefix[blk[0]] - prefix[b + 1])
            blk[0] = b + 1
        st.append([a, b, colour])
        bit_add(colour, prefix[b + 1] - prefix[a])
        v = parent[h]

answer = [0] * q
painted = 0
for i in sorted(range(q), key=qr.__getitem__):
    r = qr[i]
    while painted < r:
        painted += 1
        paint(painted, painted)
    l = ql[i]
    answer[i] = bit_sum(n) - bit_sum(l - 1) - depth[lca(l, r)]
sys.stdout.write("\n".join(map(str, answer)) + "\n")

if __name__ == "__main__":
    main()

```